

Azure Databricks Data Engineer: Complete Study and Interview Guide

Exam DP-750: Azure Fundamentals to Job-Ready, From Scratch

Prepared for Sai

August 2026

Contents

1	Part 1 — Orientation, Azure Fundamentals & Full Environment Setup	3
1.1	1. How to Use This Guide	3
1.2	2. Azure Fundamentals Primer	5
1.3	3. Complete Environment & Installation Setup Guide	12
2	Part 2 — Databricks Workspace, Compute & Unity Catalog Security and Governance	22
2.1	How to use this chapter	22
2.2	1. Databricks Workspace & Compute Fundamentals	23
2.3	2. Unity Catalog Object Model	28
2.4	3. Securing Unity Catalog	34
2.5	4. Governing Unity Catalog	37
2.6	5. Mini Quiz — Domains 1 & 2	41
3	Part 3 — Data Engineering Core: Delta Lake, Ingestion, Transformation & Data Quality	46
3.1	Why this chapter matters most	46
3.2	1. Delta Lake Deep Dive	47
3.3	2. Medallion Architecture & Data Modeling in Unity Catalog	52
3.4	3. Data Ingestion	56
3.5	4. Transformations at Scale	61
3.6	5. Data Quality Constraints	67
3.7	6. Mini Quiz — Domain 3 (Original Practice Questions)	69
4	Part 4 — Orchestration, CI/CD, Monitoring & Performance	75
4.1	1. Pipeline & Job Orchestration with Lakeflow Jobs	76
4.2	2. Scheduling, Triggers & Alerting	80
4.3	3. Development Lifecycle: Git, Repos & CI/CD	84
4.4	4. Monitoring, Troubleshooting & Performance	88
4.5	5. Mini Quiz — Domain 4	92
5	Part 5 — Capstone Project: A Production-Style Retail Lakehouse on Azure Databricks	96
5.1	1. Project Overview & Architecture	97
5.2	2. Phase 1 — Set Up the Source Systems	99
5.3	3. Phase 2 — Bronze Layer: Raw Ingestion	106
5.4	4. Phase 3 — Silver Layer: Clean, Conform & Govern	110

5.5	5. Phase 4 — Gold Layer: Business-Ready Aggregates	115
5.6	6. Phase 5 — Orchestrate It All	118
5.7	7. Phase 6 — Ship It with CI/CD	121
5.8	8. Phase 7 — Monitor & Optimize	125
5.9	9. What to Put on Your Resume and GitHub	127
6	Part 6 — Exam Practice Question Bank	131
6.1	Domain 1 Practice Questions — Set Up and Configure an Azure Databricks Environment	131
6.2	Domain 2 Practice Questions — Secure and Govern Unity Catalog Objects	136
6.3	Domain 3 Practice Questions — Prepare and Process Data	142
6.4	Domain 4 Practice Questions — Deploy and Maintain Data Pipelines and Workloads	152
6.5	Final Mixed Mock Exam (20 Questions)	163
6.6	Answer Key — Final Mixed Mock Exam	167
7	Part 7 — Interview Preparation, Quick Reference & Your 30-Day Study Calendar	170
7.1	1. 30 Real-World Interview Questions & Model Answers	170
7.2	2. Scenario / System-Design Style Questions	178
7.3	3. Command & Syntax Cheat Sheet	183
7.4	4. Glossary of Key Terms	188
7.5	5. Your 30-Day Study Calendar	191

Chapter 1

Part 1 — Orientation, Azure Fundamentals & Full Environment Setup

1.1 1. How to Use This Guide

Welcome. This book is a self-contained study path for the **DP-750: Implementing Data Engineering Solutions Using Azure Databricks** exam — Microsoft’s “Azure Databricks Data Engineer Associate” certification. You have a data science background: you already know Python and SQL cold, you understand what a DataFrame is, you’ve trained models, you’ve written joins and window functions. What you *don’t* have yet is Databricks, Azure, or the “platform engineering” muscle memory of provisioning cloud resources, configuring access control, and building production pipelines. This book exists to close that gap in about a month, without wasting your time re-explaining pandas.

1.1.1 The seven parts

The book is organized into seven parts, roughly mapping onto the exam’s four skill domains plus the scaffolding around them:

Part	Title	Maps to
Part 1	Orientation, Azure Fundamentals & Full Environment Setup	Prerequisite knowledge, not directly scored
Part 2	Setting Up and Configuring the Azure Databricks Environment	Exam Domain 1 (15–20%)
Part 3	Securing and Governing Data with Unity Catalog	Exam Domain 2 (15–20%)
Part 4	Preparing and Processing Data (Modeling, Ingestion, Transformation, Quality)	Exam Domain 3 (30–35%)
Part 5	Deploying and Maintaining Pipelines (Lakeflow Jobs & Declarative Pipelines)	Exam Domain 4 (30–35%)

Part	Title	Maps to
Part 6	End-to-End Capstone Projects	Integration across all domains
Part 7	Practice Exams, Final Review, and the 30-Day Study Calendar	Exam readiness

This chapter — Part 1 — is deliberately **not** exam-scored material. Nothing here is a DP-750 objective by itself. Instead, it’s the foundation everything else stands on: the Azure vocabulary you need in order to understand sentences like “create an external location backed by an ADLS Gen2 container secured with a storage credential” without flinching, and a fully working setup on your own machine and in the cloud so that every hands-on lab in Parts 2–6 just works when you get there. Time spent here is not a detour from exam prep — it’s what makes the rest of the book move fast instead of stalling on “wait, what’s a resource group again?”

1.1.2 How hands-on labs are marked

Throughout the book, hands-on exercises are marked with a clear label so you always know what you’re about to do and what you need available:

- **[LAB - FREE EDITION]** — Can be completed entirely on Databricks Free Edition, no Azure subscription required. This covers roughly 80% of the hands-on content: Unity Catalog basics, notebooks, ingesting sample data, transformations, Lakeflow Declarative Pipelines, Lakeflow Jobs, SQL warehouses.
- **[LAB - AZURE REQUIRED]** — Needs a real Azure-deployed Azure Databricks workspace (Premium tier) because it touches things Free Edition intentionally does not support: external locations and storage credentials against real Azure Data Lake Storage Gen2, classic (non-serverless) cluster configuration and cluster policies, Microsoft Entra ID principal mapping, private networking.
- **[MINI-QUIZ]** — A short set of 5–10 questions at the end of each major section, styled like the exam (multiple choice, multiple response, occasionally a drag-and-drop-style ordering question rendered as a list-ordering question). These are *original* questions written to test the same concepts the real exam tests — never real or leaked exam questions. Answers and explanations follow immediately after each mini-quiz.
- **[CHECKPOINT]** — A short “can you explain this out loud” self-check before moving to the next section. If you can’t, go back and reread — don’t push forward on a shaky foundation, because later parts build on earlier ones constantly.

By the end of the book, Part 7 also contains a **full-length practice exam bank** — several 45–60 question practice tests built to the same domain weightings as the real DP-750 (15–20% / 15–20% / 30–35% / 30–35%), with full rationale for every answer, both correct and incorrect options.

1.1.3 Which parts need what environment

Set expectations now so you're not surprised later:

- **Part 1 (this chapter):** You'll set up *both* Databricks Free Edition and a real Azure Databricks workspace. Do both now, even though you won't touch the Azure workspace again until it's needed, because Azure resource provisioning occasionally takes time (region capacity, permission propagation) and you don't want that blocking a lab later.
- **Part 2:** Mixed. Compute concepts and cluster policies are Azure-only; Unity Catalog object hierarchy basics work on Free Edition.
- **Part 3:** Mixed, leaning Azure-only for anything involving external locations, storage credentials, and Microsoft Entra ID group mapping. Catalog/schema/table-level grants work fine on Free Edition.
- **Part 4:** Mostly Free Edition. Ingestion from real ADLS Gen2 via Auto Loader against your own storage account is Azure-only; ingestion from sample/managed data and Lakeflow Connect UI concepts work on Free Edition.
- **Part 5:** Mostly Free Edition. Lakeflow Jobs and Lakeflow Declarative Pipelines both run on Free Edition's serverless compute. Git integration (Databricks Repos ↔ GitHub) works on Free Edition too.
- **Part 6 (capstones):** Mixed by design — each capstone tells you upfront which environment it needs.
- **Part 7:** No environment needed — it's exams and review.

1.1.4 The 30-day calendar

Part 7 contains a day-by-day, week-by-week study calendar that sequences all seven parts, the labs, the mini-quizzes, and two full practice exams into a 30-day plan, with built-in slack days for the two Azure-provisioning-heavy labs. It assumes roughly 1.5–2.5 hours per weekday and longer sessions on weekends. We won't build that calendar here — just know it exists, and that everything you set up in this chapter (Section 3) is calendar “Day 1” work.

CHECKPOINT: Before moving on, make sure you can answer: which of the two environments (Free Edition vs. real Azure workspace) will most of your hands-on time be spent in, and why does the book still have you set up both?

1.2 2. Azure Fundamentals Primer

Azure Databricks is a product that runs *inside* Microsoft Azure — technically, it's a first-party Azure service, but under the hood it's Databricks' platform deployed into Azure's infrastructure, deeply integrated with Azure's identity and storage systems. You cannot understand the “Azure” half of “Azure Databricks” — or answer the Azure-flavored questions on DP-750 — without a working mental model of core Azure concepts. This section gives you exactly that, no more. It intentionally overlaps with the free **Microsoft Azure Fundamentals** learning path (the material behind the AZ-900

exam) — if any of this feels familiar or you’ve already done AZ-900, skim it and jump to the checkpoints.

1.2.1 2.1 Cloud computing, in one paragraph

“The cloud” is just someone else’s computers, rented by the minute or the gigabyte, reachable over the internet, managed through a web console (or code) instead of a server room you walk into. Instead of buying a physical machine, racking it, and maintaining it, you click a button (or run a command) and a virtual machine, a storage account, or a database appears within minutes, billed for exactly what you use. Azure is Microsoft’s cloud platform — one of the three dominant public clouds alongside AWS and Google Cloud. Everything below is Azure-specific vocabulary built on that one idea.

1.2.2 2.2 Tenant and subscription: the “who” and the “billing account”

A **Microsoft Entra ID tenant** (formerly called an “Azure Active Directory tenant” or “Azure AD tenant” — you will see both names in older material) is your organization’s dedicated, isolated instance of Microsoft’s identity system. Think of it as *the building*: it’s where all your organization’s user accounts, groups, and security policies live. When you sign up for a personal Azure account, Microsoft creates a tenant for you automatically.

An **Azure subscription** is your **billing container** — think of it as a credit card account tied to that building. Every Azure resource you create (a storage account, a virtual machine, a Databricks workspace) belongs to exactly one subscription, and that subscription is what gets billed. A single tenant can contain multiple subscriptions (e.g., “Production,” “Development,” “Sandbox” — separate billing buckets for separate purposes), and subscriptions can also be organized differently within the tenant for permission purposes, described next.

1.2.3 2.3 The resource hierarchy: management groups → subscriptions → resource groups → resources

Azure organizes everything in a nested hierarchy, and permissions and policies flow *downward* through it (a permission granted at a higher level applies to everything below it, unless explicitly overridden). Picture a filing cabinet:

Management Group (optional, enterprise-scale – a “division”)

└─ Subscription (the “billing account”)

└─ Resource Group (a “folder” inside that billing account)

└─ Resources (the actual “files”: storage accounts, Databricks workspaces, VMs.)

- **Management groups** sit above subscriptions and let large organizations apply policy and access rules across many subscriptions at once (e.g., “every subscription in the EU division must keep data in EU regions”). As an individual learner, you likely won’t touch these — they matter for enterprise governance, but you should recognize the term.
- **Subscriptions**, as above, are the billing boundary.

- **Resource groups** are the practical unit you'll live in day to day. A resource group is literally just a **folder** — a logical container that holds related resources so you can manage, deploy, and delete them together. There's no technical requirement that resources in a group be related, but good practice groups things by project or lifecycle (e.g., a resource group called `rg-dp750-lab` holding your Databricks workspace, its managed storage account, and anything else for this course). **Deleting a resource group deletes everything inside it** — which is both a convenience (easy cleanup) and a hazard (be careful what you put where).
- **Resources** are the actual services: a storage account, a virtual network, an Azure Databricks workspace, a key vault, and so on. Every resource has a full **resource ID** — a path-like string that encodes subscription → resource group → resource type → resource name, similar in spirit to a file path.

CHECKPOINT: If someone deletes the resource group `rg-dp750-lab`, what happens to the Azure Databricks workspace and storage account inside it?

1.2.4 2.4 Regions and availability zones

An Azure **region** is a physical geographic location containing one or more data centers — e.g., “East US,” “West Europe,” “UK South.” When you create a resource, you choose the region it lives in. This matters for three reasons, and DP-750 (and real projects) will expect you to reason about all three:

1. **Latency** — a workspace in a region physically close to your users/data sources responds faster than one on the other side of the planet.
2. **Compliance/data residency** — some regulations (e.g., data-sovereignty rules in certain industries or countries) require data to stay within a specific country or geography. Choosing the wrong region can be a compliance violation, not just a performance issue.
3. **Cost and feature availability** — prices differ slightly by region, and not every Azure service or VM size is available in every region (new features often roll out to a handful of regions first).

An **availability zone** is a further subdivision *within* a region: physically separate data centers (with independent power, cooling, and networking) inside the same region, connected by low-latency links. Spreading resources across availability zones protects against a data-center-level failure (a power outage in one building doesn't take down a zone-redundant deployment). Not all regions have availability zones; this is a “know the concept” item for the exam rather than something you'll configure by hand for Azure Databricks labs.

Practical guidance for this book: pick one region close to you (e.g., “East US 2,” “West Europe,” whatever your Azure free account defaults toward or whatever is closest to you) and use it consistently for every resource in every Azure lab in this book. Mixing regions across resources that need to talk to each other (a Databricks workspace and its storage account, for instance) adds latency and, in some configurations, extra cost or complexity — keep it simple and keep it all in one region.

1.2.5 2.5 Azure Resource Manager (ARM) and infrastructure as code

Azure Resource Manager (ARM) is the management layer that sits behind *everything* you do in Azure — the Azure Portal (the web UI), the Azure CLI, PowerShell modules, and Terraform’s Azure provider are all just different front doors to the same ARM API. Every “create a resource” action, no matter which tool triggers it, ultimately becomes an ARM API call.

Because ARM is a well-defined API, Azure resources can be described declaratively as code — you write a file describing *what you want to exist* (a storage account with these properties, a resource group with that name), and a tool makes it so, rather than you clicking through a UI by hand every time. This is called **infrastructure as code (IaC)**. The two mechanisms you’ll hear about:

- **ARM templates / Bicep** — Bicep is Microsoft’s own, more readable authoring language that compiles down to JSON ARM templates. You describe resources in a .bicep file and deploy it with the Azure CLI.
- **Terraform** — a popular third-party (HashiCorp) IaC tool with an Azure provider; widely used in real-world Databricks-on-Azure deployments, including by Databricks’ own official Terraform provider for managing workspaces, clusters, and Unity Catalog objects as code.

For this book, you’ll create resources by hand through the Azure Portal so every click is visible and explainable while you’re learning — but recognize that in production environments, Azure Databricks workspaces and their Unity Catalog objects are very commonly provisioned via Terraform or Bicep instead of manual portal clicks. DP-750 expects you to *recognize* IaC concepts, not to write Bicep from memory.

1.2.6 2.6 Storage accounts and ADLS Gen2

An Azure **storage account** is a top-level container for Azure’s storage services — blobs (unstructured files), file shares, queues, and tables. For data engineering, the piece that matters is **Blob Storage**, and specifically a special configuration of it called **Azure Data Lake Storage Gen2 (ADLS Gen2)**.

Plain Blob Storage stores objects in a flat namespace — every blob’s “path” is really just a string prefix inside a container, not a real folder structure, even though the portal fakes folder-like browsing. **ADLS Gen2** is Blob Storage with an extra feature switched on called **hierarchical namespace (HNS)**. With HNS enabled, the storage account maintains *real* directories and files, not just prefix-simulated ones. This matters enormously for data lakes because:

- Renaming or moving a “folder” is a fast, atomic metadata operation instead of copying every object with the new prefix and deleting the old ones.
- File-system-style permissions (POSIX-like access control lists) can be applied per directory.
- Big-data engines (Spark included) that expect a real file system perform much better against it.

Inside a storage account, data lives in **containers** — the top-level grouping (compa-

rable to a bucket in AWS S3), inside of which you have your directory structure. A typical ADLS Gen2 layout for a Databricks project might be a container named `data`, with directories like `data/raw/`, `data/bronze/`, `data/silver/`, `data/gold/` underneath it.

This is the storage layer that real Azure Databricks workspaces sit on top of, and it's exactly what Free Edition **cannot** connect you to (no external storage locations there) — which is precisely why Part 3's external-location and storage-credential labs require the real Azure workspace you're about to create in Section 3.

1.2.7 2.7 Microsoft Entra ID

Microsoft Entra ID (the current name for what used to be called Azure Active Directory / Azure AD — you'll still see "Azure AD" all over older docs, videos, and even some current product screens) is Azure's identity and access management service. It answers the question "who is this, and what are they allowed to do?" for every resource in your tenant. Four identity concepts you need cold:

- **Users** — individual human accounts (you, signing in with an email address).
- **Groups** — collections of users (and sometimes other groups) that let you assign permissions once to a group instead of repeatedly to individuals. Best practice, both in Azure generally and specifically in Unity Catalog (Part 3), is to grant access to *groups*, not individual users, so that adding/removing a person from a group automatically grants/revokes their access everywhere that group is used.
- **Service principals** — an identity for an *application or automated process*, not a human. When a CI/CD pipeline, a script, or an external application needs to authenticate to Azure or to Databricks without a human typing a password, it uses a service principal — essentially an application's own username/credential pair, which can be granted permissions just like a user.
- **Managed identities** — a special, even more convenient flavor of service principal that Azure manages *for* you, automatically, tied to a specific Azure resource (e.g., a virtual machine or an Azure Databricks workspace can have a managed identity). You never see or handle its credentials — Azure rotates and protects them internally — which makes managed identities the preferred way for one Azure resource to authenticate to another.

Why Databricks needs Entra ID: Azure Databricks doesn't maintain its own separate universe of user accounts. It delegates identity to Microsoft Entra ID — when you log into an Azure Databricks workspace, you're authenticating as an Entra ID identity, and when Unity Catalog grants a group access to a table, that group is an Entra ID group. This is also the mechanism behind service-principal-driven automation (a Lakeflow Job running unattended authenticates as a service principal, not as you), and it's why Part 3 spends real time on mapping Entra ID principals into Unity Catalog grants.

1.2.8 2.8 Azure networking basics (conceptual)

You don't need to become a network engineer for DP-750, but you do need the vocabulary, because Azure Databricks workspaces can be deployed with network isolation:

- A **virtual network (VNet)** is your own private, isolated network inside Azure — a logical boundary within which resources can talk to each other privately, the way computers on your home Wi-Fi can see each other but strangers on the internet can't reach them directly.
- **Subnets** divide a VNet into smaller segments; an Azure Databricks workspace deployed into your own VNet (a configuration called **VNet injection**) uses two dedicated subnets for its compute.
- A **private endpoint** is a network interface that gives a resource (like a storage account or a Databricks workspace's control-plane connectivity) a private IP address *inside your VNet*, instead of that resource being reachable only over the public internet. This is how organizations lock down a Databricks workspace and its storage so that no traffic ever crosses the public internet.
- **Network security groups (NSGs)** are firewall-like rule sets that allow or deny traffic to/from subnets or individual network interfaces based on source, destination, port, and protocol.

For this book's labs, you'll use the default, publicly-reachable networking configuration (simplest to set up, fine for learning) — but recognize these terms, because “secure networking” scenarios (private endpoints, VNet injection) do show up conceptually in the exam's governance-adjacent content.

1.2.9 2.9 Pricing, cost management, and support tiers

Two separate cost dimensions apply to Azure Databricks, and it's a common point of confusion worth clearing up now:

1. **Azure infrastructure cost** — the underlying virtual machines, storage, and networking that Databricks compute runs on, billed by Azure like any other VM/storage usage.
2. **Databricks Units (DBUs)** — a Databricks-specific consumption metric billed *on top of* the Azure infrastructure cost. Different compute types and workload types (all-purpose compute vs. jobs compute vs. serverless, and different Databricks tiers) consume DBUs at different rates. This is why picking the right cluster/compute configuration (Part 2) has direct cost consequences, and why the exam cares about compute-type selection.

Azure billing itself follows a largely **consumption-based (pay-as-you-go)** model — you pay for what you use, by the second or minute for compute, by the gigabyte for storage — though **reservations** (committing to 1 or 3 years of usage for a discount) and **Azure Hybrid Benefit**-style licensing discounts exist for steady, predictable workloads. You won't need any of that for this course.

Cost management basics worth knowing conceptually: Azure provides a **Cost Management + Billing** area in the portal where you can see spend broken down by resource group or resource, and you can set **budgets** with alert thresholds (e.g., “email me at 80% of \$50/month”) so a forgotten running cluster doesn't silently rack up a large bill. When you set up your Azure free account in Section 3, set a budget alert — it costs nothing and protects you from surprises, especially important since

your Azure free-tier credit and always-free allowances are limited (details in Section 3.2).

Support tiers (Basic, Developer, Standard, Professional Direct, Premier/Unified) are Azure’s paid support-plan levels, distinct from your subscription type — Basic (free, community/self-service support) comes with every subscription; higher tiers add faster response-time SLAs and phone/chat access to Microsoft engineers. You need only recognize that these tiers exist and that “how fast Microsoft will respond to a support ticket” is a separate axis from “what services you can use.”

MINI-QUIZ 1 — Azure Fundamentals

1. (Single choice) Which statement correctly orders the Azure resource hierarchy from broadest to narrowest? A. Resource Group → Subscription → Management Group → Resource B. Management Group → Subscription → Resource Group → Resource C. Subscription → Management Group → Resource → Resource Group D. Resource → Resource Group → Subscription → Management Group
2. (Single choice) You need to guarantee that customer data for a European client never leaves the European Union for regulatory reasons. Which Azure concept most directly addresses this? A. Availability zone selection B. Region selection C. Resource group naming convention D. Support tier
3. (Single choice) What does enabling “hierarchical namespace” on an Azure Blob Storage account give you? A. Automatic data replication to a second region B. Azure Data Lake Storage Gen2 — real directories/files instead of a flat, prefix-simulated namespace C. Free storage up to 5 TB D. Built-in row-level security for files
4. (Multiple response — select two) Which of the following are valid Microsoft Entra ID identity types? A. Service principal B. Resource group C. Managed identity D. Availability zone
5. (Single choice) An automated Lakeflow Job needs to authenticate to Azure resources without a human being involved and without anyone handling a password. Which identity type is the best fit? A. A shared user account with a saved password B. A service principal (ideally with a managed identity where possible) C. A management group D. A support ticket
6. (Single choice) What is the best analogy for an Azure resource group? A. A credit card / billing account B. A folder that groups related resources for management and lifecycle purposes C. A firewall rule D. A user’s login identity
7. (Single choice) Which Azure networking feature gives a resource a private IP address inside your own VNet instead of only being reachable over the public internet? A. Network security group B. Availability zone C. Private endpoint D. Resource Manager

Answers: 1. **B** — Management Group → Subscription → Resource Group → Resource. Permissions and policy flow downward through this hierarchy. 2. **B** — Region selection determines the physical/geographic location of the data, which is what data-

residency and compliance rules generally care about. 3. **B** — Hierarchical namespace turns flat Blob Storage into ADLS Gen2, giving you a real directory/file structure, atomic renames, and POSIX-like ACLs. 4. **A and C** — Service principals and managed identities are both Entra ID identity types (alongside users and groups). Resource groups and availability zones are not identity concepts at all. 5. **B** — Service principals (backed by a managed identity when the caller is itself an Azure resource) are the standard way for unattended automation to authenticate securely. 6. **B** — A resource group is a logical folder for related resources; the subscription is the billing account analogy. 7. **C** — A private endpoint gives a resource a private IP inside your VNet; NSGs are firewall rules, not IP assignment mechanisms.

1.3 3. Complete Environment & Installation Setup Guide

This is the longest, most mechanical section in this chapter, and that’s intentional — do this once, carefully, and every lab in Parts 2–6 will just work. Set aside 60–90 uninterrupted minutes. Follow every step in order; later steps assume earlier ones are done.

You will end this section with: a Databricks Free Edition account, a real Azure Databricks Premium workspace with Unity Catalog enabled, the Azure CLI and Databricks CLI installed and authenticated, VS Code wired up to Databricks, a local Python virtual environment, Git and GitHub configured with a token Databricks Repos can use, and a smoke test proving it all works.

1.3.1 3.1 Sign up for Databricks Free Edition

Databricks **Community Edition** — the old free tier many older tutorials and YouTube videos (including freeCodeCamp-style courses) reference — was **retired on January 1, 2026**. If you follow an older video and it tells you to sign up for “Community Edition,” that product no longer exists; the direct replacement is **Databricks Free Edition**, and the steps below are current as of this writing (August 2026).

Steps:

1. Go to https://login.databricks.com/?dbx_source=docs&intent=CE_SIGN_UP in your browser. This is the official sign-up URL for Free Edition — no Azure account, no credit card, and no cloud subscription of any kind is required.
2. Enter your email address and complete the sign-up form (name, and either a password or a “sign up with Google” style option, depending on what’s offered at the time).
3. Verify your email address via the confirmation link Databricks sends you.
4. On first login, Databricks will provision a single **workspace** for you automatically — this takes anywhere from a few seconds to a couple of minutes. A workspace is Databricks’ term for your working environment: the notebooks, the compute, the catalog, all live inside one workspace.
5. You’ll land on the workspace home page. Expect a short guided tour/tooltip walk-through highlighting the left sidebar (Workspace/Catalog/Workflows/Compute

- icons), the “New” button (create notebook, create query, etc.), and a few sample notebooks or a “Get started” tile. Click through it or skip it — either is fine.
6. Take a moment to note what’s *already there*: Free Edition ships with **one Unity Catalog metastore** already attached to your one workspace, and a default catalog. You did not have to configure a metastore yourself — this is one of the biggest conveniences of Free Edition versus a real Azure workspace, where metastore assignment is a manual one-time step you’ll do yourself in Section 3.2.

What to expect functionally: Free Edition is **serverless-only** — there is no “create cluster” option with instance types and node counts; compute just starts when you run something. You get one workspace, one metastore, support for Jobs (up to 5 concurrent job tasks), Lakeflow Declarative Pipelines (one active pipeline per pipeline type), and SQL warehouses (capped at a single 2X-Small warehouse). You will **not** find classic cluster configuration, cluster policies, R or Scala language support, SSO configuration, or the ability to register external cloud storage (no “external locations” or “storage credentials” pointing at your own ADLS Gen2 account) — those live behind the real Azure workspace you’ll build next.

1.3.2 3.2 Sign up for an Azure free account and create an Azure Databricks Premium workspace

This is the more involved half of setup. You’re creating a *real* cloud resource, billed to a real (if free-credit-covered) account, so read each step rather than skimming.

Step group A — Create the Azure free account

1. Go to <https://azure.microsoft.com/en-us/free/> in your browser.
2. Click **“Start free”** (or equivalent current call-to-action) and sign in with a Microsoft account, or create one if you don’t have one.
3. Complete identity verification — this typically requires a phone number and a credit card for identity verification purposes. **Important:** a card is required to verify you’re a real person and to prevent fraud, but Azure will not auto-charge you when your free credit runs out; free services stop or you’re prompted to upgrade. Still, treat this as real money territory and set a budget alert (Step 7 below).
4. Because exact free-tier terms (credit amount, which services are included as “12 months free” vs. “always free,” and regional eligibility) can and do change, **check the current offer details on that same page at sign-up time** rather than trusting a fixed number here. As of recent offers this has generally included an upfront credit usable in your first 30 days plus a set of popular services free for 12 months and a longer list of services that are always free regardless — but confirm the live terms yourself.
5. Complete sign-up. You now have an Azure subscription (commonly named “Azure subscription 1” by default — you can rename it) inside a new Microsoft Entra ID tenant created for you.
6. In the Azure Portal search bar (top of the page once you’re logged into <https://portal.azure.com>), search for **“Cost Management + Billing”**, open it, go to **Budgets**, and create a budget (e.g., \$10/month) with an alert at 80% sent to

your email. This takes two minutes and prevents nasty surprises.

Step group B — Create a resource group

7. In the Azure Portal, click “**Resource groups**” in the left-hand menu (or search for it in the top search bar).
8. Click “**+ Create**”.
9. Choose your subscription (there’s likely only one).
10. Enter a resource group name — use something specific and memorable, e.g., `rg-dp750-lab`.
11. Choose a region — pick one close to you and **remember it**, you’ll use the same region for the workspace next. (e.g., “East US 2” or “West Europe.”)
12. Click “**Review + create**”, then “**Create**”. This finishes in a few seconds.

Step group C — Create the Azure Databricks workspace resource

13. In the Azure Portal search bar, search for “**Azure Databricks**” and click the service.
14. Click “**+ Create**” → “**Azure Databricks Service**” (naming may appear simply as “Create an Azure Databricks workspace”).
15. Fill in the **Basics** tab:
 - **Subscription:** your free-account subscription.
 - **Resource group:** select the `rg-dp750-lab` group you just created.
 - **Workspace name:** something identifiable, e.g., `dbx-dp750-lab`.
 - **Region:** the **same region** you used for the resource group.
 - **Pricing tier:** select **Premium** (or the tier labeled “Premium — Unity Catalog, RBAC, ADB SQL, etc.” depending on current portal wording). This is the critical choice: **Standard tier does not support Unity Catalog**, and Unity Catalog is a hard requirement for a large share of DP-750’s content (all of Domain 2, plus governance-adjacent parts of Domains 1, 3, and 4). Do not pick Standard.
16. Move through the remaining tabs (Networking, Encryption, Tags) — for this course, accept the defaults (default/public networking, Microsoft-managed keys). You don’t need to check “Deploy Azure Databricks workspace with Secure Cluster Connectivity (No Public IP)” or configure your own VNet for these labs; the default is simpler and sufficient.
17. Click “**Review + create.**” Azure validates the configuration; once validation passes, click “**Create.**”
18. **Wait for deployment.** This typically takes a few minutes (commonly 3-10 minutes, occasionally longer). You’ll see a “Deployment in progress” screen; you can navigate away and come back.
19. Once deployment completes, click “**Go to resource**” to land on the workspace’s Azure Portal resource page. This page shows resource metadata (region, pricing tier, resource ID) — it is **not** the Databricks workspace UI itself.
20. Click the “**Launch Workspace**” button on that page. This single-sign-on’s you (via your Microsoft Entra ID identity) directly into the actual Databricks workspace UI, opening in a new browser tab at a URL like `https://adb-<some-number>.<n>.azuredatabricks.net`.

Step group D — One-time Unity Catalog metastore assignment

Unlike Free Edition (where a metastore is pre-attached for you), a brand-new Azure-deployed workspace needs Unity Catalog switched on explicitly. Depending on exactly when and how your tenant was provisioned, one of two situations applies:

- **If your tenant has no metastore yet** (typical for a fresh Azure free account, first Databricks workspace in the region): the first time you access certain admin areas, Azure Databricks auto-provisions a metastore for the region automatically and assigns it to the workspace. Newer accounts increasingly see this happen automatically with no manual step.
- **If it doesn't happen automatically, or you want to do it explicitly** (and to understand the concept, since it's DP-750-relevant regardless):
 21. From inside the Databricks workspace, click your username in the top-right corner → **"Manage Account"** (this opens the **account console**, a separate admin surface at <https://accounts.azuredatabricks.net>, distinct from the workspace UI — a distinction the exam cares about).
 22. In the account console, go to **Catalog** (or "Data") in the left sidebar → **Meta-stores**.
 23. If no metastore exists for your region, click **"Create metastore,"** give it a name (e.g., metastore-dp750), choose the **same region** as your workspace, and specify a default storage location (Azure auto-creates a managed storage container for this, or you point it at one you've prepared — for a first workspace, letting it auto-create is simplest).
 24. Once the metastore exists, go to its **"Workspaces"** tab and **assign** your dbx-dp750-lab workspace to it (or confirm it's already assigned, which is common on newer accounts).
 25. Back in the workspace UI, open **Catalog** in the left sidebar — you should now see a default catalog (often named workspace or similar) rather than a "Unity Catalog not configured" message.

CHECKPOINT: Explain, in your own words, the difference between the Azure Portal resource page for your Databricks workspace and the Databricks workspace UI itself — and where the account console fits relative to both.

1.3.3 3.3 Install and configure the Azure CLI

The **Azure CLI** (az) is a command-line tool for managing Azure resources — the scriptable equivalent of clicking around the portal. You'll use it occasionally in later parts for quick resource lookups and for some IaC-adjacent examples.

Windows:

```
winget install -e --id Microsoft.AzureCLI
```

(Alternatively, download and run the MSI installer from Microsoft's official Azure CLI page if winget isn't available.)

macOS:

```
brew update && brew install azure-cli
```

Linux (Debian/Ubuntu):

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

(Other distributions: Microsoft publishes distro-specific one-liners for RHEL/CentOS, openSUSE, etc. — search “install azure cli linux” for your specific distribution if you’re not on Debian/Ubuntu.)

Verify installation, then log in:

```
az --version  
az login
```

az login opens your default browser, prompts you to sign into the Microsoft account you used for your Azure free account, and once authenticated, prints your available subscription(s) back in the terminal.

Verify you’re authenticated to the right subscription:

```
az account show
```

Expected output is a JSON block including "name" (your subscription name), "id" (a GUID — your subscription ID), "tenantId", and "user" showing your login email. If you have multiple subscriptions and need to switch:

```
az account set --subscription "<subscription-name-or-id>"
```

1.3.4 3.4 Install and configure the Databricks CLI

The **Databricks CLI** (databricks) is a separate tool from the Azure CLI — it talks to the Databricks workspace API (notebooks, jobs, clusters, Unity Catalog, secrets), not the Azure Resource Manager API. You’ll use it heavily from Part 4 onward for asset bundles, jobs, and Git-based deployment workflows.

Install (all platforms — official install script):

macOS/Linux:

```
curl -fsSL  
  ↪ https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh  
  ↪ | sh
```

Windows (PowerShell):

```
winget install Databricks.DatabricksCLI
```

Verify:

```
databricks --version
```

Generate a personal access token in the workspace UI:

1. Open your Azure Databricks workspace UI in the browser (<https://adb-...azuredatabricks.net>).
2. Click your username in the top-right corner → “Settings.”

3. In Settings, go to “**Developer**” (sometimes labeled under a “User” or “Advanced” section depending on UI version).
4. Find “**Access tokens**” → click “**Manage**” → “**Generate new token.**”
5. Give it a description (e.g., local-cli-token) and a lifetime (e.g., 90 days — don’t leave it set to “no expiration” for a learning environment; short-lived tokens are better security hygiene, and the exam’s governance content agrees with this instinct).
6. Click “**Generate.**” **Copy the token immediately** — Databricks shows it exactly once and cannot show it to you again after you close the dialog. If you lose it, you’ll just generate a new one.

Configure the CLI with that token:

```
databricks configure --token
```

This prompts you for two things: - **Databricks Host:** the workspace URL, e.g., `https://adb-1234567890123456.7.azure.databricks.net` - **Token:** paste the personal access token you just generated.

This writes a profile into `~/.databrickscfg` (on macOS/Linux) or `%USERPROFILE%\databrickscfg` (Windows). You can maintain multiple named profiles in this file (useful later if you also want a profile pointing at your Free Edition workspace).

Verify it works:

```
databricks workspace list /
```

Expected output: a listing of top-level items in your workspace’s file browser (folders like Users, Shared, possibly Repos), confirming the CLI can authenticate and talk to your workspace successfully.

1.3.5 3.5 Install VS Code and the Databricks extension

1. Download and install **Visual Studio Code** from <https://code.visualstudio.com/> for your OS if you don’t already have it.
2. Open VS Code, go to the **Extensions** panel (the icon on the left sidebar, or Ctrl+Shift+X / Cmd+Shift+X).
3. Search for and install:
 - “**Python**” (Microsoft’s official Python extension — syntax highlighting, IntelliSense, debugging for local .py files).
 - “**Databricks**” (the official Databricks extension for VS Code, published by Databricks) — this gives you workspace browsing, the ability to run notebooks/files against a remote cluster or serverless compute directly from VS Code, and Databricks Asset Bundle support.
4. After installing the Databricks extension, open the **Command Palette** (Ctrl+Shift+P / Cmd+Shift+P) and run “**Databricks: Configure workspace**” (exact wording may vary slightly by extension version).
5. When prompted, choose to authenticate using the same profile/token approach as the CLI — the extension can read your existing `~/.databrickscfg` profile, or

you can paste the workspace URL and token directly if prompted fresh.

6. Once connected, the Databricks extension's sidebar panel should show your workspace's folder structure, and a status-bar item should show the connected workspace and the currently selected compute (cluster or serverless) to run code against.
7. Open or create a .py file, and you should see an option (either a code-lens link above the file or a run button in the top-right) to **"Run on Databricks"** against your configured compute — this is what lets you write code locally in VS Code but actually execute it on real Databricks compute against real Unity Catalog data, which we'll exercise in the smoke test below.

1.3.6 3.6 Set up a local Python environment

Why bother with local Python at all, if the real execution happens in the cloud? Two reasons: (1) for quick syntax checks, linting, and writing/testing pure-Python helper logic without waiting on cloud compute to spin up, and (2) because the databricks-sdk lets you write Python *locally* that calls out to the Databricks REST APIs to control your workspace (create jobs, manage clusters, query Unity Catalog) — genuinely useful automation you run from your own laptop. Note the distinction clearly: **your local Python interpreter never runs Spark itself against real cluster data** — pyspark installed locally gives you a local, single-machine Spark instance for learning the API and unit-testing logic offline; it is not connected to your Databricks workspace's actual data or compute. When you want code to actually run against your workspace's real compute and real Unity Catalog tables, you either run it in a Databricks notebook, or you use VS Code's "Run on Databricks" (Section 3.5) which ships your local file up to execute remotely.

1. Verify Python 3.10+ is installed:

```
python3 --version
```

If not installed, get it from <https://www.python.org/downloads/> (Windows/macOS) or your package manager (`sudo apt install python3 python3-venv` on Debian/Ubuntu).

2. Create a project folder and a virtual environment inside it:

```
mkdir dp750-lab && cd dp750-lab
python3 -m venv .venv
```

3. Activate the virtual environment:

```
# macOS/Linux
source .venv/bin/activate
```

```
# Windows (PowerShell)
.venv\Scripts\Activate.ps1
```

Your terminal prompt should now show (.venv) at the start of the line.

4. Install the packages you'll need through this book:

```
pip install --upgrade pip
pip install databricks-sdk pyspark
```

5. Quick sanity check that both installed correctly:

```
python3 -c "import databricks.sdk; import pyspark;
↪ print('databricks-sdk and pyspark import OK')"
```

Expected output: databricks-sdk and pyspark import OK

1.3.7 3.7 Install Git, set up GitHub, and generate a token for Databricks Repos

Part 4 (and real-world Databricks development generally) uses **Databricks Repos** — Git integration built into the workspace — to version-control notebooks and pipeline code against a real GitHub repository. Set this up now.

1. **Install Git**, if you don't already have it:
 - Windows: download from <https://git-scm.com/download/win>.
 - macOS: `brew install git`, or it may already be present (`git --version` to check).
 - Linux: `sudo apt install git` (Debian/Ubuntu) or your distro's equivalent.
 - Verify: `git --version`.
2. **Create a free GitHub account** if you don't have one already, at <https://github.com/>.
3. **Generate a Personal Access Token (PAT)** for Databricks to use when connecting to your GitHub account:
 - Log into GitHub, click your profile photo (top-right) → **"Settings."**
 - Scroll to **"Developer settings"** (bottom of the left sidebar).
 - Click **"Personal access tokens"** → **"Tokens (classic)"** (fine-grained tokens work too, but classic tokens are simplest for the Databricks Repos integration and what most current Databricks documentation walks through).
 - Click **"Generate new token"** → **"Generate new token (classic)."**
 - Give it a note (e.g., `databricks-repos-integration`), set an expiration (e.g., 90 days), and check the scope **repo** (full control of private repositories) — this is the scope Databricks Repos needs to clone, pull, and push.
 - Click **"Generate token"** and **copy it immediately** — like the Databricks PAT earlier, GitHub shows it exactly once.
4. **Link the token in your Databricks workspace:**
 - In the Databricks workspace UI, click your username → **"Settings"** → **"Linked accounts"** (sometimes under a **"Git integration"** heading).
 - Choose **GitHub** as the Git provider.
 - Paste your GitHub username and the PAT you just generated.
 - Save.

5. You're now ready, in Part 4, to use **Repos** in the workspace sidebar to clone a GitHub repository directly into your workspace, commit and push notebook changes back to GitHub, and work with branches — the same way you'd expect from any Git-backed development workflow, just wired into the workspace UI.

1.3.8 3.8 Final smoke test — prove everything works

Run this once, in a **new notebook inside your Azure Databricks workspace** (not Free Edition — this test specifically exercises things worth confirming on the environment you just built; feel free to repeat it in Free Edition too, minus the storage-path specifics which will differ).

1. In the workspace UI, click “+ New” → “**Notebook.**” Give it a name like smoke-test, leave the default language as **Python**.
2. At the top, confirm the compute selector (top-right of the notebook) shows an active cluster or **Serverless** compute — if nothing is attached, choose **Serverless** from the dropdown (the simplest option, and the same compute style Free Edition uses exclusively).
3. Paste and run the following in a cell:

```
# 1. Confirms compute is alive and Spark is responding
```

```
print("Spark version:", spark.version)
```

```
# 2. Confirms Unity Catalog is configured and reachable
```

```
print("Current catalog:", spark.sql("SELECT
```

```
  ↪ current_catalog()").collect()[0][0])
```

```
spark.sql("SHOW CATALOGS").show()
```

```
# 3. Confirms you can create objects in Unity Catalog and read/write data
```

```
spark.sql("CREATE SCHEMA IF NOT EXISTS main.smoke_test")
```

```
df = spark.createDataFrame(
    [(1, "alpha"), (2, "beta"), (3, "gamma")],
    ["id", "label"]
)
```

```
df.write.mode("overwrite").saveAsTable("main.smoke_test.demo_table")
```

```
result = spark.sql("SELECT * FROM main.smoke_test.demo_table ORDER BY id")
```

```
result.show()
```

```
# 4. Confirms basic file access via the Databricks File System abstraction
```

```
print(dbutils.fs.ls("/"))
```

What success looks like: - Line 1 prints something like Spark version: 3.5.x (or whatever current version your workspace runs) — proves compute (cluster or serverless) is genuinely running and executing Spark code, not just idling. - Line 2 prints a catalog name (commonly main or your workspace's default catalog name) and a table listing multiple catalogs (at least main, system, and samples are typical defaults) — proves Unity Catalog is attached and queryable. - Line 3 prints a 3-row table: +---+-----+ | id|label| +---+-----+ | 1|alpha| | 2| beta| |

3|gamma| +---+-----+ — proves you can create a schema, write a managed Delta table into Unity Catalog, and read it back. (Note: every table you just created is a **Delta Lake** table by default — Delta is Databricks’ default table format, which Part 4 covers in depth.) - Line 4 prints a list of FileInfo objects representing the root of the Databricks File System (DBFS) mount — proves dbutils (Databricks’ notebook utility library) and file-system access both work.

If all four checks print without error, your environment — Free Edition, your real Azure workspace, or both — is fully ready for every hands-on lab in the rest of this book. If something fails: a Spark-version error usually means compute never actually attached (recheck the compute selector); a Unity Catalog error on step 2 or 3 almost always means the metastore assignment in Section 3.2, Step Group D wasn’t completed — go back and confirm your workspace shows a catalog under the **Catalog** sidebar icon before retrying.

CHECKPOINT: You should now be able to state, without looking back at this chapter: what URL you sign into for Free Edition, what pricing tier your Azure workspace must be for Unity Catalog to work, where personal access tokens are generated in the workspace UI, and what the four things the smoke test cell actually verifies.

You now have every tool, account, and working environment this book’s hands-on labs will assume from here forward. Part 2 begins immediately with Azure Databricks environment configuration — compute types, cluster policies, and how Unity Catalog objects (metastores, catalogs, schemas, and tables) are organized — building directly on the Azure vocabulary and the workspace you just stood up.

Chapter 2

Part 2 — Databricks Workspace, Compute & Unity Catalog Security and Governance

2.1 How to use this chapter

This chapter is the primary study material for two exam domains: **Domain 1 — Set up and configure an Azure Databricks environment** (15-20% of the exam) and **Domain 2 — Secure and govern Unity Catalog objects** (15-20% of the exam). Together that's roughly a third of the DP-750 exam, so treat this chapter as high-value real estate, not a warm-up.

You already have, from Part 1:

- A **Databricks Free Edition** account (a free, personal, serverless-only Databricks workspace — great for practicing SQL, Unity Catalog objects, notebooks, and governance features, but with real limits you'll hit below).
- A real **Azure Databricks Premium workspace**, deployed inside your own Azure subscription, with **Unity Catalog** enabled — this is the only place that behaves like a production enterprise deployment, and the only place that supports classic clusters, cluster policies, and connections to real Azure Data Lake Storage (ADLS) Gen2.
- Azure CLI, Databricks CLI, VS Code, Python, and Git configured locally.

Every hands-on lab below tells you explicitly **which environment to open** — Free Edition or your real Azure Premium workspace — so you never waste time discovering a feature is missing halfway through a lab. As a rule of thumb: **Free Edition is fine (and faster to use) for anything involving Unity Catalog objects, managed tables, permissions, dynamic views, lineage, and audit logs. Your real Azure workspace is required for anything involving classic clusters, cluster policies, external locations, or storage credentials tied to ADLS Gen2**, because Free Edition workspaces run exclusively on serverless compute and cannot attach external cloud storage.

A note on terminology before we start: what used to be called **Databricks Workflows** is now officially called **Lakeflow Jobs**. You'll see both names in older documentation and in the product itself during the transition — this book uses "Lakeflow Jobs" going

forward.

2.2 1. Databricks Workspace & Compute Fundamentals

2.2.1 1.1 What is a “workspace,” really?

Think of a Databricks **workspace** as your entire office building for data work: it’s a single web application (with a URL like `https://adb-XXXXXXXXXXXX.XX.azuredatabricks.net`) that bundles together a file system for notebooks and code, a place to spin up computers to run that code, a catalog of all your data tables, and tools to schedule and monitor jobs. Everything you do in this chapter happens inside a workspace.

Let’s break the building into its rooms.

Notebooks. A notebook is Databricks’ equivalent of a Jupyter notebook: a document made of cells that mix runnable code (Python, SQL, Scala, or R) with formatted text (Markdown), and that show output — tables, charts, printed text — directly beneath each cell. If you’ve used Jupyter or Google Colab, you already know 90% of how a Databricks notebook works. The differences that matter: Databricks notebooks are collaborative in real time (like a Google Doc — you can see a teammate’s cursor), they attach to a **cluster** or a **SQL warehouse** to actually execute code, and they support “magic” language switches — a cell that starts with `%sql` runs as SQL even inside a Python notebook, `%md` renders Markdown, `%sh` runs a shell command on the driver machine, and `%fs` lists or manipulates files.

Clusters and SQL warehouses. Neither notebooks nor tables *do* anything by themselves — you need actual computers (virtual machines) to execute your code and process data. In Databricks, that compute comes in two families, described in depth in section 1.2: **clusters** (general-purpose virtual machines you attach a notebook to, good for Python/Scala/R data engineering and machine learning) and **SQL warehouses** (compute optimized specifically for running SQL queries fast, used by the SQL editor, dashboards, and BI tools like Power BI). Every notebook cell and every SQL query needs one of these attached before it can run.

DBFS vs. Unity Catalog Volumes. DBFS (the **Databricks File System**) is a distributed file system baked into every workspace, historically used to store any file that isn’t a table — CSV files you’re about to ingest, model checkpoints, images, JAR libraries, and so on. It’s simple, but it has become a governance liability: DBFS is a *workspace-level* concept, meaning permissions on DBFS paths are hard to control centrally, files there are invisible to Unity Catalog’s lineage and audit tools, and two different workspaces can’t easily share the same governed view of the same files. **Unity Catalog Volumes** solve this. A Volume is a Unity Catalog object (it lives inside a catalog and schema, exactly like a table) that represents governed, non-tabular file storage — you GRANT access to it exactly like you grant access to a table, it shows up in lineage, and it’s visible in Catalog Explorer next to your tables. **This is why Volumes are now the recommended way to store raw files, and DBFS is considered legacy for anything beyond quick, throwaway scratch work.** In this

chapter and beyond, whenever you need to land a file (a CSV to load, a model artifact, an image), reach for a Volume, not a raw DBFS path.

Catalog Explorer. This is the UI screen (left sidebar → **Catalog**) where you browse the entire Unity Catalog object hierarchy — catalogs, schemas, tables, views, volumes, functions, and models — inspect their schemas and sample data, see lineage graphs, manage permissions and tags, and view audit history. You’ll live in this screen throughout section 2-4.

Jobs / Lakeflow Jobs UI. The left sidebar’s **Jobs & Pipelines** area is where you define, schedule, and monitor automated, multi-step runs of your notebooks/scripts (formerly branded “Databricks Workflows,” now **Lakeflow Jobs**). We cover this in depth in a later chapter on orchestration — for now, just know where it lives.

Repos (Git folders). Databricks integrates directly with Git. A **Repo** (now often labeled “Git folder” in the UI) is a clone of a Git repository (GitHub, Azure DevOps, GitLab, etc.) living inside your workspace file tree, so you can edit notebooks with real version control — branches, commits, pull requests — instead of relying on notebook auto-save. As a Data Scientist moving into engineering, treat this as non-negotiable: production data engineering code belongs in a Repo, not in a loose personal notebook.

2.2.2 1.2 Classic clusters vs. serverless compute

This is one of the most exam-relevant distinctions in Domain 1, so let’s be precise.

A **classic cluster** is a set of actual virtual machines (one “driver” node plus zero or more “worker” nodes) that Databricks provisions in your cloud subscription on your behalf, running Apache Spark. You choose the VM size, how many workers, the Databricks Runtime version, and various other settings — and Databricks bills you for that VM time in your Azure subscription, on top of the Databricks-specific charge (measured in **DBUs**, or Databricks Units — a unit of processing capability billed per second of use). Classic clusters take roughly 3-8 minutes to start (“cold start”) because real cloud VMs have to boot.

Serverless compute flips the model: Databricks maintains a warm, shared, pre-provisioned pool of compute capacity in its own managed infrastructure, and when you run something, your workload borrows capacity from that pool almost instantly (often just a few seconds), scales automatically, and then releases it back — you never choose a VM size, and you never see the underlying machine. You’re billed purely for the compute-seconds you actually use, at a serverless DBU rate. Databricks Free Edition runs **exclusively** on serverless compute — that’s why it has no “Create Cluster” screen with VM types: there’s nothing to configure, there is only “run.”

When to pick which, in practice:

Situation	Recommendation
Ad-hoc exploration, fast iteration, unpredictable/bursty usage	Serverless — no cold start, no idle cluster costs
Tight cost control with predictable, steady workloads and negotiated VM discounts (e.g., Azure Reserved Instances)	Classic clusters can be cheaper at scale

Situation	Recommendation
Need a specific VM family (GPU for ML training, memory-optimized for huge shuffles)	Classic clusters — you pick the exact instance type
Need init scripts, custom Docker containers, or exotic Spark configs	Classic clusters — serverless restricts these for security/isolation reasons
Fast-turnaround BI dashboards / SQL queries	SQL warehouses (serverless or classic SQL warehouses)
Org requires strict network isolation, VNet injection, custom security groups	Classic clusters in your own Azure VNet

The exam expects you to know the cost/performance tradeoff conceptually: **serverless trades some cost-per-second efficiency and configurability for near-zero startup latency and zero infrastructure management; classic clusters trade a slower start and more operational responsibility for full control over the underlying VM and networking.**

2.2.3 1.3 Cluster access modes and why Unity Catalog cares

When you create a classic cluster, you must choose an **access mode**. This single dropdown is one of the most exam-relevant settings in this entire chapter, because Unity Catalog governance (everything in sections 2–4) is only fully enforced on certain access modes.

- **Single user (Assigned).** The cluster is assigned to exactly one user (or, for automated jobs, one service principal). Full Unity Catalog governance is enforced. Supports Python, SQL, Scala, and R. This is the default and recommended mode for a single data engineer’s interactive work and for most production jobs.
- **Shared.** Multiple users can attach to and use the same cluster simultaneously, with Unity Catalog governance still fully enforced *per user* — two different people on the same shared cluster only see the tables/rows they individually have permission to see. This is the right choice for a team cluster where several analysts share compute to save cost. Historically, Shared clusters had language restrictions (no Scala, limited R); recent Databricks Runtime versions have relaxed most of these, but Shared clusters still disallow certain low-level RDD APIs and some legacy library patterns for isolation reasons.
- **No isolation shared (legacy “No Isolation Shared”).** An older mode that does **not** enforce Unity Catalog data governance at all — any user on the cluster effectively runs with the cluster’s own broad permissions, not their own. This mode is disabled by default in Unity Catalog-enabled workspaces and you should never intentionally choose it for governed work; if you ever see it, treat it as a red flag on the exam (“this configuration does NOT enforce Unity Catalog permissions”).

The rule the exam wants you to internalize: to get Unity Catalog’s row/column-level security and audit logging on a classic cluster, the cluster must run in Single User or Shared access mode. A cluster without Unity Catalog access mode enforcement (legacy “No Isolation Shared,” or a cluster

not attached to a Unity Catalog metastore) cannot be trusted to enforce GRANT/REVOKE. Serverless compute (both serverless clusters and serverless SQL warehouses) always enforces Unity Catalog governance by design — there’s no equivalent “no isolation” escape hatch on serverless.

2.2.4 1.4 Hands-on lab: cluster policies, autoscaling, and auto-termination

Environment for this lab: your real Azure Databricks Premium workspace. Free Edition is serverless-only and has no “Create Cluster” screen, no VM picker, and no cluster policies UI. If you only have Free Edition available today, read through this lab carefully for the exam and revisit it hands-on once you’re at your Azure workspace.

A **cluster policy** is a set of rules an administrator defines that constrains what regular users are allowed to configure when they create a cluster — for example, capping the maximum VM size, forcing a specific Databricks Runtime version, or forbidding a user from disabling auto-termination. Policies exist so that a well-meaning-but-inexperienced user (like you, three weeks into learning Databricks) can’t accidentally spin up a 64-node GPU cluster and leave it running all weekend. On the exam, remember: **cluster policies control what users are *permitted* to select when creating a cluster; they do not themselves create a cluster.**

Step 1 — Create a cluster policy (admin task).

1. In your Azure Databricks Premium workspace, click the gear icon → **Admin Settings** (or, in older UI, left sidebar → **Compute** → **Policies** tab). You need to be a workspace admin — if you deployed the workspace yourself in Part 1, you are.
2. Go to **Compute policies** → **Create policy**.
3. Name it dp750-study-policy.
4. In the policy definition editor, you write JSON rules. Enter the following, which caps worker count, forces auto-termination, and restricts the VM family to keep costs low:

```
{
  "num_workers": {
    "type": "range",
    "minValue": 0,
    "maxValue": 4,
    "defaultValue": 1
  },
  "autotermination_minutes": {
    "type": "range",
    "minValue": 10,
    "maxValue": 60,
    "defaultValue": 20,
    "isOptional": false
  },
  "node_type_id": {
```

```

    "type": "allowlist",
    "values": ["Standard_DS3_v2", "Standard_DS4_v2"]
  },
  "spark_version": {
    "type": "unlimited",
    "defaultValue": "auto:latest-lts"
  },
  "data_security_mode": {
    "type": "fixed",
    "value": "SINGLE_USER"
  }
}

```

5. Click **Create**. This policy now appears as a selectable option to any user creating a cluster.

Step 2 — Create a cluster using that policy, and understand every field.

1. Left sidebar → **Compute** → **Create compute**.
2. Under **Policy**, select `dp750-study-policy`. Notice the form below immediately restricts itself to what the policy allows.
3. Walk through every field (this is exactly the kind of screen the exam expects you to know cold):
 - **Cluster name** — a human-readable label, purely cosmetic.
 - **Policy** — the policy you just picked; constrains everything below it.
 - **Access mode** — choose **Single user** and pick yourself as the assigned user (see section 1.3 for why this matters for Unity Catalog).
 - **Databricks Runtime version** — the pre-built combination of Apache Spark version + OS + common libraries (e.g., Delta Lake, ML libraries) that runs on each node. Prefer an “LTS” (Long Term Support) runtime for stability.
 - **Use Photon Acceleration** — Photon is Databricks’ native, vectorized query engine (written in C++) that transparently accelerates SQL and DataFrame operations on top of Spark. Toggle it on for most SQL-heavy or Delta Lake workloads; it typically costs a slightly higher DBU rate but finishes faster.
 - **Worker type / Driver type** — the Azure VM size for worker nodes and for the driver node respectively. The driver coordinates the job and collects results; workers do the distributed processing. For small/interactive work these are often the same size.
 - **Enable autoscaling** — checked, with **Min workers** and **Max workers** fields. Set **Min workers = 1** and **Max workers = 4** (the ceiling your policy allows). With autoscaling on, Databricks automatically adds workers when a job is queued/waiting on Spark stages and removes them when idle, rather than you having to guess a fixed cluster size.
 - **Terminate after ___ minutes of inactivity (auto-termination)** — set this to **20** (matching the policy’s default). This is the single most important cost-control setting for a beginner: it automatically shuts the cluster down (stops billing) after that many idle minutes. **Always set this. A cluster left running all night is the single most common way a Databricks beginner**

accidentally burns through a training/free-credit budget.

- **Spark config / Environment variables (Advanced options)** — key-value pairs injected into the Spark session or OS environment at cluster startup, used for tuning or for setting secrets/paths.
 - **Init scripts (Advanced options)** — shell scripts that run on every node during cluster startup, typically used to install custom OS packages or libraries not available through normal means.
4. Click **Create compute**. Watch the state go **Pending** → **Running** over a few minutes — that startup delay is the “cold start” referenced in section 1.2, and it’s exactly what serverless compute avoids.
 5. Once running, attach a scratch notebook to it and run `%sql SELECT current_user(), current_catalog();` to confirm it’s alive and see which identity and catalog context you’re in.

Cost/study tip: after this lab, either delete the cluster (**Compute** → select it → **Delete**) or trust the 20-minute auto-termination to do it for you. Don’t leave it running while you read the next section.

2.3 2. Unity Catalog Object Model

2.3.1 2.1 The hierarchy, and why it exists

Before Unity Catalog, permissions and data organization in Databricks were scattered per-workspace, which made it painful to run a large organization with many workspaces sharing the same data safely. **Unity Catalog** is a single, centralized governance layer that sits *above* your workspace(s) and gives every data and AI asset one consistent home, one consistent permission model, and one audit trail — regardless of which workspace or cluster touches it.

The object hierarchy, from the top down:

```
Metastore (one per Azure region, shared across workspaces in that region)
├─ Catalog (top-level container – e.g., "dev", "prod", "finance")
│   └─ Schema (a.k.a. database – e.g., "bronze", "silver", "gold")
│       ├── Table (managed or external Delta table, or other formats)
│       │   ├── View (a saved query over one or more tables)
│       │   ├── Volume (governed non-tabular file storage)
│       │   └─ Function (a SQL or Python UDF registered for reuse)
│       └─ Model (a registered ML model, via Unity Catalog Model Registry)
```

Think of it like a filing system: the **metastore** is the entire filing *building* for a region (you typically have exactly one per Azure region, and it’s created once when Unity Catalog is first enabled — Part 1 already did this for your Azure workspace; Free Edition provisions one for you automatically). A **catalog** is a filing *cabinet* inside that building — often one per environment (dev, staging, prod) or per major business unit. A **schema** (the terms “schema” and “database” are interchangeable in Databricks SQL) is a *drawer* inside a cabinet — typically grouping related tables, like a medallion-

architecture layer (bronze, silver, gold) or a subject area (sales, hr). Inside a drawer you have the actual *folders and documents*: tables, views, volumes, functions, and models.

Every object has a fully qualified **three-level name**: `catalog.schema.object` — for example `prod.sales.orders` or `dev.bronze.raw_events`. This three-level naming is one of Unity Catalog's headline features versus the old two-level (database.table) Hive Metastore model, and you should get comfortable typing it, because almost every SQL statement in this chapter uses it.

2.3.2 2.2 Hands-on lab: create a catalog, schema, managed table, and Volume

Environment for this lab: either environment works — Free Edition is fine and faster to spin up. Use whichever is open. If you want this catalog to persist for later chapters, consider doing it in your Azure workspace since that's the one meant for longer-term project work; either is acceptable for practicing the mechanics.

Via SQL (open a SQL editor or a notebook cell with %sql):

```
-- Create a catalog. You need the CREATE CATALOG privilege (metastore
↳ admins have it by default).
CREATE CATALOG IF NOT EXISTS dp750_lab
  COMMENT 'Sandbox catalog for DP-750 study exercises';

-- Set it as your working catalog for convenience
USE CATALOG dp750_lab;

-- Create a schema (a.k.a. database) inside that catalog
CREATE SCHEMA IF NOT EXISTS dp750_lab.bronze
  COMMENT 'Raw / landing layer for study exercises';

USE SCHEMA bronze;

-- Create a managed Delta table
CREATE TABLE IF NOT EXISTS dp750_lab.bronze.customers (
  customer_id    BIGINT,
  full_name      STRING,
  email          STRING,
  region         STRING,
  signup_date    DATE
)
COMMENT 'Sample customer table for governance exercises';

-- Insert a few rows so later labs (dynamic views, lineage) have something
↳ to work with
INSERT INTO dp750_lab.bronze.customers VALUES
```

```

(1, 'Amara Chen',      'amara.chen@example.com',      'EU',
↪ DATE'2023-01-15'),
(2, 'Diego Fernandez', 'diego.fernandez@example.com', 'US',
↪ DATE'2023-03-02'),
(3, 'Priya Natarajan', 'priya.natarajan@example.com', 'APAC',
↪ DATE'2023-06-21'),
(4, 'Liam O'Brien',    'liam.obrien@example.com',      'EU',
↪ DATE'2024-02-10');

```

-- Create a Unity Catalog Volume for raw file storage

```

CREATE VOLUME IF NOT EXISTS dp750_lab.bronze.raw_files
COMMENT 'Landing zone for incoming CSV/JSON files';

```

Verify the table is a genuine Delta table and inspect its storage location:

```
DESCRIBE DETAIL dp750_lab.bronze.customers;
```

Look at the format column (should read delta) and the location column (an internal, managed storage path Databricks chose for you — you didn't specify one, which is the hallmark of a *managed* table; more on this in 2.3).

Via the UI (equivalent actions):

1. Left sidebar → **Catalog** (this opens Catalog Explorer).
2. Click **Create Catalog**, name it `dp750_lab_ui`, leave the default storage location (Unity Catalog manages it), click **Create**.
3. Inside the new catalog, click **Create schema**, name it `bronze`.
4. Inside the schema, click **Create table**. You'll see two paths here: **upload a file** (CSV/JSON — Databricks infers a schema and creates a managed Delta table from it) or open a **notebook/SQL editor** to run `CREATE TABLE` yourself. Try uploading a small CSV to see the no-code path — this is genuinely how many analysts create their first table.
5. Inside the schema, click **Create** → **Volume**, name it `raw_files`, and note that the UI form is asking for exactly the same information as `CREATE VOLUME` — the UI is just a thin skin over the same SQL commands.

You now have two ways to produce identical Unity Catalog objects — know both for the exam, since some questions describe UI clicks and expect you to recognize the resulting SQL, and vice versa.

2.3.3 2.3 Managed vs. external tables

This distinction is tested repeatedly on the exam, so let's be extremely precise about what actually differs.

A **managed table** is one where you did *not* specify a storage location when creating it. Unity Catalog picks the storage path itself (inside the catalog's or schema's managed storage location) and — critically — **owns the full lifecycle of the underlying data files**. If you `DROP TABLE` a managed table, Unity Catalog deletes the underlying Delta files too. There's no "orphaned data" left behind. This is simpler and safer for most

day-to-day engineering work, which is why **managed tables are the default and the generally recommended choice** unless you have a specific reason to do otherwise.

An **external table** is one where you explicitly provide a `LOCATION` pointing at a path Unity Catalog does *not* own — typically a path inside an existing ADLS Gen2 container that other systems also read from or write to (maybe a legacy pipeline, or a shared data lake used by tools outside Databricks). Unity Catalog registers the table’s metadata (schema, statistics, permissions) but treats the files as owned by “someone/something else.” If you `DROP TABLE` an external table, **only the Unity Catalog registration is removed — the underlying data files are left completely untouched on ADLS Gen2.**

	Managed table	External table
LOCATION specified at creation?	No	Yes (points at an external location)
Who “owns” the storage lifecycle	Unity Catalog	The external storage / whoever else writes there
Effect of <code>DROP TABLE</code>	Deletes underlying data files	Deletes only the catalog registration; data files remain
Typical use case	New tables you’re building as part of your pipeline	Data that must remain in a specific ADLS location, shared with non-Databricks tools, or data governed under an existing lake structure you can’t move
Prerequisite	None beyond <code>CREATE TABLE</code> privilege on the schema	A Storage Credential + External Location already configured (section 2.4)

Memorize this sentence, it is exam gold: “Dropping a managed table deletes both the metadata and the underlying data; dropping an external table deletes only the metadata, leaving the data in place.”

2.3.4 2.4 Hands-on lab: external location, storage credential, and an external table on ADLS Gen2

Environment for this lab: your real Azure Databricks Premium workspace — mandatory. Databricks Free Edition does not support external cloud storage locations or storage credentials at all; there is no equivalent lab you can run there. If you only have Free Edition today, read this section closely for the exam and come back to actually execute it once you’re in your Azure workspace.

To let Unity Catalog read/write a real folder in ADLS Gen2, you need two Unity Catalog objects working together, plus one piece of Azure-side identity:

- A **storage credential** — an Azure identity (with permission to access storage) that Unity Catalog is allowed to *act as* when talking to ADLS Gen2.

- An **external location** — a Unity Catalog object that pairs a specific ADLS Gen2 path (abfss://container@account.dfs.core.windows.net/path) with a storage credential, and which you then GRANT access to like any other securable object.

There are two ways to build the storage credential's underlying identity:

1. **Databricks-managed (access connector) identity** — Azure Databricks can create and manage an **Access Connector for Azure Databricks** resource on your behalf, which is essentially a first-class Azure **managed identity** wired specifically for this purpose. Databricks handles credential rotation and lifecycle for you. **This is the simpler, recommended path for a beginner** — fewer moving parts, nothing to expire or rotate manually.
2. **Microsoft Entra ID app registration (service principal) with a client secret/certificate** — you register an app in Entra ID yourself, grant it a role on the storage account, and hand Unity Catalog its client ID + secret. This gives you more control (e.g., reusing one identity across multiple non-Databricks systems) but means *you* manage secret rotation and expiry. Use this only when you have a specific cross-system reason to.

We'll use option 1 (managed identity via Access Connector) below.

Step 1 — Create an Access Connector for Azure Databricks (Azure Portal).

1. In the Azure Portal, search for **Access Connector for Azure Databricks** and click **Create**.
2. Choose the same **Resource Group** and **Region** as your Azure Databricks workspace.
3. Give it a name, e.g. ac-dp750-lab.
4. Click **Review + create**, then **Create**. Wait for deployment to finish, then open the resource and copy its **Resource ID** (you'll need it in Step 3) — it looks like /subscriptions/<sub-id>/resourceGroups/<rg>/providers/Microsoft.Databricks/accessConnectors/ac-dp750-lab.

Step 2 — Grant that connector's managed identity access to your ADLS Gen2 container.

1. Go to your **Storage account** → the container you want Databricks to read/write (or create a new container, e.g. dp750data, if you don't have one yet).
2. Go to **Access Control (IAM)** on the *storage account* (role assignments at the storage-account level apply to all containers within it; you can instead scope this to just the container if you want tighter isolation).
3. Click **Add** → **Add role assignment**.
4. Role: **Storage Blob Data Contributor** (read + write; use **Storage Blob Data Reader** instead if you only need read access).
5. Assign access to: **Managed identity** → **Select members** → find your Access Connector (ac-dp750-lab) under "Access connector for Azure Databricks" → select it.
6. **Review + assign**.

Step 3 — Create the storage credential in Unity Catalog (Catalog Explorer or SQL).

Via Catalog Explorer: **Catalog** → gear/settings icon → **Credentials** → **Create credential** → **Storage credential** → name it dp750-storage-cred → paste the Access Connector's **Resource ID** from Step 1 → **Create**.

Equivalent SQL (run as a metastore admin):

```
CREATE STORAGE CREDENTIAL dp750_storage_cred
  AZURE_MANAGED_IDENTITY '/subscriptions/<sub-
  ↳ id>/resourceGroups/<rg>/providers/Microsoft.Databricks/accessConnectors/ac-
  ↳ dp750-lab'
  COMMENT 'Managed identity credential for the DP-750 lab storage account';
```

Step 4 — Create the external location.

```
CREATE EXTERNAL LOCATION dp750_ext_location
  URL 'abfss://dp750data@<your-storage-account>.dfs.core.windows.net/'
  WITH (STORAGE CREDENTIAL dp750_storage_cred)
  COMMENT 'External location pointing at the dp750data container';
```

(Or, in Catalog Explorer: **Catalog** → settings icon → **External Locations** → **Create location**, fill in the same URL and pick the storage credential you just made.)

Grant yourself permission to actually use it (a metastore admin has this by default, but it's good practice to see the explicit grant):

```
GRANT READ FILES, WRITE FILES ON EXTERNAL LOCATION dp750_ext_location TO
  ↳ `account users`;
```

Step 5 — Create an external table using that external location.

```
CREATE TABLE dp750_lab.bronze.customers_external (
  customer_id    BIGINT,
  full_name      STRING,
  email          STRING,
  region         STRING,
  signup_date    DATE
)
LOCATION 'abfss://dp750data@<your-storage-
  ↳ account>.dfs.core.windows.net/customers_external/'
COMMENT 'External table stored outside of Unity Catalog-managed storage';
```

```
INSERT INTO dp750_lab.bronze.customers_external
SELECT * FROM dp750_lab.bronze.customers;
```

```
DESCRIBE DETAIL dp750_lab.bronze.customers_external;
```

Check the location field in the output — unlike the managed table from section 2.2, this now points at the exact ADLS Gen2 path you specified, inside your own storage account. Now prove the earlier claim to yourself:

```
DROP TABLE dp750_lab.bronze.customers_external;
```

Go back to the Azure Portal (or use `az storage fs file list`) and confirm the Parquet/Delta files under `customers_external/` are **still sitting in the container** — only the Unity Catalog registration was removed. Then, for comparison, drop the managed table from section 2.2 and note in Catalog Explorer / via `DESCRIBE DETAIL` (before dropping) that its files live under Unity Catalog’s own managed root — those *would* be deleted, unlike the external table’s files.

2.4 3. Securing Unity Catalog

2.4.1 3.1 Principals: users, groups, and service principals

Everything in Unity Catalog security boils down to answering one question for every request: “**who** is asking, and what have we told Unity Catalog they’re allowed to do?” The “who” is always one of three kinds of **principal**:

- **User** — an individual human identity, synced in from Microsoft Entra ID (Azure’s identity provider, formerly called Azure Active Directory). Identified by an email/UPN like `amara.chen@yourcompany.com`. Use direct user grants sparingly — they don’t scale and get forgotten when someone changes teams.
- **Group** — a named collection of users (and even other groups), also synced in from Entra ID (or created directly in the Databricks account console). Grant privileges to **groups**, not individual users, as the default pattern — when someone joins the data engineering team, you add them to the `data-engineers` group in Entra ID and they instantly inherit every Unity Catalog privilege that group already has, with zero Databricks-side changes.
- **Service principal** — a non-human, application identity used by automated processes: a scheduled Lakeflow Job, a CI/CD pipeline, an external application calling into Databricks via API. Service principals also originate from Microsoft Entra ID (as “app registrations”) and are then added to the Databricks account. **Never run a production, unattended pipeline under a real human’s personal identity** — if that person leaves the company or their password/token is rotated, the pipeline silently breaks (and worse, it’s an audit/security anti-pattern, since you can’t tell “did Amara run this manually” from “did the automated job run this”). Give every scheduled job its own service principal.

Rule of thumb for the exam: interactive human work → user (usually via group membership); scheduled/automated work → service principal; and privileges are almost always granted to **groups**, rarely to individual users directly.

2.4.2 3.2 Hands-on lab: GRANT and REVOKE

Environment for this lab: either environment — Free Edition is fine and faster. Permissions, GRANT/REVOKE, and dynamic views all work identically on Free Edition since they’re pure Unity Catalog metadata operations, not tied to external storage.

Unity Catalog privileges follow a simple but strict **inheritance and cascading requirement**: to SELECT from a table, a principal needs SELECT on the table itself **and** USE SCHEMA on its parent schema **and** USE CATALOG on its parent catalog. Missing any one link in that chain means “access denied,” even if the table-level grant looks correct — this trips up almost everyone the first time, so remember it explicitly.

First, create a group to practice with. Group creation happens in the **account console** (accounts.azuredatabricks.net) or, for Free Edition, in the workspace’s simplified admin settings — for this lab we’ll assume a group called analysts already exists (create it via **Admin Settings** → **Identity and access** → **Groups** → **Add group** if it doesn’t).

```
-- 1. Let the analysts group even "enter" the catalog and schema.
--    Without these two, table-level SELECT grants below do nothing.
GRANT USE CATALOG ON CATALOG dp750_lab TO `analysts`;
GRANT USE SCHEMA ON SCHEMA dp750_lab.bronze TO `analysts`;

-- 2. Read-only access to one table
GRANT SELECT ON TABLE dp750_lab.bronze.customers TO `analysts`;

-- 3. A broader grant: let analysts SELECT on every current AND future
--    ↪ table in the schema
GRANT SELECT ON SCHEMA dp750_lab.bronze TO `analysts`;

-- 4. Revoke a privilege
REVOKE SELECT ON TABLE dp750_lab.bronze.customers FROM `analysts`;

-- 5. Grant privileges suited to a pipeline's service principal
--    (assume the service principal's application ID is registered as a
--    ↪ principal
--    named 'sp-dp750-pipeline' in the workspace)
GRANT USE CATALOG ON CATALOG dp750_lab TO `sp-dp750-pipeline`;
GRANT USE SCHEMA ON SCHEMA dp750_lab.bronze TO `sp-dp750-pipeline`;
GRANT CREATE TABLE ON SCHEMA dp750_lab.bronze TO `sp-dp750-pipeline`;
GRANT MODIFY ON SCHEMA dp750_lab.bronze TO `sp-dp750-pipeline`;

-- 6. Inspect what's currently granted
SHOW GRANTS ON TABLE dp750_lab.bronze.customers;
SHOW GRANTS ON SCHEMA dp750_lab.bronze;
SHOW GRANTS TO `analysts`;
```

A quick glossary of the privileges you just used, plus a few more you’ll see on the exam:

- **USE CATALOG / USE SCHEMA** — the “door key” privileges; required just to reference the object in a query at all, but grant no rights by themselves.
- **SELECT** — read rows.
- **MODIFY** — insert, update, delete, merge rows into an existing table.
- **CREATE TABLE / CREATE SCHEMA / CREATE VOLUME / CREATE FUNCTION** — permission

to create new objects of that type inside the parent container.

- **ALL PRIVILEGES** — shorthand granting everything available at that object level; use cautiously.
- **OWNERSHIP** — every object has exactly one owner (a user, group, or service principal) who can always grant/revoke on it and change its owner; ownership is set at creation (creator becomes owner by default) and can be transferred with `ALTER ... OWNER TO`.

Common role → privilege combinations (a table you should be able to reproduce from memory for the exam):

Role	Typical grants
Read-only analyst	USE CATALOG on the catalog, USE SCHEMA on the schema(s) they need, SELECT on specific tables/schemas. Nothing else.
Data engineer building a schema	USE CATALOG, USE SCHEMA, SELECT, MODIFY, CREATE TABLE, CREATE VIEW on the target schema(s).
Pipeline service principal (ETL job)	USE CATALOG, USE SCHEMA, CREATE TABLE, MODIFY, SELECT on the schema(s) it writes to; often also READ FILES/WRITE FILES on an external location or a Volume it lands raw files into.
BI / dashboard reader	USE CATALOG, USE SCHEMA, SELECT on gold-layer tables/views only — typically granted through a dynamic view (section 4.4), never the raw table.
Catalog administrator	ALL PRIVILEGES (or OWNERSHIP) on the catalog, so they can manage schemas, grants, and objects underneath.
Metastore admin	Not a Unity Catalog grant at all — a special account-level role that can manage every metastore, catalog, and grant across the whole org; use very sparingly, ideally as a group of 2–3 trusted people, not one individual.

2.4.3 3.3 How Microsoft Entra ID identity flows into Unity Catalog

It's worth being explicit about the plumbing here, because “where does identity actually come from?” is a natural exam question.

1. Your organization's users, groups, and application (service principal) identities live in **Microsoft Entra ID** — this is the single source of truth, shared across all of Azure (VMs, storage, Azure AD-integrated apps) and Databricks.
2. At the **Databricks account level** (accounts.azure.databricks.net), an account admin either manually adds Entra ID users/groups/service principals, or — far more commonly in real deployments — turns on **SCIM provisioning**, which automatically and continuously syncs users and group memberships from Entra

ID into the Databricks account (add someone to an Entra ID group, and within minutes they appear, correctly grouped, in Databricks — no manual step).

3. Those account-level identities are then assignable to one or more **workspaces**, and it's those same identities that Unity Catalog's GRANT/REVOKE statements reference.
4. When a user logs into a workspace, Databricks authenticates them against Entra ID (single sign-on), and every subsequent query they run — whether on a classic cluster or serverless — carries their identity through to Unity Catalog, which checks it against the metastore's grants before returning any data.

The upshot: **you should almost never manage users/groups by hand directly inside Databricks.** Manage group membership in Entra ID, turn on SCIM sync, and grant Unity Catalog privileges to the resulting Databricks groups. This keeps one system (Entra ID) as the single place where “who is on the analysts team” is answered, instead of two systems that can drift out of sync.

2.5 4. Governing Unity Catalog

Securing (section 3) answers “who can access what.” **Governing** is the broader set of tools that let you understand, audit, and control data *after* access has been granted: where did this table's data come from, who actually queried it, what does this column mean, and can I automatically restrict what a query returns instead of relying on separate views for every audience.

2.5.1 4.1 Data lineage

Lineage is an automatically captured graph showing, for any table, view, or even individual column: what upstream tables/columns fed into it, and what downstream tables/queries/dashboards/jobs consumed it — captured automatically by Unity Catalog for any query that runs on a Unity Catalog-enabled cluster or SQL warehouse, with **zero extra code** required from you.

Why it matters for a real job: imagine you need to change the data type of a column, or deprecate a table entirely. Without lineage, you'd have to grep through every notebook and job in the workspace hoping you find every consumer. With lineage, you open the table in Catalog Explorer, click the **Lineage** tab, and see a visual graph of exactly which downstream tables, dashboards, and jobs depend on it — this is called **impact analysis**, and it's the main real-world reason lineage exists.

To view it: Catalog Explorer → navigate to `dp750_lab.bronze.customers` → **Lineage** tab. Run a couple of queries first so there's something to see, e.g.:

```
CREATE OR REPLACE TABLE dp750_lab.bronze.customers_by_region AS
SELECT region, COUNT(*) AS customer_count
FROM dp750_lab.bronze.customers
GROUP BY region;
```

Now revisit the Lineage tab on customers — you'll see customers_by_region appear as a downstream table, and if you open customers_by_region's own Lineage tab, customers appears upstream of it. Click **column lineage** to drill down to which specific source columns (e.g., region) fed which specific destination columns — this column-level detail is a Unity Catalog headline feature and worth remembering by name for the exam.

2.5.2 4.2 Audit logging via system tables

Unity Catalog automatically records every significant action — logins, queries, grants/revokes, table creations/drops, and more — as structured audit events, queryable directly with SQL through **system tables**: a set of Unity Catalog tables Databricks maintains for you automatically under the built-in system catalog, so you never have to set up separate log shipping just to ask “who did what, and when.”

The most important one for security/governance is system.access.audit. Example query — “show me every SELECT-relevant read against our lab schema in the last 7 days, and who ran it”:

```
SELECT
  event_time,
  user_identity.email AS performed_by,
  action_name,
  request_params.full_name_arg AS object_name,
  source_ip_address
FROM system.access.audit
WHERE service_name = 'unityCatalog'
  AND action_name IN ('getTable', 'createTable', 'deleteTable')
  AND event_time >= now() - INTERVAL 7 DAYS
ORDER BY event_time DESC;
```

Or, focused specifically on permission changes — arguably the single highest-value audit query for a security-minded engineer:

```
SELECT
  event_time,
  user_identity.email AS granted_by,
  action_name,
  request_params
FROM system.access.audit
WHERE action_name IN ('createGrant', 'removeGrant')
ORDER BY event_time DESC
LIMIT 50;
```

Two exam-relevant notes: (1) system tables live under the reserved system catalog and schemas like system.access, system.billing, system.compute — you don't create them, Databricks populates them automatically once enabled; (2) some system schemas need to be explicitly enabled once per metastore by an account admin (via **Catalog Explorer** → system catalog → enable the specific schema, e.g. access), so if system.access.audit appears empty or missing, that's the first thing to check.

2.5.3 4.3 Tags

A **tag** is a small key (and optional value) label you attach to a catalog, schema, table, column, or volume — e.g., `pii=true`, `cost_center=finance`, `data_classification=confidential`. Tags don't change security by themselves, but they make governance *searchable and automatable*: you can search Catalog Explorer for every object tagged `pii=true` to build a data classification inventory, feed tags into external governance/cataloging tools, or (in more advanced setups) drive attribute-based access policies off of them.

-- Tag a table and one sensitive column

```
ALTER TABLE dp750_lab.bronze.customers SET TAGS ('data_classification' =  
  ⇨ 'confidential');
```

```
ALTER TABLE dp750_lab.bronze.customers ALTER COLUMN email SET TAGS ('pii' =  
  ⇨ 'true');
```

-- Tag a schema and catalog too

```
ALTER SCHEMA dp750_lab.bronze SET TAGS ('layer' = 'bronze');
```

```
ALTER CATALOG dp750_lab SET TAGS ('environment' = 'study-lab');
```

You can also add/edit tags visually: Catalog Explorer → open the table → **Tags** section → **Add tag**.

2.5.4 4.4 Hands-on lab: dynamic views with a row filter and a column mask

Environment for this lab: either environment — Free Edition is fine and faster. Dynamic views are pure SQL/Unity Catalog metadata features with no dependency on external storage.

A **dynamic view** is an ordinary Unity Catalog view whose SELECT statement includes functions that inspect *who is running the query right now* — most commonly `current_user()` and `is_account_group_member()` — so the exact same view returns different rows, or masks different columns, depending on who queries it. This is Unity Catalog's built-in mechanism for **row-level security** and **column-level masking**, without needing to maintain separate physical tables per audience.

Step 1 — set up the scenario. We'll say: the analysts group should only see rows for their own region, a special eu-analysts group should see full, unmasked email addresses (they're EU-based and cleared for PII), and everyone else should see emails masked. First, create the extra group and add yourself to both, so you can test both branches:

Admin Settings → Identity and access → Groups → create eu-analysts → add your user to both analysts and eu-analysts for testing (in a real org these would be mutually exclusive groups, but for this lab we want to see both behaviors from the same account by temporarily removing yourself from one group between tests).

Step 2 — the row filter function. A row filter is a SQL function (returning BOOLEAN) that Unity Catalog applies as an implicit WHERE clause on every query against the view.

```
CREATE OR REPLACE FUNCTION dp750_lab.bronze.region_row_filter(row_region  
  ⇨ STRING)
```

RETURNS **BOOLEAN**

RETURN

```
is_account_group_member('metastore_admins')           -- admins see
↳ everything
OR is_account_group_member('eu-analysts') AND row_region = 'EU'
OR NOT is_account_group_member('analysts');           -- non-analysts
↳ unaffected by this rule
```

Step 3 — the column mask function. A column mask is a SQL function that transforms one column's value per row, again based on who's asking.

CREATE OR REPLACE FUNCTION dp750_lab.bronze.mask_email(email STRING)

RETURNS STRING

RETURN

CASE

```
WHEN is_account_group_member('eu-analysts') THEN email
ELSE CONCAT(LEFT(email, 2), '***@***MASKED***')
```

END;

Step 4 — build the dynamic view, attaching the row filter and column mask directly to the underlying table. (Row filters and column masks can be attached either to the base table itself with `ALTER TABLE ... SET ROW FILTER ...` / `ALTER TABLE ... ALTER COLUMN ... SET MASK ...`, or wrapped inside a separate view — we'll do the view approach here since it keeps the base table completely untouched and is easier to reason about for a beginner.)

CREATE OR REPLACE VIEW dp750_lab.bronze.customers_secure **AS**

SELECT

```
customer_id,
full_name,
dp750_lab.bronze.mask_email(email) AS email,
region,
signup_date
```

FROM dp750_lab.bronze.customers

WHERE dp750_lab.bronze.region_row_filter(region);

-- Let analysts query the view (not the raw table)

GRANT SELECT ON VIEW dp750_lab.bronze.customers_secure **TO** `analysts`;

Step 5 — test it. Query the view:

SELECT * FROM dp750_lab.bronze.customers_secure;

While you're in both analysts and eu-analysts, you should see EU rows only (from the row filter) with real, unmasked emails (from the column mask, since eu-analysts bypasses masking). Now go to Admin Settings, temporarily remove yourself from eu-analysts (keep analysts), re-run the same query — you should still see only EU rows, but now the email column shows the masked form like am***@***MASKED***. This proves the row filter and column mask are both being evaluated live, per-user, against the exact same view definition — nobody had to write two different views.

Why this matters for the exam and for real jobs: dynamic views (and their table-

attached equivalents, row filters/column masks) let one team maintain a single physical table while safely serving many different audiences with different visibility — a read-only analyst, an EU-compliance-cleared analyst, and an admin all query the same object name and each gets exactly what they’re entitled to see, enforced centrally instead of trusted-to-remember in application code.

2.5.5 4.5 Delta Sharing (conceptual overview)

Delta Sharing is an open, vendor-neutral protocol (originally created by Databricks, now a Linux Foundation project) for sharing live data — tables, views, volumes, even ML models — with another organization or another compute platform **without copying the data**. The recipient reads directly from your storage over a secure REST protocol, using standard connectors available even outside Databricks (Python’s delta-sharing library, Power BI, Tableau, Apache Spark, pandas, etc.), so the party you’re sharing with doesn’t need to be a Databricks customer at all.

The core idea, in a sentence: a **provider** (you) creates a **Share** (a named bundle of Unity Catalog objects) and grants it to a **Recipient** (another org, identified either by a Databricks-to-Databricks connection or by a portable credential file for non-Databricks recipients); the recipient then mounts that share and queries it like any other data source, always seeing the current, live data — no nightly export job, no duplicated copy drifting out of sync, and no need to give the recipient direct access to your workspace or cloud storage account.

Typical use cases: selling/licensing a dataset to customers, sharing curated data with a partner company, or distributing data to a subsidiary that runs its own separate Databricks account or an entirely different platform. You won’t be asked to build a full Delta Sharing lab for this exam, but you should recognize the concept, know the vocabulary (**provider**, **recipient**, **share**), and know the one-line differentiator: **Delta Sharing shares data live, in place, across organizational/platform boundaries, without copying it** — as opposed to a GRANT (section 3.2), which only controls access *within* a single metastore.

2.6 5. Mini Quiz — Domains 1 & 2

These are **original practice questions written for this book** — not real, leaked, or memorized exam questions — designed to check whether you’ve internalized the concepts above. Cover the answers and reason through each one before checking.

1. (Multiple choice) A team lead wants users to be able to query data interactively with sub-second startup time and doesn’t want to manage VM sizing or worry about idle clusters running up cost overnight. Which compute option best fits this requirement?

- A. A classic cluster with autoscaling enabled and a 10-minute auto-termination
- B. A classic cluster with a fixed number of workers and no auto-termination
- C. Serverless compute

- D. A cluster policy alone, without creating any actual compute

Answer: C. Serverless compute has near-instant startup (no VM cold start), scales automatically without you sizing anything, and only bills for actual compute-seconds used — directly matching all three stated requirements. A classic cluster (A) could get close with good settings but still has a multi-minute cold start on each new session; B is actively wrong (no auto-termination risks runaway cost); D is a distractor — a policy constrains creation, it isn't compute itself.

2. (Multiple choice) You create a classic cluster in a Unity Catalog-enabled workspace and set its access mode to a legacy “No Isolation Shared” mode. What is the most accurate consequence?

- A. Unity Catalog governance (GRANT/REVOKE enforcement) is not reliably enforced on that cluster
- B. Nothing changes — Unity Catalog governance is enforced identically on every access mode
- C. The cluster will refuse to start
- D. Only column masks stop working; row filters and table grants still work normally

Answer: A. “No Isolation Shared” is a legacy mode that does not enforce per-user Unity Catalog data governance — it's disabled by default in Unity Catalog workspaces precisely because it can't guarantee that GRANT/REVOKE decisions are honored per user. Single User and Shared access modes are the ones that properly enforce Unity Catalog governance.

3. (Multiple choice) You run `DROP TABLE sales.gold.external_orders`, where `external_orders` was created with an explicit `LOCATION` pointing at an existing ADLS Gen2 path. What happens to the underlying Parquet/Delta data files?

- A. They are permanently deleted along with the metadata
- B. They remain in ADLS Gen2 untouched; only the Unity Catalog registration is removed
- C. They are moved to a Unity Catalog-managed trash location for 30 days, then deleted
- D. The drop fails unless you pass a `FORCE` flag

Answer: B. This is the defining behavior of an external table: Unity Catalog only owns the metadata registration for external tables, never the storage lifecycle, so dropping one leaves the actual files exactly where they were. (Contrast with a managed table, where dropping it does delete the underlying data.)

4. (Multiple answer — select two) Which two Unity Catalog privileges, at minimum, must a principal hold on the *ancestors* of a table before a `SELECT` grant on the table itself will actually let them query it?

- A. `USE CATALOG` on the parent catalog
- B. `OWNERSHIP` on the parent catalog
- C. `USE SCHEMA` on the parent schema

- D. CREATE TABLE on the parent schema

Answer: A and C. USE CATALOG and USE SCHEMA are the “door key” privileges required simply to reference an object through its parents; without both, a table-level SELECT grant is inert. OWNERSHIP (B) is far more than necessary, and CREATE TABLE (D) is unrelated to reading an existing table.

5. (Scenario / short answer) Your organization runs a nightly Lakeflow Job that ingests vendor data into a Unity Catalog table. Currently, the job is configured to run as your colleague Sam’s personal user identity. What’s wrong with this, and what should change?

Answer: Running unattended production pipelines under a personal human identity is an anti-pattern: if Sam leaves the company, changes teams, or their credentials are rotated/disabled, the job silently breaks; it’s also impossible to audit-distinguish “Sam ran this manually” from “the scheduled job ran.” The fix is to create a **service principal** (an application identity in Microsoft Entra ID, added to the Databricks account), grant it exactly the Unity Catalog privileges the job needs (e.g., USE CATALOG, USE SCHEMA, CREATE TABLE, MODIFY on the target schema), and reconfigure the job to run as that service principal instead of as Sam.

6. (Multiple choice) Which statement correctly distinguishes DBFS from a Unity Catalog Volume?

- A. DBFS and Volumes are simply two names for the same underlying feature
- B. Volumes are governed Unity Catalog objects with GRANT-based permissions and lineage visibility; DBFS is a workspace-level file system without equivalent centralized governance
- C. DBFS can only store Delta tables; Volumes can only store raw files
- D. Volumes are exclusively available on Databricks Free Edition

Answer: B. This is exactly why Volumes are now the recommended way to store non-tabular files: they live inside a catalog/schema, support GRANT/REVOKE like tables, and appear in Catalog Explorer and lineage, none of which DBFS paths natively provide. A, C, and D all misstate the relationship.

7. (Scenario / short answer) A view definition includes the function call `is_account_group_member('eu-analysts')` inside a CASE expression applied to an email column, and the same view’s WHERE clause calls a row-filter function keyed off region. What is this pattern called, and what two distinct governance behaviors does it combine?

Answer: This is a **dynamic view**. It combines a **column mask** (the CASE/`is_account_group_member` logic that transforms the email column differently depending on the querying user’s group membership) with a **row filter** (the WHERE-clause function that limits which rows a query returns, again based on the caller’s identity/group). Both are evaluated live per query, letting one physical table/view definition safely serve multiple audiences with different visibility, instead of maintaining separate copies.

8. (Multiple choice) Where would you look to answer the question, “Which downstream tables and dashboards will break if I change the schema of `sales.gold.monthly_revenue`?”

- A. `system.access.audit`
- B. The Lineage tab for that table in Catalog Explorer
- C. `SHOW GRANTS ON TABLE sales.gold.monthly_revenue`
- D. The cluster's Spark UI

Answer: B. This is precisely what data lineage is for — impact analysis. Catalog Explorer's Lineage tab shows every downstream table, view, dashboard, and job that consumed this table, captured automatically with no extra code. The audit log (A) tells you *who did what*, not *what depends on what*; `SHOW GRANTS` (C) tells you permissions, not dependencies; the Spark UI (D) is about job execution internals, unrelated to cross-object lineage.

9. (Multiple choice) Which Azure-side identity option is recommended for a beginner setting up a storage credential for an external location, and why?

- A. An Entra ID app registration with a manually managed client secret, because it offers the most granular control
- B. A Databricks-managed identity via an Access Connector for Azure Databricks, because Databricks handles credential rotation and lifecycle automatically
- C. A personal Azure user account's own credentials, because it's the fastest to set up
- D. No identity is required — external locations authenticate anonymously

Answer: B. The Access Connector approach uses a first-class Azure managed identity that Databricks provisions and manages for you, with no secrets to rotate manually — the simplest and most beginner-friendly path, and the one this chapter's lab used. A is valid for advanced/cross-system scenarios but adds operational burden (secret rotation). C is not how storage credentials work and is not a supported/secure pattern. D is false — external locations always require an explicit storage credential.

10. (Multiple choice) You need to grant a group read access to every table currently in a schema *and* to any table added to that schema in the future, using a single statement. What should you grant?

- A. `GRANT SELECT ON TABLE <every table individually> TO group`
- B. `GRANT SELECT ON SCHEMA catalog.schema TO group`
- C. `GRANT USE SCHEMA ON SCHEMA catalog.schema TO group`
- D. `GRANT ALL PRIVILEGES ON CATALOG catalog TO group`

Answer: B. Granting a privilege at the schema level applies it to every current *and future* object of that type within the schema, which is exactly the “set it once” behavior described. A doesn't cover future tables and doesn't scale; C only grants the ability to reference the schema, not to read data; D is far broader than requested (grants everything on the whole catalog, not just read access to one schema).

Chapter wrap-up: you've now covered the full mechanics of Domain 1 (workspace anatomy, classic vs. serverless compute, access modes, cluster policies) and Domain 2 (the Unity Catalog object hierarchy, managed vs. external tables, storage credentials and external locations, principals, `GRANT/REVOKE`, lineage, audit logs, tags,

dynamic views, and Delta Sharing). Together these two domains are worth roughly a third of your exam score — if the hands-on labs in this chapter feel comfortable to redo from memory, you are in genuinely good shape for that third of the exam. Next, Part 3 moves into building actual data pipelines with Delta Lake and Spark on top of the environment you just secured.

Chapter 3

Part 3 — Data Engineering Core: Delta Lake, Ingestion, Transformation & Data Quality

3.1 Why this chapter matters most

Domain 3, “Prepare and process data,” is worth 30–35% of the DP-750 exam — more than any other domain. It is also the most *hands-on* domain: the questions describe a scenario (a nightly batch job, a folder full of JSON files landing in cloud storage, a table that needs an upsert) and ask you to pick the right Databricks mechanism. You cannot memorize your way through this domain; you need to have actually typed the commands. Every lab in this chapter is designed to be run for real, either in Databricks Free Edition (serverless — completely sufficient for everything here) or in your Azure Databricks Premium workspace.

One terminology note before we start, because the exam and the internet disagree on names: **Delta Live Tables (DLT)** was renamed **Lakeflow Declarative Pipelines**. They are the exact same product. Older YouTube tutorials, blog posts, and even some still-current documentation pages say “DLT” and use `@dlt.table` / `@dlt.expect` decorators in Python. That syntax is *still correct and still current* — only the marketing name changed. Databricks also ships **Lakeflow Connect**, which is the name for managed, point-and-click ingestion connectors (databases, SaaS apps like Salesforce or Workday, etc.), as opposed to Auto Loader or `COPY INTO`, which are things *you* configure by hand. Keep these three names straight:

Old name	Current name	What it is
Delta Live Tables (DLT)	Lakeflow Declarative Pipelines	Declarative framework for building bronze→silver→gold pipelines with <code>@dlt.table</code>
(no old name — newer feature)	Lakeflow Connect	Managed connectors for databases/SaaS sources
Auto Loader	Auto Loader (unchanged)	Incremental, scalable file ingestion via <code>cloudFiles</code>

You will see both “DLT” and “Lakeflow Declarative Pipelines” used interchangeably for the rest of this book, matching how you’ll see them used in the wild.

3.2 1. Delta Lake Deep Dive

3.2.1 1.1 The problem Delta Lake solves

Imagine you have a folder in cloud storage (Azure Data Lake Storage, S3, whatever) full of Parquet or CSV files, and you treat that folder as a “table” — this is the classic data lake pattern, and it has real problems:

- **No transactions.** If a Spark job writing new files crashes halfway through, you can be left with half-written files and no way to know if the “table” is in a consistent state. Two jobs writing to the same folder at the same time can corrupt each other’s output.
- **No schema enforcement.** Nothing stops someone from writing a file with a price column typed as a string into a folder where every other file has it typed as a double. Your downstream `SELECT * FROM sales` query blows up or silently gives you garbage.
- **No easy updates or deletes.** Parquet files are immutable. If you need to update one row (say, correcting a customer’s address) or comply with a “right to be forgotten” deletion request, your only real option is to rewrite the *entire* file (or the entire table) that row lives in. There is no `UPDATE` or `DELETE` statement for a plain Parquet folder.
- **No history.** Once you overwrite a file, the old version is gone. If a bad job corrupts your data, there’s no built-in “undo.”

Delta Lake fixes all four problems by adding a transaction log on top of Parquet files. Think of it this way: a plain Parquet folder is like a folder of separate photos with no album. **Delta Lake is like a pandas DataFrame that lives on disk, except it also keeps a full undo history of every operation ever done to it** — every insert, update, delete, and schema change is a numbered, recorded transaction. A Delta table is still, physically, just a folder of Parquet files — but it also has a hidden subfolder called `_delta_log` that tracks exactly which files are “currently part of the table” and every change that has ever happened.

3.2.2 1.2 ACID transactions and the transaction log, explained simply

ACID is a set of four guarantees relational databases have offered for decades, and Delta Lake brings them to your data lake:

- **Atomicity** — an operation either fully happens or doesn’t happen at all. No half-finished writes.
- **Consistency** — the table always satisfies its schema and constraints after each transaction.

- **Isolation** — concurrent readers and writers don’t see each other’s half-finished work.
- **Durability** — once a transaction is committed, it’s permanently recorded, even if the cluster dies a second later.

Here’s the mechanism, in plain terms. Every Delta table has a folder named `_delta_log` sitting right alongside the data files:

```
my_table/
├── _delta_log/
│   ├── 00000000000000000000000000000000.json
│   ├── 00000000000000000000000000000001.json
│   ├── 00000000000000000000000000000002.json
│   ├── 00000000000000000000000000000010.checkpoint.parquet
│   └── ...
├── part-0001-....snappy.parquet
├── part-0002-....snappy.parquet
└── ...
```

Each JSON file in `_delta_log` represents one **commit** (one transaction), numbered sequentially starting at 0. A commit is just a list of simple actions, the most important being:

- `add` — “this new Parquet file is now part of the table”
- `remove` — “this Parquet file is no longer part of the table” (it is *marked* removed, not physically deleted yet)
- `metaData` — schema, partitioning, table properties
- `commitInfo` — who did what operation, when (this is what powers `DESCRIBE HISTORY`)

Crucially: **Delta never edits or deletes an existing Parquet file to perform an UPDATE or DELETE.** Instead, it writes brand-new Parquet files containing the corrected data, and then writes a new commit JSON that says “remove these old files, add these new files.” The old files usually stick around on disk (until you clean them up with `VACUUM`, see below) — which is exactly what makes **time travel** possible: to read the table “as of version 5,” Delta just replays commits 0 through 5 and figures out which set of Parquet files was active at that point, then reads those specific files.

Every 10 commits (by default), Delta writes a `.checkpoint.parquet` file that summarizes the full table state at that point, so that reading the current version of a table with a long history doesn’t require replaying thousands of tiny JSON files from the beginning — it can just start from the nearest checkpoint.

This is also how Delta gets **isolation** for concurrent writers: two jobs racing to write commit #47 will both try to atomically create a file named `00000000000000000000000000000047.json`. Cloud storage guarantees only one of them wins that atomic create; the loser detects the conflict, and (depending on the operation) either retries against the new version or fails cleanly. You never end up with a corrupted, half-merged table.

3.2.3 1.3 Hands-on lab: transactions, history, and time travel

Open a new notebook attached to a serverless compute (Free Edition is fine) or a cluster in your Premium workspace. Make sure you have a catalog and schema to work in — reuse the ones from Part 2, or create a scratch one:

```
CREATE CATALOG IF NOT EXISTS dp750_demo;  
CREATE SCHEMA IF NOT EXISTS dp750_demo.core;  
USE CATALOG dp750_demo;  
USE SCHEMA core;
```

Step 1 — create a Delta table and insert some rows.

```
CREATE OR REPLACE TABLE orders (  
  order_id INT,  
  customer STRING,  
  product STRING,  
  quantity INT,  
  status STRING  
) USING DELTA;  
  
INSERT INTO orders VALUES  
  (1, 'Alice', 'Keyboard', 1, 'PENDING'),  
  (2, 'Bob', 'Mouse', 2, 'PENDING'),  
  (3, 'Charlie', 'Monitor', 1, 'PENDING'),  
  (4, 'Alice', 'Mouse', 1, 'PENDING');
```

USING DELTA is actually the default table format in Databricks/Unity Catalog, so you'll rarely need to type it — but it's worth writing it out at least once so you know it's there.

Step 2 — run UPDATE, DELETE, and MERGE.

```
-- UPDATE: mark one order shipped  
UPDATE orders SET status = 'SHIPPED' WHERE order_id = 1;  
  
-- DELETE: remove a cancelled order  
DELETE FROM orders WHERE order_id = 3;  
  
-- MERGE: upsert — update Bob's order if it exists, insert a new one if not  
MERGE INTO orders AS target  
USING (SELECT 2 AS order_id, 'Bob' AS customer, 'Mouse' AS product, 5 AS  
  ⇨ quantity, 'SHIPPED' AS status  
  UNION ALL  
  SELECT 5, 'Dana', 'Webcam', 1, 'PENDING') AS source  
ON target.order_id = source.order_id  
WHEN MATCHED THEN UPDATE SET *  
WHEN NOT MATCHED THEN INSERT *;  
  
SELECT * FROM orders ORDER BY order_id;
```

Each of UPDATE, DELETE, and MERGE is its own atomic commit in `_delta_log` — under the hood, each one rewrites only the Parquet files that contain affected rows and leaves the rest untouched (this file-level rewrite is why Delta operations on huge tables can still be fast — a table-wide UPDATE doesn't rewrite the *whole* table, just the files holding matching rows).

Step 3 — inspect the history.

```
DESCRIBE HISTORY orders;
```

You'll get a row per commit, including columns like `version`, `timestamp`, `operation` (WRITE, UPDATE, DELETE, MERGE), `operationParameters`, and `readVersion`. This is a direct, human-readable view into `_delta_log`.

Step 4 — time travel.

```
-- By version number
```

```
SELECT * FROM orders VERSION AS OF 0;
```

```
-- By timestamp (use a timestamp you saw in DESCRIBE HISTORY, adjust to  
↪ your run)
```

```
SELECT * FROM orders TIMESTAMP AS OF '2026-08-28T00:00:00.000Z';
```

```
-- The PySpark equivalent
```

```
df_v0 = spark.read.format("delta").option("versionAsOf",  
↪ 0).table("dp750_demo.core.orders")  
df_v0.display()
```

Version 0 shows you the table exactly as it was right after the initial INSERT — before any of the UPDATE/DELETE/MERGE happened. This is genuinely useful in production: if a bad pipeline run corrupts a table, you can inspect (and even restore) a prior version with `RESTORE TABLE orders TO VERSION AS OF 0;`.

Exam tip: `DESCRIBE HISTORY`, `VERSION AS OF`, `TIMESTAMP AS OF`, and `RESTORE TABLE` are all classic exam keywords for the “time travel” concept. Know that time travel is *not* infinite — it's bounded by two things: how far back your `VACUUM` retention window goes (see 1.5) and the transaction log retention (default 30 days for the log itself, separate from the `VACUUM` retention for data files).

3.2.4 1.4 Schema enforcement and schema evolution

Schema enforcement (a.k.a. schema-on-write) means: by default, Delta Lake refuses to write data that doesn't match the target table's existing schema. Try it:

```
from pyspark.sql import Row
```

```
bad_row = spark.createDataFrame([Row(order_id=6, customer="Eve",  
↪ product="Headset", quantity=1, status="PENDING", extra_col="oops")])
```

```
bad_row.write.format("delta").mode("append").saveAsTable(
  ↪ "dp750_demo.core.orders")
```

This throws an `AnalysisException` — something like “*A schema mismatch detected... Table insertion contains extra column(s).*” This is a *feature*, not a bug: it’s exactly the guardrail that plain Parquet folders don’t have. It stops garbage columns or wrong types from silently corrupting your table.

Sometimes, though, you genuinely *want* the table to grow — a new column got added upstream and you want it to flow through. That’s **schema evolution**, and you opt into it explicitly with `mergeSchema`:

```
bad_row.write.format("delta") \
  .mode("append") \
  .option("mergeSchema", "true") \
  .saveAsTable("dp750_demo.core.orders")
```

```
spark.table("dp750_demo.core.orders").printSchema()
```

Now `extra_col` is added to the table’s schema (existing rows get `NULL` for it), and the write succeeds. You can also enable this table-wide so you don’t have to pass the option on every write:

```
ALTER TABLE orders SET TBLPROPERTIES ('delta.autoOptimize.autoMerge' =
  ↪ 'true');
-- or, more commonly seen:
ALTER TABLE orders SET TBLPROPERTIES ('delta.schema.autoMerge.enabled' =
  ↪ 'true');
```

Exam tip: Know the difference between the *default* (strict enforcement, write fails on mismatch) and *opt-in* evolution (`mergeSchema` option, or `spark.databricks.delta.schema.autoMerge.enabled` at the session level). The exam likes to test “what happens when incoming data has an extra/missing/differently-typed column and you did/didn’t set `mergeSchema`.”

3.2.5 1.5 Table maintenance: OPTIMIZE, ZORDER BY, VACUUM

As you run many small `INSERT/UPDATE/MERGE` operations over time, a Delta table accumulates lots of small Parquet files (a classic “small file problem” — the same one you’d get from many small `pandas.to_parquet()` calls into the same folder). Small files hurt read performance because Spark has to open and plan around each one separately.

OPTIMIZE compacts many small files into fewer, larger files:

```
OPTIMIZE orders;
```

ZORDER BY goes a step further: it also physically co-locates rows that have similar values in the specified column(s), so that queries filtering on that column can skip reading files that don’t contain matching data (this is called **data skipping**, and it relies on Delta storing min/max statistics per file in the transaction log).

```
OPTIMIZE orders ZORDER BY (customer);
```

Use ZORDER BY on columns you frequently filter or join on — but don't overdo it; Z-ordering more than 3–4 columns gives rapidly diminishing returns because it's fundamentally a way of interleaving data along multiple dimensions into one physical sort order.

VACUUM physically deletes old, no-longer-referenced Parquet files from storage — the ones that got removed by an UPDATE/DELETE/MERGE/OPTIMIZE and are now safe to garbage-collect:

```
VACUUM orders;                                -- uses default retention (7 days)
VACUUM orders RETAIN 168 HOURS;                -- explicit: 168 hours = 7 days
```

Why 7 days by default, and why you should be very careful shortening it: any file still referenced by a version within your retention window is needed for time travel and for any *currently running* long query that started reading an older snapshot. If you VACUUM with too short a retention window, you can break time travel queries and even cause failures for readers mid-flight. Databricks deliberately makes you jump through a hoop to go below the default:

```
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM orders RETAIN 0 HOURS;  -- dangerous, only for a demo/dev table
```

Exam tip: Memorize the three commands and what each does — OPTIMIZE (compact small files), ZORDER BY (co-locate related data for skipping), VACUUM (physically delete old unreferenced files, default 7-day retention). A very common scenario question: “queries against a heavily-updated Delta table have gotten slow — what should you run?” → OPTIMIZE (with ZORDER BY if there's a well-known filter column).

3.3 2. Medallion Architecture & Data Modeling in Unity Catalog

3.3.1 2.1 Bronze, silver, gold — explained with a concrete example

The **medallion architecture** is simply a naming convention and a set of expectations for organizing tables into progressive stages of quality and refinement. You've already been implicitly doing something like this if you've ever written a `raw_df`, then a `clean_df`, then a `summary_df` in a pandas/notebook workflow — medallion architecture just formalizes that into permanent tables.

Picture a retail company streaming raw sales events from its point-of-sale systems:

- **Bronze (raw)** — data landed *as-is*, exactly as it arrived from the source, with no transformation beyond maybe adding ingestion metadata (a load timestamp, source filename). Nothing is deleted, cleaned, or deduplicated. This is your audit trail / source of truth for “what did we actually receive.” Example: bronze

.sales_events_raw — one row per raw JSON event, including any malformed or duplicate ones, plus columns like `_ingested_at` and `_source_file`.

- **Silver (cleaned & conformed)** — data is deduplicated, bad rows are filtered or quarantined, types are cast correctly, and it's joined/conformed against reference data as needed. This is the “trustworthy, queryable” layer analysts and other engineers build on. Example: `silver.sales_events` — deduplicated, valid rows only, `event_time` cast to a proper timestamp, `amount` cast to `DECIMAL`.
- **Gold (business-level aggregates)** — data is aggregated and shaped specifically for consumption: BI dashboards, ML feature tables, executive reports. Example: `gold.daily_sales_by_store` — one row per store per day, with total revenue, order count, etc.

Data flows one direction: bronze → silver → gold. Each layer typically lives in its own schema (or even its own catalog, depending on your governance design) within Unity Catalog — e.g., `retail.bronze.sales_events_raw`, `retail.silver.sales_events`, `retail.gold.daily_sales_by_store`.

Why bother with three layers instead of just cleaning data once? Two big reasons the exam expects you to know:

1. **Reprocessing.** If you discover a bug in your cleaning logic three weeks later, you still have the untouched bronze data to reprocess from — you didn't destroy information by cleaning “in place.”
2. **Incremental efficiency.** Each layer can be processed incrementally (only new/changed records) rather than every layer re-scanning the entire raw history every time, which is exactly what Structured Streaming and Lakeflow Declarative Pipelines are built to do well.

3.3.2 2.2 Fact vs. dimension tables — the basics you need for the exam

This is classic Kimball-style dimensional modeling, and the lakehouse gold layer is a very natural place to apply it.

- A **fact table** holds *events* or *measurements* — things that happened, with numeric values you want to aggregate (sum, count, average). Rows are typically append-only and there are a lot of them. Example: one row per order line item, with `quantity`, `unit_price`, foreign keys to `customer_id` and `product_id`.
- A **dimension table** holds *descriptive attributes* about the entities referenced by a fact table — the “who/what/where” context. Rows change relatively rarely and there are far fewer of them. Example: `dim_customer` (name, region, signup date), `dim_product` (category, brand, price tier).

The classic mental model: fact tables are *narrow-but-tall and numeric*, dimension tables are *wide-but-short and descriptive*. You join a fact table to its dimensions to answer questions like “total revenue by region” (join `fact_orders` → `dim_customer` → group by region).

Two dimension-modeling terms worth knowing for scenario questions:

- **Slowly Changing Dimension (SCD) Type 1** — when an attribute changes (a

customer moves to a new address), you just overwrite the old value. No history kept.

- **Slowly Changing Dimension (SCD) Type 2** — when an attribute changes, you keep the old row (often with `effective_date/end_date` or `is_current` columns) and insert a new row for the new value, preserving history. `MERGE INTO` is the standard tool for implementing SCD Type 2 upserts in Delta Lake — you'll see this exact pattern in section 4.3.

3.3.3 2.3 Hands-on exercise: bronze/silver/gold DDL for a retail dataset

Let's design real schemas for a small retail scenario: raw orders, customers, and products land as bronze, get cleaned into silver, and get aggregated into a gold fact/dimension model.

```
CREATE CATALOG IF NOT EXISTS retail;  
CREATE SCHEMA IF NOT EXISTS retail.bronze;  
CREATE SCHEMA IF NOT EXISTS retail.silver;  
CREATE SCHEMA IF NOT EXISTS retail.gold;
```

Bronze — raw, untyped-ish, everything kept, plus ingestion metadata:

```
CREATE OR REPLACE TABLE retail.bronze.orders_raw (  
  order_id      STRING,  
  customer_id   STRING,  
  product_id    STRING,  
  quantity      STRING,  -- kept as STRING on purpose: source system is  
  ↪ messy  
  order_ts      STRING,  
  raw_payload   STRING,  -- original JSON string, for full auditability  
  _ingested_at  TIMESTAMP,  
  _source_file  STRING  
) USING DELTA;
```

```
CREATE OR REPLACE TABLE retail.bronze.customers_raw (  
  customer_id   STRING,  
  full_name     STRING,  
  email         STRING,  
  region        STRING,  
  signup_date   STRING,  
  _ingested_at  TIMESTAMP,  
  _source_file  STRING  
) USING DELTA;
```

```
CREATE OR REPLACE TABLE retail.bronze.products_raw (  
  product_id    STRING,  
  product_name  STRING,  
  category      STRING,  
  unit_price    STRING,  
  _ingested_at  TIMESTAMP,
```

```
  _source_file STRING
) USING DELTA;
```

Silver — cleaned, typed, deduplicated, with constraints:

```
CREATE OR REPLACE TABLE retail.silver.orders (
  order_id      STRING NOT NULL,
  customer_id   STRING NOT NULL,
  product_id    STRING NOT NULL,
  quantity      INT,
  order_ts      TIMESTAMP,
  CONSTRAINT positive_quantity CHECK (quantity > 0)
) USING DELTA;
```

```
CREATE OR REPLACE TABLE retail.silver.customers (
  customer_id   STRING NOT NULL,
  full_name     STRING,
  email         STRING,
  region        STRING,
  signup_date   DATE
) USING DELTA;
```

```
CREATE OR REPLACE TABLE retail.silver.products (
  product_id    STRING NOT NULL,
  product_name  STRING,
  category      STRING,
  unit_price    DECIMAL(10,2)
) USING DELTA;
```

Gold — dimensional model built for reporting: one fact table plus conformed dimensions, and a pre-aggregated summary:

```
-- Dimension tables (rename/reshape from silver, add surrogate keys if
  ↳ desired)
```

```
CREATE OR REPLACE TABLE retail.gold.dim_customer (
  customer_id   STRING NOT NULL,
  full_name     STRING,
  region        STRING,
  signup_date   DATE
) USING DELTA;
```

```
CREATE OR REPLACE TABLE retail.gold.dim_product (
  product_id    STRING NOT NULL,
  product_name  STRING,
  category      STRING,
  unit_price    DECIMAL(10,2)
) USING DELTA;
```

```
-- Fact table: one row per order line
```

```
CREATE OR REPLACE TABLE retail.gold.fact_orders (
```

```

order_id      STRING NOT NULL,
customer_id   STRING NOT NULL,
product_id    STRING NOT NULL,
quantity      INT,
order_ts      TIMESTAMP,
line_amount   DECIMAL(12,2) -- quantity * unit_price, computed at load
↪ time
) USING DELTA;

```

-- Pre-aggregated business-ready table, refreshed on a schedule

```

CREATE OR REPLACE TABLE retail.gold.daily_sales_by_region (
  order_date    DATE,
  region        STRING,
  total_orders  BIGINT,
  total_revenue DECIMAL(14,2)
) USING DELTA;

```

Notice the deliberate design choices: bronze keeps everything as loosely typed strings plus the raw payload (nothing is lost, nothing fails to load); silver enforces real types and basic constraints (NOT NULL, CHECK); gold is shaped explicitly around how the data will be *queried* (a fact table for flexible slicing, plus a pre-aggregated table for a specific known dashboard need). This progressive tightening is exactly what exam scenario questions probe: given a description of a layer's purpose, pick the right layer, or predict what a query against a given layer will/won't contain.

3.4 3. Data Ingestion

3.4.1 3.1 Hands-on lab: Auto Loader (cloudFiles) in PySpark

The problem Auto Loader solves: you have a cloud storage folder that new files keep landing in (every few minutes, from some upstream export job), and you want a bronze table that stays continuously up to date *without* re-scanning the entire folder from scratch every time you run the job — that would get slower and more expensive every single day. Auto Loader solves this by keeping track of which files it has already processed, so each run only picks up what's new.

Think of Auto Loader as a mail carrier who remembers exactly which letters they already delivered — each run, they only bring you the new mail, not the entire mailbox history again.

First, let's create some sample files to ingest. In a notebook cell:

```

import json, os

sample_dir = "/Volumes/dp750_demo/core/landing/orders_json"
dbutils.fs.mkdirs(sample_dir)

batch1 = [

```

```

    {"order_id": "1001", "customer_id": "C1", "product_id": "P1",
    ↪ "quantity": 2, "order_ts": "2026-08-20T10:15:00"},
    {"order_id": "1002", "customer_id": "C2", "product_id": "P3",
    ↪ "quantity": 1, "order_ts": "2026-08-20T10:22:00"},
]

```

```

with open("/tmp/orders_batch1.json", "w") as f:
    for row in batch1:
        f.write(json.dumps(row) + "\n")

```

```

dbutils.fs.cp("file:/tmp/orders_batch1.json",
    ↪ f"{sample_dir}/orders_batch1.json")

```

(This assumes a Unity Catalog Volume named landing exists under dp750_demo.core — create one first with `CREATE VOLUME IF NOT EXISTS dp750_demo.core.landing`; if you haven't. Volumes are Unity Catalog's governed way of storing and referencing non-tabular files like this.)

Now the Auto Loader ingestion job:

```

checkpoint_path =
    ↪ "/Volumes/dp750_demo/core/landing/_checkpoints/orders_autoloader"
schema_location =
    ↪ "/Volumes/dp750_demo/core/landing/_schemas/orders_autoloader"

bronze_stream = (
    spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", schema_location)      # where Auto
    ↪ Loader remembers/infers the schema
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaHints", "quantity INT, order_ts TIMESTAMP")
    ↪ # nudge inference for tricky columns
        .load(sample_dir)
    )

(bronze_stream.writeStream
    .format("delta")
    .option("checkpointLocation", checkpoint_path)      # where Auto Loader
    ↪ remembers which files it already processed
        .trigger(availableNow=True)                      # process everything
    ↪ currently available, then stop (good for a demo / batch-like run)
        .toTable("dp750_demo.core.orders_bronze_autoloader")
    )

spark.table("dp750_demo.core.orders_bronze_autoloader").display()

```

Two option names are worth burning into memory because they are exactly the kind of thing multiple-choice questions test:

- `cloudFiles.schemaLocation` — where Auto Loader persists the schema it has inferred (and evolved) over time, so restarts don't have to re-infer from scratch.
- `checkpointLocation` (a Structured Streaming option, not Auto-Loader-specific) — where the *stream itself* tracks progress: which files/offsets it has already committed, so a restart resumes exactly where it left off rather than reprocessing or skipping data.

Now drop a *second* file into the same folder and re-run the same `writeStream` cell — you'll see only the new file's rows get appended, because Auto Loader remembers what it already ingested via the checkpoint:

```
batch2 = [{ "order_id": "1003", "customer_id": "C1", "product_id": "P2",
  ↪  "quantity": 3, "order_ts": "2026-08-21T09:00:00" }]
with open("/tmp/orders_batch2.json", "w") as f:
    for row in batch2:
        f.write(json.dumps(row) + "\n")
dbutils.fs.cp("file:/tmp/orders_batch2.json",
  ↪  f"{sample_dir}/orders_batch2.json")

# re-run the same readStream/writeStream cell above — only
  ↪  orders_batch2.json gets picked up
```

`trigger(availableNow=True)` is a good default for “batch-like” incremental loads (process everything new, then stop — great for a scheduled Job). If instead you want a continuously running stream that picks up files as they arrive, use `.trigger(processingTime="1 minute")` (or no trigger argument at all, which runs as fast as possible) and don't expect the cell to finish — it keeps running until you cancel it.

3.4.2 3.2 Hands-on lab: COPY INTO in SQL

`COPY INTO` is the SQL-native, idempotent way to load new files into a Delta table — simpler to reason about than Auto Loader for smaller-scale or less-frequent loads, and it's pure SQL, no PySpark required.

```
CREATE TABLE IF NOT EXISTS dp750_demo.core.orders_bronze_copyinto (
  order_id      STRING,
  customer_id   STRING,
  product_id    STRING,
  quantity      INT,
  order_ts      TIMESTAMP
) USING DELTA;
```

```
COPY INTO dp750_demo.core.orders_bronze_copyinto
FROM '/Volumes/dp750_demo/core/landing/orders_json'
FILEFORMAT = JSON
FORMAT_OPTIONS ('inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
```

Run that statement twice in a row — the second run loads **zero** rows, because `COPY INTO` automatically tracks (inside the Delta table's own transaction log, no separate

checkpoint needed) which source files it has already loaded, and skips them. Drop a third file into the folder and run `COPY INTO` again — only that new file gets picked up.

3.4.3 3.3 Auto Loader vs. `COPY INTO` vs. batch reads — when to use which

	Auto Loader (cloud Files)	<code>COPY INTO</code>	Plain batch read (<code>spark.read</code>)
Interface	PySpark (<code>readStream/writeStream</code>)	SQL	PySpark or SQL
Scale	Millions of files, high file-arrival rate	Best for thousands of files, not millions	Whatever you point it at, re-read every time
Tracks “already processed”	Yes — via checkpoint + internal RocksDB-backed state	Yes — via Delta transaction log metadata	No — you must build your own logic
Can run continuously (streaming)	Yes	No — always a discrete, triggered batch statement	No
Schema inference & evolution	Strong — <code>cloudFiles.inferColumnTypes</code> , schema hints, schema Location, automatic schema evolution handling	Basic — <code>inferSchema</code> option, less sophisticated	Manual — you pass or infer schema yourself
Best for	Ongoing/recurring ingestion of a large or fast-growing set of files (the mainstream production choice)	Simple, ad hoc, or lower-frequency loads where you don’t need a streaming source	One-off exploration, small static datasets, prototyping
Typical exam framing	“Continuously/incrementally ingest a folder that gets thousands of new files a day, scalably”	“Load new files into a table with a single idempotent SQL statement”	“Read a dataset once, no need to track incremental state”

Exam tip: if the scenario says “millions of files” or “needs to scale” or mentions **streaming**, the answer is Auto Loader. If it says a straightforward SQL statement that just needs to be idempotent and periodically re-run, `COPY INTO` is usually the intended answer. If it’s clearly a one-time load of a known, bounded dataset, plain batch `spark.read` is enough and Auto Loader/`COPY INTO`’s incremental-tracking machinery would be overkill.

3.4.4 3.4 Lakeflow Connect — managed connectors

Everything above is ingestion *you* build: you point Auto Loader or `COPY INTO` at a folder of files. But a lot of real ingestion isn’t “files landing in storage” — it’s “pull incremental changes out of a Salesforce org,” “replicate a SQL Server table,” or “sync a ServiceNow table.” Writing and maintaining hand-rolled connectors for every SaaS API and database is a lot of undifferentiated work.

Lakeflow Connect is Databricks' name for the managed, UI/config-driven connectors that handle this for you — you pick a source (e.g., a database via CDC, or a SaaS application), authenticate, choose the tables/objects you want, and Databricks manages the ongoing incremental ingestion (including schema drift handling) into Unity Catalog tables. Conceptually, it sits at the same “ingestion” layer as Auto Loader/COPY INTO, but it targets **structured, already-cataloged sources** (databases, SaaS applications) rather than raw files in a storage account, and it's designed to need far less custom pipeline code.

When to reach for Lakeflow Connect instead of hand-rolling ingestion:

- The source is a database (e.g., SQL Server, PostgreSQL, Salesforce, Workday, ServiceNow) with a supported native connector, rather than a folder of exported files.
- You want change data capture (CDC)-style incremental replication without writing your own CDC logic.
- You'd rather configure a connector through the UI/API and let Databricks manage schema drift and scheduling than own that code yourself.

When you still reach for Auto Loader / COPY INTO instead:

- Your data already arrives as files in cloud storage (the overwhelmingly common case for exam scenarios involving “a folder of CSV/JSON/Parquet files”).
- The source system isn't one of the natively supported Lakeflow Connect connectors.
- You need fine-grained custom control over the ingestion logic.

Exam tip: think of it as a spectrum — “raw files in storage” → Auto Loader/COPY INTO; “a live source system (database or SaaS app) you want kept in sync” → Lakeflow Connect. You won't need to know deep configuration details of specific connectors for the exam, just this decision boundary and the name.

3.4.5 3.5 Batch vs. streaming ingestion

This is a conceptual distinction the exam expects you to reason about, not just define:

- **Batch ingestion** processes a bounded, finite chunk of data — “everything in this file/folder/table right now” — and then the job finishes. Re-running it later processes whatever new data has accumulated since then, as a brand-new discrete run. Think: a nightly job that reads yesterday's exported CSV.
- **Streaming ingestion** treats the data source as unbounded — a folder that new files keep landing in, or a message bus like Event Hubs/Kafka — and processes it incrementally and continuously (or on a recurring “micro-batch” trigger), maintaining state (via a checkpoint) about exactly how much has been consumed so far.

Importantly, in Databricks, **Auto Loader and Structured Streaming can be run in a “batch-like” way** using `.trigger(availableNow=True)`: the underlying engine is

a streaming engine (state, checkpoints, incremental processing), but it processes everything currently available and then stops, rather than running forever. This gives you the *efficiency* benefits of streaming (only process what's new) with the *operational simplicity* of a batch job you schedule to run on a timer. This hybrid pattern is extremely common in real Databricks pipelines and shows up in exam scenarios as “process new data incrementally, but only needs to run once an hour” — the answer is Auto Loader/Structured Streaming with `trigger(availableNow=True)`, not a continuously running stream.

Use true continuous/near-real-time streaming (`trigger(processingTime=...)` with a short interval, or default trigger) only when you genuinely need low end-to-end latency — e.g., a dashboard that must reflect events within seconds.

3.5 4. Transformations at Scale

3.5.1 4.1 PySpark DataFrame patterns — the Spark-specific gotchas

You already know `select`, `filter/where`, `groupBy`, `join` from pandas-style thinking. The syntax in PySpark is close enough that we won't re-teach it from zero — instead, here's what's genuinely *different* from pandas and shows up on the exam:

Lazy evaluation. In pandas, `df[df.amount > 100]` executes immediately. In PySpark, `df.filter(df.amount > 100)` builds up a query plan but does **not** run anything yet. Spark only actually executes work when you call an **action** — `.display()`, `.count()`, `.collect()`, `.write...save()`, etc. Everything else (`select`, `filter`, `groupBy`, `join`, `withColumn...`) is a **transformation** that just extends the plan. This matters because Spark can look at your *entire* chain of transformations before running anything and optimize it as a whole (the Catalyst optimizer) — e.g., pushing a filter down before a join instead of after, even though you wrote the filter after the join in your code.

```
df = spark.table("dp750_demo.core.orders")
```

```
# Nothing has executed yet after this line — it's just a plan:
```

```
plan = df.filter(df.quantity > 1).groupBy("customer").count()
```

```
# THIS line triggers actual execution:
```

```
plan.display()
```

Narrow vs. wide transformations. A **narrow transformation** (e.g., `filter`, `select`, `withColumn`) can be computed on each partition independently — no data has to move between machines. A **wide transformation** (e.g., `groupBy`, `join`, `distinct`, `orderBy`) requires a **shuffle** — data with matching keys has to be physically moved across the cluster so it ends up on the same machine/partition. Shuffles are the single biggest performance cost in most Spark jobs; recognizing which operations trigger one is a core skill (and a recurring exam-flavored concept, even if the exam won't ask you to hand-tune shuffle partitions).

```
df.filter(df.quantity > 1)
```

```
# narrow — no shuffle
```

```
df.groupBy("customer").sum("quantity") # wide – shuffle required
df.join(other_df, "customer_id")      # wide – shuffle required
```

Partitions. A Spark DataFrame is split into partitions distributed across the cluster’s workers, processed in parallel. Too few partitions underutilizes the cluster; too many creates overhead from managing lots of tiny tasks. With serverless / modern Databricks compute, adaptive query execution (AQE) automatically tunes partition counts and join strategies at runtime, which is a big part of why the “just use serverless and don’t hand-tune Spark configs” story is realistic for most workloads today — but you should still recognize the concept when a question describes symptoms of it (e.g., “the job produced thousands of tiny output files” → too many output partitions, fixable with OPTIMIZE after the fact, or repartition()/AQE tuning beforehand).

3.5.2 4.2 PySpark vs. Spark SQL — the same operations, side by side

Every DataFrame operation has a SQL equivalent, and Databricks notebooks let you freely mix both (register a DataFrame as a temp view, then query it with %sql, or embed SQL strings inside PySpark). Know both forms — the exam presents code in either.

```
# PySpark
from pyspark.sql import functions as F

result = (
    spark.table("dp750_demo.core.orders")
    .filter(F.col("quantity") > 1)
    .groupBy("customer")
    .agg(F.sum("quantity").alias("total_qty"),
    ↪ F.count("*").alias("num_orders"))
    .orderBy(F.desc("total_qty"))
)
result.display()

-- Equivalent Spark SQL
SELECT customer,
       SUM(quantity) AS total_qty,
       COUNT(*)      AS num_orders
FROM dp750_demo.core.orders
WHERE quantity > 1
GROUP BY customer
ORDER BY total_qty DESC;
```

Window functions — used constantly for things like “latest record per customer” or running totals — also have direct equivalents:

```
from pyspark.sql.window import Window

w = Window.partitionBy("customer").orderBy(F.desc("order_ts"))

latest_orders = (
```

```

    spark.table("dp750_demo.core.orders")
      .withColumn("rn", F.row_number().over(w))
      .filter(F.col("rn") == 1)
      .drop("rn")
  )

WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer ORDER BY order_ts
    ↪ DESC) AS rn
  FROM dp750_demo.core.orders
)
SELECT * EXCEPT (rn) FROM ranked WHERE rn = 1;

```

This “keep only the latest row per key” pattern is exactly what you use to deduplicate bronze data into silver — file it away, you’ll want it constantly.

3.5.3 4.3 Hands-on lab: MERGE INTO for CDC-style upserts

A very common real pattern: a staging table receives a batch of changed records (inserts and updates mixed together, as you’d get from a change-data-capture feed), and you need to upsert them into your silver table — updating rows that already exist by key, inserting rows that don’t.

```

-- Staging table simulating an incoming CDC batch
CREATE OR REPLACE TABLE dp750_demo.core.orders_staging (
  order_id      STRING,
  customer_id   STRING,
  product_id    STRING,
  quantity      INT,
  order_ts      TIMESTAMP,
  is_deleted    BOOLEAN
) USING DELTA;

INSERT INTO dp750_demo.core.orders_staging VALUES
  ('1001', 'C1', 'P1', 5,    TIMESTAMP '2026-08-20T10:15:00', false), --
  ↪ update: qty changed from 2 to 5
  ('1004', 'C3', 'P2', 1,    TIMESTAMP '2026-08-22T14:00:00', false), --
  ↪ brand new order
  ('1002', 'C2', 'P3', NULL, TIMESTAMP '2026-08-20T10:22:00', true);  --
  ↪ deleted order

MERGE INTO dp750_demo.core.orders_bronze_autoloader AS target
USING dp750_demo.core.orders_staging AS source
ON target.order_id = source.order_id
WHEN MATCHED AND source.is_deleted = true THEN DELETE
WHEN MATCHED THEN UPDATE SET
  target.quantity = source.quantity,
  target.order_ts = source.order_ts

```

```

WHEN NOT MATCHED AND source.is_deleted = false THEN INSERT (order_id,
↪ customer_id, product_id, quantity, order_ts)
VALUES (source.order_id, source.customer_id, source.product_id,
↪ source.quantity, source.order_ts);

```

```

SELECT * FROM dp750_demo.core.orders_bronze_autoloader ORDER BY order_id;

```

This is the canonical Delta Lake CDC pattern: one MERGE statement, three branches (UPDATE for matched non-deletes, DELETE for matched deletes, INSERT for unmatched non-deletes), driven entirely by a `is_deleted` flag on the source. This exact shape — matched/not-matched, optionally with extra AND conditions on each branch — is worth being able to write from memory.

3.5.4 4.4 Hands-on lab: a full Lakeflow Declarative Pipeline (DLT), bronze → silver → gold

Lakeflow Declarative Pipelines let you *declare* what each table should contain — Databricks figures out execution order, incremental processing, retries, and cluster management for you. You write it as a notebook (or .py file) containing `@dlt.table` (or the newer `@Dlt.table/dlt` decorators — you will see `import dlt` used both ways; `import dlt` remains completely valid) decorated functions, and then you point a **Lakeflow Pipeline** (previously “DLT Pipeline”) resource at that notebook — you don’t run the notebook’s cells directly like a normal notebook.

Create a new notebook, e.g. `retail_pipeline`, with this content:

```

import dlt
from pyspark.sql import functions as F

# ----- BRONZE: a streaming table reading incrementally from a landing
↪ folder -----
@dlt.table(
    name="orders_bronze",
    comment="Raw orders ingested incrementally via Auto Loader."
)
def orders_bronze():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "json")
            .option("cloudFiles.inferColumnTypes", "true")
            .load("/Volumes/dp750_demo/core/landing/orders_json")
            .withColumn("_ingested_at", F.current_timestamp())
    )

# ----- SILVER: cleaned, deduplicated, with data quality expectations
↪ -----
@dlt.table(
    name="orders_silver",

```

```

        comment="Cleaned, deduplicated orders with quality checks."
    )
    @dlt.expect("valid_quantity", "quantity > 0")
    @dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL")
    def orders_silver():
        return (
            dlt.read_stream("orders_bronze")
            .withColumn("order_ts", F.to_timestamp("order_ts"))
            .dropDuplicates(["order_id"])
            .select("order_id", "customer_id", "product_id", "quantity",
↪      "order_ts")
        )

# ----- GOLD: a materialized view aggregating silver -----
@dlt.table(
    name="daily_order_summary",
    comment="Business-ready daily order aggregates."
)
def daily_order_summary():
    return (
        dlt.read("orders_silver")
        .withColumn("order_date", F.to_date("order_ts"))
        .groupBy("order_date")
        .agg(
            F.count("*").alias("num_orders"),
            F.sum("quantity").alias("total_quantity")
        )
    )

```

A few things to notice, because each one is exam-testable:

- `dlt.read_stream("orders_bronze")` vs. `dlt.read("orders_silver")` — `read_stream` reads the upstream table incrementally as a **streaming table** (only new rows since last run); plain `dlt.read` reads it as a complete, static snapshot each run, which is how you build a **materialized view** — a table that’s fully recomputed from its inputs on each pipeline run (Lakeflow Declarative Pipelines are smart enough to do this incrementally under the hood where possible, but conceptually you should think of it as “always reflects the current full state of its query,” unlike a streaming table which only ever appends).
- `orders_bronze` here is a **streaming table** (built from a `readStream` source); `daily_order_summary` is a **materialized view** (built from a complete-snapshot aggregation) — the lab explicitly asked for one of each, and this is exactly that: one streaming table (bronze) and one materialized view (gold aggregate).
- The three `@dlt.expect*` decorators on `orders_silver` are data quality expectations — covered in depth in section 5.2 below.

To actually run this, go to **Workflows → Pipelines → Create Pipeline** (or **Jobs & Pipelines** in the newer UI), choose **Lakeflow Declarative Pipelines**, point the

source code at your `retail_pipeline` notebook, set the destination catalog/schema (e.g., `dp750_demo.core`), and click **Start**. Databricks reads your `@dlt.table` functions, builds a dependency graph automatically (it can tell `orders_silver` depends on `orders_bronze` just from the `dlt.read_stream("orders_bronze")` call — you never explicitly wire the DAG yourself), and runs everything in the right order. The pipeline UI shows you a live graph of `bronze → silver → gold` with row counts and data quality metrics per table.

Exam tip: the DAG (dependency graph) between tables in a Lakeflow Declarative Pipeline is inferred automatically from which `dlt.read/dlt.read_stream` calls appear inside each table’s function body — you never manually declare “run this after that.” Also know: pipelines can be run in **Triggered** mode (process available data once, then stop — like `availableNow`) or **Continuous** mode (keep running, processing new data as it arrives), mirroring the batch-vs-streaming trigger distinction from section 3.5.

3.5.5 4.5 Structured Streaming basics — a minimal hands-on touch

Even outside a full Lakeflow pipeline, you should have directly run at least one plain Structured Streaming job, because the exam tests the `readStream/writeStream` vocabulary on its own.

```
# A minimal streaming job: read new files from a folder, write them
↳ straight to a Delta table
stream_df = (
    spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.inferColumnTypes", "true")
        .load("/Volumes/dp750_demo/core/landing/orders_json")
)

query = (
    stream_df.writeStream
        .format("delta")
        .option("checkpointLocation",
↳ "/Volumes/dp750_demo/core/landing/_checkpoints/minimal_stream")
        .trigger(availableNow=True)
        .toTable("dp750_demo.core.orders_streaming_demo")
)

query.awaitTermination() # blocks until the availableNow batch finishes
↳ (harmless with this trigger)
```

Key vocabulary to have hands-on familiarity with:

- `spark.readStream` — returns a *streaming* DataFrame, distinguishable from a normal (batch) DataFrame by the fact that you cannot call actions like `.collect()` or `.count()` directly on it — you must attach a `writeStream` sink.

- `.writeStream` — configures the **sink** (format), the **checkpoint** (mandatory for any real streaming job — this is the “where am I in the source” bookmark), and the **trigger** (how often to check for new data / whether to run forever).
 - `query.awaitTermination()` — blocks the calling cell/thread until the stream stops (only meaningful for continuously running triggers; with `availableNow=True` it just waits for the current batch to finish).
 - `query.stop()` — manually stop a continuously running stream.
-

3.6 5. Data Quality Constraints

3.6.1 5.1 Hands-on lab: table-level NOT NULL and CHECK constraints

Delta tables in Unity Catalog support two kinds of declarative constraints enforced automatically on every write — think of them as the table itself refusing bad data, the same way a database schema does, rather than relying on you remembering to filter it out in every downstream query.

```
CREATE OR REPLACE TABLE dp750_demo.core.customers_dq (
  customer_id STRING NOT NULL,
  full_name   STRING,
  email       STRING NOT NULL,
  age         INT
);
```

```
ALTER TABLE dp750_demo.core.customers_dq
  ADD CONSTRAINT valid_age CHECK (age > 0 AND age < 130);
```

-- A valid insert succeeds:

```
INSERT INTO dp750_demo.core.customers_dq VALUES ('C1', 'Alice',
  ↳ 'alice@example.com', 34);
```

-- Violates NOT NULL — fails:

```
INSERT INTO dp750_demo.core.customers_dq VALUES ('C2', 'Bob', NULL, 40);
```

-- Violates the CHECK constraint — fails:

```
INSERT INTO dp750_demo.core.customers_dq VALUES ('C3', 'Charlie',
  ↳ 'charlie@example.com', -5);
```

Both failing inserts raise an error and the write is fully rejected (atomically — remember section 1.2: it’s all-or-nothing) — no partial data gets written, and the table’s guarantee (“every row here satisfies these rules”) is never violated even for a moment. This is a fundamentally different failure mode from the `@dlt.expect*` decorators you’ll see next: a table CHECK/NOT NULL constraint always stops the write, full stop. There’s no “keep the row but log a warning” option at the plain table-constraint level — that flexibility is exactly what Lakeflow pipeline expectations add.

You can inspect existing constraints with:

```
DESCRIBE TABLE EXTENDED dp750_demo.core.customers_dq;
```

...and drop one with ALTER TABLE ... DROP CONSTRAINT valid_age;.

Exam tip: NOT NULL and CHECK are the two Delta table constraint types tested. A CHECK constraint's boolean expression can reference any column(s) in the row (like age > 0 AND age < 130, or even a cross-column comparison like end_date > start_date). Adding a constraint to a table that *already* has violating data will fail — you must clean the data first (or fix the constraint) before Databricks lets you add it.

3.6.2 5.2 Lakeflow pipeline expectations: @dlt.expect, @dlt.expect_or_drop, @dlt.expect_or_fail

Inside a Lakeflow Declarative Pipeline specifically, you get a *third*, more flexible option beyond hard table constraints: **expectations**. An expectation is a named boolean rule attached to a @dlt.table function, and — unlike a plain CHECK constraint — you get to choose exactly what happens to a row that violates it.

Decorator	Violating row is...	Pipeline behavior	Typical use case
@dlt.expect("name", "condition")	Kept in the output table	Row is written normally; violation is counted and shown in the pipeline's data quality metrics/UI	"I want visibility into how much bad data exists, but it's not severe enough to throw away or halt over"
@dlt.expect_or_drop("name", "condition")	Dropped (excluded from the output table)	Row is silently filtered out; violation count still shown in metrics	"This row is unusable downstream — silently exclude it, but keep the pipeline running"
@dlt.expect_or_fail("name", "condition")	Causes the entire pipeline run to fail	The pipeline update stops with an error the moment a violating row is encountered	"This condition must never be violated — if it is, something is seriously wrong upstream and a human needs to look before anything downstream is trusted"

You can stack multiple expectations on one table (as the lab in section 4.4 already did with two of them). Here's a single table demonstrating all three together, so you can see the contrast directly:

```
import dlt
from pyspark.sql import functions as F

@dlt.table(name="orders_silver_full_dq")
```

```

@dlt.expect("reasonable_quantity", "quantity <= 1000")           # keep +
    ↳ log: unusual but not impossible
@dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL") # drop:
    ↳ unusable without a customer
@dlt.expect_or_fail("no_negative_price", "unit_price IS NULL OR unit_price
    ↳ >= 0") # fail: should never happen
def orders_silver_full_dq():
    return (
        dlt.read_stream("orders_bronze")
        .withColumn("order_ts", F.to_timestamp("order_ts"))
        .withColumn("unit_price", F.lit(19.99)) # illustrative
    )

```

You can also group multiple conditions into one named expectation with `expect_all` / `expect_all_or_drop` / `expect_all_or_fail`, passing a dictionary of {name: condition} pairs, if you want several related checks to share one violation policy rather than writing a separate decorator line for each.

The pipeline UI (the graph view when a Lakeflow pipeline runs) shows, per table, how many rows passed and how many rows failed each named expectation on every run — this is your built-in, no-extra-code data quality monitoring.

Exam tip — this exact comparison table is a favorite exam pattern. Memorize the verb for each: `expect` = **keep**, `expect_or_drop` = **drop**, `expect_or_fail` = **fail the whole pipeline update**. A common scenario question gives you a business rule (“orders with negative quantities should never reach the silver table, but the pipeline should keep running” → `expect_or_drop`; “if we ever see a duplicate primary key, stop everything immediately, it means upstream is broken” → `expect_or_fail`; “flag suspiciously large orders for review but don’t lose them” → `expect`).

3.7 6. Mini Quiz — Domain 3 (Original Practice Questions)

These are original practice questions written for this book to reinforce the concepts above. They are not real, leaked, or memorized exam questions — treat them as a self-check, not a source of actual exam content.

Q1. A colleague says, “Delta Lake tables are really just Parquet files, so they have all the same limitations as a plain Parquet-based data lake.” What is the single biggest reason this statement is wrong?

- A) Delta tables cannot be read using standard Parquet readers
- B) Delta Lake adds a transaction log (`_delta_log`) that provides ACID transactions, schema enforcement, and time travel on top of the underlying Parquet files
- C) Delta Lake tables are stored in a proprietary binary format, not Parquet
- D) Delta Lake tables cannot be partitioned

Answer: B. Delta Lake’s data files genuinely are Parquet — the transformation is the

`_delta_log` transaction log sitting alongside them, which records every commit and is what enables ACID guarantees, enforced schema, and time travel. None of A, C, or D are true.

Q2. You run `DESCRIBE HISTORY sales` and see versions 0 through 12. You need to see exactly what the table contained immediately after version 4's commit. Which statement does that?

- A) `SELECT * FROM sales WHERE version = 4;`
- B) `SELECT * FROM sales VERSION AS OF 4;`
- C) `RESTORE TABLE sales TO VERSION AS OF 4;`
- D) `VACUUM sales RETAIN 4 HOURS;`

Answer: B. `VERSION AS OF` (or `TIMESTAMP AS OF`) is the time-travel read syntax — it queries a past snapshot without changing the table. `RESTORE TABLE` (C) would actually roll the table back to that version, going further than “just look.” A and D are not valid/appropriate for this.

Q3. You append data to a Delta table and get an `AnalysisException` because the incoming `DataFrame` has a column the table doesn't have. You've confirmed this new column is intentional and should be added permanently. What's the correct fix?

- A) Delete the table and recreate it with the new column
- B) Add `.option("mergeSchema", "true")` to the write
- C) Cast the `DataFrame` to drop the extra column before writing
- D) Use `INSERT OVERWRITE` instead of append

Answer: B. `mergeSchema` (or the table property `delta.schema.autoMerge.enabled`) is the supported, controlled way to opt into schema evolution on write — it adds the new column to the table schema and backfills `NULL` for existing rows. A is destructive and unnecessary; C throws away real data instead of using it; D doesn't address the schema mismatch at all.

Q4. Query performance against a large, frequently-updated Delta table has degraded significantly over the past few months, and you suspect it's because of an accumulation of many small files. Which single command directly addresses this, and additionally improves data-skipping performance for queries that filter heavily on region?

- A) `VACUUM sales RETAIN 168 HOURS;`
- B) `ANALYZE TABLE sales COMPUTE STATISTICS;`
- C) `OPTIMIZE sales ZORDER BY (region);`
- D) `ALTER TABLE sales SET TBLPROPERTIES ('delta.schema.autoMerge.enabled' = 'true');`

Answer: C. `OPTIMIZE` compacts small files, and `ZORDER BY` additionally co-locates rows by the specified column so file-level min/max statistics let queries filtering on region

skip irrelevant files. VACUUM only deletes old unreferenced files (doesn't compact small *current* files); the others are unrelated to file layout.

Q5. In a medallion architecture, which layer is most appropriate for storing raw, unmodified event data exactly as received — including any malformed or duplicate records — so that cleaning logic can be fixed and safely re-run later without losing information?

- A) Bronze
- B) Silver
- C) Gold
- D) It doesn't matter, all layers should have identical content

Answer: A. Bronze is the raw, as-received landing layer — nothing is filtered, deduplicated, or corrected there, precisely so it can serve as a reliable source to reprocess from if downstream logic changes. Silver is the cleaned/deduplicated layer; gold is aggregated for consumption.

Q6. In a lakehouse dimensional model, which of the following is the best description of a fact table, as opposed to a dimension table?

- A) A small, slowly-changing table of descriptive attributes like customer name or product category
- B) A large, frequently-appended table of numeric, measurable events, typically referencing dimension tables via foreign keys
- C) A table that only ever contains exactly one row
- D) A table that cannot be joined to any other table

Answer: B. Fact tables hold events/measurements (order lines, transactions, sensor readings) — numeric, append-heavy, and usually much larger row-count-wise than the dimension tables (A describes a dimension table) they reference.

Q7. You need to ingest a folder that regularly receives large numbers of new JSON files (potentially millions over time) and load them incrementally into a bronze Delta table, with the job scheduled to run once per hour and pick up only what's new each time. Which approach is best suited to this?

- A) `spark.read.format("json").load(folder)` re-run every hour
- B) Auto Loader (`cloudFiles`) with `.trigger(availableNow=True)`, scheduled hourly
- C) `TRUNCATE TABLE` followed by a full reload every hour
- D) A manual Python script that lists all files in the folder and compares timestamps against a local variable

Answer: B. Auto Loader is built exactly for large-scale, growing sets of files and tracks what's already been processed via its checkpoint; `trigger(availableNow=True)`

gives batch-like “run once and stop” semantics ideal for an hourly schedule. A re-reads and reprocesses everything every run (doesn’t scale); C destroys history and is wasteful; D reinvents (poorly, and non-durably) what Auto Loader already tracks for you.

Q8. Which statement correctly distinguishes COPY INTO from Auto Loader?

- A) COPY INTO can run as a continuously running stream, while Auto Loader can only run as a one-time batch
- B) COPY INTO is a SQL statement well suited to simpler/lower-frequency idempotent loads, while Auto Loader is a PySpark streaming source better suited to large-scale or continuous file ingestion
- C) They are functionally identical and differ only in syntax
- D) COPY INTO requires a checkpoint location, while Auto Loader does not

Answer: B. COPY INTO is a discrete, idempotent SQL load statement (tracks processed files via Delta table metadata, not a checkpoint) best for simpler/less frequent loads; Auto Loader is a full streaming source (via readStream/writeStream, uses a checkpoint) suited to larger scale and can run continuously. A and D reverse the facts; C is false — they differ in more than syntax (see the comparison table in section 3.3).

Q9. A company wants to replicate several tables from a Salesforce org into Unity Catalog on an ongoing basis, without writing and maintaining custom API integration code. Which Databricks capability is designed specifically for this?

- A) COPY INTO
- B) Auto Loader
- C) Lakeflow Connect
- D) VACUUM

Answer: C. Lakeflow Connect provides managed, native connectors for sources like Salesforce and other SaaS/database systems, handling authentication, incremental sync, and schema drift without hand-rolled ingestion code. Auto Loader and COPY INTO are for files already landing in cloud storage, not live SaaS/database sources.

Q10. In a Lakeflow Declarative Pipeline, what determines the execution order (the DAG) between tables like bronze, silver, and gold?

- A) The alphabetical order of the function names in the notebook
- B) An explicit depends_on=[...] argument that must be passed to every @dlt.table decorator
- C) The order in which dlt.read() / dlt.read_stream() calls reference other tables defined in the same pipeline — the dependency graph is inferred automatically
- D) Pipelines execute all tables simultaneously in parallel with no ordering guarantees

Answer: C. Lakeflow Declarative Pipelines automatically infer the dependency graph from which tables each `@dlt.table` function reads via `dlt.read/dlt.read_stream` — there's no manual `depends_on` argument to set (B is a distractor) and no reliance on definition order (A) or unordered parallel execution (D).

Q11. You add three expectations to a `@dlt.table` function: `@dlt.expect("q1", "amount > 0")`, `@dlt.expect_or_drop("q2", "customer_id IS NOT NULL")`, and `@dlt.expect_or_fail("q3", "order_id IS NOT NULL")`. A batch of incoming rows includes: Row A (amount = -5, otherwise valid), Row B (customer_id is NULL, otherwise valid), Row C (order_id is NULL). What happens?

- A) All three rows are written to the table, with warnings logged
- B) Row A is written and flagged in metrics, Row B is silently dropped, and Row C causes the entire pipeline update to fail
- C) All three rows are silently dropped
- D) The pipeline ignores expectations unless explicitly told to enforce them

Answer: B. `expect` keeps the row but logs the violation in pipeline metrics (Row A: negative amount, still written); `expect_or_drop` filters the row out of the output silently (Row B: null customer_id, dropped); `expect_or_fail` halts the entire pipeline run the moment it's violated (Row C: null order_id, causing failure) — this is exactly the keep/drop/fail distinction the three decorators are designed around.

Q12. You want to add a rule to an existing Delta table so that no row can ever have `end_date` earlier than `start_date`. Which of the following correctly adds this rule directly at the table level (not inside a Lakeflow pipeline)?

- A) `ALTER TABLE events ADD CONSTRAINT valid_dates CHECK (end_date >= start_date);`
- B) `@dlt.expect_or_fail("valid_dates", "end_date >= start_date")`
- C) `ALTER TABLE events SET TBLPROPERTIES ('delta.schema.autoMerge.enabled' = 'true');`
- D) `VACUUM events;`

Answer: A. `CHECK` constraints support arbitrary boolean expressions over the row's columns, including cross-column comparisons like `end_date >= start_date`, and are added directly to a Delta table (any writer, any workload) via `ALTER TABLE ... ADD CONSTRAINT`. B is a Lakeflow-pipeline-only mechanism, not a plain table constraint; C and D are unrelated to constraints.

3.7.1 Chapter summary — what to walk away knowing cold

- Delta Lake = Parquet files + `_delta_log` transaction log → ACID transactions, enforced schema, time travel (`VERSION AS OF/TIMESTAMP AS OF`), and safe concurrent `UPDATE/DELETE/MERGE`.

- OPTIMIZE (compact files) + ZORDER BY (co-locate for skipping) + VACUUM (delete old unreferenced files, 7-day default retention) are the three maintenance commands.
- Medallion architecture: bronze (raw, untouched) → silver (cleaned, typed, deduplicated) → gold (aggregated, business-ready); fact tables hold numeric events, dimension tables hold descriptive attributes.
- Ingestion: Auto Loader (cloudFiles, PySpark, streaming, scales to huge file counts) vs. COPY INTO (SQL, idempotent, simpler/lower-frequency) vs. Lakeflow Connect (managed connectors for databases/SaaS, not files).
- “Lakeflow Declarative Pipelines” = the current name for “Delta Live Tables” / DLT — same `@dlt.table`, `@dlt.expect*` syntax either way. Streaming tables (`dlt.read_stream`) are incremental; materialized views (plain `dlt.read`) recompute from the current full state of their inputs.
- Data quality: table-level NOT NULL/CHECK constraints always reject the whole write on violation. Pipeline expectations give three graduated responses: `expect` (keep + log), `expect_or_drop` (silently filter out), `expect_or_fail` (halt the whole pipeline run).

Chapter 4

Part 4 — Orchestration, CI/CD, Monitoring & Performance

By now you can build Delta tables and wire up a Bronze → Silver → Gold Lakeflow Declarative Pipeline. That pipeline is one piece of machinery. A real data platform has dozens of these pieces — ingestion notebooks, transformation pipelines, data quality checks, machine learning retraining jobs, reports that need to refresh every morning — and none of them are useful in isolation. Something has to decide *when* each piece runs, *in what order*, *what happens if one fails*, and *who gets a text message at 3 a.m. when it does*. That “something” is the subject of this chapter, and it maps almost exactly onto **Domain 4 of the DP-750 exam**: Deploy and maintain data pipelines and workloads. At 30–35% of the exam, this is tied for the single largest domain — bigger than Unity Catalog, bigger than pipeline transformations. If you only had time to master one part of this book deeply, this would be the one.

We’ll cover four things, in order: orchestrating multi-step work with **Lakeflow Jobs** (the current name for what Databricks used to call **Databricks Workflows** — you’ll still see “Workflows” in older tutorials, YouTube videos, and even some freeCodeCamp-style courses, but the product and the exam now call it Lakeflow Jobs); scheduling and alerting on those jobs; the Git-based development lifecycle and CI/CD with Databricks Asset Bundles; and monitoring, troubleshooting, and tuning performance once things are running in production. Every section has a hands-on lab. Do them — Domain 4 rewards muscle memory over memorized facts, because the exam likes to describe a UI screen or a JSON snippet and ask what it does.

A quick platform note before we start: everything in this chapter works on **Databricks Free Edition**, which is serverless-only (jobs run on serverless compute, capped around 5 concurrent job tasks, and one active Lakeflow Declarative Pipeline per pipeline type). A few concepts — custom cluster policies, choosing an instance type for a job cluster, cluster pools — only *fully* apply on a real Azure Databricks Premium workspace with classic compute enabled. Where that matters, we’ll say so explicitly and explain the concept anyway, because the exam still tests it even if your free lab environment abstracts it away.

4.1 1. Pipeline & Job Orchestration with Lakeflow Jobs

4.1.1 1.1 What “orchestration” actually means

Imagine you have three notebooks:

1. `01_ingest.py` — pulls raw sales data from a source and lands it as a Bronze Delta table.
2. `02_transform.py` — cleans it and produces a Silver table.
3. `03_quality_check.py` — runs assertions on the Silver table (no nulls in `order_id`, no negative prices, etc.).

You could open each notebook every morning and click “Run All” in order, waiting for one to finish before starting the next. That works for exactly one person, exactly once, as long as nobody forgets and nobody goes on vacation. It falls apart the moment you need this to run every day at 6 a.m. whether or not you’re awake, or the moment step 2 needs to *only* run if step 1 actually succeeded, or the moment you want someone to be notified automatically if the quality check fails.

Orchestration is the automation layer that solves this: it is the system that runs your tasks in the correct order, on a schedule or in response to an event, retries them if they fail, and tells the right people what happened — without a human clicking anything. Think of a **job** as a recipe card: it lists the steps, the order they go in, what ingredient (output) each step needs from the step before it, and what to do if a step burns. **Lakeflow Jobs** is Databricks’ built-in orchestration engine — the place where you assemble notebooks, Python scripts, SQL queries, and Lakeflow Declarative Pipelines into a single controlled workflow with dependencies between them.

This is a different layer from the Lakeflow Declarative Pipeline you built in Part 3. A Declarative Pipeline orchestrates the *flow of data between tables inside one pipeline* (bronze → silver → gold, using `dlt/pipeline` decorators and dependency inference). A **Lakeflow Job** orchestrates *tasks* — and one of those tasks can itself be “run this entire Declarative Pipeline.” A job is the outer conductor; a pipeline is one instrument in the orchestra. In the exam, expect questions that test whether you know a Job can *contain* a pipeline run as one task among several — ingestion notebook, then pipeline, then a dbt task, then a Python wheel task, then a notification — all in one job graph.

4.1.2 1.2 Job clusters vs. all-purpose clusters vs. serverless

Every task in a job needs compute to run on. Databricks gives you three flavors, and knowing the difference — including the cost implications — is a favorite exam topic.

All-purpose clusters are the clusters you spin up for interactive work: attach a notebook, run cells, explore data, iterate. They stay running until you (or an inactivity auto-termination setting) shut them down, and multiple people/notebooks can share one. Because they sit idle waiting for you to think between cell executions, they are billed at a higher DBU (Databricks Unit) rate than job compute, and leaving one running overnight is a classic way to blow through your Azure budget for no reason.

Job clusters are compute that Databricks spins up automatically *when a job run*

starts, uses only for that run, and **terminates automatically the moment the job finishes** (success or failure). Nobody else can attach a notebook to them for interactive poking around. Because they’re single-purpose and short-lived, they’re billed at a cheaper job-compute DBU rate than all-purpose clusters, which is precisely why production jobs should almost never be pointed at a long-running all-purpose cluster — it’s both more expensive and it couples your production job’s fate to whatever else that shared cluster is doing (someone else’s messy interactive notebook could OOM it).

Serverless compute is the modern default and the direction the whole platform is moving: you don’t configure an instance type, node count, or Spark version at all — you just say “run this task on serverless,” and Databricks manages a warm, auto-scaling pool of compute behind the scenes, billed per second of actual usage. It starts faster than a classic job cluster (no multi-minute cluster boot time) and removes an entire category of tuning decisions (VM size, autoscaling min/max, Photon on/off — Photon is on by default). On **Databricks Free Edition**, serverless is the *only* option — there is no classic compute at all, so “job cluster vs. all-purpose cluster” is a bit academic there; every task simply runs serverless. On a real **Azure Databricks Premium workspace**, you have the full choice: you can still attach a job to a classic job cluster and control its instance type, autoscaling range, Spark config, and even apply a **cluster policy** (an admin-defined template that restricts, say, which VM sizes junior engineers are allowed to pick, or forces a maximum node count to cap cost) — none of which exists as a concept on serverless. For the exam, know the conceptual differences (cost, lifecycle, sharing, tunability) even though your Free Edition labs won’t let you touch cluster policies hands-on.

	All-purpose cluster	Job cluster (classic)	Serverless
Lifecycle	Manually started/stopped, or idle-timeout	Created at job start, terminated at job end	Always available, no boot wait
Sharing	Multiple users/notebooks	Dedicated to one job run	Dedicated per task run
Cost	Higher DBU rate	Lower DBU rate	Pay per second used, no idle cost
Configurable (VM size, policy, autoscaling range)	Yes	Yes	No — fully managed
Available on Free Edition	No	No	Yes (only option)
Typical use	Interactive dev/exploration	Scheduled production jobs (Premium workspaces)	Production jobs (modern default) and Free Edition

4.1.3 1.3 Hands-on lab: build a multi-task Lakeflow Job

We’ll build a job with four tasks: **ingest** → **transform** → **quality_check** → **notify**, where `quality_check` only proceeds if `transform` succeeded, and `notify` runs regardless of outcome (a common “always send a summary” pattern).

Step 1 — Create the notebooks. In your workspace, create a folder /Workspace/Users/<you>/dp7 and add three notebooks:

01_ingest:

```
# 01_ingest
from pyspark.sql.functions import current_timestamp

catalog = "workspace"
schema = "dp750_lab"

spark.sql(f"CREATE CATALOG IF NOT EXISTS {catalog}")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.{schema}")

data = [(1, "Widget", 19.99), (2, "Gadget", 29.99), (3, "Gizmo", -5.00)]
df = spark.createDataFrame(data, ["order_id", "product", "price"]) \
    .withColumn("ingested_at", current_timestamp())

df.write.mode("overwrite").saveAsTable(f"{catalog}.{schema}.orders_bronze")
print(f"Ingested {df.count()} rows into {catalog}.{schema}.orders_bronze")
```

02_transform:

```
# 02_transform
catalog = "workspace"
schema = "dp750_lab"

df = spark.table(f"{catalog}.{schema}.orders_bronze") \
    .filter("product IS NOT NULL")

df.write.mode("overwrite").saveAsTable(f"{catalog}.{schema}.orders_silver")
print(f"Wrote {df.count()} rows to {catalog}.{schema}.orders_silver")
```

03_quality_check:

```
# 03_quality_check
catalog = "workspace"
schema = "dp750_lab"

df = spark.table(f"{catalog}.{schema}.orders_silver")
bad_price_count = df.filter("price < 0").count()

assert bad_price_count == 0, f"Found {bad_price_count} rows with negative
    ↪ price!"
print("Quality check passed: no negative prices.")
```

Notice 03_quality_check will actually **fail on purpose** right now, because our sample data has a -5.00 price for “Gizmo” and the transform step doesn’t filter it out. That’s deliberate — we want to see a real failure and a real alert later in this chapter. For now, build the job structure.

Step 2 — Create the job in the UI. 1. In the left sidebar, click **Jobs & Pipelines**

→ **Create** → **Job**. 2. Name it `dp750_part4_orders_pipeline`. 3. You land on the task graph canvas with one empty task box. Configure **Task 1**: - Task name: `ingest` - Type: Notebook - Path: browse to `01_ingest` - Compute: select **Serverless** (or a job cluster if you're on a Premium workspace and want to practice classic compute — click “New Job Cluster,” pick the smallest available node type, single-node, and leave autoscaling off for a lab this small) - Click **Create task**. 4. Click the **+** below the `ingest` box to add **Task 2**: - Task name: `transform` - Type: Notebook, path → `02_transform` - Under **Depends on**, select `ingest` (this is what draws the arrow and enforces “only run after `ingest` succeeds”) - Compute: same serverless/job cluster as before 5. Add **Task 3**: - Task name: `quality_check` - Type: Notebook, path → `03_quality_check` - Depends on: `transform` 6. Add **Task 4**: - Task name: `notify` - Type: Notebook (or you could use a simple SQL task) — for now point it at any tiny notebook that just prints “Pipeline run complete”. - Depends on: `quality_check` - Click the dependency's **run condition** dropdown and change it from the default “All succeeded” to “**All done**” — this makes `notify` run whether `quality_check` passed or failed, which is exactly the “always notify” behavior a real pipeline needs.

You now have a diamond-free, straight-line DAG: `ingest` → `transform` → `quality_check` → `notify`. Real pipelines are often wider than this — e.g., two independent `ingest` tasks feeding one `transform` task, or one `transform` task fanning out into three parallel `quality_check`s. You build fan-in/fan-out exactly the same way: just point multiple tasks' “Depends on” field at the same upstream task, or list multiple upstream tasks as dependencies for one downstream task (the task only starts once *all* listed dependencies have finished, by default only if they all succeeded).

Step 3 — Run it and watch it fail correctly. Click **Run now**. Watch the graph: `ingest` and `transform` turn green, `quality_check` turns red (the assertion fires), and — because we set its run condition to “All done” — `notify` still turns green and executes anyway. This is orchestration doing its job: a downstream failure was contained, didn't crash the whole run silently, and the notification step still ran to tell someone about it.

The equivalent job JSON. Every job you build in the UI has an underlying JSON definition — you can view/copy it via the “**more options**” (...) menu → **View JSON**, and it's exactly what you'd hand to the REST API, the Databricks CLI, or an Asset Bundle (Section 3). For our job it looks like this:

```
{
  "name": "dp750_part4_orders_pipeline",
  "tasks": [
    {
      "task_key": "ingest",
      "notebook_task": {
        "notebook_path":
          ↪ "/Workspace/Users/you@example.com/dp750_part4/01_ingest"
      }
    },
    {
      "task_key": "transform",
      "depends_on": [{ "task_key": "ingest" }],

```

```

    "notebook_task": {
      "notebook_path":
        ↪ "/Workspace/Users/you@example.com/dp750_part4/02_transform"
    }
  },
  {
    "task_key": "quality_check",
    "depends_on": [{ "task_key": "transform" }],
    "notebook_task": {
      "notebook_path":
        ↪ "/Workspace/Users/you@example.com/dp750_part4/03_quality_check"
    }
  },
  {
    "task_key": "notify",
    "depends_on": [{ "task_key": "quality_check" }],
    "run_if": "ALL_DONE",
    "notebook_task": {
      "notebook_path":
        ↪ "/Workspace/Users/you@example.com/dp750_part4/04_notify"
    }
  }
]
}

```

Key fields the exam likes: `task_key` (the unique name a task is referenced by), `depends_on` (an array, because a task can depend on *multiple* upstream tasks), and `run_if` (values like `ALL_SUCCESS` (default), `ALL_DONE`, `AT_LEAST_ONE_FAILED`, `NONE_FAILED`, etc. — this is the JSON name for the “run condition” dropdown you clicked in the UI). Recognize this JSON shape on sight; you’ll see it again in Section 3 when we deploy jobs as code with Asset Bundles.

4.2 2. Scheduling, Triggers & Alerting

A job that only runs when you click “Run now” isn’t automated yet — it’s just a saved recipe. To make it truly hands-off, you attach a **trigger**: a rule that says *when* to start a new run automatically.

4.2.1 2.1 The three trigger types

- **Scheduled (cron)** — run on a fixed time-based schedule: every day at 6 a.m., every hour, every Monday at 9 a.m. Best for predictable, time-based batch workloads (nightly ETL, weekly reports).
- **File arrival** — run automatically whenever a new file lands in a specified Unity Catalog volume or cloud storage location. Best for event-driven ingestion where data shows up unpredictably (a vendor drops a CSV whenever they feel like it)

and you don't want to poll on a schedule and either waste runs finding nothing new or introduce needless latency.

- **Continuous** — the job runs, and the instant one run finishes, Databricks immediately starts the next one (with a short built-in idle wait if there's truly nothing new to do). Best for near-real-time / streaming-style workloads where you want the lowest possible latency between new data arriving and it being processed, and where you're willing to keep compute running essentially all the time.

4.2.2 2.2 Hands-on lab: cron schedule

1. Open your job → **Schedules & Triggers** tab → **Add trigger**.
2. Trigger type: **Scheduled**.
3. You'll see a simple picker (e.g., "Every day at 06:00") for common cases, and an **Advanced** option that exposes the raw cron expression. Databricks Jobs uses **Quartz cron syntax**, which has **6 or 7 fields** (seconds, minutes, hours, day-of-month, month, day-of-week, optional year) — this is different from standard Unix/Linux 5-field cron, and it's a detail the exam likes to probe because it trips up people who already know Unix cron.

A few examples: | Quartz expression | Meaning |
0 0 6 * * ?	Every day at 6:00:00 AM
0 30 8 * * MON-FRI	8:30 AM, Monday through Friday
0 0 0 1 * ?	Midnight on the 1st of every month
0 */15 * * * ?	Every 15 minutes
0 0 9 ? * MON	9:00 AM every Monday

The ? is Quartz-specific: it means "no specific value" and is required in either the day-of-month or day-of-week field whenever the other one is specified with an actual value, because Quartz won't let both be simultaneously meaningful.

4. Set your job to 0 0 6 * * ? (daily at 6 a.m.), choose your workspace time zone, and save. You'll see the job's next scheduled run time reflected immediately in the Schedules tab.

4.2.3 2.3 Hands-on lab: file arrival trigger

1. First you need a location to watch. If you completed Part 1/3's Unity Catalog setup, you already have a volume, e.g. workspace.dp750_lab.landing_zone. If not, create one:

```
CREATE VOLUME IF NOT EXISTS workspace.dp750_lab.landing_zone;
```

2. In the job → **Schedules & Triggers** → **Add trigger** → Trigger type: **File arrival**.
3. Storage location: point it at the volume's path, e.g. /Volumes/workspace/dp750_lab/landing_zone/.
4. Optionally set a **minimum time between triggers** (e.g., don't fire more than once every 5 minutes) so that a burst of 50 files landing at once doesn't spawn 50 separate runs — Databricks coalesces arrivals inside that window into a single run.

5. Save, then test it: upload any small file into that volume path (via the Catalog Explorer UI's "Upload to volume" button, or `dbutils.fs.cp`). Within a couple of minutes you should see a new job run appear automatically in the **Runs** tab, with the trigger type shown as `FILE_ARRIVAL`.

When to use which. Cron is right when *you* control the tempo (a nightly batch load you know finishes upstream by 5 a.m.). File arrival is right when an external system controls the tempo and you want to react quickly without wasteful polling. Continuous is right when latency matters more than cost and the source is a genuine stream (e.g., a task wrapping an Auto Loader/Structured Streaming read) — remember continuous triggers effectively keep compute running non-stop, so on Free Edition's serverless task-count limits and on any cost-sensitive Azure workspace, only reach for it when the business actually needs sub-minute freshness, not because it's the "cool" option.

4.2.4 2.4 Hands-on lab: retries and failure alerts

Real pipelines hit transient failures — a source API rate-limits you for 30 seconds, a cloud storage call times out. You don't want a human paged for every blip; you want the platform to retry automatically and only escalate to a human if it *keeps* failing.

1. Open your job, click into the **quality_check** task specifically (task-level settings live on each task, while notifications can be set at both the job level and the task level).
2. Under the task's **Retry policy**, set:
 - **Maximum retries:** 2
 - **Minimum interval between retries:** e.g. 60 seconds (you can also enable exponential backoff so each retry waits longer than the last)
 - Optionally set a **Timeout** for the task (e.g., 3600 seconds) — if the task runs longer than this, Databricks kills it and counts it as a failure, which then *also* becomes eligible for retry under your retry policy. This is what people mean informally by "retry on timeout": a timeout is just one of the ways a task can fail, and your retry policy applies to any failure, timeout included.
3. Now set alerting at the job level: go to the job's **Notifications** tab → **Add notification**.
 - Choose the event: **On failure** (also available: On start, On success, On streaming backlog exceeded, On duration warning threshold exceeded).
 - Destination: add an email address (or a pre-configured Slack/Teams web-hook or system notification destination if your workspace admin has set one up).
 - Save.
4. Force a real failure to see it end to end: temporarily edit `03_quality_check` to always fail (it already does, thanks to our seeded bad row), run the job, and confirm you get the email after retries are exhausted. Note the retry runs are visible in the **Runs** tab as repeated attempts of the same task before the job is ultimately marked failed and the notification fires.

Exam angle: know that retries are configured **per task**, not just per job, and that

a task exhausting its retries is what finally triggers an “On failure” notification for the run — the notification doesn’t fire on the first transient failure if retries are still available and one of them succeeds.

4.2.5 2.5 Hands-on lab: passing parameters into a notebook task

Hard-coding `catalog = "workspace"` and `schema = "dp750_lab"` inside every notebook (like we did above) is exactly the kind of thing that breaks the moment you promote code from dev to prod. **Job parameters** let you define values at the job level and inject them into tasks at run time, so the same notebook code works unchanged across environments.

1. Edit `02_transform` to read its inputs via **widgets** instead of hard-coded strings:

```
dbutils.widgets.text("catalog", "workspace")
dbutils.widgets.text("schema", "dp750_lab")

catalog = dbutils.widgets.get("catalog")
schema = dbutils.widgets.get("schema")

df = spark.table(f"{catalog}.{schema}.orders_bronze").filter("product
↪ IS NOT NULL")
df.write.mode("overwrite").saveAsTable(f"{catalog}.{schema}
↪ .orders_silver")
print(f"Wrote to {catalog}.{schema}.orders_silver")

dbutils.widgets.text(name, default) declares a parameter with a default
value (used if nobody supplies one — handy for interactive testing); dbutils.
widgets.get(name) reads whatever value was actually passed in at run time.
```

2. In the job UI, go to the job’s **Job parameters** section (job-level, not task-level) and add:

- `catalog = workspace`
- `schema = dp750_lab`

3. On the transform task itself, open **Parameters** and map the task’s notebook parameters to the job parameters using Databricks’ dynamic value reference syntax:

```
catalog -> {{job.parameters.catalog}}
schema  -> {{job.parameters.schema}}
```

4. Run the job. Databricks resolves `{{job.parameters.catalog}}` to `workspace` at run time and passes it in as if you’d typed it into the widget yourself. Now if you want to point the exact same job at a staging catalog next month, you change one job parameter — not the notebook code.

Other useful dynamic references you’ll see on the exam: `{{job.id}}`, `{{job.run_id}}`, `{{task.name}}`, and date/time helpers like `{{job.start_time.[argument]}}` — all re-

solved automatically at run time and commonly used to, say, write output into a date-partitioned path or tag a run for traceability.

4.3 3. Development Lifecycle: Git, Repos & CI/CD

So far every notebook in this chapter has lived loosely in your workspace. That's fine for a solo lab, but it's not how a real data engineering team works. Teams need version control (so changes are tracked and reversible), code review (so a second person checks logic before it hits production), and a repeatable way to promote the *exact same* tested code from a dev environment into staging and then production — without anyone manually re-typing or re-clicking anything. That's the development lifecycle, and it's the second big piece of Domain 4.

4.3.1 3.1 Hands-on lab: Databricks Repos day-to-day workflow

You connected Databricks to your GitHub account back in Part 1, and you should already have a Databricks Repo cloned from a GitHub repository. If not: **Workspace** → **Repos** → **Add Repo** → paste your GitHub repo's HTTPS URL → **Create Repo**. This is the day-to-day loop you'll use constantly:

1. **Open the repo** in the workspace file browser. Note the branch indicator near the top — it shows the branch you're currently on (probably main).
2. **Create a feature branch.** Click the branch name → **Create branch** → name it something like feature/add-quality-check, based off main. Databricks checks you out onto the new branch immediately; nothing you do next touches main.
3. **Make a change.** Open one of your job's notebooks inside the repo (or add a new one) and edit it — for example, tighten 03_quality_check to actually filter out bad rows instead of just asserting and failing:

```
df_clean =  
  ↪ spark.table("workspace.dp750_lab.orders_silver").filter("price >=  
  ↪ 0")  
df_clean.write.mode("overwrite").saveAsTable(  
  ↪ "workspace.dp750_lab.orders_silver_clean")
```

4. **Commit and push.** Click the branch/Git status icon again → you'll see a diff view of every changed file with additions/deletions highlighted, a commit message box, and buttons to **Commit & Push**. Write a real commit message ("Filter negative prices in quality check"), and click **Commit & Push**. Under the hood this is a normal git add, git commit, git push origin feature/add-quality-check — Databricks is just giving you a UI over the same Git operations, and the commit really does land on GitHub.
5. **Open a pull request.** The Databricks Git dialog shows a "**Create Pull Request**" link once you've pushed a branch that's ahead of main — clicking it takes

you straight to GitHub’s compare/PR page with the branches pre-filled. Alternatively, go to GitHub directly, and you’ll see the “Compare & pull request” banner for your just-pushed branch. Fill in a description and open the PR.

6. **Review and merge.** This step happens on GitHub itself (or wherever your org’s review process lives) — a teammate reviews the diff, comments, approves, and the PR gets merged into main. This is the checkpoint that keeps a bad change out of production; it’s a human-in-the-loop gate that orchestration alone doesn’t give you.
7. **Pull the update back.** Back in Databricks, switch your Repo back to main and click **Pull** to bring the merged change down into the workspace.

Two details the exam likes: Databricks Repos syncs a **folder in your workspace to a Git branch**, and switching branches in the Repos UI actually swaps the files on disk in that folder to match — so any job pointed at a *specific Repos path* is effectively pointed at whatever branch that folder currently has checked out. This is exactly why production jobs shouldn’t just point at “the repo folder” casually; the safer pattern (and the one Asset Bundles formalizes, next) is for a job to reference a specific Git ref (branch, tag, or commit) explicitly, so it can’t accidentally start running someone’s half-finished feature branch code.

4.3.2 3.2 Databricks Asset Bundles (DABs)

Repos solve *code* version control. But a job also has *configuration* — which cluster it uses, its schedule, its parameters, its notifications — and that configuration needs the same rigor: reviewed, versioned, and promotable across environments, instead of hand-clicked once in the UI and then quietly drifting between dev and prod. **Databricks Asset Bundles** are the answer: YAML files, stored in the same Git repo as your code, that declare your jobs and pipelines as **infrastructure-as-code**. Instead of clicking through the Jobs UI to build a job in each environment separately, you define it once in YAML and the Databricks CLI deploys it — deterministically, repeatably — to dev, staging, or prod based on a `--target` flag.

Why this matters in practice: it’s the difference between “someone manually recreated the job in prod and typo’d a table name” and “the exact job definition tested in dev, byte-for-byte, is what’s running in prod, because it was deployed from the same YAML file.” This is also literally what CI/CD means for Databricks — a GitHub Actions (or Azure DevOps) pipeline can run `databricks bundle deploy` automatically whenever a PR is merged, giving you real continuous deployment.

4.3.3 3.3 Hands-on lab: minimal Asset Bundle — define, deploy, run

You’ll need the **Databricks CLI** (v0.200+, which is the unified CLI with bundle support — confirm with `databricks -v`; install/upgrade instructions are at docs.databricks.com/dev-tools/cli if you don’t have it yet) and to be authenticated to your workspace (`databricks auth login --host <your-workspace-url>`).

1. On your local machine (or in a Databricks CLI-enabled terminal), scaffold a bundle:

```
databricks bundle init
```

This offers templates; for a from-scratch minimal example, just create the files by hand instead — it's more instructive. Make a folder:

```
mkdir dp750-bundle && cd dp750-bundle
```

2. Create databricks.yml:

```
bundle:
  name: dp750-part4-bundle

include:
  - resources/*.yaml

targets:
  dev:
    mode: development
    default: true
    workspace:
      host: https://<your-workspace-url>
    variables:
      catalog: workspace
      schema: dp750_lab_dev

  prod:
    mode: production
    workspace:
      host: https://<your-workspace-url>
    variables:
      catalog: workspace
      schema: dp750_lab_prod

variables:
  catalog:
    description: Unity Catalog catalog name
  schema:
    description: Unity Catalog schema name
```

3. Create resources/orders_job.yml:

```
resources:
  jobs:
    orders_pipeline_job:
      name: dp750-part4-orders-pipeline-${bundle.target}
      tasks:
        - task_key: ingest
          notebook_task:
```

```

        notebook_path: ../src/01_ingest.py
    - task_key: transform
      depends_on:
        - task_key: ingest
      notebook_task:
        notebook_path: ../src/02_transform.py
        base_parameters:
          catalog: ${var.catalog}
          schema: ${var.schema}
  trigger:
    periodic:
      interval: 1
      unit: DAYS

```

Put 01_ingest.py and 02_transform.py (the widget-based version from Section 2.5) in a src/ folder alongside resources/. Note the `${bundle.target}` and `${var.catalog}` substitutions — this single YAML file produces a *differently named, differently configured* job depending on which target you deploy to, which is the whole point.

4. **Validate** before deploying — this catches YAML/schema errors and shows you a summary of what will be created, without touching the workspace:

```
databricks bundle validate -t dev
```

Expected output is a clean summary like:

```

Name: dp750-part4-bundle
Target: dev
Workspace:
  Host: https://<your-workspace-url>
Validation OK!

```

5. **Deploy** to dev:

```
databricks bundle deploy -t dev
```

This uploads your notebooks and creates/updates the job definition in your workspace under a bundle-managed folder (typically /Workspace/Users/<you>/ .bundle/dp750-part4-bundle/dev/). Expected output ends with something like:

```

Uploading bundle files to /Workspace/Users/you@example.com/.bundle/dp750-
part4-bundle/dev/files...
Deploying resources...
Updating deployment state...
Deployment complete!

```

6. **Run** the deployed job without touching the UI at all:

```
databricks bundle run orders_pipeline_job -t dev
```

This kicks off a real job run and streams status back to your terminal, ending with a link to the run in the Databricks UI. Go check the Jobs UI — you’ll see dp

750-part4-orders-pipeline-dev sitting there, created entirely from YAML.

7. When you're happy, deploy the *same* bundle to prod — same files, different target:

```
databricks bundle deploy -t prod
databricks bundle run orders_pipeline_job -t prod
```

This creates a second, independent job, dp750-part4-orders-pipeline-prod, pointed at the dp750_lab_prod schema — with zero manual re-clicking, and zero risk of the prod job silently differing from what was tested in dev.

8. Clean up if you want: `databricks bundle destroy -t dev` (and `-t prod`) removes everything the bundle created in that target.

4.3.4 3.4 Environment promotion patterns

The pattern you just built by hand is the general shape used everywhere in production Databricks: **one bundle, multiple targets**, where each target maps to both a **Unity Catalog catalog** (dev, staging, prod — remember from Part 2 that catalogs are the natural environment boundary in Unity Catalog's three-level namespace) and a **bundle deployment mode** (development gives you things like run-as-current-user and a [dev you] prefix on resource names to avoid clashes; production enforces stricter checks, like requiring a fixed run_as service principal rather than defaulting to whoever deployed it).

A typical real-world flow: a data engineer works in a dev target against a dev catalog with made-up or sampled data; a merge to a staging/release branch triggers a CI pipeline that runs `databricks bundle deploy -t staging` against a staging catalog for integration testing; a merge/tag on main triggers `databricks bundle deploy -t prod` against the prod catalog, often gated by a manual approval step in GitHub Actions/Azure DevOps. The bundle YAML's variables block is what lets one set of job/pipeline definitions parameterize cleanly across all three without forked code — exactly the `${var.catalog}` pattern from the lab above. Know this shape for the exam: **Unity Catalog catalogs for data isolation, bundle targets for job/pipeline configuration isolation, one Git repo and one set of YAML files as the single source of truth for both.**

4.4 4. Monitoring, Troubleshooting & Performance

Deploying a job is not the end of the job's story — it's the beginning. Something will eventually break, run slowly, or cost more than expected, and Domain 4 explicitly tests whether you know where to look.

4.4.1 4.1 Hands-on lab: reading the Spark UI query profile

1. Run your `dp750_part4_orders_pipeline` job again (or any job with a Spark workload).

2. From the **Runs** tab, click into the specific run, then click the transform task. You'll see a **Spark UI** link (and, for SQL/DataFrame workloads, a **Query Profile / Query details** view) — open it.
3. The query profile shows your job as a graph of **stages**, each broken into **tasks** (small units of parallel work — not to be confused with a *job task* from Section 1; this is Spark-internal terminology, and the exam does test that you can tell “Spark task” and “job task” apart from context). For each stage, look at:
 - **Duration** — which stage took disproportionately long relative to the data volume involved? That's your starting point.
 - **Skew** — in the stage's task metrics, compare the max task duration to the median. If one or two tasks take 10x longer than the rest while processing roughly the same *expected* amount of data, that's **data skew**: one partition (often caused by a join or group-by key with a very uneven value distribution, like 90% of orders belonging to one `customer_id`) is doing dramatically more work than its peers, and Spark can't finish the stage faster than its single slowest task.
 - **Spill (memory/disk)** — the query profile / Spark UI surfaces “Spill (Memory)” and “Spill (Disk)” metrics. Spill means a task ran out of allotted execution memory mid-operation (typically during a large sort, aggregation, or join) and had to write intermediate data to disk and read it back — much slower than staying in memory. Heavy spill is a sign you need more memory per task (bigger nodes, fewer concurrent tasks per node) or a smarter partitioning/join strategy to shrink what any single task has to hold at once.
 - **Shuffle read/write** — a shuffle happens whenever Spark has to redistribute data across the cluster (a `GROUP BY`, a `JOIN` on non-co-partitioned data, a `repartition()`). Shuffles are expensive because they involve network I/O and serialization across every node. A stage with enormous “Shuffle Read” relative to its input size is often the actual bottleneck, and it's the first thing to investigate when a job that used to be fast suddenly gets slow after a data volume increase.
4. Put it together: a stage that's slow *and* shows skew *and* shows spill is almost always a join or aggregation on a skewed key that's also memory-starved — the classic combination the exam likes to describe in a scenario question and ask you to diagnose from a description of symptoms.

4.4.2 4.2 Hands-on lab: break a job on purpose and diagnose it like a real incident

1. Edit `01_ingest` temporarily to point at a location that doesn't exist:

```
df = spark.read.format("csv").load(  
    ↪ "/Volumes/workspace/dp750_lab/does_not_exist/")
```
2. Run the job. It fails. Now, instead of just reading the obvious error, walk it like you would in production:
 - **Step 1 — Job Runs history.** Go to **Jobs & Pipelines** → your job → **Runs** tab. The failed run shows a red status. Click into it.

- **Step 2 — Identify which task failed.** The DAG view highlights ingest in red; downstream tasks show as **Skipped** (their upstream dependency never succeeded, so by default ALL_SUCCESS condition kept them from starting — a good sanity check that dependency logic is working as designed, not a separate bug).
 - **Step 3 — Read the task’s error output.** Click the failed ingest task → **Output/Logs**. You’ll see the actual Spark exception, something like `AnalysisException: Path does not exist: dbfs:/Volumes/workspace/dp750_lab/does_not_exist/`. This is almost always enough on its own to diagnose a “bad path” class of failure.
 - **Step 4 — Check cluster event logs when the error is *not* obviously about your code.** If instead the failure looked like a cluster problem (e.g., “cluster terminated,” “could not launch,” a driver-unresponsive error), you’d go to the cluster used by that run (linked from the run’s detail page for a job-cluster run) → **Event log** tab. This shows lifecycle events: CREATING, RUNNING, TERMINATING, and — critically — a **termination reason** (e.g., INACTIVITY, SPOT_INSTANCE_TERMINATION, DRIVER_ERROR, cloud-provider quota errors). This is the layer to check when the question is “why did the *compute* die” rather than “why did *my code* throw an exception.”
3. Fix it by reverting the path, rerun, confirm green. The habit to build here: **Runs tab for “what failed and what was skipped,” task Output/Logs for “what did my code say,” cluster Event log for “did the infrastructure itself mis-behave.”** The exam frequently gives you a symptom and asks which of these three you’d check first.

4.4.3 4.3 System tables for monitoring, auditing, and billing

Unity Catalog exposes operational data about your entire account as regular queryable Delta tables, under the built-in system catalog. You don’t have to build your own logging pipeline for basic governance/cost questions — you can just SELECT them.

-- Total DBU/\$ spend by SKU over the last 30 days

```
SELECT
  sku_name,
  SUM(usage_quantity) AS total_dbus
FROM system.billing.usage
WHERE usage_date >= current_date() - INTERVAL 30 DAYS
GROUP BY sku_name
ORDER BY total_dbus DESC;
```

-- Which clusters are currently running, and their configured autoscaling

```
↪ range
SELECT
  cluster_id,
  cluster_name,
  min_autoscale_workers,
```

```

    max_autoscale_workers,
    cluster_source
FROM system.compute.clusters
WHERE delete_time IS NULL;

-- Recent job run outcomes and durations (schema names such as
  ↳ system.lakeflow.*
-- have shifted as this feature has matured – always confirm the exact
  ↳ table/column
-- names available in YOUR workspace via `SHOW TABLES IN system.lakeflow`
  ↳ before
-- relying on this in an exam-adjacent real task)
SELECT
  workspace_id,
  job_id,
  run_id,
  result_state,
  period_start_time,
  period_end_time
FROM system.lakeflow.job_run_timeline
WHERE period_start_time >= current_date() - INTERVAL 7 DAYS
ORDER BY period_start_time DESC;

```

For the exam, know the *categories* of system tables that exist and roughly what each answers, more than exact column names: **system.billing.usage** (what did this cost, broken down by SKU/workspace/tag — the go-to table for “which job is burning our budget” questions), **system.compute.clusters** (an inventory/history of clusters and their configuration), and the **job/pipeline run tables under system.lakeflow** (run history, durations, and outcomes queryable in bulk, instead of clicking through the Jobs UI one run at a time — useful for building your own SLA dashboards). These system tables are themselves governed by Unity Catalog permissions, so a workspace admin has to grant you USE/SELECT on the system catalog schemas before you can query them.

4.4.4 4.4 Performance and cost levers

These are the concrete dials you reach for once monitoring has told you *where* the problem is:

- **Photon acceleration** — a vectorized query engine (written in native code, not the JVM) that Databricks can substitute in for standard Spark execution on SQL and DataFrame workloads, especially aggregations, joins, and scans over Delta/Parquet. It’s a checkbox on classic clusters and is on by default for serverless SQL warehouses and serverless job compute. Reach for it whenever a workload is CPU-bound on relational-style operations; it typically doesn’t help much on workloads dominated by Python/UDF logic that Photon can’t accelerate.
- **Partitioning strategy** — choosing a column (often a date) to physically segment a Delta table’s files by, so queries that filter on that column can skip reading ir-

relevant files entirely (partition pruning). Reach for it on large, append-heavy tables that are almost always queried with a filter on a low-cardinality, frequently-filtered column (like `event_date`); avoid over-partitioning on high-cardinality columns, which creates too many small files and *hurts* performance instead of helping it — small-file overhead is a very common Databricks anti-pattern the exam likes to test.

- **Caching** — keeping a working dataset in cluster memory (via `.cache()/persist()`, or the Delta cache on classic compute) so repeated queries against the same data skip re-reading it from cloud storage. Reach for it when the *same* dataset is scanned repeatedly in one session (iterative notebook development, a BI dashboard hitting the same table constantly) — it's wasted effort (and wasted memory) for a table that's scanned exactly once in a linear pipeline.
 - **Cluster right-sizing and autoscaling** — matching node count/size (or, on serverless, letting the platform matches it for you automatically) to the actual workload instead of guessing too big (wasted cost, idle capacity) or too small (spill, long run times, OOM errors). Reach for autoscaling on jobs with variable/unpredictable data volume, and reach for a fixed, right-sized cluster when a workload's volume is stable and predictable enough that autoscaling's ramp-up lag would just add latency for no benefit.
-

4.5 5. Mini Quiz — Domain 4

These are **original practice questions** written for this book, not real exam questions. Cover the material, don't memorize the specific wording.

1. (Multiple choice) A data engineering team wants a job to run automatically every weekday at 7:30 AM in the Databricks Jobs UI. Which cron expression, using the syntax Databricks Jobs actually requires, achieves this? - A. `30 7 * * 1-5` - B. `0 30 7 * * MON-FRI` - C. `7:30 * * MON-FRI` - D. `0 30 7 ? * MON-FRI`

Answer: D. Databricks Jobs uses Quartz cron syntax, which requires 6 fields (seconds, minutes, hours, day-of-month, month, day-of-week) and requires a `?` in whichever of day-of-month/day-of-week is *not* the one being constrained — here day-of-month is unconstrained, so it must be `?`, not `*`. Option A is standard 5-field Unix cron, not valid Quartz syntax; option B incorrectly uses `*` for day-of-month instead of `?`; option C isn't valid cron syntax at all.

2. (Multiple choice) Which compute option is the *only* one available for running Lakeflow Jobs on Databricks Free Edition? - A. All-purpose clusters - B. Classic job clusters with a custom cluster policy - C. Serverless compute - D. SQL warehouses only, no job compute

Answer: C. Free Edition is serverless-only — there is no classic compute (all-purpose or job clusters) and therefore no cluster policies to configure. All job tasks run on Databricks-managed serverless compute.

3. (Scenario / multiple choice) A job has four tasks: A, B (depends on A), C (depends on A), and D (depends on both B and C). Task A fails. What happens to B, C, and D if

all tasks use the default dependency run condition? - A. B and C run in parallel since neither depends on the other; D is skipped - B. B, C, and D are all skipped, because they transitively depend on A which failed - C. B and C fail explicitly; D is skipped - D. Only D is skipped; B and C run because they don't directly reference A's output

Answer: B. The default run condition for a dependency is `ALL_SUCCESS` — a task only starts once every task it depends on has *succeeded*. Since A failed, B and C never start (they're marked Skipped, not Failed), and since D depends on B and C which never succeeded, D is skipped too. This cascades down the whole DAG unless a task's run condition is explicitly changed (e.g., to `ALL_DONE`).

4. (Multiple answer — select two) Which of the following are true about job clusters compared to all-purpose clusters? (Select two.) - A. Job clusters are billed at a lower DBU rate than all-purpose clusters - B. Job clusters stay running after the job finishes so the next scheduled run starts faster - C. Job clusters are created automatically when a job run starts and terminate when it ends - D. All-purpose clusters are recommended for production job workloads because they're already warm

Answer: A and C. Job clusters are ephemeral (created at run start, terminated at run end) and cheaper per DBU than all-purpose clusters, which are meant to stay running for interactive/shared use. B is false (that describes all-purpose cluster behavior, and it's the opposite of what makes job clusters cost-efficient); D is a common misconception — using an all-purpose cluster for production jobs is generally discouraged specifically because of its higher cost and shared-resource risk.

5. (Multiple choice) A job's notify task should run regardless of whether the upstream `quality_check` task succeeds or fails, so the team always gets a summary. What should be configured? - A. Remove the dependency between `quality_check` and `notify` entirely - B. Set `notify`'s run condition (`run_if`) to `ALL_DONE` on its dependency on `quality_check` - C. Add a retry policy of 0 retries to `notify` - D. Set `quality_check`'s timeout to 0

Answer: B. The run condition on a dependency (JSON field `run_if`, UI dropdown under the dependency) controls whether the dependent task requires success (`ALL_SUCCESS`, the default) or simply requires the upstream task to have finished in any outcome (`ALL_DONE`). Removing the dependency (A) would let `notify` start immediately, in parallel, without waiting for `quality_check` at all — not the desired behavior.

6. (Multiple choice) Which trigger type is most appropriate for a job that should start ingesting a file the moment a third-party vendor uploads it to a Unity Catalog volume, at unpredictable times during the day? - A. Scheduled (cron), running every minute - B. File arrival trigger - C. Continuous trigger - D. Manual trigger only

Answer: B. A file arrival trigger fires when new files land in the specified location, reacting to the actual event rather than polling on a fixed interval. Running cron every minute (A) wastes runs when nothing has arrived and still introduces up to a minute of latency; continuous (C) is meant for streaming-style workloads that never really stop, which isn't what's described here.

7. (Scenario) After a job run fails, an engineer opens the Runs tab and sees the failing task's error is `ClusterTerminatedException`, with no clear application-level

stack trace pointing to a code bug. What should they check next to find the root cause?

- A. The notebook's cell output only
- B. The cluster's Event log, for a termination reason
- C. The job's Notifications tab
- D. `system.billing.usage`

Answer: B. A cluster-level failure (as opposed to a code exception) is diagnosed through the cluster's Event log, which records lifecycle events and termination reasons (e.g., spot instance reclaimed, inactivity timeout, driver unresponsive, cloud quota exceeded). The notebook output (A) won't show infrastructure-level events if the cluster died before/during execution; the Notifications tab (C) only shows whether an alert was configured/sent, not why the failure happened; billing usage (D) is for cost, not incident diagnosis.

8. (Multiple choice) In the Spark UI query profile for a slow stage, one task in a group of 200 takes 8 minutes while the rest take about 20 seconds each, even though every partition is supposed to hold roughly the same amount of data. This is most likely an example of:

- A. Shuffle write
- B. Data skew
- C. Photon acceleration
- D. Partition pruning

Answer: B. Data skew is exactly this pattern: one or a few tasks (usually tied to an uneven join/group-by key distribution) take drastically longer than their peers even though the stage was expected to split work evenly, and the whole stage can't finish faster than its single slowest task.

9. (Multiple choice) What is the primary purpose of a Databricks Asset Bundle's targets section in `databricks.yml`?

- A. To list the notebooks that belong to the bundle
- B. To define environment-specific configuration (e.g., workspace host, variables) so the same job/pipeline definitions can be deployed differently to dev, staging, and prod
- C. To specify which Git branch the bundle was created from
- D. To configure retry policies for individual tasks

Answer: B. targets is what lets one bundle produce differently-configured deployments per environment — different workspace, different variable values (like catalog/schema names) — using the same underlying job/pipeline resource definitions, which is the core value of infrastructure-as-code for promotion across dev/staging/prod.

10. (Multiple choice) Which CLI command actually creates or updates a job in the target workspace based on a bundle's YAML definitions?

- A. `databricks bundle validate`
- B. `databricks bundle init`
- C. `databricks bundle deploy`
- D. `databricks bundle run`

Answer: C. validate only checks the YAML for errors without touching the workspace; init scaffolds a new bundle from a template; deploy is what actually creates/updates the resources (jobs, pipelines, uploaded files) in the target workspace; run executes an already-deployed job/pipeline, it doesn't create it.

11. (Scenario / multiple answer — select two) A data engineer wants a transform notebook task to use a Unity Catalog schema name that differs between the dev and prod bundle targets, without maintaining two copies of the notebook. Which two things should they do?

- A. Hard-code the schema name as `"prod_schema"` directly in the notebook
- B. Read the schema name inside the notebook via `dbutils.widgets.get("schema")`
- C. Define a bundle variable (e.g., `schema`) with

different values per target, and pass it into the task's `base_parameters` in the YAML - D. Create a completely separate notebook file per environment

Answer: B and C. The notebook should read its configuration via a widget parameter rather than a hard-coded value, and the bundle YAML should supply that parameter's value per target through variables and `base_parameters` — this is exactly the pattern from the Section 3.3 lab, and it's what lets one notebook and one job definition serve every environment safely.

12. (Multiple choice) A team notices their nightly job's cost has crept up significantly even though data volume hasn't grown much. Querying `system.billing.usage`, they see most of the spend is on an all-purpose cluster that several engineers also use for ad hoc notebook work, and that this same cluster is what the nightly job is configured to run on. What is the most appropriate fix? - A. Disable Unity Catalog on that cluster - B. Switch the nightly job to run on a dedicated job cluster (or serverless) instead of the shared all-purpose cluster - C. Reduce the job's retry count to 0 - D. Move the job to a continuous trigger

Answer: B. Running scheduled production jobs on a shared, always-on all-purpose cluster is both more expensive (higher DBU rate, plus it never fully idles down because people are using it) and riskier (someone else's interactive workload can starve or crash the job). Moving the job onto its own job cluster or serverless compute isolates its cost and blast radius and is billed at the cheaper job-compute rate.

Summary — Part 4 checklist. You should now be able to: explain what orchestration solves and why it's a distinct layer from a Declarative Pipeline; build a multi-task Lakeflow Job with real dependencies and run conditions, in both the UI and JSON; choose between job clusters, all-purpose clusters, and serverless, and know why Free Edition only offers the last one; configure cron (Quartz syntax), file-arrival, and continuous triggers, plus retries and failure notifications; pass parameters into a notebook via widgets and job parameters; work the full Git loop in Databricks Repos from feature branch to pull request; write a minimal Databricks Asset Bundle and deploy/run it across targets; and diagnose a broken or slow job using run history, cluster event logs, the Spark UI query profile, and system tables — plus know when to reach for Photon, partitioning, caching, and right-sizing. That's the whole of Domain 4 — the largest single chunk of the exam — covered hands-on.

Chapter 5

Part 5 — Capstone Project: A Production-Style Retail Lakehouse on Azure Databricks

Every earlier part of this book handed you one skill at a time: Unity Catalog governance in isolation, Auto Loader in isolation, a Lakeflow Declarative Pipeline in isolation, a Lakeflow Job in isolation, a Databricks Asset Bundle in isolation. The DP-750 exam does not test you that way — a scenario question will hand you a messy situation that touches governance, ingestion, transformation, orchestration, and cost all at once, and ask what you’d do. Real jobs work the same way.

This chapter closes that gap. You are going to build one coherent, end-to-end lakehouse — from two raw source systems all the way to governed, business-ready gold tables, wired together by a scheduled, monitored, CI/CD-deployed job — and you are going to do it with your own hands, typing every command. When you’re done you will have:

- A working project you can demo live in an interview.
- A public GitHub repository with a README that explains what you built and why.
- A concrete, first-hand answer to almost every “how would you...” question DP-750 can throw at you, because you will have actually done it, not just read about it.

Budget a full weekend (or three or four weeknight evenings) to work through this chapter once, and a second, faster pass in the days before your exam to re-run it from memory. That second pass is worth more to your exam score than almost anything else in your prep — it’s the closest thing to a full practice exam this book can give you.

Cost note before you start. This project is designed to run comfortably inside Azure free-tier / pay-as-you-go budgets — a few dollars at most if you tear resources down when you finish each session — and every compute-heavy step tells you explicitly how to keep clusters small and how to shut things down afterward. Section “Phase 1” also gives you a genuinely free, zero-Azure-cost alternative for the source database if you’d rather not spend anything at all.

5.1 1. Project Overview & Architecture

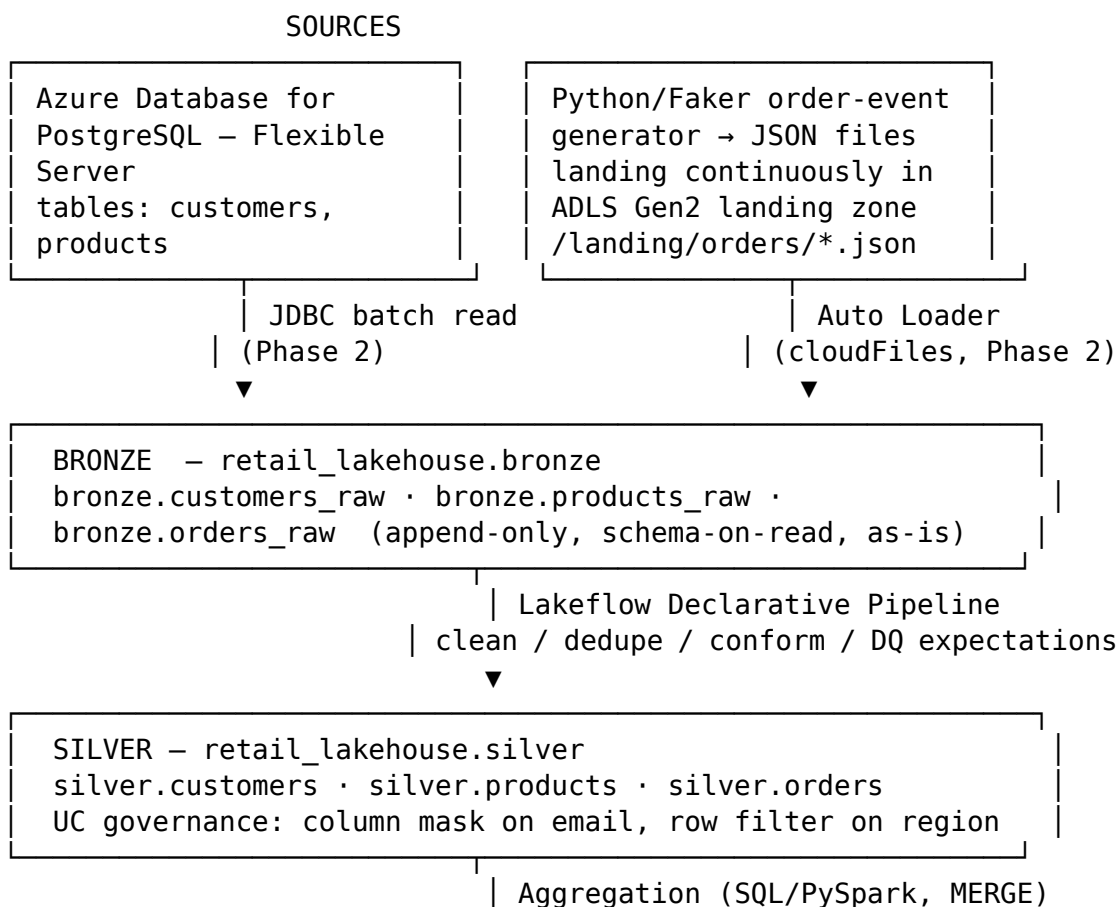
5.1.1 1.1 The scenario

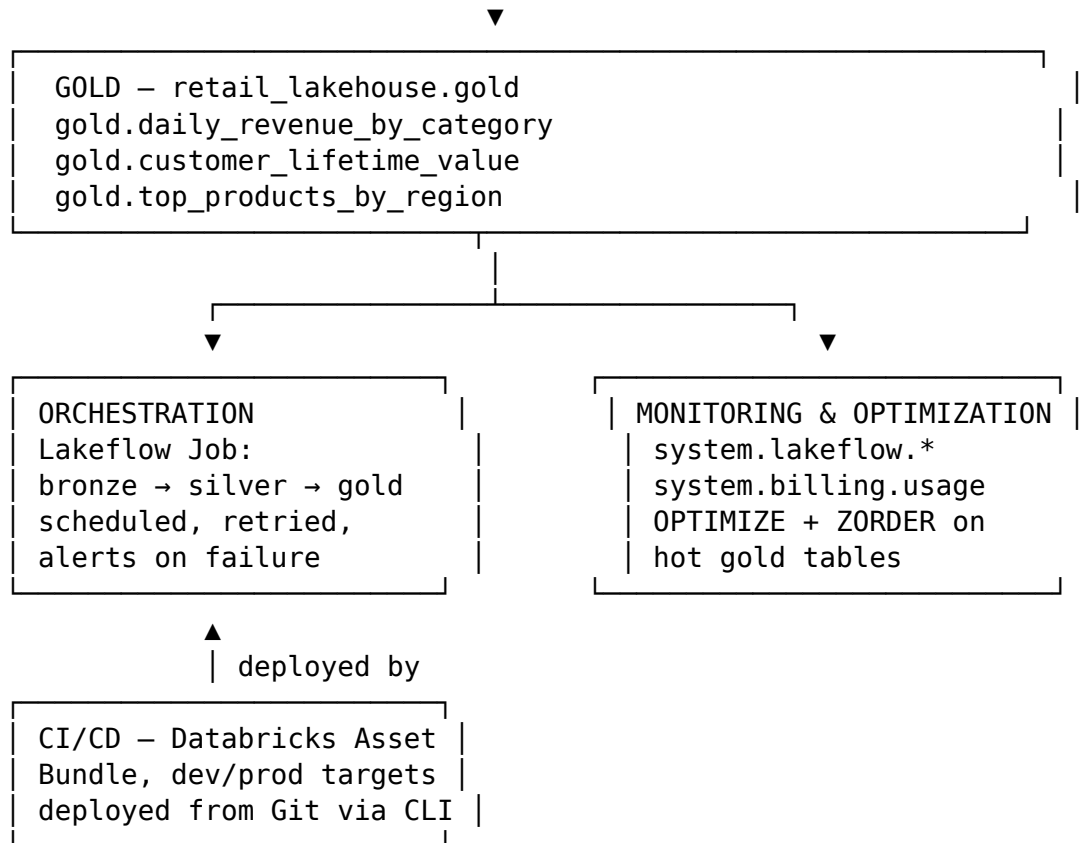
You are the data engineer at **NorthWind Retail**, a small (fictional) online retailer. The business runs two kinds of systems today:

1. An **operational relational database** — the system of record for customers and products — the kind of thing a web application writes to directly.
2. A **stream of order events** — every checkout emits a JSON record, landing continuously as files in cloud storage, the kind of thing an application, IoT device, or message-queue-to-storage sink would produce.

Nobody at NorthWind Retail can currently answer simple questions like “what was yesterday’s revenue by category?” or “who are our highest-value customers?” without emailing someone who runs a manual SQL query against production — which is slow, risky, and ungoverned (anyone with database access can see every customer’s raw email address). Your job is to build a lakehouse that ingests both sources, cleans and governs the data, aggregates it into answers the business can query safely and cheaply, and keeps itself running on a schedule without your intervention.

5.1.2 1.2 Architecture





5.1.3 1.3 What this project proves you can do (mapped to exam domains)

DP-750 organizes its content into domains covering ingestion & processing, pipeline construction/deployment, orchestration, monitoring/analysis, and governance. Use this table to see exactly where each phase of this project lands, and refer back to it once you finish — it doubles as a self-check that you’ve genuinely covered the exam’s ground.

Project phase	Exam domain area exercised	Concrete skill demonstrated
Phase 1 — Source systems	(Prerequisite/context)	Understanding source system shapes: relational OLTP vs. event files
Phase 2 — Bronze ingestion	Data ingestion and processing	Auto Loader (cloudFiles), schema inference/evolution, JDBC batch ingestion, Unity Catalog object creation
Phase 3 — Silver cleaning & governance	Data ingestion and processing; Data governance and security	Lakeflow Declarative Pipelines, DLT/pipeline expectations (expect_or_drop, expect), deduplication, UC grants, column masks, row filters

Project phase	Exam domain area exercised	Concrete skill demonstrated
Phase 4 — Gold aggregates	Data ingestion and processing	Incremental MERGE, star-schema-style business aggregation, CREATE OR REPLACE vs. incremental patterns
Phase 5 — Orchestration	Deploying and orchestrating data workflows	Multi-task Lakeflow Jobs, task dependencies, schedules (cron), retries, failure notifications
Phase 6 — CI/CD	Deploying and orchestrating data workflows	Databricks Asset Bundles, databricks.yml, dev/prod targets, CLI-driven deployment, Git integration
Phase 7 — Monitor & optimize	Monitoring and analyzing data pipelines for performance	System tables (system.lakeflow.*, system.billing.usage), OPTIMIZE, ZORDER, cost/performance tuning

If you can rebuild this project from memory, you have — in the most literal sense — practiced every domain DP-750 tests.

5.1.4 1.4 What you'll need before starting

- The workspace and tooling from earlier parts of this book: an Azure Databricks **Premium** workspace with Unity Catalog enabled, the Databricks CLI (databricks command) authenticated to that workspace, the Azure CLI (az) logged in, VS Code, Git, and a GitHub account.
- A Unity Catalog metastore already attached to your workspace, and a UC **external location** (backed by an ADLS Gen2 storage account + storage credential) you can create volumes/tables in — Part 1 walked through setting this up. This chapter reuses that storage; if you don't have it yet, go back and complete that setup first.
- Python 3.10+ locally, with `pip install faker psycpg2-binary` available (used only for the Phase 1 seed/generator scripts, run from your own laptop — not inside Databricks).
- About \$5-15 of Azure spend if you follow the Azure PostgreSQL path and tear it down promptly after finishing (exact figures in Phase 1).

5.2 2. Phase 1 — Set Up the Source Systems

5.2.1 2.1 Choosing the operational database: Azure Database for PostgreSQL - Flexible Server

We're using **Azure Database for PostgreSQL - Flexible Server** as the primary path in this chapter, for two reasons specific to your exam goals:

1. DP-750 scenarios frequently describe ingesting from “an external relational database” — practicing a real JDBC connection from Databricks to a real cloud Postgres instance, including firewall/networking configuration, is exactly the muscle memory those questions expect.
2. It’s genuinely cheap if you use the smallest burstable SKU and delete the server when you’re done — a few dollars for the time it takes to work through this chapter.

Zero-cost alternative. If you’d rather spend nothing, run Postgres locally in Docker instead: `docker run --name northwind-pg -e POSTGRES_PASSWORD=Northwind123! -e POSTGRES_DB=northwind -p 5432:5432 -d postgres:16`. Everything in Phase 2’s JDBC ingestion code works identically — you’ll just need Databricks to reach localhost, which means either running your Databricks cluster against a tunnel (e.g., `ngrok tcp 5432`) or, more realistically, using this local Postgres only to generate CSV/Parquet exports that you upload to your ADLS landing zone and ingest with Auto Loader like the orders feed. Because reachability from a cloud cluster to a laptop-local database is genuinely awkward, **the Azure path below is what we recommend you actually follow** if your budget allows it at all — it’s a more honest rehearsal of what the exam (and a real job) expects.

5.2.1.1 2.1.1 Create the resource group (if you don’t already have one for this project)

```
az group create \
  --name rg-retail-lakehouse \
  --location eastus
```

5.2.1.2 2.1.2 Create the Flexible Server (smallest/cheapest burstable SKU)

```
az postgres flexible-server create \
  --resource-group rg-retail-lakehouse \
  --name pg-northwind-retail \
  --location eastus \
  --admin-user pgadmin \
  --admin-password 'N0rthWind!Retail2026' \
  --sku-name Standard_B1ms \
  --tier Burstable \
  --storage-size 32 \
  --version 16 \
  --public-access 0.0.0.0-255.255.255.255 \
  --yes
```

Notes on the values above:

- `Standard_B1ms` is the cheapest burstable compute tier (1 vCore, 2 GiB RAM) — plenty for this project’s small seed tables.

- `--storage-size 32` is the minimum allowed (32 GiB) and still costs only cents/month prorated.
- `--public-access 0.0.0.0-255.255.255.255` opens the server's firewall to all IPs **for this exercise only** — it is the simplest way to guarantee Azure Databricks' cluster nodes (whose outbound IPs you don't control on serverless/most compute) can reach the server without you hunting down a specific IP range. **This is not something you'd do in a real production environment** — call this out explicitly if you're demoing this project in an interview, and mention that in production you'd instead use VNet injection / a private endpoint and put the Databricks workspace on the same virtual network, which Part 1 discusses. For a home-lab exercise like this, the tradeoff is fine as long as you delete the server when you're done (Phase 7 reminds you).
- Replace the password with your own strong value if you like — just remember it, you'll need it in Phase 2.

Wait for the command to finish (a few minutes); it prints the server's connection details when done.

5.2.1.3 2.1.3 Confirm you can connect, and create the database

```
az postgres flexible-server db create \
  --resource-group rg-retail-lakehouse \
  --server-name pg-northwind-retail \
  --database-name northwind
```

Test connectivity from your laptop with psql (install it if needed — on macOS brew install libpq, on Ubuntu sudo apt install postgresql-client):

```
psql "host=pg-northwind-retail.postgres.database.azure.com port=5432
↳ dbname=northwind user=pgadmin password=N0rthWind!Retail2026
↳ sslmode=require"
```

If you land at a `northwind=>` prompt, you're connected. Type `\q` to exit for now.

5.2.2 2.2 Create the customers and products tables with seed data

Save the following as `sql/01_seed_source_db.sql` in your project folder, then run it with `psql -f sql/01_seed_source_db.sql "<same connection string as above>"`.

```
-- sql/01_seed_source_db.sql
-- Run against the northwind database on pg-northwind-retail
```

```
DROP TABLE IF EXISTS customers;
CREATE TABLE customers (
  customer_id SERIAL PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(120) NOT NULL,
  region VARCHAR(20) NOT NULL, -- 'North
↳ America', 'Europe', 'APAC', 'LATAM'
```

```

        signup_date      DATE          NOT NULL,
        updated_at       TIMESTAMP     NOT NULL DEFAULT now()
    );

```

```

DROP TABLE IF EXISTS products;
CREATE TABLE products (
    product_id          SERIAL PRIMARY KEY,
    product_name         VARCHAR(120) NOT NULL,
    category            VARCHAR(40)  NOT NULL,    --
    ⇨ 'Electronics', 'Home', 'Apparel', 'Sports', 'Books'
    unit_price          NUMERIC(10,2) NOT NULL,
    updated_at          TIMESTAMP     NOT NULL DEFAULT now()
);

```

```

INSERT INTO customers (first_name, last_name, email, region, signup_date)
⇨ VALUES
('Aria', 'Nakamura', 'aria.nakamura@example.com', 'APAC', '2023-02-11'),
('Liam', 'O'Connor', 'liam.oconnor@example.com', 'Europe', '2022-11-02'),
('Sofia', 'Reyes', 'sofia.reyes@example.com', 'LATAM', '2023-06-19'),
('Ethan', 'Walker', 'ethan.walker@example.com', 'North
⇨ America', '2021-09-14'),
('Noor', 'Haddad', 'noor.haddad@example.com', 'Europe', '2023-01-05'),
('Priya', 'Shah', 'priya.shah@example.com', 'APAC', '2022-04-27'),
('Marco', 'Rossi', 'marco.rossi@example.com', 'Europe', '2023-08-30'),
('Chidi', 'Okonkwo', 'chidi.okonkwo@example.com', 'North
⇨ America', '2022-12-15'),
('Isla', 'Campbell', 'isla.campbell@example.com', 'North
⇨ America', '2023-03-22'),
('Diego', 'Fernandez', 'diego.fernandez@example.com', 'LATAM', '2021-07-08'),
('Yuki', 'Tanaka', 'yuki.tanaka@example.com', 'APAC', '2023-05-01'),
('Emma', 'Schmidt', 'emma.schmidt@example.com', 'Europe', '2022-02-19'),
('Kwame', 'Mensah', 'kwame.mensah@example.com', 'North
⇨ America', '2023-07-11'),
('Valentina', 'Moreno', 'valentina.moreno@example.com', 'LATAM', '2022-10-06')
⇨ ),
('Hana', 'Kobayashi', 'hana.kobayashi@example.com', 'APAC', '2021-12-25');

```

```

INSERT INTO products (product_name, category, unit_price) VALUES
('Wireless Earbuds Pro', 'Electronics', 89.99),
('4K Streaming Stick', 'Electronics', 39.99),
('Mechanical Keyboard', 'Electronics', 74.50),
('Espresso Machine', 'Home', 199.00),
('Non-Stick Cookware Set', 'Home', 129.99),
('Memory Foam Pillow', 'Home', 34.99),
('Running Shoes', 'Sports', 84.99),
('Yoga Mat', 'Sports', 24.99),
('Adjustable Dumbbell Set', 'Sports', 149.00),

```

```
( 'Men''s Denim Jacket', 'Apparel', 59.99),
( 'Women''s Wool Coat', 'Apparel', 129.00),
( 'Graphic T-Shirt', 'Apparel', 19.99),
( 'Data Engineering with Databricks', 'Books', 44.99),
( 'The Pragmatic Programmer', 'Books', 34.50),
( 'Designing Data-Intensive Applications', 'Books', 49.99);
```

You should now have 15 customers and 15 products in Postgres. Confirm:

```
psql "host=pg-northwind-retail.postgres.database.azure.com port=5432
↳ dbname=northwind user=pgadmin password=N0rthWind!Retail2026
↳ sslmode=require" \
-c "SELECT count(*) FROM customers; SELECT count(*) FROM products;"
```

5.2.3 2.3 Generate a stream of order-event files with Python + Faker

Orders represent the “streaming-like” source: a real e-commerce checkout system would emit one small JSON file (or a Kafka/Event Hubs message, sink’d to storage) per order. We’ll simulate that by writing a Python script that generates a batch of JSON files every time it runs, referencing the real customer_ids and product_ids you just seeded, and uploading them into your ADLS Gen2 landing zone.

5.2.3.1 2.3.1 Install dependencies locally

```
python3 -m venv .venv
source .venv/bin/activate # on Windows: .venv\Scripts\activate
pip install faker azure-storage-file-datalake
```

5.2.3.2 2.3.2 The generator script

Save as scripts/generate_orders.py:

```
"""
generate_orders.py
Simulates a stream of order-event JSON files landing in an ADLS Gen2
container, one file per small batch of orders, mimicking what an
e-commerce checkout system would produce.

Usage:
python scripts/generate_orders.py --batches 5 --orders-per-batch 40
"""

import argparse
import json
import random
import uuid
from datetime import datetime, timedelta, timezone
from pathlib import Path
```

```

from faker import Faker
from azure.storage.filedatalake import DataLakeServiceClient

fake = Faker()

# Must match the seed data you loaded into Postgres in section 2.2
CUSTOMER_IDS = list(range(1, 16))          # customer_id 1..15
PRODUCT_CATALOG = [
    (1, 89.99), (2, 39.99), (3, 74.50), (4, 199.00), (5, 129.99),
    (6, 34.99), (7, 84.99), (8, 24.99), (9, 149.00), (10, 59.99),
    (11, 129.00), (12, 19.99), (13, 44.99), (14, 34.50), (15, 49.99),
] # (product_id, unit_price) – mirrors the products table

STATUSES = ["PLACED", "PLACED", "PLACED", "PLACED", "CANCELLED"] # ~20%
    ↪ cancelled, for DQ practice

def make_order_event() -> dict:
    """Build one realistic (and occasionally intentionally messy)
    ↪ order-event dict."""
    product_id, unit_price = random.choice(PRODUCT_CATALOG)
    quantity = random.randint(1, 4)
    order_ts = datetime.now(timezone.utc) -
    ↪ timedelta(minutes=random.randint(0, 4320))

    event = {
        "order_id": str(uuid.uuid4()),
        "customer_id": random.choice(CUSTOMER_IDS),
        "product_id": product_id,
        "quantity": quantity,
        "unit_price": unit_price,
        "order_status": random.choice(STATUSES),
        "order_timestamp": order_ts.isoformat(),
        "shipping_country": fake.country_code(),
    }

    # Intentionally inject a small amount of dirty data so Phase 3's
    # data-quality expectations have something real to catch.
    roll = random.random()
    if roll < 0.03:
        event["order_id"] = None                # missing primary key
    elif roll < 0.06:
        event["quantity"] = -1                  # invalid quantity
    elif roll < 0.09:
        event["customer_id"] = 9999             # orphaned customer_id

    return event

```

```

def write_batch_locally(batch: list[dict], out_dir: Path) -> Path:
    out_dir.mkdir(parents=True, exist_ok=True)
    file_name =
    ↪ f"orders_{datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%S%f')}.json"
    file_path = out_dir / file_name
    with file_path.open("w") as f:
        for event in batch:
            f.write(json.dumps(event) + "\n")    # newline-delimited JSON
    return file_path

def upload_to_adls(local_path: Path, account_name: str, container: str,
    ↪ directory: str):
    service_client = DataLakeServiceClient(
        account_url=f"https://{account_name}.dfs.core.windows.net",
        credential=None, # uses `az login` credentials via
    ↪ DefaultAzureCredential if configured;
        # simplest for this project: use account_key
        ↪ instead, see note below
    )
    fs_client = service_client.get_file_system_client(container)
    dir_client = fs_client.get_directory_client(directory)
    file_client = dir_client.create_file(local_path.name)
    with local_path.open("rb") as f:
        data = f.read()
    file_client.append_data(data, offset=0, length=len(data))
    file_client.flush_data(len(data))

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--batches", type=int, default=5, help="how many
    ↪ files to generate")
    parser.add_argument("--orders-per-batch", type=int, default=40)
    parser.add_argument("--local-dir", default="./data/orders_landing")
    parser.add_argument("--upload", action="store_true", help="also upload
    ↪ each file to ADLS")
    parser.add_argument("--account-name", default="")
    parser.add_argument("--container", default="landing")
    parser.add_argument("--directory", default="orders")
    args = parser.parse_args()

    out_dir = Path(args.local_dir)

    for i in range(args.batches):
        batch = [make_order_event() for _ in range(args.orders_per_batch)]
        local_path = write_batch_locally(batch, out_dir)

```

```

print(f"[{i+1}/{args.batches}] wrote {local_path} ({len(batch)}
↳ events)")

if args.upload:
    upload_to_adls(local_path, args.account_name, args.container,
↳ args.directory)
    print(f"    uploaded to
↳ abfss://{args.container}@{args.account_name}
↳ .dfs.core.windows.net/{args.directory}/{local_path.name}")

if __name__ == "__main__":
    main()

```

Simplest upload path. Rather than wiring up azure-storage-file-datalake credentials from your laptop, the easiest and most exam-relevant approach is to run the script with `--upload off`, writing files locally, and then upload each batch with the Databricks CLI’s filesystem commands (which use the same auth you already set up in Part 1):

```

databricks fs cp -r ./data/orders_landing
↳ dbfs:/Volumes/retail_lakehouse/bronze/landing/orders

```

Or, more realistically for a Unity Catalog Volume backed by ADLS, use `az storage fs directory upload` after retrieving your storage account key with `az storage account keys list`. Either works — what matters for Phase 2 is that JSON files end up under a single landing path that Auto Loader can watch. This chapter assumes you land files under a **Unity Catalog Volume** path — see section 3.1 for why.

Run it once to produce your first batch of files:

```
python scripts/generate_orders.py --batches 5 --orders-per-batch 40
```

You should see five `orders_*.json` files appear under `./data/orders_landing`. You’ll re-run this script (or schedule it, e.g., via cron or Task Scheduler, if you want continuous “streaming” during your testing) any time you want new order events to flow into bronze — that’s what makes Auto Loader’s incremental behavior visible and demonstrable.

5.3 3. Phase 2 — Bronze Layer: Raw Ingestion

5.3.1 3.1 Set up the Unity Catalog structure

Open a Databricks SQL editor (or a notebook cell with `%sql`) against your Premium workspace and run:

```

-- Top-level catalog for the whole capstone project
CREATE CATALOG IF NOT EXISTS retail_lakehouse

```

```

MANAGED LOCATION 's3://replace-with-nothing' -- omit this clause
↳ entirely if your

-- metastore already has a
↳ default
-- storage location
↳ configured (typical
-- for a workspace you set
↳ up in Part 1)

;

```

If your workspace's Unity Catalog metastore already has a default managed storage location (the normal case if you followed Part 1's setup), just run:

```

CREATE CATALOG IF NOT EXISTS retail_lakehouse
  COMMENT 'Capstone project: NorthWind Retail lakehouse';

CREATE SCHEMA IF NOT EXISTS retail_lakehouse.bronze
  COMMENT 'Raw, as-landed data – no cleaning applied';

CREATE SCHEMA IF NOT EXISTS retail_lakehouse.silver
  COMMENT 'Cleaned, deduplicated, conformed data with DQ enforcement';

CREATE SCHEMA IF NOT EXISTS retail_lakehouse.gold
  COMMENT 'Business-ready aggregates for reporting and analytics';

-- A managed Unity Catalog Volume to act as the "landing zone" that the
-- order-generator script writes into and Auto Loader reads from.
CREATE VOLUME IF NOT EXISTS retail_lakehouse.bronze.landing
  COMMENT 'Landing zone for raw order-event JSON files';

```

Confirm the structure:

```

SHOW SCHEMAS IN retail_lakehouse;
SHOW VOLUMES IN retail_lakehouse.bronze;

```

Upload your generated order files into this volume now (repeat the databricks fs cp command from section 2.3, pointed at dbfs:/Volumes/retail_lakehouse/bronze/landing/orders):

```

databricks fs mkdir dbfs:/Volumes/retail_lakehouse/bronze/landing/orders
databricks fs cp -r ./data/orders_landing
↳ dbfs:/Volumes/retail_lakehouse/bronze/landing/orders

```

5.3.2 3.2 Bronze ingestion of orders with Auto Loader

Create a Databricks notebook (e.g., notebooks/01_bronze_orders_autoloader.py), attach it to a small **single-node, small-instance cluster or serverless compute**, and run:

```

# notebooks/01_bronze_orders_autoloader.py
from pyspark.sql import functions as F

```

```

landing_path = "/Volumes/retail_lakehouse/bronze/landing/orders"
checkpoint_path = "/Vol-
↳ umes/retail_lakehouse/bronze/landing/_checkpoints/orders_autoloader"
schema_location =
↳  "/Volumes/retail_lakehouse/bronze/landing/_schemas/orders_autoloader"

bronze_orders_df = (
    spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", schema_location)
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load(landing_path)
        .withColumn("_ingested_at", F.current_timestamp())
        .withColumn("_source_file", F.col("_metadata.file_path"))
)

(
    bronze_orders_df.writeStream
        .format("delta")
        .option("checkpointLocation", checkpoint_path)
        .option("mergeSchema", "true")
        .trigger(availableNow=True)  # process everything currently in the
↳  volume, then stop
        .toTable("retail_lakehouse.bronze.orders_raw")
)

```

A few things worth understanding for the exam, not just for this project:

- `cloudFiles.format` tells Auto Loader the file format to parse (json here — it also supports csv, parquet, avro, binaryFile, text).
- `cloudFiles.schemaLocation` is where Auto Loader persists the schema it infers, so subsequent runs don't need to re-infer from scratch and can detect new columns.
- `cloudFiles.schemaEvolutionMode = "addNewColumns"` means if the order-event JSON gains a new field later (e.g., a `promo_code`), the stream fails once with an `UnknownFieldException`, updates the schema, and picks the new column up on the next run — rather than silently dropping it.
- `trigger(availableNow=True)` is the right choice here: it processes every file currently sitting in the volume and then stops, which suits a scheduled batch-style job (Phase 5) far better than a truly continuous `trigger(processingTime=...)` stream would for this use case, and it avoids leaving a cluster running (and billing) indefinitely.
- `_metadata.file_path` is Spark's built-in hidden metadata column, useful for lineage/debugging — always know which physical file a row came from.

Run the cell. Then verify:

```
SELECT count(*) FROM retail_lakehouse.bronze.orders_raw;
SELECT * FROM retail_lakehouse.bronze.orders_raw LIMIT 5;
DESCRIBE HISTORY retail_lakehouse.bronze.orders_raw;
```

Generate a second batch of order files (python scripts/generate_orders.py --batches 2 --orders-per-batch 40, then re-upload), re-run the same notebook cell, and confirm the row count grows only by the new files' worth of rows — that's Auto Loader's exactly-once, incremental file tracking in action, and it's a great thing to be able to describe confidently in an interview.

5.3.3 3.3 Bronze ingestion of customers and products via JDBC

First, store your Postgres credentials as a Databricks **secret** rather than hardcoding them (this is both an exam-relevant best practice and a governance story worth telling in an interview):

```
databricks secrets create-scope retail_lakehouse
databricks secrets put-secret retail_lakehouse pg_user --string-value
↪ "pgadmin"
databricks secrets put-secret retail_lakehouse pg_password --string-value
↪ "N0rthWind!Retail2026"
```

Now, in a notebook (notebooks/02_bronze_jdbc_customers_products.py):

```
# notebooks/02_bronze_jdbc_customers_products.py
from pyspark.sql import functions as F

jdbc_host = "pg-northwind-retail.postgres.database.azure.com"
jdbc_port = 5432
jdbc_db = "northwind"
jdbc_url =
↪ f"jdbc:postgresql://{jdbc_host}:{jdbc_port}/{jdbc_db}?sslmode=require"

pg_user = dbutils.secrets.get("retail_lakehouse", "pg_user")
pg_password = dbutils.secrets.get("retail_lakehouse", "pg_password")

connection_props = {
    "user": pg_user,
    "password": pg_password,
    "driver": "org.postgresql.Driver",
}

def load_table_to_bronze(source_table: str, target_table: str):
    df = (
        spark.read.jdbc(url=jdbc_url, table=source_table,
↪ properties=connection_props)
        .withColumn("_ingested_at", F.current_timestamp())
    )
    (
        df.write
```

```

        .format("delta")
        .mode("overwrite")
        .option("overwriteSchema", "true")
        .saveAsTable(target_table)
    )
    print(f"Loaded {df.count()} rows from {source_table} into
    ↪ {target_table}")

```

```

load_table_to_bronze("customers", "retail_lakehouse.bronze.customers_raw")
load_table_to_bronze("products", "retail_lakehouse.bronze.products_raw")

```

Because Azure Databricks clusters ship with the PostgreSQL JDBC driver pre-installed on the Databricks Runtime, you don't need to install anything extra — `org.postgresql.Driver` is available out of the box.

We used `mode("overwrite")` here deliberately: customers and products are small dimension-style tables where a full refresh on every run is simpler and cheap. In a bigger production system with millions of rows you'd instead do a JDBC read filtered on `updated_at > <last_watermark>` and MERGE the results — exactly the pattern we'll use for the gold layer in Phase 4, so you'll get to practice that technique too.

Verify:

```

SELECT * FROM retail_lakehouse.bronze.customers_raw LIMIT 5;
SELECT * FROM retail_lakehouse.bronze.products_raw LIMIT 5;

```

On Lakeflow Connect. Databricks also offers **Lakeflow Connect**, a managed set of ingestion connectors (including one for databases via a gateway) that removes the need to hand-write JDBC/Auto Loader code for supported sources. It's worth knowing it exists for the exam and for interview conversations ("when would you reach for a managed connector instead of writing your own ingestion code?"), but hand-rolling the JDBC and Auto Loader ingestion yourself, as we just did, is the better learning exercise — and it's the pattern you'll need for any source Lakeflow Connect doesn't cover.

You now have three bronze tables: `bronze.orders_raw`, `bronze.customers_raw`, `bronze.products_raw` — raw, unfiltered, exactly as they arrived.

5.4 4. Phase 3 — Silver Layer: Clean, Conform & Govern

5.4.1 4.1 Build a Lakeflow Declarative Pipeline for silver

Lakeflow Declarative Pipelines (the current name for what was Delta Live Tables) let you declare *what* each table should look like and let the engine handle the *how* — incremental processing, dependency ordering, and data-quality enforcement. Create a new file in your project, `pipelines/silver_pipeline.py`:

```

# pipelines/silver_pipeline.py
import dlt

```

```

from pyspark.sql import functions as F

# -----
# <- --
# SILVER: orders
# -----
# <- --
@dlt.table(
    name="orders",
    comment="Cleaned, deduplicated order events with valid business keys.",
    table_properties={"quality": "silver"},
)
@dlt.expect_or_drop("valid_order_id", "order_id IS NOT NULL")
@dlt.expect_or_drop("valid_quantity", "quantity > 0")
@dlt.expect("known_customer", "customer_id IS NOT NULL") # warn only,
# <- don't drop
def orders():
    return (
        dlt.read_stream("retail_lakehouse.bronze.orders_raw")
        .withColumn("order_timestamp", F.to_timestamp("order_timestamp"))
        .withColumn("order_date", F.to_date("order_timestamp"))
        .withColumn("line_total", F.col("quantity") * F.col("unit_price"))
        .dropDuplicates(["order_id"])
        .select(
            "order_id", "customer_id", "product_id", "quantity",
            # <- "unit_price",
            "line_total", "order_status", "order_timestamp", "order_date",
            "shipping_country",
        )
    )

# -----
# <- --
# SILVER: customers
# -----
# <- --
@dlt.table(
    name="customers",
    comment="Conformed customer dimension with validated email format.",
    table_properties={"quality": "silver"},
)
@dlt.expect_or_drop("valid_customer_id", "customer_id IS NOT NULL")
@dlt.expect_or_drop(
    "valid_email_format",
    r"email RLIKE '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'",
)

```

```

@dlt.expect("known_region", "region IN ('North
↳ America', 'Europe', 'APAC', 'LATAM')")
def customers():
    return (
        dlt.read("retail_lakehouse.bronze.customers_raw")
        .dropDuplicates(["customer_id"])
        .select(
            "customer_id", "first_name", "last_name", "email", "region",
            "signup_date",
        )
    )

# -----
↳ --
# SILVER: products
# -----
↳ --
@dlt.table(
    name="products",
    comment="Conformed product dimension.",
    table_properties={"quality": "silver"},
)
@dlt.expect_or_drop("valid_product_id", "product_id IS NOT NULL")
@dlt.expect_or_drop("valid_price", "unit_price > 0")
def products():
    return (
        dlt.read("retail_lakehouse.bronze.products_raw")
        .dropDuplicates(["product_id"])
        .select("product_id", "product_name", "category", "unit_price")
    )

```

A quick refresher on the expectation decorators, since they show up on the exam by name:

- `@dlt.expect_or_drop("name", "condition")` — rows failing the condition are **dropped** from the output but the pipeline keeps running; failure counts show up in the pipeline's event log.
- `@dlt.expect("name", "condition")` — rows failing the condition are **kept** but flagged/counted; use this for “worth watching” conditions you don't want to lose data over (like an order referencing a `customer_id` that doesn't exist yet due to timing).
- A third variant, `@dlt.expect_or_fail("name", "condition")`, would **stop the pipeline entirely** on any violation — appropriate for a condition so severe you never want bad data to reach the table at all. We don't use it here, but know it for the exam.

Notice the deliberately dirty records the Faker script injects (`order_id = null`, quan-

tity = -1, customer_id = 9999) exist specifically so you can watch these expectations catch real problems — that’s worth demonstrating live in an interview.

5.4.1.1 4.1.1 Create and run the pipeline

Create the pipeline via the CLI (or the Workflows UI under “Delta Live Tables” / “ETL Pipelines” — same underlying feature):

```
databricks pipelines create --json '{
  "name": "retail_silver_pipeline",
  "catalog": "retail_lakehouse",
  "target": "silver",
  "libraries": [
    {"notebook": {"path": "/Workspace/Users/<you@example.com>/retail-
↪ lakehouse/pipelines/silver_pipeline"}}
  ],
  "continuous": false,
  "photon": true,
  "clusters": [
    {"label": "default", "num_workers": 1, "node_type_id":
↪ "Standard_DS3_v2"}
  ]
}'
```

Replace <you@example.com> with your workspace username, and import pipelines/silver_pipeline into that Workspace path first (databricks workspace import or just save it there via the Databricks notebook UI). Then trigger a run:

```
databricks pipelines start-update --pipeline-id
↪ <pipeline-id-from-create-output>
```

Watch it run in the workspace UI (Workflows → Pipelines → retail_silver_pipeline) — you’ll see a live DAG of the three silver tables, row counts, and expectation pass/fail metrics per table. Confirm the tables landed:

```
SELECT * FROM retail_lakehouse.silver.orders LIMIT 5;
SELECT * FROM retail_lakehouse.silver.customers LIMIT 5;
SELECT * FROM retail_lakehouse.silver.products LIMIT 5;
```

```
-- Confirm the dirty rows really were dropped:
SELECT count(*) FROM retail_lakehouse.bronze.orders_raw; -- includes bad
↪ rows
SELECT count(*) FROM retail_lakehouse.silver.orders; -- bad rows
↪ removed
```

5.4.2 4.2 Apply Unity Catalog governance to silver

This is where Part 2’s governance material gets exercised for real. First, simulate an analyst group. If you have workspace admin rights, create it (via the account console,

or the CLI shown below); if you're on a trial/free tier without account-admin access, you can substitute a single test user's email everywhere analysts appears below.

```
databricks account groups create --display-name analysts
# then add yourself or a test user as a member via the account console UI,
# or: databricks account groups patch <group-id> --operations
↪ ' [{"op": "add", "path": "members", "value": [{"value": "<user-id>"}]} ] '
```

5.4.2.1 4.2.1 Grant table-level privileges

```
GRANT USE CATALOG ON CATALOG retail_lakehouse TO `analysts`;
GRANT USE SCHEMA ON SCHEMA retail_lakehouse.silver TO `analysts`;
GRANT USE SCHEMA ON SCHEMA retail_lakehouse.gold TO `analysts`;

GRANT SELECT ON TABLE retail_lakehouse.silver.orders TO `analysts`;
GRANT SELECT ON TABLE retail_lakehouse.silver.customers TO `analysts`;
GRANT SELECT ON TABLE retail_lakehouse.silver.products TO `analysts`;
GRANT SELECT ON SCHEMA retail_lakehouse.gold TO `analysts`; -- analysts
↪ can read all gold tables
```

```
-- Analysts should NOT be able to see or touch raw bronze data at all
REVOKE ALL PRIVILEGES ON SCHEMA retail_lakehouse.bronze FROM `analysts`;
```

5.4.2.2 4.2.2 Column mask on the customer email field

Analysts need to be able to join on customer_id and see aggregate behavior, but should not see raw email addresses (real PII). We'll mask emails for anyone in the analysts group, while admins/data engineers keep seeing the real value.

```
CREATE OR REPLACE FUNCTION retail_lakehouse.silver.mask_email(email STRING)
RETURNS STRING
RETURN
CASE
  WHEN is_account_group_member('analysts') THEN
    concat('***MASKED***@', split(email, '@')[1])
  ELSE
    email
END;

ALTER TABLE retail_lakehouse.silver.customers
  ALTER COLUMN email SET MASK retail_lakehouse.silver.mask_email;
```

Test it: query `SELECT email FROM retail_lakehouse.silver.customers LIMIT 3`; as yourself (an admin) and you'll see real addresses; if you switch to a test user who is only a member of analysts, they'll see `***MASKED***@example.com`-style values instead.

5.4.2.3 4.2.3 Row filter restricting by region

Suppose NorthWind Retail wants its APAC regional analyst team to only see APAC customer/order data (a common multi-region governance pattern). We'll build a row filter parameterized by group membership:

```
CREATE OR REPLACE FUNCTION retail_lakehouse.silver.region_row_filter(region
  ↪ STRING)
RETURNS BOOLEAN
RETURN
  is_account_group_member('admins')                                -- admins
  ↪ see everything
  OR is_account_group_member('analysts_apac') AND region = 'APAC' -- APAC
  ↪ analysts see only APAC
  OR NOT is_account_group_member('analysts_apac');                --
  ↪ everyone else unaffected by this filter
```

```
ALTER TABLE retail_lakehouse.silver.customers
  SET ROW FILTER retail_lakehouse.silver.region_row_filter ON (region);
```

(If you didn't create an analysts_apac group, skip actually attaching this filter in your live workspace — but keep the SQL in your repo. It's the exact shape of answer DP-750 governance questions expect, and it's a great thing to walk an interviewer through even if you demo it against a single test table rather than a fully populated multi-region org.)

Confirm masks and filters are attached:

```
DESCRIBE TABLE EXTENDED retail_lakehouse.silver.customers;
```

Look for the Column Masks and Row Filter sections in the output.

5.5 5. Phase 4 — Gold Layer: Business-Ready Aggregates

Gold tables answer specific business questions directly — no joins required for a BI tool or analyst to get an answer. We'll build three, each using MERGE so the tables update **incrementally** rather than being fully recomputed on every run (a technique the exam calls out explicitly, and one that matters enormously at real data volumes).

Create notebooks/03_gold_aggregates.py:

```
# notebooks/03_gold_aggregates.py
from pyspark.sql import functions as F
from delta.tables import DeltaTable
```

```
spark.sql("USE CATALOG retail_lakehouse")
```

```
# -----
# --
```

```

# GOLD 1: daily_revenue_by_category
# "What was our revenue by product category, per day?"
# -----
↪ --
daily_revenue = (
    spark.table("silver.orders").filter("order_status = 'PLACED'")
    .join(spark.table("silver.products"), "product_id")
    .groupBy("order_date", "category")
    .agg(
        F.sum("line_total").alias("revenue"),
        F.count("order_id").alias("order_count"),
        F.sum("quantity").alias("units_sold"),
    )
)

spark.sql("""
CREATE TABLE IF NOT EXISTS gold.daily_revenue_by_category (
    order_date DATE,
    category STRING,
    revenue DECIMAL(12,2),
    order_count BIGINT,
    units_sold BIGINT
) USING DELTA
""")

target = DeltaTable.forName(spark, "gold.daily_revenue_by_category")
(
    target.alias("t")
    .merge(daily_revenue.alias("s"), "t.order_date = s.order_date AND
↪ t.category = s.category")
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)

# -----
↪ --
# GOLD 2: customer_lifetime_value
# "How much has each customer spent overall, and when did they last order?"
# -----
↪ --
clv = (
    spark.table("silver.orders").filter("order_status = 'PLACED'")
    .groupBy("customer_id")
    .agg(
        F.sum("line_total").alias("lifetime_revenue"),
        F.count("order_id").alias("total_orders"),
        F.max("order_date").alias("last_order_date"),
    )
)

```

```

    )
    .join(spark.table("silver.customers"), "customer_id")
    .select(
        "customer_id", "first_name", "last_name", "region",
        "lifetime_revenue", "total_orders", "last_order_date",
    )
)

spark.sql("""
CREATE TABLE IF NOT EXISTS gold.customer_lifetime_value (
    customer_id INT,
    first_name STRING,
    last_name STRING,
    region STRING,
    lifetime_revenue DECIMAL(12,2),
    total_orders BIGINT,
    last_order_date DATE
) USING DELTA
""")

target = DeltaTable.forName(spark, "gold.customer_lifetime_value")
(
    target.alias("t")
    .merge(clv.alias("s"), "t.customer_id = s.customer_id")
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)

# -----
# <- --
# GOLD 3: top_products_by_region
# "What are the best-selling products in each region?"
# -----
# <- --
top_products = (
    spark.table("silver.orders").filter("order_status = 'PLACED'")
    .join(spark.table("silver.customers").select("customer_id", "region"),
    <- "customer_id")
    .join(spark.table("silver.products"), "product_id")
    .groupBy("region", "product_id", "product_name", "category")
    .agg(
        F.sum("line_total").alias("revenue"),
        F.sum("quantity").alias("units_sold"),
    )
)

spark.sql("""

```

```

CREATE TABLE IF NOT EXISTS gold.top_products_by_region (
  region STRING,
  product_id INT,
  product_name STRING,
  category STRING,
  revenue DECIMAL(12,2),
  units_sold BIGINT
) USING DELTA
""")

target = DeltaTable.forName(spark, "gold.top_products_by_region")
(
  target.alias("t")
  .merge(
    top_products.alias("s"),
    "t.region = s.region AND t.product_id = s.product_id",
  )
  .whenMatchedUpdateAll()
  .whenNotMatchedInsertAll()
  .execute()
)

print("Gold layer refreshed.")

```

Run it, then check the results:

```

SELECT * FROM retail_lakehouse.gold.daily_revenue_by_category ORDER BY
  ⇨ order_date DESC, revenue DESC;
SELECT * FROM retail_lakehouse.gold.customer_lifetime_value ORDER BY
  ⇨ lifetime_revenue DESC LIMIT 10;
SELECT * FROM retail_lakehouse.gold.top_products_by_region ORDER BY region,
  ⇨ revenue DESC;

```

You now have three tables that a BI tool (Power BI, Tableau, or even the Databricks SQL dashboard UI) could point at directly, with no further transformation needed.

5.6 6. Phase 5 — Orchestrate It All

A pipeline you have to run cell-by-cell isn't a data engineering project — it's a notebook. Now wire bronze → silver → gold into one scheduled, monitored, self-healing **Lakeflow Job**.

5.6.1 6.1 Design the job's tasks

Task key	Type	What it does	Depends on
bronze_orders	Notebook	Runs 01_bronze_orders_autoloader	—
bronze_customers_products	Notebook	Runs 02_bronze_jdbc_customers_products	—
silver_pipeline	Pipeline	Triggers retail_silver_pipeline	bronze_orders, bronze_customers_products
gold_aggregates	Notebook	Runs 03_gold_aggregates	silver_pipeline

5.6.2 6.2 Create the job with the CLI

```

databricks jobs create --json '{
  "name": "retail_lakehouse_daily",
  "tags": {"project": "capstone", "layer": "orchestration"},
  "email_notifications": {
    "on_failure": ["you@example.com"]
  },
  "max_concurrent_runs": 1,
  "schedule": {
    "quartz_cron_expression": "0 0 6 * * ?",
    "timezone_id": "UTC",
    "pause_status": "UNPAUSED"
  },
  "tasks": [
    {
      "task_key": "bronze_orders",
      "notebook_task": {
        "notebook_path": "/Workspace/Users/<you@example.com>/retail-
↪ lakehouse/notebooks/01_bronze_orders_autoloader"
      },
      "new_cluster": {
        "spark_version": "15.4.x-scala2.12",
        "node_type_id": "Standard_DS3_v2",
        "num_workers": 1
      },
      "timeout_seconds": 1800,
      "max_retries": 2,
      "min_retry_interval_millis": 60000
    },
    {
      "task_key": "bronze_customers_products",
      "notebook_task": {
        "notebook_path": "/Workspace/Users/<you@example.com>/retail-
↪ lakehouse/notebooks/02_bronze_jdbc_customers_products"
      },

```

```

    "new_cluster": {
      "spark_version": "15.4.x-scala2.12",
      "node_type_id": "Standard_DS3_v2",
      "num_workers": 1
    },
    "timeout_seconds": 1800,
    "max_retries": 2,
    "min_retry_interval_millis": 60000
  },
  {
    "task_key": "silver_pipeline",
    "depends_on": [
      {"task_key": "bronze_orders"},
      {"task_key": "bronze_customers_products"}
    ],
    "pipeline_task": {
      "pipeline_id": "<pipeline-id-from-section-4.1.1>"
    },
    "timeout_seconds": 1800
  },
  {
    "task_key": "gold_aggregates",
    "depends_on": [
      {"task_key": "silver_pipeline"}
    ],
    "notebook_task": {
      "notebook_path": "/Workspace/Users/<you@example.com>/retail-
↪ lakehouse/notebooks/03_gold_aggregates"
    },
    "new_cluster": {
      "spark_version": "15.4.x-scala2.12",
      "node_type_id": "Standard_DS3_v2",
      "num_workers": 1
    },
    "timeout_seconds": 1800,
    "max_retries": 2,
    "min_retry_interval_millis": 60000
  }
]
}'

```

Key details worth knowing cold for the exam:

- `depends_on` is how you declare task-level dependencies inside one job — `silver_pipeline` will not start until **both** bronze tasks succeed; `gold_aggregates` waits on `silver_pipeline`.
- `quartz_cron_expression`: "`0 0 6 * * ?`" runs the job daily at 06:00 UTC. Quartz cron syntax has six fields (seconds included) — a detail the exam likes to test.

- `max_retries / min_retry_interval_millis` are set **per task**, so a transient JDBC connection blip in `bronze_customers_products` gets retried without re-running the whole job from scratch.
- `email_notifications.on_failure` fires an alert email the moment *any* task in the job fails — that’s your “failure-email alert” requirement satisfied at the job level; you can also set task-level `email_notifications` if you want different recipients per task.
- Each notebook task here uses its own small `new_cluster` (job clusters, billed only for the run’s duration) rather than an always-on interactive cluster — the right default for a scheduled production job, and much cheaper.

Grab the job ID from the create response, then run it once manually to confirm the whole DAG works end to end:

```
databricks jobs run-now --job-id <job-id>
```

Watch it in Workflows → Jobs → `retail_lakehouse_daily`. You should see all four tasks execute in the right order, with the dependency arrows drawn in the UI exactly as designed.

5.7 7. Phase 6 — Ship It with CI/CD

Running `databricks jobs create` by hand from your laptop got you a working job — but it’s not reproducible, not version-controlled, and not something a teammate (or an interviewer) could redeploy from scratch. Now package the whole project as a **Databricks Asset Bundle** and deploy it from Git, the way Part 4 taught.

5.7.1 7.1 Project layout

```
retail-lakehouse/
├─ databricks.yml
├─ resources/
│  └─ jobs.yml
│  └─ pipelines.yml
├─ notebooks/
│  └─ 01_bronze_orders_autoloader.py
│  └─ 02_bronze_jdbc_customers_products.py
│  └─ 03_gold_aggregates.py
├─ pipelines/
│  └─ silver_pipeline.py
├─ scripts/
│  └─ generate_orders.py
└─ sql/
   └─ 01_seed_source_db.sql
```

5.7.2 7.2 databricks.yml

```
# databricks.yml
bundle:
  name: retail-lakehouse

include:
  - resources/*.yml

variables:
  notebook_root:
    description: Workspace path prefix for notebooks in this bundle
    default:
      ↪ /Workspace/${workspace.current_user.userName}/.bundle/${bundle.name}/${bundle.target}

targets:
  dev:
    mode: development
    default: true
    workspace:
      host: https://adb-xxxxxxxxxxxxxxxx.xx.azuredatabricks.net
      root_path:
        ↪ /Workspace/Users/${workspace.current_user.userName}/.bundle/${bundle.name}/dev
    run_as:
      user_name: ${workspace.current_user.userName}

  prod:
    mode: production
    workspace:
      host: https://adb-xxxxxxxxxxxxxxxx.xx.azuredatabricks.net
      root_path: /Workspace/Shared/.bundle/${bundle.name}/prod
    run_as:
      service_principal_name: <your-service-principal-application-id>
    permissions:
      - level: CAN_MANAGE
        group_name: admins
```

Replace `https://adb-xxxxxxxxxxxxxxxx.xx.azuredatabricks.net` with your actual workspace URL (databricks auth profiles or your workspace's browser URL will show it). For the prod target's `run_as`, use a service principal if you set one up in Part 4's CI/CD material; if you haven't, you can temporarily comment that line out and let it default to your own user for this exercise — just note in your README that a real production deployment should run as a service principal, not a human identity.

5.7.3 7.3 resources/pipelines.yml

```
# resources/pipelines.yml
resources:
```

```

pipelines:
  retail_silver_pipeline:
    name: retail_silver_pipeline_${bundle.target}
    catalog: retail_lakehouse
    target: silver
    photon: true
    continuous: false
    libraries:
      - notebook:
          path: ../pipelines/silver_pipeline.py
    clusters:
      - label: default
        num_workers: 1
        node_type_id: Standard_DS3_v2

```

5.7.4 7.4 resources/jobs.yml

```

# resources/jobs.yml
resources:
  jobs:
    retail_lakehouse_daily:
      name: retail_lakehouse_daily_${bundle.target}
      tags:
        project: capstone
        layer: orchestration
      email_notifications:
        on_failure:
          - you@example.com
      max_concurrent_runs: 1
      schedule:
        quartz_cron_expression: "0 0 6 * * ?"
        timezone_id: UTC
        pause_status: >-
          ${bundle.target == "prod" ? "UNPAUSED" : "PAUSED"}
      tasks:
        - task_key: bronze_orders
          notebook_task:
            notebook_path: ../notebooks/01_bronze_orders_autoloader.py
            new_cluster: &small_job_cluster
            spark_version: 15.4.x-scala2.12
            node_type_id: Standard_DS3_v2
            num_workers: 1
            timeout_seconds: 1800
            max_retries: 2
            min_retry_interval_millis: 60000
        - task_key: bronze_customers_products
          notebook_task:

```

```

    notebook_path: ../notebooks/02_bronze_jdbc_customers_products.py
    new_cluster: *small_job_cluster
    timeout_seconds: 1800
    max_retries: 2
    min_retry_interval_millis: 60000

- task_key: silver_pipeline
  depends_on:
    - task_key: bronze_orders
    - task_key: bronze_customers_products
  pipeline_task:
    pipeline_id: ${resources.pipelines.retail_silver_pipeline.id}
    timeout_seconds: 1800

- task_key: gold_aggregates
  depends_on:
    - task_key: silver_pipeline
  notebook_task:
    notebook_path: ../notebooks/03_gold_aggregates.py
    new_cluster: *small_job_cluster
    timeout_seconds: 1800
    max_retries: 2
    min_retry_interval_millis: 60000

```

Two bundle-specific tricks worth calling out for the exam:

- `${resources.pipelines.retail_silver_pipeline.id}` lets the job task reference the *bundle-managed* pipeline’s ID directly — you never have to hardcode or hunt down a pipeline ID by hand; the bundle wires it for you at deploy time.
- The `pause_status` expression pauses the schedule automatically in dev (so your personal testing deploys don’t quietly start running on a cron) while keeping it `UNPAUSED` in prod — a clean, one-file way to express “same job definition, different behavior per environment,” which is exactly what dev/prod targets are for.

5.7.5 7.5 Validate and deploy

```
cd retail-lakehouse
```

```

databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run retail_lakehouse_daily -t dev

```

`validate` catches YAML/reference errors before anything touches your workspace. `deploy` uploads the notebooks/pipeline source and creates (or updates) the job and pipeline objects in the dev target’s workspace path. `run` triggers an on-demand run of the freshly deployed job, exactly like `jobs run-now` but bundle-aware.

When you’re satisfied dev works, promote to prod the same way:

```
databricks bundle validate -t prod
databricks bundle deploy -t prod
```

5.7.6 7.6 Commit it to Git

```
git init
git add .
git commit -m "Capstone: retail lakehouse bundle (bronze/silver/gold +
  ↳ orchestration)"
git branch -M main
git remote add origin
  ↳ https://github.com/<your-username>/retail-lakehouse.git
git push -u origin main
```

From here, Part 4’s CI/CD material applies directly: a GitHub Actions workflow that runs `databricks bundle validate` and `databricks bundle deploy -t dev` on every pull request, and `deploy -t prod` on merge to main, turns this from “a project I deployed once” into “a project with a real deployment pipeline” — exactly what you want to be able to point to in an interview. (Don’t forget to add a `.gitignore` excluding `.venv/`, `data/orders_landing/`, and any files containing secrets — never commit the Postgres password; it only ever belongs in the Databricks secret scope you created in section 3.3.)

5.8 8. Phase 7 — Monitor & Optimize

5.8.1 8.1 Check job run history and cost with system tables

Unity Catalog’s **system tables** — schemas under the system catalog that Databricks maintains automatically — let you monitor everything you just built with plain SQL, no separate tooling required. Confirm system tables are enabled for your metastore (usually on by default in a Premium workspace with Unity Catalog); if a query below returns “table not found,” an account admin needs to enable the relevant system schema once via the account console (Settings → System schemas).

-- Every run of every task in your job, most recent first

```
SELECT
  job_id,
  run_id,
  task_key,
  period_start_time,
  period_end_time,
  result_state
FROM system.lakeflow.job_task_run_timeline
WHERE job_id = '<job-id>'
ORDER BY period_start_time DESC
LIMIT 25;
```

```

-- Roll up run outcomes to spot flaky tasks
SELECT
    task_key,
    result_state,
    count(*) AS run_count
FROM system.lakeflow.job_task_run_timeline
WHERE job_id = '<job-id>'
GROUP BY task_key, result_state
ORDER BY task_key;

-- What did this job actually cost in DBUs over the last 7 days?
SELECT
    usage_date,
    sku_name,
    sum(usage_quantity) AS total_dbus
FROM system.billing.usage
WHERE usage_metadata.job_id = '<job-id>'
    AND usage_date >= current_date() - INTERVAL 7 DAYS
GROUP BY usage_date, sku_name
ORDER BY usage_date DESC;

```

system.billing.usage is the table the exam expects you to reach for whenever a scenario asks “how would you find out what this pipeline is costing us” — know its name and that usage_metadata carries job/pipeline/cluster identifiers you can filter and group by.

5.8.2 8.2 Optimize the busiest gold table

gold.daily_revenue_by_category is the table your MERGE touches on every run and the one a BI dashboard would query constantly, filtered heavily by order_date and grouped by category — a textbook OPTIMIZE ... ZORDER BY candidate.

```

OPTIMIZE retail_lakehouse.gold.daily_revenue_by_category
ZORDER BY (order_date, category);

```

```

-- Confirm the file layout improved
DESCRIBE DETAIL retail_lakehouse.gold.daily_revenue_by_category;

```

OPTIMIZE compacts the table’s small files (an inevitable side effect of frequent small MERGE operations — every incremental run can leave behind more small files than a batch write would) into fewer, larger ones; ZORDER BY (order_date, category) additionally co-locates rows with similar values in those two columns on disk, so a query like WHERE order_date = '2026-08-27' AND category = 'Electronics' skips far more data via Delta’s file statistics than it could on an un-optimized table. Worth also setting up **predictive optimization** (covered in Part 3) at the catalog or table level so this happens automatically going forward rather than requiring you to remember to run it by hand:

```

ALTER TABLE retail_lakehouse.gold.daily_revenue_by_category

```

```
SET TBLPROPERTIES (delta.autoOptimize.optimizeWrite = true);
```

5.8.3 8.3 What you'd tune next at real scale

Be ready to talk through this list in an interview — it signals you understand this project is a scaled-down teaching model, not a finished production system:

- **Partition the bronze/silver order tables by order_date** once volumes grow past tens of millions of rows, so OPTIMIZE and query pruning operate on a manageable number of files per partition instead of one flat table.
 - **Move the source database connection to a private network path** (VNet injection + private endpoint) instead of the public-access firewall rule used in Phase 1 — appropriate for a learning exercise, not for a real customer database.
 - **Switch the customers/products JDBC ingestion from full overwrite to incremental (updated_at-watermarked) reads** once those tables are too large to reasonably re-read in full on every run — the MERGE pattern from Phase 4 applies directly.
 - **Add a proper streaming trigger** (trigger(processingTime="5 minutes") with a continuous job cluster, or a Lakeflow Job scheduled far more frequently) if the business genuinely needs near-real-time order visibility rather than the daily batch cadence this project uses.
 - **Right-size and autoscale job clusters** based on the system-table cost data from section 8.1 rather than the fixed single-worker clusters used throughout this chapter, which are deliberately minimal to keep your learning costs low.
 - **Add pipeline-level alerting on expectation failure rates**, not just job-level failure emails — a silver table that's silently dropping 40% of incoming orders due to a bad expectation is a very different problem from a task that failed outright, and deserves its own alert threshold.
-

5.9 9. What to Put on Your Resume and GitHub

5.9.1 9.1 Resume bullet points

Pick two or three of these, adapt the specifics to what you actually built, and never claim a number you can't defend if asked about it in an interview:

- Designed and built an end-to-end lakehouse on Azure Databricks, ingesting data from a relational Azure PostgreSQL source and a simulated streaming JSON feed into a governed, three-layer (bronze/silver/gold) Unity Catalog architecture.
- Implemented automated data-quality enforcement using Lakeflow Declarative Pipelines with column- and table-level expectations, reducing malformed records reaching downstream analytics tables to zero for enforced fields.
- Applied Unity Catalog fine-grained governance — column masking on PII fields and row-level security filters — to enforce least-privilege access for analyst roles without duplicating data.

- Orchestrated a four-stage, dependency-aware pipeline with Lakeflow Jobs (scheduled, retried, and alerting on failure) and deployed it via Databricks Asset Bundles with separate dev/prod CI/CD targets from GitHub.
- Reduced query latency on high-traffic gold aggregate tables through file compaction and Z-order clustering (OPTIMIZE/ZORDER), and used Unity Catalog system tables to monitor job run health and compute cost on an ongoing basis.

5.9.2 9.2 Suggested GitHub README outline

Structure your public repo's README.md like this — it's what a hiring manager or interviewer will actually open first:

Retail Lakehouse on Azure Databricks

One-paragraph summary: what the project is, what business problem it solves, and the stack (Azure Databricks, Unity Catalog, Delta Lake, Lakeflow Declarative Pipelines, Lakeflow Jobs, Databricks Asset Bundles).

Architecture

(Paste/link the architecture diagram from section 1.2, or a screenshot of the job's DAG from the Databricks Workflows UI.)

What it does

- Ingests customer/product data from Azure PostgreSQL and simulated order-event streams from cloud storage
- Cleans, deduplicates, and enforces data quality with declarative pipeline expectations
- Applies Unity Catalog governance (column masking, row filters)
- Produces three business-ready gold tables via incremental MERGE
- Orchestrates the whole pipeline on a schedule with retries and failure alerting
- Deploys via CI/CD using Databricks Asset Bundles (dev/prod targets)

Repo structure

(Short description of each top-level folder – notebooks/, pipelines/, resources/, scripts/, sql/.)

How to run it

Link back to this chapter's steps, or summarize the CLI commands:

```
`databricks bundle deploy -t dev` / `databricks bundle run
  ↪ retail_lakehouse_daily -t dev`
```

Sample output

(A screenshot or markdown table of a few rows from each gold table.)

What I'd do differently at scale

(Adapt section 8.3 of this chapter – this section is a strong signal to

reviewers that you understand production tradeoffs, not just the happy
↪ path.)

Include a couple of real screenshots — the Workflows DAG view, a pipeline’s expectation metrics, and a query result from one of the gold tables — a repo with visual proof of a working system reads very differently from one with code alone.

5.9.3 9.3 Likely interview questions about this project, with model answers

“Walk me through how a single order event flows from source to a dashboard-ready number in your project.”

An order event starts as a JSON file written by the order generator into an ADLS Gen2 landing volume, simulating what a checkout service would emit. Auto Loader picks it up incrementally — it tracks which files it’s already processed so re-running never double-counts — and lands it as-is into a bronze Delta table. A Lakeflow Declarative Pipeline then reads that bronze table, applies data-quality expectations (dropping rows with a null order ID or a negative quantity, for example), deduplicates on `order_id`, and writes the result to a silver table alongside similarly cleaned customer and product dimension tables. From there, a notebook task joins silver orders to the product dimension, aggregates revenue by day and category, and MERGEs the result into a gold table incrementally, so only the days that actually changed get rewritten. All four stages run as tasks in one Lakeflow Job with explicit dependencies, so silver never starts before both bronze sources finish, and gold never starts before silver is validated.

“How did you handle a scenario where the source data was messy or incomplete?”

I intentionally built dirtiness into my test data — a small percentage of order events with a missing order ID, a negative quantity, or a customer ID that doesn’t exist in the customer dimension — specifically so I could design real handling for it rather than assuming clean input. I used two different expectation strategies depending on severity: `expect_or_drop` for conditions that make a row genuinely unusable in silver, like a null primary key, so those rows are removed but the pipeline keeps running and the drop is logged; and a plain `expect` for conditions I want visibility into without losing data, like an order referencing a customer ID that might just not have synced yet, so those rows stay in the table but get counted so I can monitor the failure rate over time. If a rate spiked unexpectedly, that’s exactly the kind of thing I’d wire an alert onto rather than only alerting on outright task failure.

“How would you extend this if the company started generating 100x the order volume, or wanted near-real-time reporting?”

A few changes, roughly in the order I’d make them. First, I’d partition the order tables by `order_date` so `OPTIMIZE` and downstream queries prune files instead of scanning a flat table, and I’d rely on predictive optimization to keep file layout healthy automatically rather than running `OPTIMIZE` by hand. Second, I’d move the bronze ingestion from a scheduled `trigger(availableNow=True)` batch to a continuously running Auto Loader stream with a short `processingTime` trigger, and correspondingly increase

the pipeline schedule's frequency — the architecture doesn't change, just the trigger and cadence. Third, at genuinely high volume I'd stop doing a full-table JDBC read for customers and products and switch to a watermark-based incremental read plus MERGE, the same pattern I already use for the gold layer, rather than overwriting those tables wholesale on every run. And I'd lean harder on the system tables I already used for monitoring — `system.billing.usage` and the job/pipeline run-timeline tables — to catch cost or latency regressions before they became a problem, rather than after.

Chapter 6

Part 6 — Exam Practice Question Bank

A note on academic integrity before you start. Every question in this chapter was written from scratch for this book, modeled on the publicly published DP-750 skills-measured outline (compute setup, Unity Catalog, data engineering with Delta Lake/Lakeflow, and pipeline deployment/monitoring). None of these questions is copied or paraphrased from a real exam, a leaked question bank, or a “dump” site. Using leaked or braindumped questions instead of genuine practice would violate the Microsoft Certification Exam NDA you agree to at check-in and can lead to score cancellation or a certification ban — so treat any site claiming to have “real DP-750 questions” as a risk to your certification, not a shortcut. Everything below is here to build the same skills the real exam tests, not to memorize.

Work through each domain section in order, cover the answer before you commit to a choice, and read every explanation — including for questions you got right, since the distractor analysis often contains a second fact worth knowing. The final section is a 20-question mock exam formatted for timed, closed-book practice.

6.1 Domain 1 Practice Questions — Set Up and Configure an Azure Databricks Environment

Q1. Your team’s data scientists each need their own interactive cluster to run ad hoc PySpark and pandas exploration in notebooks. Each person will only work with data they personally have access to, and no other user should be able to attach to another person’s cluster. Which cluster access mode should you configure?

- A. Shared (Unity Catalog enabled), configured for the whole team
- B. Single user, one cluster per data scientist
- C. No isolation shared
- D. High Concurrency without credential passthrough

Correct answer: B Explanation: Single user access mode assigns a cluster to exactly one user (or one service principal) and enforces that only that identity can attach to it, while still supporting Unity Catalog governance and all languages including Scala. Shared clusters (option A) are designed for multiple concurrent users and would let teammates attach to the same cluster, which violates the isolation requirement. “No isolation shared” is a legacy mode that does not enforce Unity Catalog data governance at all and is disallowed in most governed workspaces.

Q2. You are a workspace admin at a company that wants to prevent analysts from spinning up oversized, expensive clusters while still letting them choose a runtime version and enable autoscaling. Which Azure Databricks feature should you use to enforce this?

- A. A storage credential
- B. A cluster policy
- C. An external location
- D. A Unity Catalog schema-level grant

Correct answer: B Explanation: Cluster policies let an admin define rules and limits (allowed VM types/families, max autoscale workers, allowed Databricks Runtime versions, DBU/hour caps, tagging requirements) that constrain what users can configure when creating a cluster, while still exposing a simplified UI for the fields they are allowed to set. Storage credentials and external locations are Unity Catalog objects for governing access to cloud storage, not for constraining compute size, and a schema-level grant controls data access, not cluster shape.

Q3. Which of the following are true about Azure Databricks serverless compute for notebooks and jobs? (Select two.)

- A. Serverless compute requires you to select a VM instance type and node count before it will start
- B. Serverless compute starts in seconds because Databricks manages a pool of pre-warmed, elastic capacity behind the scenes
- C. Serverless compute still runs inside a customer-managed VNet with cluster nodes visible in the Azure subscription
- D. Serverless compute removes the need to size or manage clusters — Databricks handles instance selection and scaling automatically
- E. Serverless compute cannot be governed by Unity Catalog

Correct answer: B, D Explanation: Serverless compute is a fully managed offering where Databricks provisions and scales the underlying infrastructure in its own compute plane, so users never pick VM types or node counts, and startup is typically near-instant because capacity is kept warm. It is fully compatible with Unity Catalog governance (in fact modern serverless SQL warehouses and serverless jobs enforce Unity Catalog), and unlike classic compute its nodes are not visible or manageable as VMs in the customer’s own Azure subscription.

Q4. A workspace has been attached to a Unity Catalog metastore. In what order, from broadest to narrowest, does the Unity Catalog object hierarchy nest?

- A. Catalog → Metastore → Schema → Table
- B. Metastore → Catalog → Schema → Table/View/Volume/Function
- C. Schema → Catalog → Metastore → Table
- D. Workspace → Metastore → Table → Schema

Correct answer: B Explanation: A Unity Catalog metastore is the top-level container for metadata (one per region, attachable to one or more workspaces); it contains catalogs, each catalog contains schemas (databases), and each schema contains the actual data objects — tables, views, volumes, functions, and models. Understanding this hierarchy is essential because grants, default locations, and object names (a three-level namespace catalog.schema.table) all follow it.

Q5. You create a table with `CREATE TABLE sales.transactions.orders (...) USING DELTA`; without specifying a `LOCATION`. What kind of table is this, and what happens to its underlying data files if you run `DROP TABLE sales.transactions.orders`;

- A. It is an external table; the data files remain in cloud storage after the drop
- B. It is a managed table; the data files are deleted along with the metadata
- C. It is a managed table; the data files remain until manually deleted
- D. It is an external table; dropping the table is blocked unless `CASCADE` is specified

Correct answer: B Explanation: When no `LOCATION` is given, Unity Catalog creates a **managed table**, storing its data in the managed storage location associated with the schema/catalog/metastore; Unity Catalog owns the full lifecycle of that data, so `DROP TABLE` deletes both the metadata and the underlying data files. An **external table** is created with an explicit `LOCATION` pointing at an external location, and dropping it only removes the metastore registration — the files themselves stay in place, which is the key operational difference tested on the exam.

Q6. Which two Unity Catalog objects must exist before you can create an external table pointing at a path in an Azure Data Lake Storage Gen2 container that Unity Catalog has never accessed before? (Select two.)

- A. A storage credential that lets Unity Catalog authenticate to the storage account (e.g., via a managed identity)
- B. A Delta Sharing recipient
- C. An external location that binds a storage path to a storage credential and grants permission to use it
- D. A cluster policy scoped to that storage account
- E. A row filter function

Correct answer: A, C Explanation: A **storage credential** encapsulates the authentication mechanism (typically an Azure managed identity) Unity Catalog uses to reach a storage account, and an **external location** combines a storage credential with a specific container/path and is the object you actually grant `READ FILES/WRITE FILES/CREATE EXTERNAL TABLE` permissions on. Delta Sharing recipients, cluster policies, and row filters are unrelated to authorizing raw storage access for external tables.

Q7. A data engineer needs to land raw, unstructured PDF files and images in a gov-

erned location so that PySpark code and other tools can read them by path, without registering the files as rows in a table. Which Unity Catalog object type is designed for this?

- A. A managed table
- B. A volume
- C. A view
- D. A Delta Sharing share

Correct answer: B Explanation: Volumes are the Unity Catalog object for governing access to non-tabular data — arbitrary files such as PDFs, images, CSVs to be loaded later, or model artifacts — accessible by path (`/Volumes/catalog/schema/volume/...`) with full governance and audit logging, and they come in managed or external flavors just like tables. Tables and views model structured, row-based data and cannot hold arbitrary files; a Delta Sharing share is for distributing data to another organization or metastore, not for local file storage.

Q8. Your organization’s compliance team requires that every job cluster used for production ETL be created only from an approved, fixed list of VM instance types and Databricks Runtime LTS versions, and that autoscaling never exceed 8 workers. What is the single most effective way to enforce this consistently across every new job that is created?

- A. Ask every job author to manually type the correct configuration in the cluster JSON each time
- B. Create a cluster policy encoding these limits and require jobs to use it
- C. Set an Azure Resource Manager quota on the subscription
- D. Restrict the workspace to only Single User access mode

Correct answer: B Explanation: A cluster policy is the purpose-built control for standardizing and capping cluster configuration (instance types, runtime versions, max autoscale workers) and can be made mandatory for job clusters, removing the risk of human error from manual configuration. An Azure subscription quota limits total consumed cores across the subscription but cannot express “only these SKUs, only these runtimes,” and access mode governs data isolation, not instance type or autoscaling limits.

Q9. Which statement correctly compares “No Isolation Shared” access mode with “Shared (Unity Catalog)” access mode on a classic Azure Databricks cluster?

- A. Both modes fully enforce Unity Catalog table and row/column-level security for every user on the cluster
- B. No Isolation Shared allows multiple users to attach and share the cluster but does not enforce Unity Catalog data governance, while Shared access mode supports multiple concurrent users with per-user process isolation and full Unity Catalog enforcement
- C. No Isolation Shared is the only mode that supports Python, Scala, and SQL simultaneously
- D. Shared access mode can only be used by exactly one user at a time, like Single User mode

Correct answer: B **Explanation:** “No Isolation Shared” (the legacy high-concurrency-style mode) lets multiple users attach to one cluster but cannot guarantee user isolation and is therefore blocked from enforcing Unity Catalog fine-grained governance in most configurations; “Shared” access mode is the modern, UC-governed multi-user mode that isolates each user’s code execution and correctly enforces catalog, schema, table, row, and column-level permissions. Both modes support multiple languages, so option C is not a real distinguishing factor.

Q10. A finance analyst needs to run occasional interactive SQL queries against gold-layer Delta tables using a BI tool connected over ODBC, with sub-second-to-a-few-seconds query latency and automatic scaling to handle bursts of concurrent dashboard refreshes, while minimizing idle cost. What compute should you provision?

- A. A Single User all-purpose cluster
- B. A Databricks SQL warehouse (ideally serverless), with auto-stop enabled
- C. A job cluster attached to a scheduled Lakeflow Job
- D. A shared all-purpose cluster running Databricks Runtime for Machine Learning

Correct answer: B **Explanation:** SQL warehouses are purpose-built for BI/SQL workloads: they expose a JDBC/ODBC endpoint, support multi-cluster autoscaling for concurrency bursts, and — especially in the serverless flavor — start almost instantly and auto-stop when idle, minimizing cost for intermittent analyst usage. An all-purpose cluster is priced and sized for notebook development work and lacks the query-routing/autoscaling behavior of a warehouse, and a job cluster is ephemeral compute tied to a scheduled job run, not an always-available interactive endpoint.

Q11. True or False style — Select all statements about Unity Catalog metastores that are correct. (Select two.)

- A. A single Unity Catalog metastore can be assigned to multiple workspaces within the same Azure region
- B. Each Azure region can have any number of metastores, and a workspace can be attached to more than one metastore simultaneously
- C. Identities (users, groups, service principals) and grants defined at the metastore level can apply consistently across every workspace attached to that metastore
- D. A metastore must be recreated separately for every catalog you create
- E. Table and file access audit logs are stored only inside the workspace, never at the metastore/account level

Correct answer: A, C **Explanation:** One of Unity Catalog’s core value propositions is that a single metastore (normally one per region) can be attached to many workspaces, giving those workspaces a shared, consistent view of catalogs, permissions, and identities — a user granted access in one workspace has that same access from any other workspace sharing the metastore. A workspace can only be attached to **one** metastore at a time, catalogs are created inside an existing metastore rather than requiring new metastores, and Unity Catalog audit/access logs are captured centrally via system tables at the account/metastore level, not siloed per workspace.

Q12. During initial Unity Catalog setup, an admin creates a new catalog named `bronze_raw` without specifying a managed storage location for the catalog. Where will managed tables created inside schemas of this catalog store their data by default?

- A. They cannot be created until a catalog-level location is explicitly set
- B. In the default managed storage location configured at the metastore level (or an inherited schema-level location, if set)
- C. In the DBFS root of the workspace that created the catalog
- D. In a randomly chosen storage account picked by Unity Catalog

Correct answer: B Explanation: Managed storage locations can be set at the metastore, catalog, or schema level, and they are inherited downward — if a catalog does not specify its own managed location, its schemas’ managed tables fall back to the metastore’s default managed location (or a schema-level override if one is set), rather than failing outright. Storing managed data in the workspace’s legacy DBFS root is discouraged and is not the Unity Catalog default behavior; it does not fall back to DBFS automatically.

Q13. Which two actions should you take to give a group of five new data engineers standardized, cost-controlled interactive compute for daily development work, in line with Azure Databricks best practices? (Select two.)

- A. Give each engineer full “Can Manage” permission on an unrestricted all-purpose cluster
- B. Create a cluster policy that caps VM size, runtime version, and auto-termination, and grant the engineers’ group “Can Use” permission on that policy
- C. Have each engineer create clusters from scratch with no policy applied
- D. Enable autoscaling within the policy-defined bounds and set a short auto-termination (inactivity) timeout
- E. Require every engineer to share a single Single User cluster simultaneously

Correct answer: B, D Explanation: Best practice is to define a cluster policy that constrains cost-relevant settings and standardizes the runtime, then grant the team “Can Use” on that policy so they can self-serve compact clusters within guardrails; combining that with autoscaling and a short auto-termination timeout controls idle spend automatically. Giving unrestricted “Can Manage” access invites cost overruns and configuration drift, and Single User clusters are — by definition of that access mode — restricted to one attached identity, so multiple engineers cannot share one.

6.2 Domain 2 Practice Questions — Secure and Govern Unity Catalog Objects

Q14. You want to grant every member of the analysts Entra ID group read access to all current and future tables in the `gold` schema of the sales catalog, without having to re-run a grant every time a new table is added. Which statement accomplishes this

most efficiently?

- A. `GRANT SELECT ON TABLE sales.gold.* TOanalysts;`
- B. `GRANT SELECT ON SCHEMA sales.gold TOanalysts;`
- C. Manually run a `GRANT SELECT` statement on each existing table, and repeat it for every new table created
- D. `GRANT USAGE ON CATALOG sales TOanalysts;`

Correct answer: B Explanation: Granting `SELECT` at the **schema** level applies to all tables and views currently in that schema and automatically extends to any new ones created later, because Unity Catalog privileges are inherited down the object hierarchy — this is the efficient, low-maintenance pattern the exam expects you to recognize. Wildcard syntax like `sales.gold.*` is not valid grant syntax, and granting only `USAGE` on the catalog lets the group traverse the namespace but does not by itself grant `SELECT` rights on the tables inside.

Q15. A nightly Lakeflow Job needs to write to a Unity Catalog table with no human interactively signing in, and its identity must be auditable and independently revocable if compromised, without disabling any human user’s account. What should you use to run the job?

- A. A shared personal access token belonging to the data engineering team lead
- B. A service principal registered in Microsoft Entra ID, added to the workspace and granted the needed Unity Catalog privileges
- C. The workspace admin’s own Azure Databricks login credentials embedded in the job
- D. A group named “automation” containing every engineer’s personal account

Correct answer: B Explanation: A **service principal** is a non-human Entra ID identity purpose-built for automation: it can be granted precisely the Unity Catalog privileges the job needs, its activity is separately audited in system tables, and it can be disabled or have its credentials rotated without touching any human account. Using a personal token or a human’s login for automation is an anti-pattern the exam explicitly tests against, because it ties production automation to an individual’s identity lifecycle and defeats least-privilege auditing.

Q16. An engineer who owns table `finance.reporting.revenue` runs `REVOKE SELECT ON TABLE finance.reporting.revenue FROMcontractors;` after previously granting it. What is the effect if the `contractors` group also independently inherited `SELECT` on `finance.reporting.revenue` through a grant on the parent schema `finance.reporting`?

- A. The revoke fails with an error because of the conflicting inherited grant
- B. The group loses direct table-level access, but members can still query the table because of the schema-level grant that still applies
- C. The schema-level grant is automatically revoked too, cascading upward
- D. Unity Catalog ignores schema-level grants entirely, so table-level revokes are always sufficient

Correct answer: B Explanation: Unity Catalog privileges are additive across the hierarchy — a principal effectively has access if it holds a grant at *any* level (table,

schema, or catalog) that covers the object — so revoking a table-level grant removes only that specific grant and does not touch a separately-issued schema-level grant, meaning the contractors group can still `SELECT` the table until the schema-level grant is also revoked. This is a common real-world gotcha: to fully remove access, you must revoke at every level where a grant was made, not just the most specific one.

Q17. Which mechanism should you use to ensure that when the `support_agents` group queries the `customers` table, they see the email column with only the first two characters visible (e.g., `jo***@***.com`), while the `data_engineers` group sees the full value?

- A. A row filter
- B. A column mask implemented as a SQL function and applied with `ALTER TABLE ... ALTER COLUMN email SET MASK`
- C. A Delta Sharing recipient-level restriction
- D. A cluster policy limiting which columns can be selected

Correct answer: B Explanation: A **column mask** is a SQL user-defined function that Unity Catalog applies transparently at query time to transform (redact, hash, or partially obscure) a column's value based on the querying principal's group membership, which is exactly this partial-redaction requirement; you attach it to a column with `ALTER TABLE ... ALTER COLUMN <col> SET MASK <function>`. A row filter instead controls which *entire rows* are visible/hidden based on a predicate, not how an individual column's value is displayed, so it cannot achieve partial masking on its own.

Q18. A retail company wants regional managers to query the same `orders` table but have each manager automatically see only rows for their own region, based on their group membership, with no separate physical copy of the table per region. Which Unity Catalog feature is designed for exactly this?

- A. Column mask
- B. Row filter
- C. Delta Sharing share
- D. Table clone

Correct answer: B Explanation: A **row filter** is a SQL function attached to a table (via `ALTER TABLE ... SET ROW FILTER`) that Unity Catalog evaluates for every query, dynamically restricting which rows a given principal can see — a standard way to implement this is a function that checks `is_member('region_east')`-style group membership against a row's region column. Column masks redact specific field values rather than filtering whole rows, a Delta Sharing share distributes data externally rather than restricting rows for internal users, and a table clone would create a static, disconnected copy rather than dynamic per-user filtering.

Q19. Where would you look to answer the question “which user ran a `DROP TABLE` command against `finance.reporting.revenue` last week, and from which IP address?”

- A. The Spark UI's SQL tab on the cluster that ran the command

- B. The `system.access.audit` table (Unity Catalog audit logs exposed as a system table)
- C. The workspace's cluster event log
- D. The Delta table's transaction log (`_delta_log`)

Correct answer: B Explanation: Unity Catalog audit logs — including who performed an action, what action, on what object, when, and from where — are exposed through **system tables** under the `system.access.audit` schema, which you can query with ordinary SQL once system tables are enabled for the metastore. The Delta transaction log records data/schema changes to that one table for ACID and time travel purposes, but it does not record the identity or source IP of the user who issued the command, and cluster event logs cover cluster lifecycle events (start/stop/resize), not data-plane SQL commands.

Q20. A governance team wants a visual, auto-generated map showing that column `customers.raw.ssn` flows into `customers.silver.customer_clean.ssn_masked` via a specific notebook, and from there into a downstream dashboard. What Unity Catalog capability provides this without any manual documentation effort?

- A. Tags
- B. Automatically captured data lineage, viewable in Catalog Explorer as a lineage graph down to the column level
- C. A Delta Sharing share definition
- D. A row filter's predicate definition

Correct answer: B Explanation: Unity Catalog automatically captures **lineage** for tables and, at a finer grain, columns as data moves through notebooks, Lakeflow Jobs/pipelines, and dashboards — with no manual tagging required — and surfaces it as an interactive graph in Catalog Explorer, plus it is queryable via `system.access.table_lineage` / `column_lineage` system tables. Tags are static key/value labels you apply for classification/search, not a record of data flow, and a row filter only governs row visibility.

Q21. Your company wants to share three curated Delta tables with an external partner organization that does not have an Azure Databricks account, using an open, vendor-neutral protocol, without copying the data into the partner's own storage. What should you set up?

- A. A Unity Catalog external location granted directly to the partner's Azure AD tenant
- B. A Delta Sharing share, containing the tables, with an open-sharing recipient (or a Databricks-to-Databricks recipient if they do have an account)
- C. A managed table clone exported to a shared storage account
- D. A row filter that lets external IP ranges query the table directly

Correct answer: B Explanation: **Delta Sharing** is the open protocol purpose-built for this scenario: a provider creates a **share** (a named collection of tables/volumes/notebooks), adds a **recipient**, and the recipient — even without any Databricks account, using the open-sharing credential file and any Delta Sharing-compatible client — can read the data directly from cloud storage without the

provider copying or duplicating it. Granting an external location directly to another organization's tenant is not how Unity Catalog cross-org sharing works and would require them to be a principal in your own metastore, which open sharing is specifically designed to avoid.

Q22. Which of the following are valid uses of tags in Unity Catalog governance? (Select two.)

- A. Applying a `pii=true` tag to a column so it can be discovered and audited for compliance reporting
- B. Using a tag as the sole enforcement mechanism to physically block a query, with no grant or row filter involved
- C. Applying tags at the catalog, schema, table, or column level to support classification, search, and downstream policy automation
- D. Tags replace the need for GRANT/REVOKE entirely
- E. Tags are immutable once applied and can never be changed or removed

Correct answer: A, C **Explanation:** Tags are key-value metadata labels you can attach at multiple levels of the Unity Catalog hierarchy (catalog, schema, table, column, and more) to support discovery, classification (e.g., marking PII), and to drive external governance tooling or automated policies. Tags themselves are descriptive metadata, not an access-control enforcement mechanism — you still need GRANT/REVOKE, row filters, or column masks to actually restrict access — and tags can be freely added, changed, or removed by users with the right permission.

Q23. A metastore admin wants to start querying billing, audit, and lineage data using SQL. What must typically happen first before `system.billing.usage` and similar tables return rows?

- A. Nothing — all system schemas are enabled and populated by default the moment a metastore is created
- B. The relevant system schema (e.g., access, billing, lineage) must be enabled for the metastore, generally by an account/metastore admin, after which the tables begin populating
- C. Every workspace must individually export CSV logs to a storage account
- D. System tables can only be queried through the Azure portal, never through SQL

Correct answer: B **Explanation:** System tables live in the reserved system catalog but are organized into schemas (`system.access`, `system.billing`, `system.lineage`, `system.compute`, etc.) that must be explicitly **enabled** (historically via a Unity Catalog API/CLI call, or through account console toggles) before Databricks begins populating them for that metastore — they are not all on by default. Once enabled, they are ordinary Delta tables queryable with standard SQL from any workspace attached to the metastore, not something requiring manual CSV export.

Q24. A contractor's Unity Catalog access needs to be revoked immediately when their contract ends, without disrupting any other user, and the change should take effect without redeploying any pipeline code. What is the most appropriate approach?

- A. Rewrite every pipeline's source code to remove references to the contractor
- B. Revoke the specific grants held by the contractor's user account (or remove them from the groups those grants were made to), which Unity Catalog enforces immediately for future queries
- C. Delete the entire catalog the contractor had access to
- D. Rotate the storage account access keys for the whole environment

Correct answer: B Explanation: Because Unity Catalog centralizes authorization as metadata-level grants evaluated at query time, revoking a user's grants (directly, or by removing them from a group that holds the grants) takes effect immediately for any future query with no code changes or redeployment needed. Deleting an entire catalog or rotating storage keys are drastic, blunt actions that would disrupt every legitimate user and pipeline relying on that data, not a targeted fix.

Q25. Select the two statements that correctly describe securable object ownership in Unity Catalog. (Select two.)

- A. Every securable object (catalog, schema, table, view, volume, function, model) has an owner, who by default can grant privileges on it to other principals
- B. Ownership can be transferred to another user, group, or service principal by someone with sufficient privilege (e.g., `ALTER ... OWNER TO`)
- C. Only the account's global admin can ever be the owner of any object
- D. Ownership has no bearing on who can grant access — only explicit `GRANT` statements matter, regardless of ownership
- E. An object can have zero owners if the creator's account is later deleted

Correct answer: A, B Explanation: Every Unity Catalog securable has an owner (initially the creator, by default), and the owner inherently has full management rights on the object including the ability to grant privileges to others, transfer ownership, and drop the object; ownership can be explicitly reassigned to a user, group, or service principal, which is a common practice to avoid tying critical production objects to one individual's account. Ownership is not restricted to a global admin, and it very much matters for authorization — an owner can act on an object even without an explicit `GRANT` to themselves — so options C and D are both incorrect.

Q26. Select the two objects below that are **securable objects you can directly GRANT privileges on** in Unity Catalog. (Select two.)

- A. A cluster policy
- B. A storage credential
- C. An external location
- D. A VNet peering connection
- E. A Databricks Runtime version

Correct answer: B, C Explanation: Storage credentials and external locations are first-class Unity Catalog securable objects, and admins grant privileges on them (e.g., `CREATE EXTERNAL LOCATION`, `CREATE EXTERNAL TABLE`, `READ FILES`) to control who can register or use paths in cloud storage. Cluster policies use their own separate "Can Use" permission model rather than Unity Catalog `GRANT` syntax, and VNet peering and

runtime versions are infrastructure/platform concepts with no Unity Catalog grant surface at all.

6.3 Domain 3 Practice Questions — Prepare and Process Data

Q27. What does the Delta Lake transaction log (`_delta_log`) primarily provide that a plain Parquet table on ADLS Gen2 does not?

- A. Automatic compression of Parquet files
- B. ACID transactions, a versioned history of all commits, and the ability to reconstruct the table's state at any past version
- C. Faster network throughput when reading from storage
- D. Automatic partitioning of the underlying files

Correct answer: B Explanation: The `_delta_log` is an ordered record of JSON (and periodically Parquet checkpoint) commit files that describe every add/remove file operation and schema change made to the table, which is what gives Delta Lake atomic, consistent, isolated, durable (ACID) transactions and lets you time-travel to any historical version — none of which plain Parquet files provide on their own, since Parquet alone has no shared transaction metadata. Compression and network throughput are unrelated properties of the storage/file format, not the transaction log, and partitioning is a table-design choice you make independently of Delta's ACID guarantees.

Q28. Which Spark SQL statement correctly queries the version of the `sales.gold.orders` table as it existed exactly at `2026-08-01 00:00:00`?

- A. `SELECT * FROM sales.gold.orders HISTORY '2026-08-01';`
- B. `SELECT * FROM sales.gold.orders TIMESTAMP AS OF '2026-08-01 00:00:00';`
- C. `SELECT * FROM sales.gold.orders SNAPSHOT '2026-08-01 00:00:00';`
- D. `RESTORE TABLE sales.gold.orders TO TIMESTAMP AS OF '2026-08-01';`

Correct answer: B Explanation: Delta Lake's time travel syntax lets you query a historical state using either `VERSION AS OF <version_number>` or `TIMESTAMP AS OF '<timestamp>'` directly in a `SELECT`, and B uses the correct clause and placement. `DESCRIBE HISTORY` (not `HISTORY`) is the command to list past versions/operations rather than query data as of a point in time, and `RESTORE TABLE` permanently reverts the table's current state to a past version rather than doing a read-only historical query, which is a different operation with a different effect.

Q29. A Structured Streaming job appending to a Delta table encounters a new column that appears in the source data for the first time. Which option, set on the writer, allows the target table's schema to automatically evolve to include this new column instead of the write failing?

- A. `.option("mergeSchema", "true")`

- B. `.option("overwriteSchema", "true")`
- C. `.option("schemaEnforcement", "strict")`
- D. `.option("ignoreChanges", "true")`

Correct answer: A **Explanation:** `mergeSchema` (settable as a write option, or enabled at the session/table level via `spark.databricks.delta.schema.autoMerge.enabled`) tells Delta Lake to reconcile new columns present in the incoming data into the target table's schema instead of throwing a schema-mismatch error, which is the intended schema evolution mechanism. `overwriteSchema` instead replaces the entire target schema (destructive, used for full overwrites), and `ignoreChanges` relates to how Structured Streaming handles upstream file rewrites, not schema evolution.

Q30. A silver table receives thousands of small files per day from a streaming Auto Loader job, and downstream analytical queries filtering on `customer_id` have gotten noticeably slower over the past few months. Which two maintenance operations together most directly address this? (Select two.)

- A. `VACUUM` the table with a 0-hour retention immediately
- B. `OPTIMIZE sales.silver.orders;` to compact small files into larger ones
- C. `OPTIMIZE sales.silver.orders ZORDER BY (customer_id);` to co-locate related data for better file pruning on that column
- D. Convert the table from Delta format to CSV for faster scans
- E. Drop and recreate the table weekly

Correct answer: B, C **Explanation:** `OPTIMIZE` compacts the accumulated small files from frequent streaming writes into fewer, right-sized files, which reduces file-listing and open/close overhead, and adding `ZORDER BY (customer_id)` co-locates rows with similar `customer_id` values within those files so that data-skipping statistics let queries filtering on `customer_id` prune far more files at scan time. `VACUUM` only removes stale, untracked files left over from prior operations and does not compact or reorganize current data (and running it at 0-hour retention is also dangerous, as it can break time travel and any concurrent readers), while converting to CSV would remove Delta's transaction log, statistics, and all associated performance benefits entirely.

Q31. A teammate runs `VACUUM sales.silver.orders RETAIN 0 HOURS` on a production table to immediately reclaim storage space. What is the most significant risk of this action?

- A. It will corrupt the Delta transaction log irreversibly
- B. It can delete data files still needed by concurrent readers or by time-travel queries to recent versions, potentially breaking those reads
- C. It converts the table to an external table automatically
- D. It has no risk; `VACUUM` only ever removes files older than the current version

Correct answer: B **Explanation:** `VACUUM` deletes data files no longer referenced by the table's *current* logical version and older than the retention threshold; the default 7-day retention exists specifically as a safety margin so that other queries already in flight, or time-travel reads against recent versions, don't have their underlying files pulled out from under them. Setting `RETAIN 0 HOURS` disables that safety margin and

is explicitly discouraged (Databricks requires disabling a safety check to even run it) because it risks exactly this kind of data loss for concurrent or historical reads; it does not corrupt the transaction log itself or change the table's managed/external status.

Q32. In a medallion architecture implemented on Azure Databricks, which statement best describes the responsibilities of the bronze, silver, and gold layers?

- A. Bronze holds raw, largely unprocessed data as ingested from source systems; silver holds cleansed, conformed, and deduplicated data; gold holds business-level aggregates and curated tables ready for reporting/ML
- B. Bronze holds final reporting tables; silver holds raw ingested data; gold holds intermediate cleansed data
- C. All three layers must always live in the same schema, differentiated only by table name prefixes
- D. Gold layer data must never be queried directly by BI tools

Correct answer: A Explanation: The medallion pattern progressively improves data quality and structure as it flows through layers: **bronze** preserves an as-ingested, often schema-flexible copy of source data (useful for reprocessing/audit), **silver** applies cleansing, deduplication, conformance, and joins to produce validated, queryable datasets, and **gold** contains highly refined, aggregated, business-ready tables optimized for consumption by dashboards, reports, and ML features. There is no requirement that all three layers share one schema (in fact separate catalogs/schemas per layer is the common Unity Catalog pattern), and gold tables are specifically designed to be queried directly by BI tools.

Q33. Which statement correctly describes Auto Loader (cloudFiles source) in Azure Databricks?

- A. It is a batch-only command that must be manually re-run each time new files land in cloud storage
- B. It incrementally and efficiently discovers new files as they arrive in cloud storage (using cloud notification services or directory listing), and can be run as a structured streaming source with schema inference and evolution support
- C. It only supports reading from Azure Data Lake Storage, never from any other listed path
- D. It requires you to manually maintain a list of already-processed file names in a separate table

Correct answer: B Explanation: Auto Loader is implemented as the cloudFiles Structured Streaming source; it tracks which files have already been processed internally (backed by a scalable, checkpoint-based mechanism), can discover new files either via efficient cloud notification queues or incremental directory listing, and supports automatic schema inference plus schema evolution as new columns appear over time. It is explicitly designed for incremental/streaming ingestion (not requiring manual re-runs) and does not require the developer to hand-maintain a processed-files table — that bookkeeping is handled by its checkpoint.

Q34. A team needs to load a known, finite batch of new CSV files that land periodically in a landing zone into a Delta table, want idempotent “exactly-once” loading behavior

(files already loaded should never be reloaded even if the job re-runs), but do not need a continuously running stream. Which ingestion approach best fits, assuming they're comfortable expressing this as SQL run on a schedule?

- A. `COPY INTO`, which tracks already-loaded files and only ingests new ones on each run
- B. A plain `spark.read.csv(...).write.mode("append")` batch job re-run every hour
- C. Auto Loader running in continuous trigger mode only
- D. `MERGE INTO` run directly against the raw CSV files

Correct answer: A **Explanation:** `COPY INTO` is a SQL command designed for exactly this pattern: idempotent, incremental batch loading where Databricks keeps track internally of which source files have already been loaded so that re-running the same `COPY INTO` statement will not duplicate previously ingested files, without requiring a persistent streaming job. A plain `spark.read...write.append` batch job has no built-in tracking and would reload (and duplicate) every file on every run, and `MERGE INTO` operates on structured tables/DataFrames for upserts, not directly on raw CSV files in storage.

Q35. Your source system is a third-party SaaS application (e.g., Salesforce) and you want a low-code, managed way to continuously replicate its objects into Unity Catalog tables without hand-writing a custom extraction pipeline. Which Azure Databricks capability is designed for this?

- A. Lakeflow Connect
- B. Delta Sharing
- C. `dbutils.fs.cp`
- D. A cluster policy

Correct answer: A **Explanation:** **Lakeflow Connect** provides managed, low-code/no-code connectors for ingesting data from common SaaS applications and databases directly into Unity Catalog, handling authentication, incremental/CDC-style replication, and schema management with minimal custom code. Delta Sharing is for distributing data you already have to other consumers, not for pulling data in from a SaaS source, and `dbutils.fs.cp` is a low-level file-copy utility with no connector, scheduling, or CDC intelligence.

Q36. In Structured Streaming, what does `.trigger(availableNow=True)` accomplish, as compared to the default (unspecified) micro-batch trigger or `.trigger(processingTime="10 seconds")`?

- A. It processes all currently available data in one or more micro-batches and then stops the stream automatically, similar in spirit to a scheduled batch job but using streaming's incremental/checkpointed semantics
- B. It runs the query continuously and never terminates, exactly like the default trigger
- C. It disables checkpointing entirely
- D. It forces the query to reprocess all historical data from the beginning every run

Correct answer: A **Explanation:** `availableNow=True` tells Structured Streaming to process everything currently available in the source (across as many micro-batches as needed, respecting rate limits) and then gracefully stop the stream once caught up — this gives you streaming’s incremental, checkpoint-based “only process new data” behavior, but scheduled and run to completion like a batch job, which is a common and cost-efficient pattern for scheduled Lakeflow Jobs. It still relies on checkpointing to know what has already been processed (so it does not reprocess from the beginning), which distinguishes it from a plain default trigger that would instead run indefinitely at the fastest possible cadence.

Q37. What is the primary purpose of the `checkpointLocation` option in a Structured Streaming write?

- A. It specifies where the final output Delta table’s data files are stored
- B. It stores the streaming query’s progress metadata (offsets processed, state information) so the query can recover and resume correctly after a restart or failure, without reprocessing or losing data
- C. It defines the schema of the incoming streaming data
- D. It sets the cluster’s autoscaling checkpoint interval

Correct answer: B **Explanation:** The checkpoint location persists a streaming query’s internal state — which offsets/files have been consumed from the source and any stateful aggregation state — so that if the query is stopped and restarted (planned or due to failure), it can resume exactly where it left off with exactly-once processing guarantees for many sinks, including Delta. It is unrelated to where the output table’s data files live (that’s the table’s own storage location) and unrelated to schema definition or cluster autoscaling.

Q38. You need to implement a Slowly Changing Dimension Type 1 (overwrite-on-change) upsert from a daily batch of changed customer records into a customers Delta table, keyed by `customer_id`. Which approach is most appropriate?

- A. `INSERT OVERWRITE` the entire table every day with only the changed records
- B. `A MERGE INTO customers AS t USING updates AS s ON t.customer_id = s.customer_id WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT * statement`
- C. `DELETE FROM customers` followed by a full reload of the source system every day
- D. `APPEND` the changed records as new rows on every run

Correct answer: B **Explanation:** `MERGE INTO` is Delta Lake’s native upsert operation: matching this pattern lets you update existing rows in place when the key matches (Type 1: overwrite with the latest values) and insert new rows when it doesn’t, in a single atomic, efficient operation that only touches the affected files. `INSERT OVERWRITE` with only the changed subset would destroy all unchanged existing rows, plain `APPEND` would create duplicate rows for existing customers rather than updating them, and a full delete-and-reload is unnecessarily expensive and loses any Type-1 history the merge approach avoids needing.

Q39. In a Lakeflow Declarative Pipeline (formerly Delta Live Tables) Python notebook,

which decorator correctly defines a new materialized table from a transformation function?

```
import dlt
```

```
@__  
def silver_orders():  
    return spark.readStream.table("bronze_orders").where("amount > 0")
```

- A. @dlt.expect
- B. @dlt.table
- C. @dlt.view
- D. @dlt.pipeline

Correct answer: B Explanation: @dlt.table registers the function's return value (a DataFrame) as a managed, materialized table within the pipeline's target schema, with Lakeflow Declarative Pipelines handling dependency resolution, incremental refresh, and orchestration automatically. @dlt.view would define a temporary, non-materialized logical view usable only within the pipeline rather than a persisted table, @dlt.expect is a data-quality decorator applied in addition to @dlt.table (not a replacement for it), and @dlt.pipeline is not a real decorator in the API.

Q40. In a Lakeflow Declarative Pipeline, what is the behavioral difference between @dlt.expect_or_drop("valid_amount", "amount > 0") and @dlt.expect_or_fail("valid_amount", "amount > 0")?

- A. There is no difference; both are aliases for the same behavior
- B. expect_or_drop silently discards rows that violate the constraint while keeping the pipeline running, and continues to load the passing rows into the table; expect_or_fail stops the pipeline update entirely the moment a violating row is encountered
- C. expect_or_drop fails the entire pipeline; expect_or_fail only logs a warning and keeps all rows, including invalid ones
- D. Both decorators drop violating rows but only expect_or_fail records violation metrics

Correct answer: B Explanation: All three expectation decorators (@dlt.expect, @dlt.expect_or_drop, @dlt.expect_or_fail) track a named data-quality metric, but they differ in enforcement: plain expect only records/reports violations while still keeping the bad rows in the output, expect_or_drop removes rows violating the condition from the output while letting the pipeline continue, and expect_or_fail (the strictest) causes the entire pipeline update to fail as soon as a violation is detected, which is appropriate for hard data-quality gates that must halt downstream processing. B correctly captures the drop-vs-fail distinction; the other options invert or misstate this behavior.

Q41. Which statement correctly distinguishes a **streaming table** from a **materialized view** in a Lakeflow Declarative Pipeline?

- A. A streaming table incrementally processes new source data (append-only,

exactly-once, defined typically via `spark.readStream`), while a materialized view is computed by (re)running its defining query — potentially incrementally or fully — over the current data, and is well-suited to aggregations that need to reflect updates/deletes in source data

- B. Streaming tables and materialized views are identical in every respect and the terms are interchangeable
- C. Materialized views can only read from other materialized views, never from streaming tables
- D. Streaming tables cannot use `@dlt.table`; only materialized views can

Correct answer: A **Explanation:** A streaming table is built to incrementally ingest new, append-only data from a growing source (e.g., files arriving via Auto Loader, or a Kafka topic), tracking what’s new since the last run, while a materialized view re-computes/refreshes its result from its defining query and is a better fit when the logic involves aggregations or needs to correctly reflect updates and deletes upstream, not just new appends. Both are declared using `@dlt.table` in the Python API (the distinction comes from whether the function reads a stream via `spark.readStream` or a static/batch read), and materialized views can read from streaming tables just as easily as from other materialized views or batch tables.

Q42. A products Delta table must guarantee that price is never negative and that sku is never null. Which SQL correctly enforces both, assuming the table already exists?

- A. `ALTER TABLE products ADD CONSTRAINT positive_price CHECK (price >= 0); ALTER TABLE products ALTER COLUMN sku SET NOT NULL;`
- B. `ALTER TABLE products ADD EXPECTATION positive_price (price >= 0);`
- C. `CREATE ROW FILTER positive_price ON products WHERE price >= 0;`
- D. `ALTER TABLE products ADD MASK sku_not_null ON sku;`

Correct answer: A **Explanation:** Delta Lake table constraints are enforced with standard SQL DDL: a named CHECK constraint (`ADD CONSTRAINT ... CHECK (<boolean expression>)`) enforces an arbitrary boolean condition on every row, and `ALTER COLUMN ... SET NOT NULL` enforces non-nullability — both reject any write that would violate them, at write time, on a plain Delta table (independent of any Lakeflow Declarative Pipeline). “Expectations” (`@dlt.expect*`) are a related but separate mechanism specific to Lakeflow Declarative Pipelines, not general `ALTER TABLE` syntax, and row filters/column masks are access-control tools, not data-quality enforcement mechanisms.

Q43. You attempt to add a `CHECK (price >= 0)` constraint to an existing Delta table that already contains several rows with a negative price. What happens?

- A. The constraint is added successfully but simply ignores the pre-existing violating rows going forward
- B. The `ALTER TABLE ADD CONSTRAINT` statement fails, because Delta Lake validates all existing rows against a new constraint before adding it, and existing violations must be fixed first
- C. The violating rows are automatically deleted when the constraint is added

- D. The violating rows are automatically updated to price = 0

Correct answer: B Explanation: When you add a CHECK (or NOT NULL) constraint to a table that already has data, Delta Lake verifies every existing row against the new condition as part of adding the constraint, and the ALTER TABLE statement fails with a violation error if any existing rows don't satisfy it — you must clean up or correct the offending rows (e.g., via UPDATE/DELETE) before the constraint can be successfully added. Delta does not silently ignore, delete, or auto-correct existing violating data on your behalf; that would risk silent, unreviewed data loss.

Q44. A streaming aggregation counts events per user_id within 10-minute tumbling windows, but late-arriving events (up to 30 minutes late, due to a mobile client buffering data offline) must still be correctly counted, while state for very old windows should eventually be dropped to bound memory usage. Which Structured Streaming feature should you apply?

- A. dropDuplicates() with no additional configuration
- B. A watermark, e.g., .withWatermark("event_time", "30 minutes"), combined with the windowed aggregation
- C. .trigger(once=True)
- D. Increasing the cluster's driver memory only, with no code change

Correct answer: B Explanation: A **watermark** tells Structured Streaming how late data is allowed to arrive relative to the max event time seen so far (here, 30 minutes) before a window is considered final; it lets the engine correctly incorporate late but still-within-tolerance events into their proper window while safely dropping old aggregation state for windows older than the watermark threshold, bounding memory growth. dropDuplicates() alone addresses duplicate records, not lateness/window-state expiry, and simply adding driver memory does not solve the fundamental need to define a lateness tolerance and state-eviction policy.

Q45. Which command shows the full operation history of a Delta table, including which operation (MERGE, UPDATE, OPTIMIZE, VACUUM, etc.) was performed at each version, by whom, and when?

- A. SHOW TABLES;
- B. DESCRIBE HISTORY sales.gold.orders;
- C. DESCRIBE DETAIL sales.gold.orders;
- D. EXPLAIN sales.gold.orders;

Correct answer: B Explanation: DESCRIBE HISTORY returns the table's version-by-version audit trail — version number, timestamp, user, operation type, and operation parameters/metrics — which is exactly the metadata needed to answer “what happened to this table and when,” and it is also what powers time travel. DESCRIBE DETAIL instead returns current-state metadata about the table (location, size, number of files, partitioning) rather than its historical operations, and EXPLAIN shows a query's physical/logical plan, unrelated to table history.

Q46. When configuring Auto Loader, what is the purpose of the cloudFiles.schemaLocation option?

- A. It specifies where the final table's data files should be written
- B. It designates a directory where Auto Loader persists the inferred schema (and tracks schema changes over time), independent of the streaming checkpoint
- C. It sets which Azure region the source storage account is located in
- D. It is required only when reading Parquet files, never for CSV or JSON

Correct answer: B Explanation: `cloudFiles.schemaLocation` tells Auto Loader where to store the schema it infers from the source data and the log of schema changes it has observed over time, which lets schema inference be consistent and evolve safely across restarts of the stream; it is a distinct concept from — though it may share a parent directory with — the streaming checkpointLocation. It has nothing to do with output table file location or storage account region, and it is generally recommended/required regardless of source file format (CSV, JSON, Parquet, etc.), not just for one format.

Q47. A source system exports one large CSV file to a fixed path every night at 2 AM, fully overwriting the previous night's file (not appending new files), and downstream consumers only need the latest full nightly snapshot rather than continuous updates throughout the day. Which ingestion approach is most appropriate and cost-efficient?

- A. A continuously running Auto Loader stream watching that path 24/7
- B. A Lakeflow Job scheduled shortly after 2 AM that runs a batch read of the file and overwrites/merges it into the target table, then the job cluster terminates
- C. A Structured Streaming job with `.trigger(processingTime="1 second")`
- D. Delta Sharing configured to pull from the source system

Correct answer: B Explanation: Since there is exactly one predictable batch snapshot per day and no need for intraday freshness, a scheduled Lakeflow Job that spins up compute, performs a straightforward batch read/overwrite (or merge) once, and then lets the job cluster terminate is the simplest and most cost-efficient design — you pay only for the few minutes of compute needed rather than running infrastructure continuously. A 24/7 streaming job (Auto Loader or a sub-second trigger) would sit idle almost all day, burning cost for no benefit, since the source doesn't produce genuinely incremental new files, and Delta Sharing is for distributing data to other consumers, not for pulling data in from a source system.

Q48. Given the DataFrame `df` with a `order_date` string column in `yyyy-MM-dd` format, which PySpark snippet correctly adds a new column `order_year` containing the year as an integer?

- A. `df.withColumn("order_year", year(to_date(col("order_date"), "yyyy-MM-dd")))`
- B. `df.select("order_year", year("order_date"))`
- C. `df.addColumn("order_year", "year(order_date)")`
- D. `df.withColumnRenamed("order_date", "order_year")`

Correct answer: A Explanation: `withColumn` is the correct method to add a new derived column without dropping existing ones, `to_date` parses the string into a proper date type using the given format, and `year()` extracts the integer year from that date value — this is standard, idiomatic PySpark for date-based feature derivation.

`df.select("order_year", ...)` would fail (no such column exists yet to select) and doesn't correctly build the expression, `addColumn` is not a real DataFrame method, and `withColumnRenamed` merely renames an existing column rather than computing a new derived value.

Q49. You need to compute, for each `order_id` row, the running total of amount per `customer_id` ordered by `order_date`, without collapsing the DataFrame into an aggregated summary (each original row must remain, with the running total added as a new column). Which approach is correct?

- A. `df.groupBy("customer_id").agg(sum("amount"))`
- B. A window function: `df.withColumn("running_total", sum("amount").over(Window.partitionBy("customer_id").orderBy("order_date")))`
- C. `df.orderBy("order_date").agg(sum("amount"))`
- D. `df.filter(col("customer_id").isNotNull()).sum("amount")`

Correct answer: B Explanation: A window function computed with `.over(Window.partitionBy(...))` computes an aggregate (here, a running sum) per row within a defined partition and order, while preserving every original row — exactly the “running total per customer, one row per order” requirement. Plain `groupBy().agg()` collapses all rows for each `customer_id` into a single summary row, which loses per-order granularity and is not what's being asked for here.

Q50. In a Lakeflow Declarative Pipeline ingesting IoT sensor readings, you want to keep and load rows where temperature is between -40 and 150 (physically plausible), silently discard rows outside that range without stopping the pipeline, and also track how many rows were discarded for a data-quality dashboard. Which decorator/expectation best fits?

- A. `@dlt.expect("valid_temp", "temperature BETWEEN -40 AND 150")` alone
- B. `@dlt.expect_or_drop("valid_temp", "temperature BETWEEN -40 AND 150")`
- C. `@dlt.expect_or_fail("valid_temp", "temperature BETWEEN -40 AND 150")`
- D. A manual `.filter()` call with no expectation decorator at all

Correct answer: B Explanation: `expect_or_drop` both enforces the condition by removing violating rows from the table's output (so implausible readings never reach downstream consumers) and — like all expectation decorators — automatically records pass/fail/dropped-row metrics that Lakeflow Declarative Pipelines exposes for monitoring/dashboards, satisfying both stated requirements in one line. Plain `@dlt.expect` would track the metric but leave the bad rows in the table (not “discard” them), `expect_or_fail` would halt the entire pipeline on any single out-of-range reading (far too strict for expected sensor noise), and a manual `.filter()` with no expectation decorator would drop the rows but lose the automatic data-quality metric tracking that expectations provide.

Q51. A regulated table must guarantee no downstream consumer ever sees a row with a null `transaction_id`, even if a future engineer forgets to add filtering logic in a query. What is the most robust way to guarantee this at the data layer itself, independent of how the table is queried?

- A. Document in a wiki page that `transaction_id` should never be null and hope engineers check
- B. Add a `NOT NULL` constraint on `transaction_id` directly on the Delta table, so any write violating it is rejected at write time
- C. Add a `WHERE transaction_id IS NOT NULL` clause to every downstream BI report
- D. Rely on Auto Loader's schema inference to exclude null values automatically

Correct answer: B Explanation: A `NOT NULL` constraint enforced directly on the Delta table is a write-time guarantee — any `INSERT`, `MERGE`, or streaming write that would introduce a null `transaction_id` is rejected outright — so the guarantee holds regardless of how any future query or report reads the table, which is strictly more robust than relying on documentation or per-query filtering that can be forgotten. Auto Loader's schema inference determines column types/structure during ingestion but does not enforce nullability rules or reject null values by itself; that enforcement must come from an explicit constraint on the target table.

6.4 Domain 4 Practice Questions — Deploy and Maintain Data Pipelines and Workloads

Q52. You are designing a Lakeflow Job with three tasks: `ingest_bronze`, `transform_silver` (which depends on `ingest_bronze` succeeding), and `build_gold` (which depends on `transform_silver` succeeding). If `transform_silver` fails, what is the default behavior for `build_gold`?

- A. `build_gold` runs anyway, ignoring the upstream failure
- B. `build_gold` is skipped, since its declared upstream dependency did not complete successfully
- C. The entire job is deleted automatically
- D. `build_gold` runs first, out of order, to avoid wasting compute

Correct answer: B Explanation: Lakeflow Jobs (multi-task jobs) execute tasks according to the dependency graph you define (`depends_on`), and by default a task whose upstream dependency did not succeed is not run — it is marked skipped — because its precondition for running was not met, which prevents downstream tasks from operating on incomplete or invalid upstream output. Tasks do not run out of declared order, and a task failure does not delete the job definition; the job remains available (with a failed/skipped run reported) for the next scheduled or manual trigger.

Q53. Which best describes the cost and lifecycle difference between a **job cluster** and an **all-purpose (interactive) cluster** used to run a nightly Lakeflow Job task?

- A. They are billed identically and both remain running indefinitely regardless of job activity
- B. A job cluster is created specifically for that job run and automatically terminates when the run completes, typically at a lower DBU rate than all-purpose

compute; an all-purpose cluster is meant for persistent interactive use and keeps running (and incurring cost) until manually or automatically terminated

- C. Job clusters can only run SQL, never Python or Scala
- D. All-purpose clusters cannot be used to run scheduled jobs at all

Correct answer: B Explanation: A **job cluster** is ephemeral, created automatically when a scheduled/triggered job run starts and torn down as soon as the run finishes, and Databricks bills job-cluster compute at a lower DBU rate than all-purpose compute, which is why job clusters are the recommended, cost-efficient choice for scheduled production workloads. An **all-purpose cluster** is meant to stay up for interactive notebook use across a workgroup and keeps incurring cost until it's terminated (manually or via inactivity auto-termination); it *can* technically be attached to a job for convenience during development, but doing so for production jobs is more expensive and not best practice.

Q54. Which trigger type should you configure on a Lakeflow Job if new files land in a cloud storage directory at unpredictable times throughout the day, and you want the job to start processing shortly after new files actually arrive, rather than on a fixed schedule?

- A. A cron-based scheduled trigger set to run every minute, all day, whether or not new files exist
- B. A file arrival trigger, which starts the job run when new files are detected in the specified storage location
- C. A manual trigger only, requiring someone to click "Run Now" each time
- D. A continuous trigger that never stops running once started, regardless of new files

Correct answer: B Explanation: A **file arrival trigger** watches a specified storage location (leveraging cloud storage event notifications) and starts a job run when it detects new files have landed, which is exactly the "unpredictable arrival, run soon after" requirement, without wasting compute polling on a fixed schedule. Running a cron trigger every minute all day would frequently execute the job with nothing new to process, wasting compute/cost, and a purely manual trigger would defeat the goal of automated, timely processing.

Q55. You configure a task in a Lakeflow Job with `max_retries = 2` and a `min_retry_interval_millis` delay between attempts. The task fails on its first attempt due to a transient network timeout, then succeeds on the second attempt. What does the job run history show for that task?

- A. The task is reported as failed, because any retry counts as an overall failure
- B. The task is reported as succeeded, with its run history showing the earlier failed attempt(s) and the eventual successful attempt
- C. The job stops entirely and does not attempt a retry, since retries are only for cluster-level failures
- D. Retries can only be configured at the whole-job level, never per task

Correct answer: B Explanation: When a task is configured with retries and a later attempt succeeds within the allowed `max_retries`, the task's overall status for that

run is **succeeded**, while the run history/UI still shows the individual attempt(s) that failed first, giving visibility into transient issues without treating the whole run as a failure. Retries in Lakeflow Jobs are commonly configured per task (each task can have its own retry policy), not only at the whole-job level, which lets you tune retry behavior for tasks known to hit transient issues (like network calls) differently from tasks where a failure should stop things immediately.

Q56. A production Lakeflow Job should notify the on-call data engineer's email and post to a Slack/Teams webhook immediately if any task fails, without requiring anyone to manually check the job run history. What should you configure?

- A. Nothing — job failures are always broadcast to every workspace user automatically
- B. Task- or job-level notifications/alerts (email and webhook destinations) triggered on failure (and optionally on start, success, or duration threshold)
- C. A cluster policy with an email field
- D. A row filter on the job's output table

Correct answer: B Explanation: Lakeflow Jobs support configuring notifications at the job and/or individual task level for events such as on start, on success, on failure, and on duration threshold exceeded, sending to destinations like email addresses or webhook-based integrations (Slack, Microsoft Teams, PagerDuty, etc.) — this is the built-in mechanism for proactive alerting instead of manually checking run history. Job failures are not broadcast to the whole workspace by default; notification destinations must be explicitly configured, and cluster policies/row filters have nothing to do with job run alerting.

Q57. In a multi-task Lakeflow Job, task B needs to read a value computed by task A (e.g., a row count or a computed date string) that isn't naturally available except by re-querying the source. What is the recommended mechanism for passing this small piece of information from task A to task B?

- A. Writing the value into a hard-coded Python file checked into the repo
- B. Task values (`dbutils.jobs.taskValues.set(...)` in task A, and `dbutils.jobs.taskValues.get(...)` in task B)
- C. Manually copying and pasting the value between notebooks after each run
- D. Job-level cluster policies

Correct answer: B Explanation: Task values are the purpose-built mechanism for passing small pieces of data (strings, numbers, small JSON-serializable structures) between tasks in the same job run: task A calls `dbutils.jobs.taskValues.set(key, value)`, and any downstream task that declares a dependency on task A can retrieve it with `dbutils.jobs.taskValues.get(taskKey="A", key=...)`. Hard-coding values in source files defeats the purpose of a dynamic, per-run computed value, and cluster policies govern compute configuration, not inter-task data passing.

Q58. A team develops notebooks and pipeline code collaboratively, wants full Git history, pull-request-based code review, and the ability to work on isolated feature branches before merging to main, directly from within the Azure Databricks workspace UI. Which capability should they use?

- A. DBFS root storage
- B. Databricks Repos, which integrates the workspace with a Git provider (e.g., Azure DevOps, GitHub) for clone/branch/commit/pull/push operations on notebooks and files
- C. Unity Catalog volumes
- D. Cluster policies

Correct answer: B Explanation: **Databricks Repos** connects a workspace folder to a remote Git repository, letting users clone, create and switch branches, commit, and push/pull directly from the workspace UI (or via CLI/API), which is exactly the collaborative, branch-based, PR-reviewed workflow described. DBFS and Unity Catalog volumes are storage locations for files/data, not source control integrations, and cluster policies are unrelated to version control.

Q59. What is the primary purpose of a Databricks Asset Bundle's `databricks.yml` file?

- A. It stores the actual Delta table data in YAML form
- B. It declaratively defines the bundle's resources (jobs, pipelines, etc.), along with deployment targets (e.g., dev, staging, prod) and their environment-specific overrides, enabling infrastructure-as-code deployment via the Databricks CLI
- C. It is an alternative name for the Unity Catalog metastore configuration file
- D. It replaces the need for a Git repository entirely

Correct answer: B Explanation: `databricks.yml` is the root configuration file of a **Databricks Asset Bundle (DAB)**: it declares the bundle's resources (Lakeflow Jobs, Declarative Pipelines, etc.) as code, defines one or more deployment targets (commonly dev/staging/prod) each with their own workspace URLs, variables, or overrides, and is what the `databricks bundle deploy/databricks bundle run` CLI commands act on — making pipeline and job deployment reproducible and version-controlled. It has nothing to do with storing actual table data or replacing Unity Catalog's metastore configuration, and it works *with* Git (bundles are normally stored in a Git repo), not as a replacement for it.

Q60. A team wants to promote a Lakeflow Job from a dev target (running against a dev catalog, with a smaller test cluster and the individual engineer's own credentials) to a prod target (running against the production catalog, a larger job cluster, and a dedicated service principal) using the same underlying Asset Bundle codebase. How should this be modeled in Databricks Asset Bundles?

- A. Maintain two entirely separate, hand-copied `databricks.yml` files with no shared code
- B. Define multiple targets in the same bundle (e.g., dev and prod), each overriding variables such as catalog name, cluster size, and `run_as` identity, and deploy the appropriate target with `databricks bundle deploy --target prod`
- C. Databricks Asset Bundles do not support more than one environment
- D. Use a single hard-coded catalog name in the job definition and manually edit it before each deployment

Correct answer: B Explanation: Asset Bundles are explicitly designed for multi-

environment promotion: a single bundle defines shared resource logic once, and separate targets (e.g., dev, staging, prod) override environment-specific settings — catalog/schema names, compute sizing, workspace URL, run_as identity — and you deploy/run against a specific target with `--target <name>` on the CLI, avoiding duplicated code and manual editing. Hand-copying files or manually editing hard-coded values before every deploy is the error-prone, non-IaC anti-pattern that Asset Bundles are meant to replace.

Q61. A Spark SQL query’s Spark UI query profile shows one task in a stage taking 40 minutes while all other ~200 tasks in the same stage finish in under 30 seconds. What performance problem does this most likely indicate, and what is a common remediation?

- A. This is normal and requires no action
- B. Data skew — a small number of partition keys hold a disproportionate share of the data (or an unfiltered/very large join key), causing one task to process far more data than the others; remediations include salting the skewed key, using adaptive query execution’s skew join optimization, or broadcasting a small side of a join
- C. The cluster’s network bandwidth to Azure Storage is the sole possible cause, unrelated to data distribution
- D. This always indicates a corrupted Delta transaction log

Correct answer: B Explanation: A single wildly outlier task duration alongside many fast, roughly-equal tasks in the same stage is the classic signature of **data skew** — one partition (often tied to a specific, disproportionately common key value) contains far more rows than the others, so the task processing it takes far longer; common remediations include enabling/relying on Adaptive Query Execution’s automatic skew join handling, “salting” the skewed key to spread it across more partitions, or broadcasting a small dimension table instead of shuffling it. This is a data distribution problem, not evidence of network bandwidth limits or transaction log corruption, and it should not be dismissed as normal since it directly extends overall job runtime.

Q62. A scheduled Lakeflow Job task failed overnight. Which two resources should you check first to diagnose the root cause? (Select two.)

- A. The job run’s task-level output/logs and error stack trace in the job run history UI
- B. The Spark UI (or query profile) for the specific run’s cluster, to inspect stages/tasks around the time of failure
- C. The Azure subscription’s cost management blade
- D. A completely unrelated workspace’s audit logs
- E. The `system.billing.usage` table filtered to a different job ID

Correct answer: A, B Explanation: The **job run history** for the specific failed run surfaces the task’s stdout/stderr, error messages, and stack trace — the first and most direct source of “why did this fail” — and the **Spark UI/query profile** for that run’s cluster lets you drill into which stage/task was running (or stuck) at the time of failure, revealing issues like OOM errors, skew, or a bad query plan. Cost management, an

unrelated workspace's logs, or billing data for a different job ID are not diagnostic sources for a specific task's runtime failure.

Q63. Where can you find a historical record of a cluster's autoscaling events — for example, when it scaled from 4 to 12 workers and why (e.g., due to pending tasks)?

- A. The Delta transaction log of any table the cluster wrote to
- B. The cluster's event log, available in the cluster UI, which records lifecycle and autoscaling events with timestamps and reasons
- C. `system.access.audit`
- D. The job's task value store

Correct answer: B Explanation: Every classic cluster exposes an **event log** in its UI that records lifecycle events — creation, start, resize/autoscaling changes (with the trigger reason, such as pending tasks), termination, and driver/executor errors — with timestamps, which is exactly where you'd look to understand scaling behavior over time. The Delta transaction log tracks table-level data operations (not cluster infrastructure events), `system.access.audit` covers Unity Catalog access/action auditing (not cluster autoscaling specifically), and task values are a small data-passing mechanism between job tasks, unrelated to cluster events.

Q64. A FinOps team wants to build a monthly chargeback report showing DBU consumption broken down by workspace, cluster, and job, using SQL, without exporting anything manually from the Azure portal. Which system table(s) should they query?

- A. `system.lineage.table_lineage`
- B. `system.billing.usage` (joined as needed with `system.compute.clusters` and job/workspace metadata)
- C. `system.access.audit`
- D. The Delta transaction log of the gold-layer reporting tables

Correct answer: B Explanation: `system.billing.usage` records granular usage records (DBUs consumed, SKU, workspace, cluster/warehouse/job identifiers, timestamps) suitable for cost attribution and chargeback reporting, and it can be joined with other system tables like `system.compute.clusters` for richer cluster metadata — all queryable with ordinary SQL from within a Databricks SQL warehouse or notebook, no manual portal export required. Lineage and audit system tables serve different governance purposes (data flow tracking and action auditing, respectively) and do not contain DBU/cost consumption data.

Q65. What is Photon in the context of Azure Databricks, and what should you generally do to benefit from it?

- A. Photon is a separate product requiring a completely different Databricks Runtime installation process
- B. Photon is a native, vectorized query execution engine (written in C++) that can dramatically accelerate SQL and DataFrame workloads; you generally enable it by selecting a Photon-enabled runtime when creating a cluster or SQL warehouse (it's on by default for many recent runtimes/serverless SQL)
- C. Photon only works with RDD-based Spark code, never DataFrames or SQL

- D. Photon must be manually coded into every PySpark transformation using a special import

Correct answer: B Explanation: Photon is Databricks' native vectorized execution engine that re-implements much of Spark's execution layer in C++ for significant performance and cost efficiency gains on SQL and DataFrame (Spark SQL engine-based) workloads; enabling it is generally as simple as selecting a Photon-capable Databricks Runtime/cluster configuration or SQL warehouse (and it's the default in many current serverless/SQL warehouse configurations), with no code changes required in your notebooks. It is not a separate product requiring different installation steps, does not target legacy RDD-only code paths, and requires no special imports in user code — it operates transparently underneath the existing Spark SQL/DataFrame APIs.

Q66. A fact table `sales.gold.transactions` has 2 billion rows spanning five years, and virtually all queries filter on a `transaction_date` range (e.g., "last 30 days"). `store_id` has only 50 distinct values queried somewhat evenly. Which physical table design decision would most effectively improve typical query performance?

- A. Partition the table by `store_id`, since it has fewer distinct values
- B. Partition the table by a coarse-grained transformation of `transaction_date` (e.g., by month or year), so date-range filters can skip/prune entire partitions rather than scanning the full table
- C. Avoid partitioning entirely and rely solely on Z-ordering by `store_id`
- D. Partition by a random UUID column added purely to improve parallelism

Correct answer: B Explanation: Because virtually every query filters on a date range, partitioning by a coarse date grain (month or year, not by raw daily timestamp which would create excessive tiny partitions) lets Delta Lake's partition pruning skip entire irrelevant partitions before even reading file-level statistics, which is the single highest-leverage physical design choice here. Partitioning by `store_id` would not help date-range queries at all (and 50 relatively evenly-distributed values is actually a reasonable Z-ORDER candidate, not necessarily a good partition column at this data volume), and partitioning by a random UUID would create enormous numbers of tiny, useless partitions with zero query-filtering benefit.

Q67. What is the difference between Delta's built-in disk cache (the "Delta cache"/"IO cache" on cluster local SSDs) and explicitly calling `.cache()` / `.persist()` on a DataFrame?

- A. They are exactly the same mechanism under a different name, with no practical difference
- B. The disk cache automatically and transparently caches remote file data (e.g., Parquet/Delta files read from ADLS Gen2) in a compressed form on the cluster's local storage to speed up repeated reads, whereas `.cache()/persist()` explicitly materializes a specific DataFrame's computed result in memory (and/or disk) for reuse across multiple actions within a Spark job/session
- C. `.cache()` only works on Delta tables, never on DataFrames derived from transformations

- D. The disk cache requires explicit code changes in every notebook to enable, exactly like `.cache()`

Correct answer: B Explanation: The disk cache (sometimes called IO cache) operates transparently at the storage-read layer — automatically caching blocks of remote cloud files on fast local cluster storage as they’re read, benefiting any subsequent reads of the same data without any code change — whereas `.cache()/persist()` is an explicit Spark API call that materializes the *result of a specific DataFrame’s transformations* (post filters/joins/etc.) in memory or on disk so that it can be reused across multiple downstream actions without recomputation. `.cache()` works on any DataFrame, not just ones backed by Delta tables, and unlike `.cache()`, the disk cache does not require any explicit code — it is a cluster-level, automatic optimization (when enabled/available on the chosen instance type).

Q68. A job cluster processes a highly variable daily data volume — some days 10 GB, some days 2 TB, unpredictably. Which cluster configuration best balances cost and performance for this workload?

- A. A fixed-size cluster sized for the 2 TB worst case, running every day regardless of actual volume
- B. Autoscaling enabled with a sensible minimum and maximum worker count, so the cluster scales up for large-volume days and scales down (reducing cost) for small-volume days
- C. A fixed-size cluster with exactly 1 worker every day, regardless of volume
- D. Disabling autoscaling and manually resizing the cluster by hand every morning based on a guess

Correct answer: B Explanation: Autoscaling, given a sensible min/max range, lets the cluster add workers when the workload actually demands more capacity (large-volume days) and shrink back down when it doesn’t (small-volume days), which directly matches unpredictable, highly variable daily volume without requiring manual intervention or paying for permanently over-provisioned capacity. A fixed cluster sized for the worst case wastes money on every smaller day, a fixed 1-worker cluster would be badly under-provisioned and slow on large-volume days, and manual daily resizing based on a guess is error-prone and doesn’t scale operationally.

Q69. A Lakeflow Job’s transform task fails at 2 AM due to a transient upstream API outage; tasks ingest (upstream, already succeeded) and publish (downstream, skipped due to the failure) are part of the same run. Once the outage is resolved, what is the most efficient way to get this run to a successful completed state without re-running unaffected steps unnecessarily?

- A. Delete the job and recreate it from scratch
- B. Use “Repair Run,” which re-runs only the failed and skipped tasks (transform and publish), reusing the successful output of tasks that already completed (ingest), rather than re-running the entire job from the beginning
- C. Manually re-run the entire job as a brand-new run, including re-running ingest from scratch
- D. Wait for the next day’s scheduled run and ignore the failed run entirely

Correct answer: B Explanation: **Repair Run** is the Lakeflow Jobs feature designed exactly for this situation: given a run with some successful and some failed/skipped tasks, it re-executes only the tasks that didn't complete successfully (here, transform and the downstream publish that was skipped because of it), leaving the already-successful ingest task's output untouched — saving compute time and cost versus a full re-run. Re-running the entire job from scratch would needlessly redo ingest's work, and simply waiting for the next scheduled run would leave last night's data gap unresolved and delay downstream consumers.

Q70. In a multi-task Lakeflow Job, tasks A and B (which do not depend on each other) both need a cluster, and you want to minimize both cost and cluster startup latency by having them share one job cluster rather than each spinning up its own. How is this configured?

- A. It is not possible; every task always gets its own dedicated cluster
- B. Define the job cluster once in the job's `job_clusters` list with a `job_cluster_key`, and reference that same `job_cluster_key` from both task A's and task B's cluster configuration
- C. Set both tasks to use "Serverless" compute, which is the only way to ever share a cluster
- D. Manually copy the exact same cluster JSON configuration into each task, hoping Databricks detects they're identical and merges them

Correct answer: B Explanation: Lakeflow Jobs let you declare a reusable cluster once under the job's `job_clusters` list (keyed by a `job_cluster_key`), and any task that references that same key will run on that shared cluster instance for the duration of the run, avoiding the cost and startup latency of spinning up a separate cluster per task. This sharing is achieved via the shared key reference, not by relying on Databricks to detect identical JSON blobs (which are treated as independent, separately-provisioned clusters unless explicitly keyed together), and it is a general job-cluster feature, not something exclusive to serverless compute.

Q71. As part of a CI/CD pipeline, a GitHub Actions workflow should automatically validate and deploy an Asset Bundle to a staging workspace whenever a pull request merges to the main branch. Which command sequence, run in that CI job (with the Databricks CLI configured/authenticated), correctly does this?

- A. `databricks bundle destroy --target staging` followed by manually recreating everything in the UI
- B. `databricks bundle validate --target staging` followed by `databricks bundle deploy --target staging`
- C. `databricks fs cp -r . dbfs:/staging/`
- D. Manually attach the CI runner to the staging workspace's cluster and copy notebook files over SSH

Correct answer: B Explanation: The standard Asset Bundle CI/CD pattern is to run `databricks bundle validate` (which checks the bundle configuration for errors before committing to a deployment) and then `databricks bundle deploy --target <target>`, both scoped to the desired target (here, staging), which deploys the de-

financed jobs/pipelines/resources to that workspace as code — a repeatable, automatable process suited to a CI runner. `bundle destroy` tears resources down rather than deploying them, and copying raw files via `databricks fs cp` or manual SSH/UI steps bypasses the Asset Bundle's resource definitions, variable substitution, and target-specific overrides entirely.

Q72. While monitoring a running Structured Streaming query in the Spark UI's Structured Streaming tab, you notice the "input rate" consistently exceeds the "processing rate," and the number of files/records waiting to be processed keeps growing run over run. What does this most likely indicate, and what is a reasonable first remediation to try?

- A. This is expected and requires no action, since streaming queries always show this pattern
- B. The query cannot keep up with the incoming data rate (falling behind, growing backlog); reasonable remediations include scaling up/out the cluster, increasing shuffle partitions appropriately, or tuning the trigger/batch size to process the backlog down
- C. The only possible cause is a corrupted checkpoint, requiring the checkpoint to be deleted immediately
- D. This metric is purely cosmetic and never indicates an actual performance problem

Correct answer: B Explanation: When the input rate (data arriving) consistently outpaces the processing rate (data actually consumed per micro-batch) and backlog keeps growing across runs, the streaming query is falling behind in a way that will keep worsening unless addressed; typical remediations include adding cluster capacity (more/larger workers), tuning Spark shuffle partition counts to better match cluster parallelism, or adjusting the micro-batch trigger interval/size so each batch processes a more manageable chunk and lets the pipeline catch up. Deleting the checkpoint is a destructive, unrelated action that would lose the query's progress tracking and is not an appropriate first response to a capacity/throughput problem.

Q73. A join between a very large fact table (billions of rows) and a small dimension table (a few thousand rows) is running slower than expected, with a large shuffle visible in the query plan. Which technique is most likely to improve performance here?

- A. Increase the number of shuffle partitions to an arbitrarily high number with no other change
- B. Use a broadcast join hint (e.g., `df_large.join(broadcast(df_small), "key")`), so the small dimension table is sent to every executor and no expensive shuffle of the large fact table is required for the join
- C. Convert both tables to CSV before joining
- D. Remove the join entirely and perform two separate, unrelated queries

Correct answer: B Explanation: When one side of a join is small enough to fit comfortably in executor memory, a **broadcast join** avoids shuffling the large table across the cluster entirely by instead sending a full copy of the small table to every execu-

tor, which is typically dramatically faster than a full shuffle-based sort-merge join for this size asymmetry; Spark’s adaptive query execution often does this automatically, but an explicit `broadcast()` hint guarantees it. Simply cranking up shuffle partition count doesn’t eliminate the fundamental shuffle cost of moving the large table’s data around, and converting to CSV would remove Parquet/Delta’s columnar and statistics benefits while doing nothing to address the join strategy itself.

Q74. A cluster policy for job clusters restricts allowed VM instance types to a short approved list and enforces a maximum of 8 autoscaling workers, in line with cost governance requirements from Domain 1. From a Domain 4 (deployment/monitoring) perspective, what is the correct way to make sure every Lakeflow Job created going forward actually uses this policy, rather than relying on each job author remembering to select it?

- A. There is no way to require a policy be used; it can only ever be optional
- B. Reference the approved cluster policy’s ID in the job’s cluster definition (or in the Asset Bundle’s job resource YAML) so it’s baked into the job’s configuration as code, and additionally restrict “Can Manage”/unrestricted cluster creation so that only policy-conformant clusters can be created in the workspace
- C. Email all engineers a PDF describing the policy and trust manual compliance
- D. Apply the policy only to all-purpose clusters, since job clusters are always exempt from cluster policies

Correct answer: B Explanation: Making a policy consistently applied is a combination of (1) referencing the approved policy ID directly in the job/task cluster definition — ideally checked into source control via an Asset Bundle, so it’s part of the reviewed, deployed configuration rather than a manual UI choice — and (2) restricting broader unrestricted cluster-creation permissions so engineers can’t simply bypass the policy by picking “unrestricted” when creating a job cluster. Job clusters are fully subject to cluster policies just like all-purpose clusters (option D is false), and relying on emailed documentation and manual compliance is exactly the failure mode governance controls and Asset Bundles are meant to eliminate.

Q75. A gold-layer table that a critical dashboard depends on is supposed to be refreshed by 6 AM daily. You want to be automatically alerted if the table has not been updated by 7 AM on any given day (e.g., due to an upstream pipeline failure that didn’t otherwise trigger a job-failure notification). What is an appropriate way to implement this freshness check?

- A. It is not possible to monitor data freshness independent of job success/failure status
- B. Create a Databricks SQL alert on a query that checks the table’s most recent update timestamp (e.g., via `DESCRIBE HISTORY` or a max timestamp column) against the current time, scheduled to run periodically and configured to notify when the freshness condition is violated
- C. Rely solely on the job’s “on success” notification, since a successful job run always guarantees the target table was actually updated with new data
- D. Manually query the table by hand every morning at 7 AM

Correct answer: B **Explanation:** A **SQL alert** in Databricks SQL can be built on a scheduled query that evaluates a freshness condition (e.g., `SELECT max(event_time) FROM gold.table` compared against `current_timestamp() - INTERVAL 1 HOUR`, or the latest `DESCRIBE HISTORY` timestamp), and configured to fire a notification when that condition is breached — this directly monitors data freshness/correctness rather than just job execution status. A job reporting “success” only confirms the job’s tasks completed without error; it does not guarantee the target table actually received the expected fresh data (e.g., an upstream source could have silently returned zero rows), so relying solely on job-success notifications can miss exactly this kind of silent freshness failure.

Q76. Which two of the following are genuine benefits of adopting Databricks Asset Bundles plus a Git-based Repos workflow for a data engineering team’s development lifecycle, as opposed to building and scheduling jobs purely by hand through the workspace UI? (Select two.)

- A. Job and pipeline definitions become version-controlled, code-reviewable, and repeatably deployable across dev/staging/prod using the same declarative source
- B. It eliminates the need for Unity Catalog governance entirely, since bundles handle all security automatically
- C. Deployments become consistent and auditable (who changed what, and when, is tracked in Git history), reducing configuration drift between environments
- D. Asset Bundles remove the need to ever test code before deploying to production
- E. Asset Bundles automatically guarantee zero-downtime deployment with no possibility of a failed deploy

Correct answer: A, C **Explanation:** The core value of combining Repos (Git-backed source control) with Asset Bundles (declarative, code-based resource definitions and multi-target deployment) is that job/pipeline configuration becomes versioned, peer-reviewable, and consistently reproducible across environments via CI/CD, and every change is traceable through ordinary Git history — directly reducing the “it worked in dev but not in prod” configuration drift problem common with manual UI-based setup. Neither Repos nor Asset Bundles replace Unity Catalog governance (which remains a separate, necessary layer), remove the need for testing, or provide any inherent zero-downtime/deployment-failure guarantee — those claims overstate what the tooling does.

6.5 Final Mixed Mock Exam (20 Questions)

Set a timer for **24 minutes** (this is 20/60ths of the real DP-750’s 120-minute allotment for its full question set, scaled to this sample) and answer all 20 questions below without looking at the Answer Key. The mix below approximates the real exam’s domain weighting: 3 questions from Domain 1, 3 from Domain 2, 7 from Domain 3, and 7 from Domain 4. Choices are given with no answers shown — check your work

against the **Answer Key** section that follows only after finishing or timing out.

M1. A workspace admin wants to guarantee that no cluster created in the workspace can ever cost more than a defined DBU/hour ceiling, regardless of who creates it or what they select in the UI. Which feature enforces this?

- A. Unity Catalog schema grants
- B. A cluster policy with a DBU/hour limit configured
- C. A Delta Sharing share
- D. A row filter

M2. Which access mode should be used for a cluster shared by multiple data engineers who all need Unity Catalog governance enforced with per-user process isolation?

- A. No Isolation Shared
- B. Single User
- C. Shared (Unity Catalog)
- D. Legacy passthrough

M3. You create `CREATE TABLE bronze.raw.events (...) USING DELTA LOCATION 'abfss://landing@acct.dfs.core.windows.net/events/';`. What kind of table is this, and what happens to the underlying files on `DROP TABLE`?

- A. Managed table; files are deleted
- B. External table; files remain in place after the drop
- C. External table; files are deleted
- D. Managed table; files remain until manually deleted

M4. Which statement should you use to give the `bi_readers` group `SELECT` access to every current and future table in the `gold` schema of the `sales` catalog with one command?

- A. `GRANT SELECT ON SCHEMA sales.gold TO bi_readers;`
- B. `GRANT SELECT ON TABLE sales.gold.* TO bi_readers;`
- C. `GRANT ALL PRIVILEGES ON METASTORE TO bi_readers;`
- D. Run individual `GRANT` statements per table, repeated for every future table

M5. A nightly automated job must write to Unity Catalog tables with an auditable, independently revocable, non-human identity. What should authenticate the job?

- A. A shared personal access token
- B. A service principal
- C. A workspace admin's own login
- D. An Entra ID group containing every engineer

M6. Which object should be attached to a table to make support group members see only rows for their assigned region, while other groups see all rows?

- A. Column mask

- B. Row filter
- C. Tag
- D. Storage credential

M7. What does `OPTIMIZE table_name ZORDER BY (col)` primarily achieve?

- A. Deletes old, unreferenced data files
- B. Compacts small files and co-locates data by the given column(s) to improve file-skipping/pruning for queries filtering on that column
- C. Encrypts the specified column at rest
- D. Converts the table from managed to external

M8. A team needs idempotent, SQL-based batch loading of new files landing periodically in cloud storage, without running a persistent stream, where already-loaded files are automatically skipped on re-run. What should they use?

- A. `COPY INTO`
- B. `MERGE INTO` directly on raw files
- C. A continuously running Auto Loader stream
- D. `dbutils.fs.cp` on a schedule

M9. In a Lakeflow Declarative Pipeline, which decorator/expectation combination causes the entire pipeline update to stop immediately when a row violates the given condition?

- A. `@dlt.expect`
- B. `@dlt.expect_or_drop`
- C. `@dlt.expect_or_fail`
- D. A plain Python `assert` statement inside the transformation

M10. Which Structured Streaming feature bounds memory usage in a stateful windowed aggregation by defining how late data is allowed to arrive before a window's state can be safely dropped?

- A. `dropDuplicates()`
- B. A watermark (`withWatermark`)
- C. `.trigger(once=True)`
- D. `checkpointLocation`

M11. A Delta table needs a guarantee, enforced at write time regardless of how it's later queried, that `order_id` is never null. What should you add?

- A. A row filter
- B. A `NOT NULL` constraint on the column
- C. A tag marked `required=true`
- D. A materialized view wrapping the table

M12. What is the effect of running `VACUUM my_table RETAIN 0 HOURS` on a busy production table?

- A. It is always perfectly safe with no downside
- B. It can delete files still needed by concurrent readers or recent time-travel queries, since it disables the normal retention safety margin
- C. It rewrites the table's schema
- D. It only ever affects the table's `_delta_log`, never data files

M13. In a medallion architecture, which layer is responsible for holding cleansed, deduplicated, conformed data — an intermediate step between raw ingestion and curated business aggregates?

- A. Bronze
- B. Silver
- C. Gold
- D. Platinum

M14. A source produces a large batch CSV file once nightly that fully overwrites the prior file, and only the latest snapshot is needed downstream. What is the most cost-efficient ingestion design?

- A. A 24/7 continuously running Auto Loader stream
- B. A Lakeflow Job scheduled shortly after the file lands, doing a batch read/overwrite, with the job cluster terminating afterward
- C. `.trigger(processingTime="1 second")` running all day
- D. A Delta Sharing share pulling from the source

M15. In a multi-task Lakeflow Job, what happens by default to a downstream task whose only upstream dependency fails?

- A. It runs anyway, independent of upstream status
- B. It is skipped
- C. It automatically retries the upstream task instead
- D. The whole job definition is deleted

M16. Which compute choice is generally most cost-efficient for a scheduled, unattended production Lakeflow Job task, as compared to attaching it to a persistent all-purpose cluster?

- A. A job cluster, which is billed at a lower DBU rate and terminates automatically when the run finishes
- B. An all-purpose cluster kept running 24/7 “just in case”
- C. They are always priced identically with no difference
- D. A SQL warehouse, since job tasks can never use job clusters

M17. Which Databricks Asset Bundle CLI commands are the correct sequence to validate and then deploy a bundle to a prod target?

- A. `databricks bundle destroy --target prod` then `databricks bundle deploy --target prod`
- B. `databricks bundle validate --target prod` then `databricks bundle deploy --target prod`

- C. `databricks fs cp -r . dbfs:/prod/`
- D. `databricks clusters create --target prod`

M18. A Spark UI query profile shows one task out of 300 taking 25 minutes while the rest finish in seconds, within the same stage. What does this most likely indicate?

- A. Data skew on a join or grouping key
- B. A corrupted `_delta_log`
- C. Normal, expected Spark behavior requiring no investigation
- D. Insufficient Azure subscription quota

M19. Which system table would you query to build a monthly DBU cost breakdown by workspace and job?

- A. `system.lineage.table_lineage`
- B. `system.billing.usage`
- C. `system.access.audit`
- D. `system.compute.warehouse_events` only, with no billing table involved

M20. A join between a 5-billion-row fact table and a 2,000-row dimension table is slow due to a large shuffle. What technique most directly addresses this?

- A. Increase shuffle partitions to an arbitrarily large number with no other change
- B. Use a broadcast join so the small dimension table is sent to every executor, avoiding a shuffle of the large fact table
- C. Convert both tables to CSV
- D. Remove the join and query each table separately

6.6 Answer Key — Final Mixed Mock Exam

M1. Correct answer: B. A cluster policy is the mechanism that enforces hard configuration/cost limits (like a DBU/hour ceiling) on every cluster created under it, regardless of who creates it.

M2. Correct answer: C. “Shared (Unity Catalog)” access mode supports multiple concurrent users on one cluster with per-user process isolation and full Unity Catalog enforcement; “No Isolation Shared” does not enforce UC governance.

M3. Correct answer: B. Specifying an explicit `LOCATION` creates an external table; `DROP TABLE` only removes the metastore registration, leaving the underlying files in cloud storage untouched.

M4. Correct answer: A. A schema-level `GRANT SELECT` applies to all current and future tables/views in that schema through Unity Catalog’s privilege inheritance; wildcard table syntax (`sales.gold.*`) is not valid grant syntax.

M5. Correct answer: B. A service principal is the non-human Entra ID identity purpose-built for automated jobs, giving auditable, independently revocable access without tying automation to a person’s account.

M6. Correct answer: B. A row filter dynamically restricts which entire rows a principal sees based on a predicate (e.g., group-to-region mapping); a column mask instead redacts individual field values, not whole rows.

M7. Correct answer: B. `OPTIMIZE ... ZORDER BY` compacts small files and co-locates data by the given column(s), improving data-skipping/file pruning for queries that filter on that column.

M8. Correct answer: A. `COPY INTO` is the SQL command designed for idempotent, incremental batch loading that automatically tracks and skips already-loaded files, without needing a persistent stream.

M9. Correct answer: C. `@dlt.expect_or_fail` halts the entire pipeline update immediately on a violation, unlike `@dlt.expect` (reports only) or `@dlt.expect_or_drop` (drops bad rows but keeps the pipeline running).

M10. Correct answer: B. A watermark defines the allowed lateness threshold for event-time data, letting Structured Streaming safely evict state for windows older than that threshold and bound memory usage.

M11. Correct answer: B. A `NOT NULL` constraint is enforced by Delta Lake at write time on any insert/merge/streaming write, guaranteeing the property independent of how the table is later queried.

M12. Correct answer: B. `RETAIN 0 HOURS` disables the normal safety retention window, risking deletion of files still needed by concurrent readers or recent time-travel queries.

M13. Correct answer: B. The silver layer holds cleansed, conformed, deduplicated data — refined beyond raw bronze ingestion but not yet aggregated into gold-layer business-ready tables.

M14. Correct answer: B. A scheduled Lakeflow Job with a job cluster that spins up, performs the batch overwrite, and terminates is far more cost-efficient than running continuous streaming infrastructure for a single predictable nightly snapshot.

M15. Correct answer: B. By default, a task whose upstream dependency did not succeed is skipped, since its declared precondition for running was not met.

M16. Correct answer: A. Job clusters are ephemeral, spun up per run and torn down automatically at completion, and are billed at a lower DBU rate than persistent all-purpose clusters — making them the cost-efficient default for scheduled production tasks.

M17. Correct answer: B. `databricks bundle validate` followed by `databricks bundle deploy --target <target>` is the standard Asset Bundle validate-then-deploy CI/CD sequence.

M18. Correct answer: A. One dramatically slower task among many fast, similar ones in the same stage is the classic signature of data skew on a join/grouping key.

M19. Correct answer: B. `system.billing.usage` contains the granular DBU/usage records (by workspace, cluster, job, etc.) needed to build cost/chargeback reporting.

M20. Correct answer: B. A broadcast join sends the small dimension table to every executor, avoiding an expensive shuffle of the large fact table and directly resolving this asymmetric join's slowness.

Chapter 7

Part 7 — Interview Preparation, Quick Reference & Your 30-Day Study Calendar

This is the chapter you keep open in two very different rooms: the room where you’re waiting to walk into a technical interview, and the room where you’re sitting down to take the DP-750 exam. Everything in the first six parts of this book taught you *how* Azure Databricks works. This part is about *performance* — saying the right things out loud under pressure, recalling syntax fast, and having a plan for the next four weeks so none of this feels rushed.

A quick honesty note before we start: none of the interview answers below are “correct” in the way a multiple-choice exam question has a correct answer. They’re **templates**. Read them, then rewrite them in your own words with your own project details swapped in. An interviewer can always tell the difference between a memorized paragraph and someone who understands what they’re saying — the goal here is to give you the skeleton so you can focus your energy on the specifics of what you actually built.

7.1 1. 30 Real-World Interview Questions & Model Answers

These are grouped the way a real Data Engineer interview loop is usually structured: a recruiter or hiring manager screen touches governance and “tell me about yourself” territory, a technical round goes deep on Delta Lake and pipelines, and a system-design or bar-raiser round covers orchestration, performance, and cost. Answers are written in first person so you can practice saying them out loud. Where it’s natural, they reference “a lakehouse project I built” — this points at the retail lakehouse capstone from Part 5 of this book; swap in your own project’s nouns (tables, data volumes, business problem) once you’ve built it.

7.1.1 A. Unity Catalog & Governance (6 questions)

1. What is Unity Catalog, and why does it matter compared to the old Hive metastore approach?

Unity Catalog is a centralized governance layer that sits above all my workspaces and gives me one place to manage permissions, lineage, and auditing for every table, view, volume, and model — instead of a separate, workspace-scoped Hive metastore per workspace with no shared identity. The big win is the three-level namespace: `catalog.schema.table`, which lets me separate environments or business units at the catalog level instead of hacking around it with naming conventions. In a lakehouse project I built, I used one catalog per environment — dev, staging, prod — so a `SELECT * FROM prod.sales.orders` was unambiguous and access-controlled from day one. It also gives me automatic lineage and a single audit trail, which matters a lot the first time a compliance team asks “who touched this table and when.”

2. Walk me through how you’d grant a data analyst read-only access to one schema, without giving them access to the whole catalog.

I’d start from the top of the namespace and grant only what’s needed at each level, because Unity Catalog privileges are hierarchical — you need `USE CATALOG` on the catalog and `USE SCHEMA` on the schema before any table-level grant actually does anything. So I’d run `GRANT USE CATALOG ON CATALOG retail TO analysts`, then `GRANT USE SCHEMA ON SCHEMA retail.gold TO analysts`, and finally `GRANT SELECT ON SCHEMA retail.gold TO analysts` to cover every current and future table in that schema in one statement. I’d deliberately avoid granting anything on `bronze` or `silver`, since those often contain PII or unvalidated data that analysts shouldn’t see directly. I always prefer granting to a group over an individual user, because it makes onboarding and offboarding a group-membership change instead of a re-audit of every object.

3. What’s the difference between a managed table and an external table in Unity Catalog, and when would you choose each?

A managed table has its data files stored and fully owned by Unity Catalog in the catalog’s managed storage location — if I `DROP TABLE`, the underlying data is deleted too. An external table just registers metadata pointing at files I already control in my own storage path (via an external location), so dropping the table only removes the registration, not the data. I default to managed tables for anything Databricks fully owns, like my silver and gold layers in a lakehouse project, because I get automatic optimization, cleaner governance, and one less thing to worry about. I reach for external tables when another system also needs to read those same files directly, or when a landing zone’s raw files need to outlive the table registration for compliance reasons.

4. How would you make sure a support engineer can see aggregated sales numbers but not the customer’s raw email address in the same table?

That’s exactly what row filters and column masks are for, and I’d apply a column mask directly on the table. I’d write a SQL function that returns the real value for privileged roles and a masked or null value otherwise, then attach it with `ALTER TABLE customers ALTER COLUMN email SET MASK mask_email`; . If they also shouldn’t see certain regions’ rows, I’d add a row filter function the same way, applied at the table level with `ALTER TABLE ... SET ROW FILTER`. The nice part is the support engineer runs the exact same query as everyone else — the masking is invisible and centrally enforced, so I’m not relying on every downstream dashboard to remember to hide the column.

5. What are Unity Catalog system tables, and how have you used them?

System tables are Databricks-managed Delta tables under the system catalog that expose operational metadata — things like `system.access.audit`, `system.billing.usage`, and `system.lineage.table_lineage` — queryable with plain SQL instead of pulling logs from somewhere else. I’ve used `system.billing.usage` to build a simple dashboard that broke down DBU spend by workspace and job, which is how I first noticed a job whose cluster was oversized for the data volume it processed. I’ve also used the lineage tables to answer “what breaks downstream if I change this column” before making a schema change, instead of guessing.

6. What’s the difference between a service principal and a user account, and when should a pipeline use which?

A user account represents a person and is tied to their identity in Microsoft Entra ID, while a service principal is a non-human identity meant for applications, jobs, and automation. I always run production Lakeflow Jobs and CI/CD deployments under a service principal, never under my own user account, because a job tied to my identity breaks the moment I leave the team or lose access, and it also makes audit logs confusing — you can’t tell if a change was a human action or an automated one. In a lakehouse project I built, I created a dedicated service principal for the deployment pipeline, granted it exactly the catalog and schema privileges the pipeline needed, and used its client secret (stored in a secret scope, never hardcoded) for the CI/CD service connection.

7.1.2 B. Delta Lake Internals (6 questions)

7. Explain Delta Lake to someone who only knows plain Parquet.

Parquet is just a columnar file format — it has no idea what other files belong to the same table, no transaction guarantees, and no way to safely handle two writers at once. Delta Lake adds a transaction log — the `_delta_log` folder — on top of Parquet files, and every write becomes an atomic, versioned JSON/Parquet commit to that log. That log is what gives me ACID transactions, time travel, schema enforcement, and the ability to run MERGE safely, none of which plain Parquet can offer on its own. Practically, it means I can have a streaming job and a batch job writing to the same table without corrupting it, which was non-negotiable in a lakehouse project I built where bronze ingestion was continuous.

8. How does time travel actually work under the hood?

Every write to a Delta table creates a new numbered JSON commit file in `_delta_log`, and periodically Delta writes a checkpoint Parquet file that summarizes all commits up to that point so readers don’t have to replay the entire log from version zero. Time travel just means reading the table as of a specific version or timestamp — `SELECT * FROM orders VERSION AS OF 12` or `TIMESTAMP AS OF '2026-08-01'` — which Delta resolves by reconstructing exactly which data files were “active” at that point from the log. I’ve used it in practice to recover from a bad MERGE — I ran `RESTORE TABLE orders TO VERSION AS OF 45` after a job wrote duplicate rows, which took seconds instead of re-running a multi-hour backfill.

9. What's the difference between OPTIMIZE and VACUUM, and why do you need both?

OPTIMIZE compacts many small files into fewer, larger files — it solves the “small file problem” that streaming and frequent-MERGE workloads create, and I can pair it with ZORDER BY to physically co-locate rows that are commonly filtered together. VACUUM is unrelated to file size — it permanently deletes data files that are no longer referenced by the current table version and are older than the retention threshold, freeing up storage and cleaning up after time travel history expires. I run OPTIMIZE regularly, often on a schedule after heavy write windows, but I'm deliberate about VACUUM because it's destructive — once files are vacuumed, time travel to versions that depended on them stops working, so I never drop the retention period below 7 days without a real reason.

10. How does MERGE INTO handle an upsert, and what can go wrong with it at scale?

MERGE INTO compares incoming rows against a target table on a matching key and applies WHEN MATCHED to update existing rows and WHEN NOT MATCHED to insert new ones, all as a single atomic transaction. What goes wrong at scale is usually one of two things: the join condition isn't selective enough, so Delta has to scan and rewrite far more files than necessary, or the source side has duplicate keys, which throws a “multiple source rows matched” error. In a lakehouse project I built, I fixed a slow silver-layer MERGE by partitioning the target table on ingestion date and always filtering the target side of the merge condition on that same date range, which let Delta's file-pruning skip most of the table instead of scanning it all.

11. What is schema evolution versus schema enforcement, and how do you control it?

Schema enforcement is Delta's default behavior of rejecting a write if the incoming data doesn't match the target table's schema — it's a safety net that stops garbage or unexpected columns from silently corrupting a table. Schema evolution is the opt-in override: adding mergeSchema (batch) or .option("mergeSchema", "true") / Auto Loader's cloudFiles.schemaEvolutionMode lets new columns be added automatically instead of failing. I keep enforcement on by default for silver and gold tables, because I want a schema change to be a deliberate decision, not an accident, but I turn evolution on for bronze ingestion with Auto Loader since raw source systems add columns without telling me and I'd rather capture them than lose data.

12. What's a Delta table checkpoint, and does it relate to Structured Streaming checkpoints?

They're two different things that share a name. A Delta *log* checkpoint is that periodic Parquet snapshot of the transaction log I mentioned earlier — purely a read-performance optimization for the table itself. A Structured Streaming *checkpoint* is a separate directory that tracks a streaming query's progress — offsets read, state store data for aggregations — so that if the job restarts, it resumes exactly where it left off instead of reprocessing or dropping data. I always give every streaming write its own dedicated checkpoint path, because reusing one checkpoint across two different streaming queries silently corrupts both of their progress tracking.

7.1.3 C. Ingestion & Lakeflow Pipelines (6 questions)

13. When would you use Auto Loader instead of a plain batch read of a folder?

Auto Loader is built for incremental, ongoing ingestion of files landing in cloud storage — it tracks which files it's already processed (using either directory listing or, at higher scale, file notifications), so re-running the job doesn't reprocess everything. A plain batch read has no memory between runs, so I'd have to build my own "what's new since last time" logic, which is exactly the kind of thing that quietly breaks. In a lakehouse project I built, bronze ingestion used `spark.readStream.format("cloudFiles")` pointed at a landing zone that received new order files throughout the day, and Auto Loader picked each one up automatically and inferred/evolved the schema as new fields appeared.

14. What's the difference between Auto Loader and COPY INTO, and when would you pick one over the other?

Both handle incremental file ingestion into Delta tables, but COPY INTO is a simple, idempotent SQL command best suited to smaller, less frequent batches of files — think a few thousand files at a time — while Auto Loader is built to scale to millions of files and supports both streaming and batch (via `trigger(availableNow=True)`) execution with more advanced schema handling. I'd reach for COPY INTO for a quick one-off load or a simple scheduled job with modest file volume, and Auto Loader for anything continuous, high-volume, or where I need fine control over schema evolution and rescue-data handling.

15. Explain the medallion architecture and why you structure a lakehouse that way.

Medallion architecture is a layered design — bronze, silver, gold — where each layer refines data further. Bronze is raw data landed as-is, with minimal transformation, so I always have an unaltered copy to reprocess from if downstream logic has a bug. Silver applies cleaning, deduplication, type casting, and joins to produce validated, conformed tables. Gold is business-level aggregation — the tables that actually feed dashboards and ML features. The reason I like this structure is separation of concerns and reprocessability: in a lakehouse project I built, I once found a bug in a silver-layer join logic weeks after deployment, and because bronze still held the untouched raw history, I could just rerun silver transformations from scratch instead of re-ingesting from the source system.

16. What are Lakeflow Declarative Pipeline expectations, and how do they differ from just writing a WHERE clause?

Expectations, declared with `@dlt.expect`, `@dlt.expect_or_drop`, and `@dlt.expect_or_fail`, are declarative data quality rules attached directly to a table definition that get tracked, logged, and surfaced in the pipeline's event log automatically — I get pass/fail counts per expectation without building that tracking myself. A plain WHERE clause silently filters rows with no record of how many were dropped or why. In a lakehouse project I built, I used `@dlt.expect_or_drop("valid_order_amount", "order_amount > 0")` on the silver orders table, which let me see in the pipeline UI exactly how many bad rows were quarantined on each run instead of just wondering

why row counts looked off.

17. What's the difference between a streaming table and a materialized view in a Lakeflow Declarative Pipeline?

A streaming table (@dlt.table backed by a streaming read) processes new data incrementally — each run only touches rows that have arrived since the last run — which is what I want for something like continuously appended raw events. A materialized view recomputes its full result based on its query logic each time its inputs change, which fits aggregations or joins where “incremental” doesn’t cleanly apply, like a daily rollup that needs to account for late-arriving updates to old rows. I pick streaming tables for bronze/silver ingestion where append-only, incremental processing is natural, and materialized views for gold-layer aggregates where I want the pipeline engine to figure out the cheapest way to keep the result current.

18. How do you handle late-arriving or out-of-order data in a streaming pipeline?

I use watermarking — `withWatermark("event_time", "30 minutes")` — which tells Structured Streaming how long to wait for late data before finalizing a windowed aggregation and dropping state for that window. The watermark threshold is a real business tradeoff: too short and I lose legitimately late events, too long and I hold more state in memory and delay results. In a lakehouse project I built, clickstream events could arrive up to 20 minutes late from mobile clients with poor connectivity, so I set a 30-minute watermark on the hourly aggregation to safely cover that plus buffer, and I validated the choice by checking the `numLateInputs` metric in the streaming query’s progress logs.

7.1.4 D. Orchestration & CI/CD (6 questions)

19. How do Lakeflow Jobs compare to a tool like Airflow, and why would a company choose one over the other?

Lakeflow Jobs is Databricks’ native orchestrator — tasks run directly on Databricks compute, with built-in support for notebooks, Python scripts, SQL, Lakeflow Declarative Pipelines, and dbt, plus native retry, alerting, and lineage inside the same platform. Airflow is a general-purpose orchestrator that can coordinate work across many systems beyond Databricks — it’s the right choice when a company’s DAG genuinely spans multiple platforms, like triggering a Databricks job, then a Snowflake load, then a Salesforce API call, all in one dependency graph. For a team whose orchestration needs live entirely inside the lakehouse, I’d default to Lakeflow Jobs because it’s simpler to operate, has no separate infrastructure to maintain, and gives tighter native integration with Unity Catalog permissions and job-run lineage.

20. Explain how you’d structure a multi-task Lakeflow Job for a bronze-silver-gold pipeline with proper dependencies.

I’d create one job with four tasks: a bronze ingestion task with no dependencies, a silver transformation task that depends on bronze completing successfully, a data-quality-check task that depends on silver, and a gold aggregation task that depends on the quality check passing. Task dependencies in Lakeflow Jobs are just DAG edges

— I set `depends_on` for each task so gold literally cannot start unless the upstream tasks succeeded. In a lakehouse project I built, I also added a “run if” condition on a notification task so a Slack alert only fired on failure, not on every successful run, since alert fatigue is a real problem once a job runs many times a day.

21. What are Databricks Asset Bundles, and why would you use them instead of clicking through the UI?

Asset Bundles (DABs) let me define jobs, pipelines, and their configuration as YAML files in source control, with separate target environments — dev, staging, prod — driven by the same codebase and deployed with `databricks bundle deploy`. Clicking through the UI works for exploration, but it isn’t reproducible, isn’t reviewable in a pull request, and makes it easy for dev and prod configurations to quietly drift apart. In a lakehouse project I built, I defined the job and pipeline in a `databricks.yml`, and a change to a schedule or cluster size went through a pull request like any other code change, which meant a teammate could review it and I had a clean rollback if it broke something.

22. Walk me through a CI/CD pipeline you’d set up for a Databricks project.

On every pull request, I’d run a CI stage that lints the code, runs unit tests against DataFrame transformation logic locally or on a small job cluster, and validates the bundle configuration with `databricks bundle validate`. On merge to the main branch, a CD stage deploys the bundle to a staging workspace with `databricks bundle deploy -t staging` and runs integration tests against real, smaller-scale data. Promotion to production is a separate, often manually approved step that deploys the same bundle with `-t prod`, so the exact code and configuration that passed staging is what reaches production — no manual re-typing of settings. I always keep environment-specific values like catalog names and cluster sizes in the bundle’s target-level variable overrides, never hardcoded in the notebook logic itself.

23. How do you monitor a production Lakeflow Job and know when something’s actually wrong versus just noisy?

I set up task-level and job-level email or webhook alerts on failure and on duration threshold breaches — a job that normally finishes in 10 minutes taking 45 is often a warning sign before it’s an outright failure. I also query `system.lakeflow.job_run_timeline` and related system tables to track run duration trends over time instead of relying only on reactive alerts. For noise, I’m deliberate about which conditions actually page someone: transient retries that succeed on the second attempt shouldn’t wake anyone up, so I configure retries at the task level and only alert on final failure after retries are exhausted.

24. What’s your strategy for testing a Lakeflow Declarative Pipeline before it hits production?

I run the pipeline in development mode against a staging catalog with a representative but smaller sample of data, which gives me fast iteration and lets me inspect the pipeline’s event log for expectation pass/fail rates before trusting it with real volume. I write unit tests for any pure transformation logic that’s factored out of the pipeline definitions into plain Python functions, so I can test edge cases without spinning up a cluster at all. Only after staging validation and a code review do I promote the same

bundle-defined pipeline to production, which guarantees the SQL/Python logic that passed testing is identical to what runs live.

7.1.5 E. Performance Tuning & Cost (4 questions)

25. A query that used to run in 5 minutes is now taking 40. How do you start investigating?

I'd start with the Spark UI's query profile for that job, specifically looking for a big jump in shuffle read/write volume or a stage with heavy data skew, where one or two tasks take far longer than the rest. Next I'd check whether the underlying table has accumulated a lot of small files — a quick `DESCRIBE DETAIL` shows `numFiles` and average file size — since that alone can wreck read performance if `OPTIMIZE` hasn't run in a while. I'd also check whether data volume itself simply grew, whether a partition column's cardinality changed, or whether someone changed the join condition or added a non-selective filter. In a lakehouse project I built, a "sudden" slowdown turned out to be a `MERGE` whose target table hadn't been `OPTIMIZE`d in weeks, so file count had exploded — a scheduled `OPTIMIZE` job fixed it permanently.

26. When would you choose serverless compute over a classic cluster, and what's the tradeoff?

Serverless compute removes cluster startup and management entirely — jobs and SQL warehouses spin up in seconds against Databricks-managed infrastructure, which is great for spiky or unpredictable workloads and for teams that don't want to tune cluster sizing themselves. Classic (job or all-purpose) clusters give me more control over instance types, cluster policies, and init scripts, and can be cheaper at sustained, predictable, high-volume workloads where I can right-size and use spot instances aggressively. I default to serverless for ad hoc SQL and smaller or bursty jobs where developer time saved outweighs marginal compute cost, and I'd move a large, steady-state production pipeline to a well-tuned classic job cluster once its resource profile is well understood.

27. What's data skew, and how do you fix it?

Skew is when the data isn't evenly distributed across partitions, so one or a few tasks end up processing far more data than the rest and become the bottleneck the whole stage waits on — I usually spot it in the Spark UI as a stage where most tasks finish in seconds but one runs for many minutes. Common causes are a join key or `GROUP BY` column with a few very high-frequency values, like a single "unknown" customer ID that half the rows share. Fixes include salting the skewed key (appending a random suffix to spread it across more partitions and removing it after aggregation), enabling adaptive query execution's skew join optimization, which Databricks turns on by default, or broadcasting the smaller side of a join with `broadcast()` if one table is small enough to fit in memory.

28. How do you control Databricks cost on a team that's new to the platform?

The biggest lever is compute: I'd push job workloads onto job clusters instead of all-purpose clusters, since job clusters exist only for the duration of the run and terminate automatically, while all-purpose clusters tend to sit running (and billing) between

interactive uses. I'd apply cluster policies to cap max cluster size and enforce auto-termination on interactive clusters so someone can't leave a large cluster running over a weekend by accident. I'd also use `system.billing.usage` to build a simple cost dashboard by workspace, job, and tag, so cost overruns get caught within days instead of at the end of the month, and I'd enable Photon selectively where I can show it actually reduces wall-clock time enough to offset its higher per-DBU rate.

7.1.6 F. Behavioral-Technical (2 questions)

29. Tell me about a data pipeline you're proud of.

I built a retail lakehouse pipeline that ingested daily order, inventory, and customer files into a medallion architecture on Databricks. What I'm actually proud of isn't the happy path — it's that I designed it to fail safely: bronze ingestion with Auto Loader kept the raw history so nothing was ever truly lost, expectations on the silver layer quarantined bad rows instead of silently corrupting downstream tables, and the whole thing was deployed through a Databricks Asset Bundle so a change went through code review before touching production. The part I'd highlight technically is a MERGE-based upsert into the silver orders table that I tuned by partitioning on order date and aligning the merge condition with that partition, which cut runtime from minutes to seconds once volume grew. It taught me that a pipeline's real quality shows up not on a normal day, but on the day the upstream data is malformed and everything downstream still behaves predictably.

30. Walk me through how you'd handle a production pipeline failure at 2 a.m.

First, I'd triage before diagnosing — check the job's alert or run page to see exactly which task failed and read the error message and stack trace rather than guessing, and check whether it's an isolated failure or a symptom of something bigger like an upstream source outage. If it's a transient issue — a brief network blip, a source system timeout — I'd let the configured retry handle it or manually retry the specific failed task rather than the whole job, since Lakeflow Jobs supports retrying just the failed task and its downstream dependents. If it's a real data or code issue, I'd assess blast radius: has bad data already landed in a table other jobs or dashboards read from? If so, I'd consider whether time travel gives me a fast, safe rollback (`RESTORE TABLE ... TO VERSION AS OF`) while I fix the root cause during business hours, rather than trying to hot-fix complex logic half-asleep. Afterward, I'd write a short postmortem — what broke, why the alert either caught it well or too late, and one concrete change (a new expectation, a better alert threshold, a data contract with the upstream team) to reduce the chance of a repeat.

7.2 2. Scenario / System-Design Style Questions

These questions don't have a single right answer — what an interviewer is actually evaluating is whether you can structure your thinking out loud: clarify the problem, propose an architecture, name the tradeoffs, and mention how you'd know it's work-

ing. Use this four-step shape for any scenario question you haven't seen before, even outside this list.

7.2.1 1. “Design a near-real-time pipeline that ingests clickstream events and serves an hourly dashboard with under 15 minutes of latency.”

- **Clarify requirements:** I'd ask about event volume (thousands vs. millions of events/hour), the source (Kafka/Event Hubs vs. files landing in storage), whether “under 15 minutes” is end-to-end from event to dashboard, and whether late/out-of-order events need to be handled or can be dropped.
- **Architecture:** Ingest from Azure Event Hubs using Structured Streaming directly into a bronze Delta table (streaming table in a Lakeflow Declarative Pipeline), append-only, minimal transformation. A silver streaming table cleans and deduplicates using `dropDuplicatesWithinWatermark`. A gold materialized view or a windowed streaming aggregation (`window("event_time", "1 hour")` with a watermark) produces the hourly rollup that a dashboard (Databricks SQL dashboard on a serverless SQL warehouse, refreshed on a short schedule) queries.
- **Tradeoffs:** A shorter watermark reduces latency but risks dropping legitimately late events; a longer one is safer but delays finalized numbers. I'd also weigh Structured Streaming's continuous processing against `trigger(availableNow=True)` running every few minutes — the latter is cheaper (no cluster running 24/7) but adds its own scheduling latency, so for a 15-minute SLA I'd likely need true continuous streaming with a small always-on cluster or serverless streaming compute.
- **Monitoring/cost:** I'd track streaming query progress metrics (input rate vs. processing rate, to catch the pipeline falling behind) and alert if `batchDuration` trends upward, plus watch DBU cost from keeping a cluster running continuously versus the business value of sub-15-minute freshness.

7.2.2 2. “A silver table's MERGE is taking 40 minutes and used to take 5 — how do you investigate?”

- **Clarify:** Did this happen gradually (data growth) or suddenly (a code or data change)? What's the current row/file count versus historical?
- **Approach:** Check `DESCRIBE DETAIL` for file count and average file size — small-file bloat is the most common culprit. Check the query profile in the Spark UI for skew in the join/shuffle stage. Check whether the MERGE condition is still selective (partition pruning working) or whether a recent change made it scan the full table. Check whether source-side duplicate keys are forcing extra matching work.
- **Tradeoffs:** A quick fix (`OPTIMIZE` now) buys time but doesn't prevent recurrence — I'd propose a scheduled `OPTIMIZE` job as the durable fix, weighed against the compute cost of running it regularly versus the cost of a slow MERGE every run.
- **Monitoring:** I'd add file-count and duration tracking to the job's own metrics or system tables so this trend is visible before it becomes an 8x slowdown next time, not after.

7.2.3 3. “You need to give three business units read access to shared gold tables, but each should only see their own region’s data. Design the access model.”

- **Clarify:** Is “region” a column already in the table, or does it need to be derived? Are there users who legitimately need to see all regions (e.g., a global finance team)?
- **Approach:** Use Unity Catalog row filters — one SQL function that checks the calling user’s group membership (via `is_account_group_member()`) and returns a boolean per row, applied with `ALTER TABLE ... SET ROW FILTER`. Create one group per business unit in Microsoft Entra ID, mapped into Databricks account groups, and grant `SELECT` on the shared gold schema to all three groups equally — the row filter, not separate grants, does the actual segmentation.
- **Tradeoffs:** Row filters centralize the logic in one place (good — one function to audit) but add a small per-query overhead and require care that the filter function itself is airtight, since a bug there is a data leak, not just a bug.
- **Monitoring:** I’d periodically query `system.access.audit` to confirm no user outside a region’s group is querying that schema, and I’d write a couple of automated tests that log in as a representative user per group and assert row counts match expectations.

7.2.4 4. “Design a pipeline to ingest data from an on-prem SQL Server database into the lakehouse, refreshed nightly.”

- **Clarify:** Full refresh or incremental (change tracking / CDC available on the source)? Network path — VPN/ExpressRoute already exists?
- **Approach:** If CDC/change-tracking is available, use Lakeflow Connect (or a JDBC-based extraction job) to pull only changed rows nightly into bronze, then `MERGE` into silver keyed on primary key. If no CDC, a full nightly extract via JDBC into bronze, followed by a `MERGE` against silver using a business key, is simpler to build though heavier on the source system.
- **Tradeoffs:** Full nightly extracts are simple and self-healing (each run is independent) but scale poorly and put more load on the source database; CDC-based incremental loads are efficient but add complexity and a dependency on the source system’s change-tracking staying enabled and reliable.
- **Monitoring/cost:** I’d track extraction duration and row counts night over night to catch a silent CDC gap (row counts should track known business volume), and alert if the nightly job doesn’t complete before the first downstream consumer expects fresh data.

7.2.5 5. “How would you migrate an existing on-prem Hadoop/Hive data warehouse to Azure Databricks with Unity Catalog?”

- **Clarify:** What’s the actual data volume and how many downstream consumers/reports depend on current table names and locations? Is there a hard cutover date or can both run in parallel?
- **Approach:** I’d design the target Unity Catalog structure first — catalogs per environment, schemas mirroring the logical domains of the old warehouse —

then migrate data in phases, starting with less business-critical tables to validate the process. Bulk-copy historical data into ADLS Gen2 (via azcopy or Data Factory), register it as external or migrate it to managed Delta tables, and stand up Lakeflow Declarative Pipelines to replace legacy ETL jobs one domain at a time, running old and new in parallel to diff results before cutting reports over.

- **Tradeoffs:** A “big bang” cutover is faster to complete but riskier; a phased, parallel-run migration takes longer and costs more (running two systems at once) but gives a much safer rollback path and lets you validate data parity table by table.
- **Monitoring/cost:** I’d build row-count and checksum comparisons between old and new systems per migrated table as an explicit go/no-go gate, and track migration cost (storage + compute for parallel running) against a target completion date to avoid an open-ended dual-run.

7.2.6 6. “Design a data quality strategy for a pipeline that feeds a financial reporting dashboard, where a wrong number is a serious problem.”

- **Clarify:** What specifically counts as “wrong” — missing rows, duplicate rows, values outside expected ranges, referential integrity against a dimension table?
- **Approach:** Use `@dlt.expect_or_fail` (not just `expect` or `expect_or_drop`) on the critical financial fields so the pipeline run itself fails loudly rather than silently publishing a partial or incorrect gold table — for financial data, I’d rather the dashboard show stale-but-correct numbers than fresh-but-wrong ones. I’d add reconciliation checks — row counts and sum totals compared against the source system — as a separate validation task in the job, gating whether the gold refresh is “published” (e.g., via table swap or a `_valid` flag) that the dashboard reads.
- **Tradeoffs:** Failing hard on any violation protects correctness but risks the dashboard going stale if a legitimate edge case trips an overly strict expectation — so I’d tune thresholds carefully and version-control the expectations alongside code review, treating them as seriously as the transformation logic itself.
- **Monitoring:** I’d alert immediately on any `expect_or_fail` violation, and separately track a daily reconciliation report so a finance stakeholder — not just engineering — has visibility into data trustworthiness.

7.2.7 7. “A stakeholder says the gold table ‘sales_by_region’ is missing yesterday’s data. Walk me through your debugging process.”

- **Clarify:** Is this specific to one region or the whole table? Did the job report success or failure?
- **Approach:** First check the job run history — did the job actually run and succeed for yesterday’s schedule, or did it fail/get skipped? If it succeeded, check the silver table it reads from for yesterday’s data — the gap might be upstream. If silver has the data but gold doesn’t, check whether the gold materialized view/aggregation logic has a filter or join that’s silently excluding rows (a classic culprit: an inner join against a dimension table missing a new key). I’d use time travel to compare yesterday’s table version against today’s to see exactly what changed.

- **Tradeoffs:** N/A for pure debugging, but I’d note the process tradeoff — checking job logs first is faster than re-running queries blind, so I always start at the orchestration layer and work down through bronze → silver → gold rather than guessing at the SQL first.
- **Monitoring:** This is exactly the gap a `expect_or_fail` and a reconciliation row-count check (scenario 6) would have caught automatically instead of a stakeholder finding it first — I’d propose adding one after resolving the immediate issue.

7.2.8 8. “Design a disaster recovery / business continuity approach for a critical Databricks pipeline.”

- **Clarify:** What’s the actual RTO/RPO requirement — is losing a few hours of data acceptable, or does it need near-zero data loss? Single-region or must it survive a regional outage?
- **Approach:** For data durability, Delta Lake on ADLS Gen2 already gives strong built-in redundancy (geo-redundant storage options), and time travel plus `RESTORE TABLE` covers most “we broke something” scenarios without needing a separate backup system. For true regional failover, I’d look at Delta Sharing or cross-region replication of critical tables, plus Asset Bundles making it fast to redeploy the same jobs/pipelines into a secondary workspace in another region.
- **Tradeoffs:** True multi-region active-active is expensive and operationally complex — for most teams, a well-tested “redploy from source control into a standby workspace” runbook, tested periodically, is a much more realistic and cost-effective RPO/RTO tradeoff than always-on duplication.
- **Monitoring/cost:** I’d track storage redundancy costs against the business cost of downtime, and I’d insist on actually running a DR drill on a schedule — an untested recovery plan is not a recovery plan.

7.2.9 9. “You’re asked to reduce a Databricks bill by 30% without hurting pipeline reliability. What do you do?”

- **Clarify:** Where is spend actually concentrated — is it compute (job/cluster DBUs), storage, or a specific handful of expensive jobs? I’d pull `system.billing.usage` first rather than guessing.
- **Approach:** Likely levers, in rough order of effort-to-impact: switch interactive-only all-purpose clusters that sit idle to auto-terminate aggressively; move recurring jobs from all-purpose to job clusters; right-size clusters that are clearly over-provisioned relative to their actual data volume (visible in the Spark UI’s low executor utilization); enable Photon selectively where it demonstrably reduces wall-clock time enough to offset cost; consolidate small, frequent jobs where reasonable to reduce cluster-startup overhead; evaluate serverless for bursty workloads instead of paying for idle capacity.
- **Tradeoffs:** Aggressive auto-termination saves cost but adds cluster startup latency to the next run — fine for a nightly batch job, potentially bad for something latency-sensitive. Down-sizing clusters too far risks reliability (OOM errors, longer runtimes) which is exactly what I was told not to hurt, so I’d change one variable at a time and measure before moving to the next.

- **Monitoring:** I'd set up a cost dashboard from system tables broken down by job/tag before making changes, so I have a real before/after number to report back, not a guess.

7.2.10 10. “Design the ingestion and modeling approach for a slowly changing dimension, like customer address, where you need to track history.”

- **Clarify:** Does the business need full history (every past address, with valid-from/valid-to dates — SCD Type 2) or just the current value (SCD Type 1, overwrite)?
- **Approach:** For SCD Type 2, I'd use `MERGE INTO` with conditional logic: when a matched row's tracked columns differ, close out the old row (`UPDATE SET valid_to = current_date, is_current = false`) and insert a new row with the new values and `is_current = true` — Delta's `MERGE` supports multiple `WHEN MATCHED/WHEN NOT MATCHED` clauses in one statement to do exactly this in one atomic operation. I'd model this as a dedicated silver or gold dimension table, keyed on a surrogate key plus the business key, so joins from fact tables are stable even as the dimension changes.
- **Tradeoffs:** SCD Type 2 is more complex to build and query (consumers need to know to filter on `is_current` or a date range) but preserves history that Type 1 permanently discards — I'd only push for Type 2 where the business genuinely needs “what did we know at the time,” since it's real added complexity.
- **Monitoring:** I'd validate periodically that exactly one row per business key has `is_current = true`, since a bug in the `MERGE` logic here tends to silently produce zero or multiple “current” rows rather than an obvious failure.

7.3 3. Command & Syntax Cheat Sheet

Keep this section bookmarked — it's meant for fast lookups during the exam's SQL/code questions and during a live coding interview, not for reading start to finish.

7.3.1 3.1 Core PySpark DataFrame Operations

Command	What it does	Example
<code>spark.read.format(...).load(path)</code>	Reads data into a DataFrame from a given format/path	<code>spark.read.format("parquet").load("/mnt/raw/orders")</code>
<code>spark.readStream.format("cloudFiles")</code>	Starts an Auto Loader streaming read	<code>.option("cloudFiles.format", "json")</code>
<code>df.select(...)</code>	Chooses a subset of columns	<code>df.select("id", "amount")</code>
<code>df.filter(...)</code> / <code>df.where(...)</code>	Keeps rows matching a condition	<code>df.filter(col("amount") > 0)</code>

Command	What it does	Example
<code>df.withColumn(...)</code>	Adds or replaces a column	<code>df.withColumn("total", col("qty")*col("price"))</code>
<code>df.drop(...)</code>	Removes column(s)	<code>df.drop("temp_col")</code>
<code>df.groupBy(...).agg(...)</code>	Groups rows and aggregates	<code>df.groupBy("region").agg(sum("sales"))</code>
<code>df.join(other, on, how)</code>	Joins two DataFrames	<code>orders.join(customers, "cust_id", "left")</code>
<code>df.dropDuplicates([...])</code>	Removes duplicate rows by key	<code>df.dropDuplicates(["order_id"])</code>
<code>df.withWatermark(col, threshold)</code>	Bounds lateness for streaming aggregation	<code>.withWatermark("event_time", "30 minutes")</code>
<code>df.write.format("delta").mode(...).save/saveAsTable(...)</code>	Writes a DataFrame out as Delta	<code>df.write.format("delta").mode("overwrite").saveAsTable("silver.orders")</code>
<code>df.writeStream.trigger(...).start()</code>	Starts a streaming write	<code>.trigger(availableNow=True).start()</code>
<code>df.repartition(n) / df.coalesce(n)</code>	Changes partition count (shuffle vs. no-shuffle)	<code>df.coalesce(4)</code> before writing to avoid small files
<code>df.cache() / df.persist()</code>	Marks a DataFrame to keep in memory/disk across actions	<code>df.cache()</code> before multiple reuses
<code>df.explain(True)</code>	Shows the physical/logical query plan	Used to debug performance
<code>df.printSchema()</code>	Prints the DataFrame's schema tree	Quick sanity check after a read
<code>pivot()</code>	Pivots values in a column into new columns	<code>df.groupBy("region").pivot("month").sum("sales")</code>
<code>broadcast(df)</code>	Hints the optimizer to broadcast a small DataFrame in a join	<code>big_df.join(broadcast(small_df), "key")</code>

7.3.2 3.2 Spark SQL / Delta SQL Commands

Command	What it does	Example
<code>CREATE TABLE ... USING DELTA</code>	Creates a managed Delta table	<code>CREATE TABLE sales.orders (id INT, amt DOUBLE) USING DELTA;</code>
<code>CREATE TABLE ... LOCATION</code>	Creates an external Delta table at a path	<code>CREATE TABLE t (...) USING DELTA LOCATION 'abfss://.../t';</code>
<code>CREATE TABLE ... AS SELECT (CTAS)</code>	Creates a table from a query result	<code>CREATE TABLE gold.summary AS SELECT region, sum(amt) FROM silver.orders GROUP BY region;</code>

Command	What it does	Example
MERGE INTO	Upsert: update matched rows, insert new ones	MERGE INTO t USING s ON t.id=s.id WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *;
OPTIMIZE	Compacts small files into larger ones	OPTIMIZE sales.orders;
OPTIMIZE ... ZORDER BY	Compacts and co-locates data by column for faster filtering	OPTIMIZE sales.orders ZORDER BY (customer_id);
VACUUM	Permanently deletes unreferenced old data files	VACUUM sales.orders RETAIN 168 HOURS;
... VERSION AS OF n	Time travel by version number	SELECT * FROM sales.orders VERSION AS OF 12;
... TIMESTAMP AS OF 'ts'	Time travel by timestamp	SELECT * FROM sales.orders TIMESTAMP AS OF '2026-08-01';
RESTORE TABLE ... TO VERSION AS OF	Rolls a table back to a prior version	RESTORE TABLE sales.orders TO VERSION AS OF 45;
DESCRIBE HISTORY	Shows the table's commit history	DESCRIBE HISTORY sales.orders;
DESCRIBE DETAIL	Shows table metadata: file count, size, location	DESCRIBE DETAIL sales.orders;
ALTER TABLE ... ADD COLUMN	Adds a column	ALTER TABLE sales.orders ADD COLUMN discount DOUBLE;
ALTER TABLE ... SET TBLPROPERTIES	Sets table-level configuration	ALTER TABLE t SET TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true');
COPY INTO	Idempotent incremental batch file load	COPY INTO sales.orders FROM '/mnt/raw' FILEFORMAT = JSON;
DELETE FROM	Deletes rows from a Delta table	DELETE FROM sales.orders WHERE amount < 0;
UPDATE	Updates matching rows	UPDATE sales.orders SET status='cancelled' WHERE id=5;
CLONE (SHALLOW/DEEP)	Copies a table's metadata (shallow) or data+metadata (deep)	CREATE TABLE t2 SHALLOW CLONE t1;

7.3.3 3.3 Databricks CLI Commands

Command	What it does	Example
databricks configure	Sets up CLI authentication	databricks configure --token
databricks workspace ls	Lists workspace objects	databricks workspace ls /Users/me
databricks workspace import	Uploads a notebook/file to the workspace	databricks workspace import ./nb.py /Users/me/nb

Command	What it does	Example
<code>databricks fs cp</code>	Copies files to/from DBFS/volumes	<code>databricks fs cp local.csv dbfs:/mnt/raw/</code>
<code>databricks clusters list</code>	Lists clusters	<code>databricks clusters list</code>
<code>databricks clusters create</code>	Creates a cluster from a JSON spec	<code>databricks clusters create --json-file cluster.json</code>
<code>databricks jobs create</code>	Creates a job from a JSON spec	<code>databricks jobs create --json-file job.json</code>
<code>databricks jobs run-now</code>	Triggers a job run immediately	<code>databricks jobs run-now --job-id 123</code>
<code>databricks jobs list-runs</code>	Lists recent runs of jobs	<code>databricks jobs list-runs --job-id 123</code>
<code>databricks bundle init</code>	Scaffolds a new Asset Bundle project	<code>databricks bundle init</code>
<code>databricks bundle validate</code>	Validates a bundle's YAML config	<code>databricks bundle validate -t dev</code>
<code>databricks bundle deploy</code>	Deploys a bundle to a target environment	<code>databricks bundle deploy -t staging</code>
<code>databricks bundle run</code>	Runs a job/pipeline defined in a bundle	<code>databricks bundle run my_job -t dev</code>
<code>databricks secrets create-scope</code>	Creates a secret scope	<code>databricks secrets create-scope my-scope</code>
<code>databricks secrets put-secret</code>	Stores a secret value	<code>databricks secrets put-secret my-scope db-password</code>
<code>databricks catalogs list</code>	Lists Unity Catalog catalogs	<code>databricks catalogs list</code>

7.3.4 3.4 Lakeflow Declarative Pipeline (DLT) Decorators

Decorator/syntax	What it does	Example
<code>@dlt.table</code>	Defines a Delta table materialized by the pipeline	<code>@dlt.table def silver_orders(): ...</code>
<code>@dlt.view</code>	Defines a temporary, non-materialized view within the pipeline	<code>@dlt.view def staging(): ...</code>
<code>@dlt.table(name=..., comment=..., table_properties={...})</code>	Table decorator with metadata options	<code>@dlt.table(comment="cleaned orders")</code>
<code>@dlt.expect("name", "condition")</code>	Flags rows violating a condition but keeps them	<code>@dlt.expect("valid_qty", "qty > 0")</code>
<code>@dlt.expect_or_drop("name", "condition")</code>	Drops rows violating the condition	<code>@dlt.expect_or_drop("valid_id", "id IS NOT NULL")</code>
<code>@dlt.expect_or_fail("name", "condition")</code>	Fails the whole pipeline run if any row violates it	<code>@dlt.expect_or_fail("no_negatives", "amount >= 0")</code>
<code>@dlt.expect_all({...})</code>	Applies multiple expectations from a dict, flagging only	<code>@dlt.expect_all({"a": "x>0", "b": "y IS NOT NULL"})</code>
<code>@dlt.expect_all_or_drop({...})</code>	Multiple expectations, dropping violators	Same shape as above, drop behavior

Decorator/syntax	What it does	Example
<code>@dlt.expect_all_or_fail({...})</code>	Multiple expectations, failing the run on any violation	Same shape as above, fail behavior
<code>dlt.read(...)</code> / <code>dlt.read_stream(...)</code>	Reads another table defined in the same pipeline (batch/stream)	<code>dlt.read_stream("bronze_orders")</code>
<code>spark.readStream.table(...)</code>	Reads a Unity Catalog table as a stream inside a pipeline	Used for cross-pipeline reads
<code>@dlt.append_flow</code>	Appends output from multiple sources into one streaming table	Used for multiplexing sources into one target
<code>APPLY CHANGES INTO (dlt.apply_changes)</code>	Declarative CDC/SCD handling (Type 1 or 2)	<code>dlt.apply_changes(target="dim_customer", source="cdc_feed", keys=["id"], sequence_by="ts")</code>

7.3.5 3.5 Unity Catalog SQL

Command	What it does	Example
<code>CREATE CATALOG</code>	Creates a new catalog	<code>CREATE CATALOG retail;</code>
<code>CREATE SCHEMA</code>	Creates a schema within a catalog	<code>CREATE SCHEMA retail.gold;</code>
<code>CREATE STORAGE CREDENTIAL</code>	Registers a cloud identity Unity Catalog can use to access storage	<code>CREATE STORAGE CREDENTIAL cred1 WITH (...);</code>
<code>CREATE EXTERNAL LOCATION</code>	Registers a governed path in cloud storage tied to a credential	<code>CREATE EXTERNAL LOCATION loc1 URL 'abfss://...' WITH (STORAGE CREDENTIAL cred1);</code>
<code>GRANT ... ON ... TO</code>	Grants a privilege to a user/group	<code>GRANT SELECT ON TABLE retail.gold.sales TO analysts;</code>
<code>REVOKE ... ON ... FROM</code>	Removes a granted privilege	<code>REVOKE SELECT ON TABLE retail.gold.sales FROM analysts;</code>
<code>GRANT USE CATALOG</code>	Required first step to allow any access within a catalog	<code>GRANT USE CATALOG ON CATALOG retail TO analysts;</code>
<code>SHOW GRANTS ON</code>	Shows current privileges on an object	<code>SHOW GRANTS ON TABLE retail.gold.sales;</code>
<code>ALTER TABLE ... SET ROW FILTER</code>	Attaches a row-level security function to a table	<code>ALTER TABLE t SET ROW FILTER region_filter ON (region);</code>
<code>ALTER TABLE ... ALTER COLUMN ... SET MASK</code>	Attaches a column masking function	<code>ALTER TABLE t ALTER COLUMN email SET MASK mask_email;</code>
<code>CREATE VOLUME</code>	Creates a Unity Catalog-governed volume for non-tabular files	<code>CREATE VOLUME retail.bronze.raw_files;</code>

Command	What it does	Example
ALTER SCHEMA ... OWNER TO	Changes ownership of a schema	ALTER SCHEMA retail.gold OWNER TO data-eng-team;
COMMENT ON TABLE	Adds a documentation comment visible in the catalog UI	COMMENT ON TABLE retail.gold.sales IS 'Daily sales rollup';

7.4 4. Glossary of Key Terms

Access mode — A cluster setting that controls what kind of workloads and users can share a cluster and whether Unity Catalog governance is enforced on it (e.g., single-user vs. shared/standard).

ACID — Short for Atomicity, Consistency, Isolation, Durability; the set of guarantees that make sure a database (or Delta table) write either fully happens or doesn't happen at all, even if something crashes mid-write.

ADLS Gen2 (Azure Data Lake Storage Gen2) — Microsoft's cloud storage service that Databricks tables typically sit on top of; it's regular blob storage with a hierarchical file system layered on for folder-like organization.

ARM (Azure Resource Manager) — The management layer of Azure that lets you deploy and organize resources (like a Databricks workspace) as code, usually via JSON or Bicep templates.

Audit log — A record of who did what and when across a workspace or account, used for security review and compliance; in Databricks this is exposed through system tables like `system.access.audit`.

Auto Loader — A Databricks feature (`cloudFiles` format) that incrementally and efficiently ingests new files as they land in cloud storage, keeping track of what it's already processed.

Bronze table — The first, rawest layer in a medallion architecture; data landed close to its original source format with minimal transformation.

Catalog — The top level of Unity Catalog's three-level namespace (`catalog.schema.table`); typically used to separate environments or major business domains.

Checkpoint (streaming) — A directory where Structured Streaming stores progress (offsets, state) so a restarted stream resumes correctly instead of reprocessing or losing data.

Checkpoint (Delta log) — A periodic snapshot of a Delta table's transaction log that speeds up reads by avoiding replaying every commit from the beginning.

Cluster — A group of virtual machines (driver + workers) that runs your Spark code; Databricks manages provisioning them for you.

Cluster policy — An admin-defined template that restricts what settings a user can pick when creating a cluster (e.g., max size, allowed instance types), used to control cost and standardize configuration.

Column mask — A Unity Catalog feature that dynamically hides or obfuscates a column's real value for certain users, while showing the true value to others, enforced centrally on the table.

COPY INTO — A simple, idempotent SQL command for incrementally loading files into a Delta table, best suited to smaller or less frequent batch loads.

Cron trigger — A schedule expressed in cron syntax (minute/hour/day/month/weekday) used to run a Lakeflow Job automatically at set times.

Data lineage — A record of how data flows and transforms from source tables through to downstream tables and dashboards, automatically tracked by Unity Catalog.

Delta Lake — An open-source storage layer that adds a transaction log on top of Parquet files, giving ACID transactions, schema enforcement, and time travel to data lake tables.

Delta Sharing — An open protocol for securely sharing live data (tables) across organizations or platforms without copying the data.

Dev/staging/prod — The standard set of separate environments a codebase moves through: development (where you build and break things), staging (a production-like environment for final testing), and production (what real users/systems depend on).

DLT (Delta Live Tables) — The former product name for what is now called Lakeflow Declarative Pipelines; the `@dlt` decorator syntax in code is unchanged even though the product name changed.

External location — A Unity Catalog object that registers a specific cloud storage path as governed, tied to a storage credential, so tables can be created there under access control.

External table — A Unity Catalog table whose data files live outside Unity Catalog's managed storage; dropping the table removes only the registration, not the underlying files.

Gold table — The most refined, business-ready layer in a medallion architecture; typically aggregated data that feeds dashboards, reports, or ML features.

Job cluster — A cluster created automatically for a specific job run and terminated automatically when it finishes; generally cheaper than leaving a cluster running.

Lakeflow Connect — A Databricks-native set of connectors for ingesting data from common source systems (databases, SaaS apps) directly into the lakehouse.

Lakeflow Declarative Pipelines — The current product name for what used to be called Delta Live Tables (DLT); a framework for declaratively defining ETL pipelines using `@dlt` decorators, with built-in data quality (expectations), lineage, and incremental processing.

Lakeflow Jobs — The current product name for what used to be called Databricks Workflows; Databricks' native orchestrator for scheduling and running notebooks, scripts, SQL, and pipelines with dependencies.

Managed table — A Unity Catalog table whose data files are fully owned and stored by Unity Catalog; dropping it deletes the underlying data too.

Medallion architecture — A layered lakehouse design pattern of bronze (raw), silver (cleaned/conformed), and gold (aggregated/business-ready) tables.

MERGE (upsert) — A single SQL/DataFrame operation that updates existing matching rows and inserts new non-matching rows in one atomic transaction.

Metastore — The central metadata store that tracks what catalogs, schemas, tables, and permissions exist; in Unity Catalog, one metastore is typically shared across an entire Azure region's workspaces.

Microsoft Entra ID — Microsoft's cloud identity service (formerly Azure Active Directory) used to manage users, groups, and service principals that Databricks authenticates against.

Notebook widget/parameter — An input control (text box, dropdown) at the top of a Databricks notebook that lets you pass a value into the notebook at run time, often used to parameterize jobs.

OPTIMIZE — A Delta command that compacts many small files into fewer, larger ones to improve read performance; can be combined with `ZORDER BY`.

Partitioning — Physically splitting a table's data files into separate folders based on a column's value (like date), so queries filtering on that column can skip irrelevant folders entirely.

Photon — Databricks' native, vectorized query engine (written in C++) that can dramatically speed up SQL and DataFrame workloads compared to standard Spark execution, at a higher per-DBU rate.

Query profile — A visual breakdown, in the Databricks UI, of how a specific query executed — stages, tasks, time spent, data scanned — used to diagnose performance problems.

Region — A physical Azure datacenter location (like East US or West Europe); choosing a region affects latency, data residency compliance, and sometimes cost.

Resource group — An Azure organizational container that groups related resources (like a Databricks workspace and its managed resource group) for management and billing purposes.

Row filter — A Unity Catalog feature that dynamically restricts which rows a user can see in a table based on a SQL function, enforced centrally at the table level.

Schema enforcement — Delta Lake's default behavior of rejecting writes that don't match a table's existing schema, preventing accidental corruption.

Schema evolution — An opt-in Delta Lake behavior that allows new columns in incoming data to be automatically added to a table's schema instead of failing.

Serverless compute — Databricks-managed compute that starts in seconds and requires no cluster configuration or management from the user, billed per usage.

Service principal — A non-human identity (in Microsoft Entra ID) used by applications, jobs, and automation pipelines instead of a real user's login.

Shuffle — The expensive process of redistributing data across partitions/executors over the network, typically triggered by operations like `groupBy`, `join`, or `repartition`.

Silver table — The middle layer of a medallion architecture; cleaned, validated, deduplicated, and conformed data, built from bronze.

Skew — An uneven distribution of data across partitions that causes a few tasks to take far longer than the rest, becoming the bottleneck for an entire stage.

Storage credential — A Unity Catalog object that securely stores the cloud identity (like a managed identity) Databricks uses to authenticate to a cloud storage account.

Structured Streaming — Spark's engine for processing data incrementally as a stream of micro-batches (or continuously), used for near-real-time pipelines.

Subscription (Azure) — A billing and access boundary in Azure; resources like a Databricks workspace live inside a specific subscription.

System tables — Databricks-managed Delta tables under the system catalog exposing operational data like billing, audit logs, and lineage, queryable with plain SQL.

Time travel — The ability to query or restore a Delta table as it existed at a previous version or timestamp, made possible by the transaction log.

Unity Catalog — Databricks' centralized governance layer providing a single place to manage access control, auditing, and lineage across all catalogs, schemas, and tables in an account.

VACUUM — A Delta command that permanently deletes data files no longer referenced by the current table version and older than a retention threshold.

Volume — A Unity Catalog-governed storage location for non-tabular files (images, PDFs, raw text) that aren't structured as a table.

Z-ORDER — A technique (used with `OPTIMIZE ... ZORDER BY`) that physically co-locates related data within files based on one or more columns, speeding up filters on those columns.

7.5 5. Your 30-Day Study Calendar

You have about a month, roughly 1-2 hours on weekdays and more on weekends. This calendar maps every part of this book onto that time, with the biggest domain (Prepare/Process Data, 30-35% of the exam) getting the most days, and enough spaced repetition of the practice bank that you're not cramming 300 questions into one panicked weekend. Adjust the exact days to your calendar, but keep the *order and rel-*

ative weighting — that reflects how the actual exam is weighted, not an arbitrary sequence.

Day	Focus	What to complete
1	Part 1 — Environment setup	Create Azure account/subscription, deploy a Databricks workspace, tour the UI
2	Part 1 — Environment setup	Clusters, cluster policies, notebooks, DBFS/volumes basics
3	Part 1 — Environment setup	Finish Part 1; do any Part 1 hands-on labs
4 (weekend)	Part 2 — Unity Catalog fundamentals	Metastore, catalogs, schemas, three-level namespace; create your first catalog/schema
5 (weekend)	Part 2 — Security & governance	Identities, groups, service principals, GRANT/REVOKE, row filters & column masks; hands-on
6	Part 2 — Governance wrap-up	External locations, storage credentials, managed vs. external tables; review Part 2
7	Part 2 review + light Part 3 preview	Skim Part 2 glossary terms; read Part 3 intro (medallion architecture)
8	Part 3 — Delta Lake fundamentals	ACID, transaction log, time travel; hands-on MERGE, OPTIMIZE, VACUUM
9	Part 3 — Ingestion	Auto Loader, COPY INTO, Lakeflow Connect basics; hands-on bronze ingestion
10	Part 3 — Transformations	PySpark/Spark SQL transformations, schema evolution/enforcement
11 (weekend)	Part 3 — Lakeflow Declarative Pipelines	@dlt.table/@dlt.view, streaming tables vs. materialized views; build a small pipeline
12 (weekend)	Part 3 — Data quality	Expectations (expect, expect_or_drop, expect_or_fail), hands-on quality pipeline
13	Part 3 — Streaming	Structured Streaming, watermarking, checkpoints
14	Part 3 review	Redo Part 3 hands-on labs from memory; skim Part 3 glossary terms
15	Part 4 — Lakeflow Jobs	Multi-task jobs, dependencies, retries, alerts; build a simple multi-task job

Day	Focus	What to complete
16	Part 4 — CI/CD	Databricks Repos, Git integration, Asset Bundles basics
17	Part 4 — CI/CD continued	databricks bundle workflow, dev/staging/prod targets; hands-on bundle deploy
18 (weekend)	Part 4 — Monitoring + start Part 5	Job monitoring, system tables for jobs; begin reading capstone project overview
19 (weekend)	Part 5 — Capstone: setup	Set up capstone environment, catalogs/schemas for the retail lakehouse
20	Part 5 — Capstone: bronze/silver	Build bronze ingestion and silver transformation layers
21	Part 5 — Capstone: gold + pipeline	Build gold aggregates, wire up the full Lakeflow Declarative Pipeline
22	Part 5 — Capstone: orchestration	Wrap the pipeline in a Lakeflow Job, add CI/CD via Asset Bundle
23	Part 5 — Capstone: finish + review	Finish capstone end-to-end, test failure/retry scenarios, review what you built
24	Part 6 — Practice bank, Domains 1-2	Timed set: Set up/configure environment + Secure/govern Unity Catalog questions
25 (weekend)	Part 6 — Practice bank, Domains 3-4 (part 1)	Timed set: Prepare/process data questions (this domain is large — take your time)
26 (weekend)	Full timed mock exam	One full 120-minute mock exam under real conditions (Part 6 bank), then score and log every miss
27	Review mock exam misses	Go through every wrong/guessed answer, re-read the relevant book section for each
28	Part 6 — Practice bank, Domain 4 (part 2)	Timed set: Deploy/maintain pipelines questions; re-check weak spots from Day 27
29	Light review only	Glossary pass (this chapter), re-skim flagged weak topics only — no new material
30	Exam day (or final light review + rest)	Exam Week Checklist below; if exam isn't today, one last light glossary/flashcard pass and rest

A note on pacing: if your mock exam score on Day 26 is comfortably above 700 (aim for

a buffer — 780–800+ is a safer cushion than a bare pass, since real exam conditions add nerves), go ahead and book the actual exam for a date in the following few days so the material stays fresh. If it's below 700, don't panic and don't book yet — push the exam back, spend two or three extra days re-working your specific weak domains using the practice bank, and re-take a second mock before scheduling. It's better to spend an extra week than to spend the (non-refundable, and not cheap) exam fee on an attempt you're not ready for.

7.5.1 Exam Week Checklist

- **Logistics, sorted early, not the morning of:** Confirm whether you're testing online (check your webcam, room setup, and government ID meet the online-proctoring requirements well in advance) or at a test center (confirm the address and arrive with time to spare). Have your photo ID ready and matching the name on your registration.
- **Do not cram new material the day before.** Anything genuinely new you learn in the last 24 hours is more likely to create confusion than add points. Use that day only for the glossary and your personal list of weak spots from mock-exam misses.
- **Sleep matters more than one more practice question.** A tired brain misreads exam questions — go to bed at a normal time; don't stay up doing "just one more" practice set.
- **The night before:** Re-skim this chapter's glossary once, lightly. Re-read your own notes on the two or three topics you got wrong most often in practice. That's it.
- **On exam day:** Eat something, arrive/log in early, and read every question carefully — DP-750 questions often hinge on one precise word ("managed" vs. "external," "expect" vs. "expect_or_fail"). If you're unsure, flag it and move on rather than burning your 120 minutes on one question.
- **One honest, important note:** Do not use exam-dump or leaked-question sites, no matter how tempting the "guaranteed pass" promise looks. Beyond being an explicit breach of the Microsoft certification NDA you agree to at registration, it puts your certification at real risk of being revoked if discovered, and it teaches you nothing that will actually help you in the job the certification is supposed to get you. Everything in this book was built to get you to a genuine pass — trust the process you just spent a month on.

Good luck — you've built a real project, you can explain your own decisions out loud, and you know exactly where the syntax lives when you need to look it up fast. That's what both the exam and the interview are actually testing.